

MEDIATEK

MT8668 Yocto AVM User Manual

版本：1.0
出版日期：2026-01-28

版本历史

版本	日期	作者	描述
1.0	2026-01-28	郑中华	正式版

目录

版本历史 3

目录 4

1 介绍 6

 1.1 功能 7

 1.2 缩略词 12

2 系统概述 13

 2.1 标定流量 16

 2.2 运行时系统 17

 2.3 生命周期 20

 2.4 显示管道 22

3 性能概览 27

 3.1 内存占用 33

 3.2 项目配置 35

4 开发套件 37

 4.1 AVM SDK 38

5 相机 40

 5.1 安装车轮轨迹线 43

6 标定过程 48

 6.1 预备知识 51

 6.2 相机标定 52

 6.3 内参生成 60

 6.4 AVM 标定 71

 6.5 在线标定 100

7 标定步骤 104

8 演示应用程序 107

 8.1 AVM 演示 107

 8.2 AVMCalarion 系统 114

 8.3 AVM 客户端实用程序 124

9 模块 138

 9.1 AVM 守护进程 138

 9.2 点检测库 144

 9.3 标定库 151

10 演示客制化 161

 10.1 布局 162

 10.2 3D 模型 185

 10.3 3D 相机 210

11 源代码 227

12 调试 AVM 229

13 版本 231

附件一 附加条款 233

1 介绍

2D/3D AVM 开发人员车轮轨迹线



MediaTek Around-View Monitor (AVM) 将最先进的环视功能集成到车载信息娱乐系统中。此整体解决方案提供易于使用的calibration toolchain、robust factory workflow和high-quality SDK。

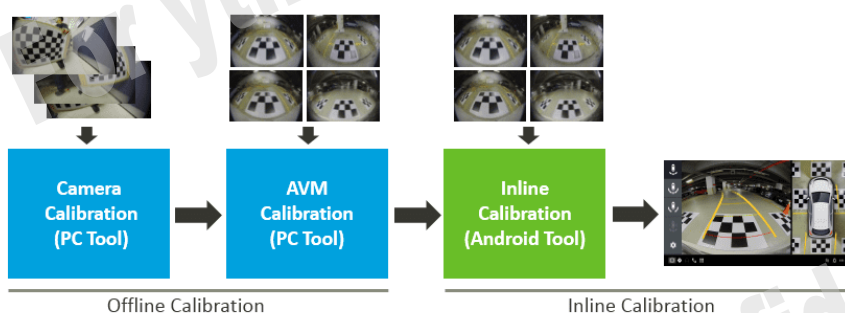
您将在此在线文档中了解以下主题：

- AVM 主要功能
- AVM 实际性能
- AVM SDK 里面有什么
- AVM 软件架构
- AVM 标定过程
- AVM 演示客制化

1.1 功能

端到端标定工具链

- 相机标定工具（PC）用于计算相机的固有参数
- AVM 标定工具（PC）用于计算 AVM 针迹参数
- 在线标定工具（Android）用于标定生产线上的每辆车



丰富的观看模式

- 流行的 AVM 视图模式的参考设计，包括：
 - 经过失真校正的相机视图
 - 鸟瞰图
 - 停车辅助系统的放大顶视图
 - 带有 3D 汽车模块和动画虚拟摄像机的 3D 视图模式
- 灵活添加自己的视图模式



高性能高品质拼接算法

- 实时算法将四幅源图像拼接成无缝拼接的鸟瞰图图像



Geometry Alignment Algorithm

to compensate each camera's installation variations and generate seamless floor image



Photometry Alignment Algorithm

to compensate each camera image's white balance & brightness variations

最先进的 3D 引擎

- 功能丰富的 3D 引擎，具有定制功能



3D Scene Generation



Transparent Windows



Dynamic Light



3D Obstacle Map (Radar)



Dynamic Objects



Exchangeable Skin



Transparent Model
+ Road Paving



3D Guidelines



OFF

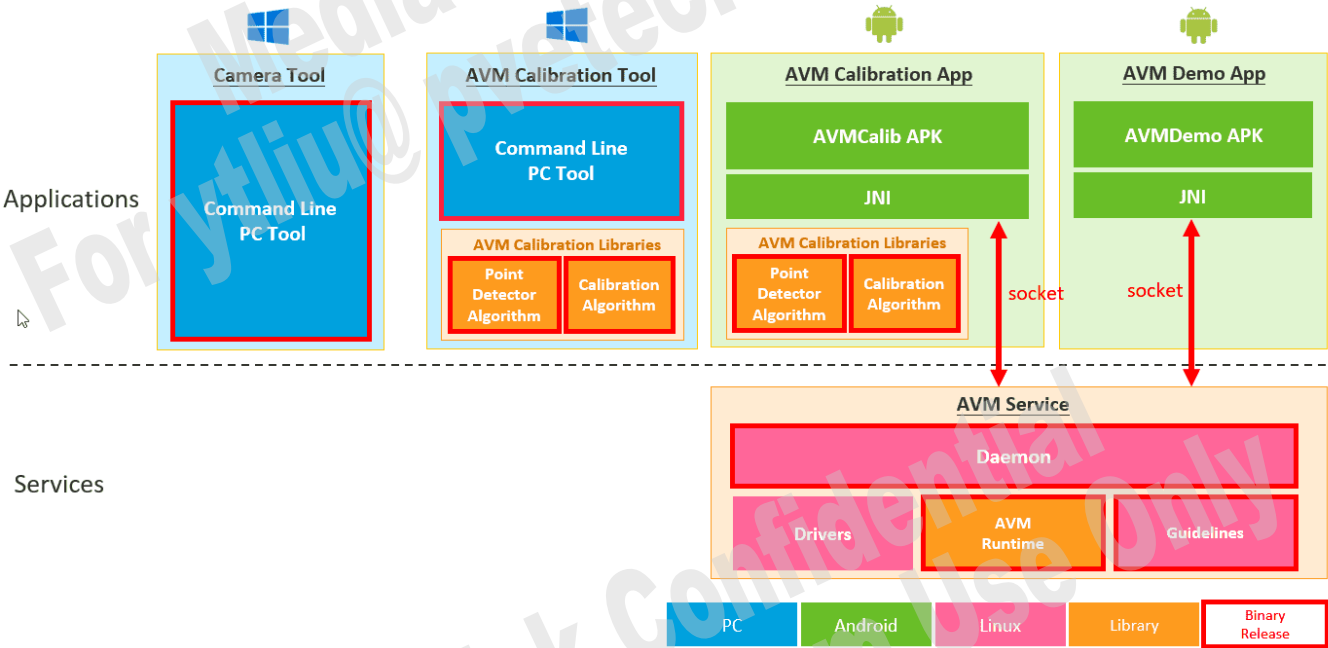


ON

Anti-Alias Support

全面的 SDK 和演示应用程序

- 提供具有高度定制灵活性的模块化库
- 提供演示应用程序作为参考代码，以加速开发过程



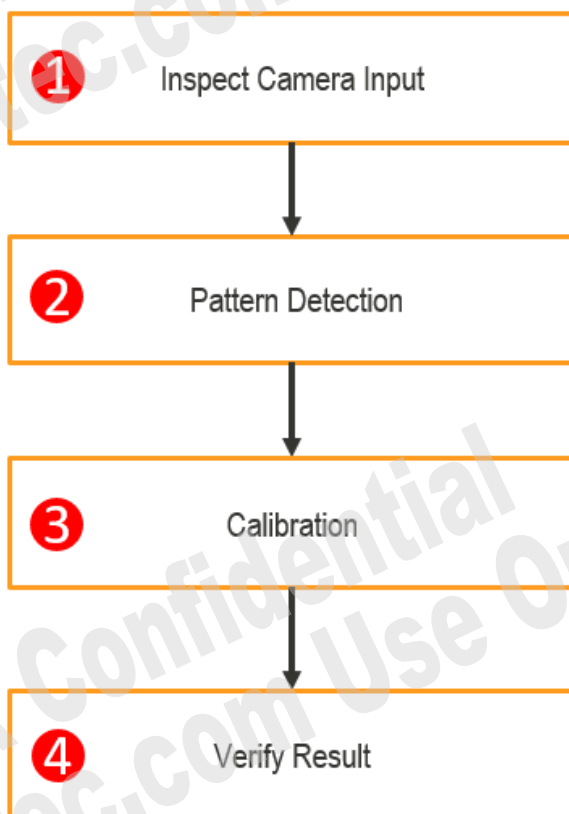
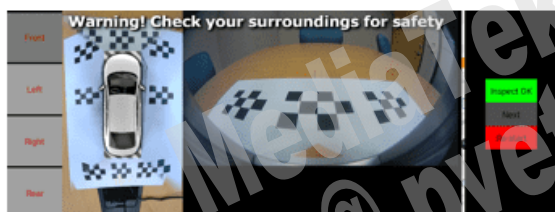
可定制车轮轨迹线

- 模拟方向盘输入角度来重现或模拟车辆的行驶轨迹
- 可定制的车轮轨迹线样式，具有颜色/纹理和 alpha 混合
- 抗锯齿支持，带来更好的视觉质量



参考在线标定流程

- 快速且强大的在线标定流程，适用于生产线上每辆车的标定
- 熟练工人可在 2 分钟内完成标定过程

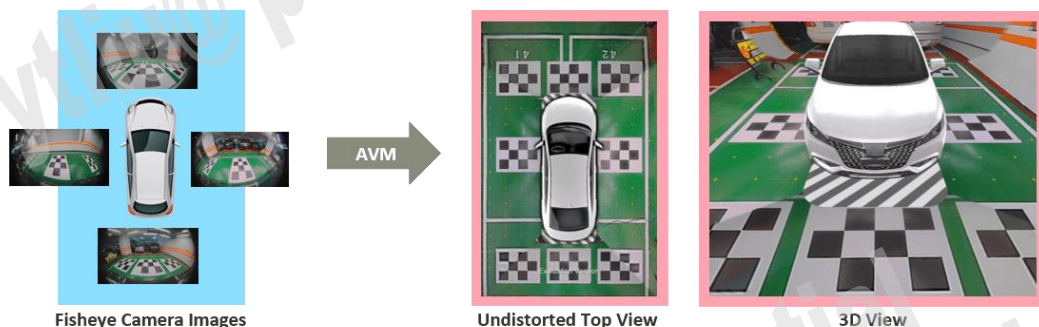


1.2 缩略词

缩略词	全称	释义
ADB	Android Debugging Bridge	是一个多功能命令行工具
AVM	Around View Monitor	全景监控器
BEV	Bird Eye View	鸟瞰图
EVS	Enhanced Voice Services	增强语音服务
FPS	Frame-per-second	每秒帧数
HAL	Hardware Abstraction Layer	硬件抽象层
OVL	MediaTek Display Overlay Unit	叠加
ROI	Region of Interest	感兴趣区域
SDK	Software Development Kit	软件开发包

2 系统概述

Around View Monitor (AVM) 系统的核心功能是将来自多个摄像头的输入图像转换为各种视图的无缝图像，为车辆驾驶员提供帮助。



为了实现这一点，系统必须获取各种参数，例如：

- 车辆上安装的每个摄像头的几何布局，通常表示为一系列矩阵，称为摄像头外参
- 镜头的光学畸变特性，这被称为畸变系数
- 每个相机的视图投影模型，也称为相机内参
- 车辆的尺寸，例如轴距、轮距等

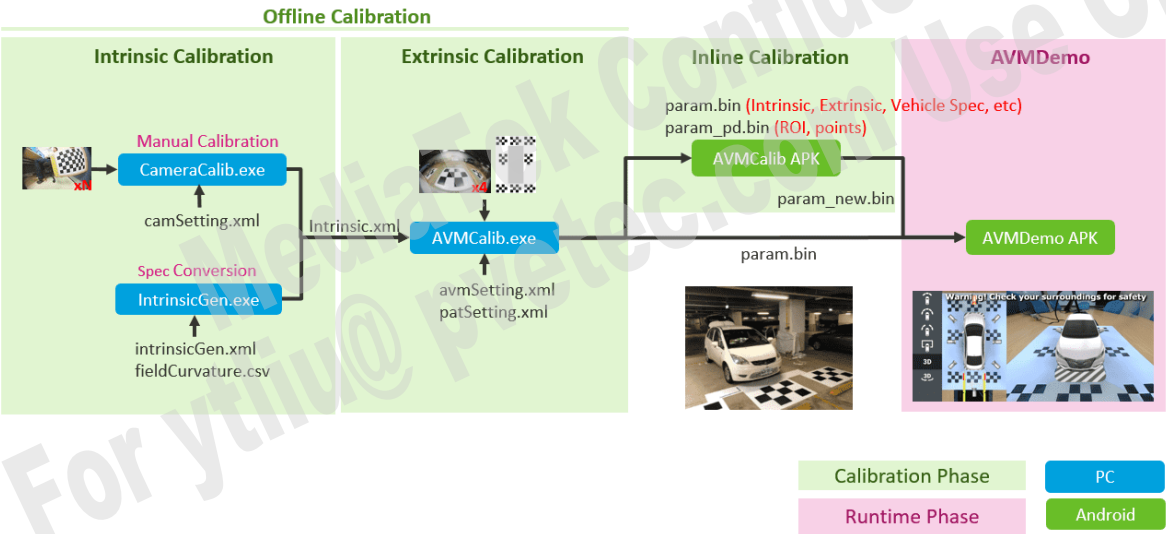
然后，AVM 系统使用这些参数将实时摄像机输入转换至车载显示屏以供用户交互。

标定和运行阶段

从最高层次来看，整个 AVM 功能的开发可以分为两个主要阶段：

- 标定阶段是获取和计算上述所有必要参数的过程。此阶段涵盖收集摄像头和车辆数据的所有活动。
- 在运行阶段，车载嵌入式硬件根据标定过程的结果执行计算，将摄像机输入图像变换拼接成各种视图，例如模拟鸟瞰摄像机或虚拟 3D 摄像机。

这两个阶段的总体工作流程如下所示：



标定过程

在标定阶段，执行标定过程。标定过程可能在不同情况下执行：

- 在系统开发过程中，采用离线标定过程通过一组输入数据确定相机内参和外参。
- 在工厂生产期间，可能会执行在线标定过程以补偿不同车辆和摄像机之间的细微差异。

然后，标定结果将存储为参数文件，如上图所示。param.bin 和 param_new.bin 文件分别是 PC 标定工具和在线标定工具生成的参数文件。

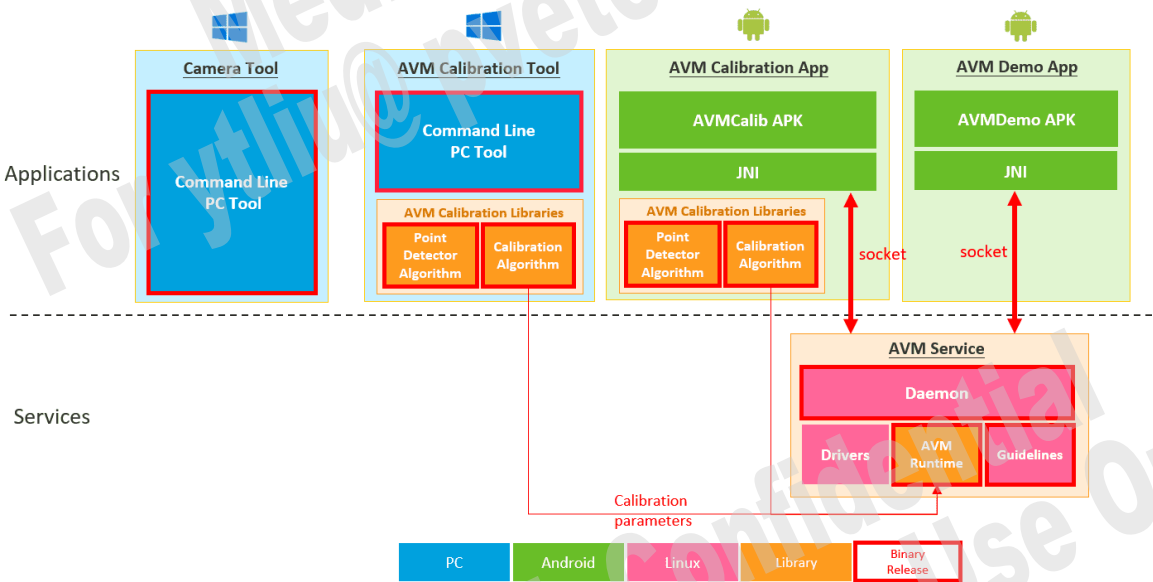
离线标定过程在一组 PC 工具上执行。除了 PC 工具外，还提供了在 MT2712 Android 平台上运行的在线标定应用程序，用于工厂生产场景中的每辆车标定。

运行时系统

这运行时系统是软件的集合，包括库、服务和示例应用程序，用于支持 AVM 运行时阶段。运行时系统完全在 MT 2712 Android 平台上运行。车辆上路后，运行时系统会根据磁盘上的参数文件 param.bin 和 param_new.bin 将摄像头图像转换为最终显示输出。运行时系统不会修改或重新计算参数文件。运行时系统采用应用程序服务架构，如下章节所述。

SDK 架构

AVM SDK 提供了一组软件工具、库和示例应用程序，以支持标定过程和运行时系统的开发。这些软件可以归类为应用程序或服务，如下所示：



应用

SDK 提供了用于相机标定、AVM 标定以及 Android 内联标定的 PC 工具。

作为标定阶段的一部分，Android AVM 标定应用程序已发布源代码，允许开发人员自定义工厂生产工作流程。开发人员还可以通过向标定工具提供自定义点检测器算法来自定义 AVM 标定中使用的图案。请参阅标定工作流程页面以了解这些标定应用程序的概述。

对于运行时阶段，提供了 AVM 演示应用程序作为 MT2712 Android 平台上 AVM 应用程序的参考设计。

在线标定和 AVM Demo 应用程序都是轻量级应用程序，仅包含 UI 按钮和交互逻辑。这些应用程序不控制 AVM 系统的实际显示。相反，它们将套接字请求发送到底层服务以满足用户请求。

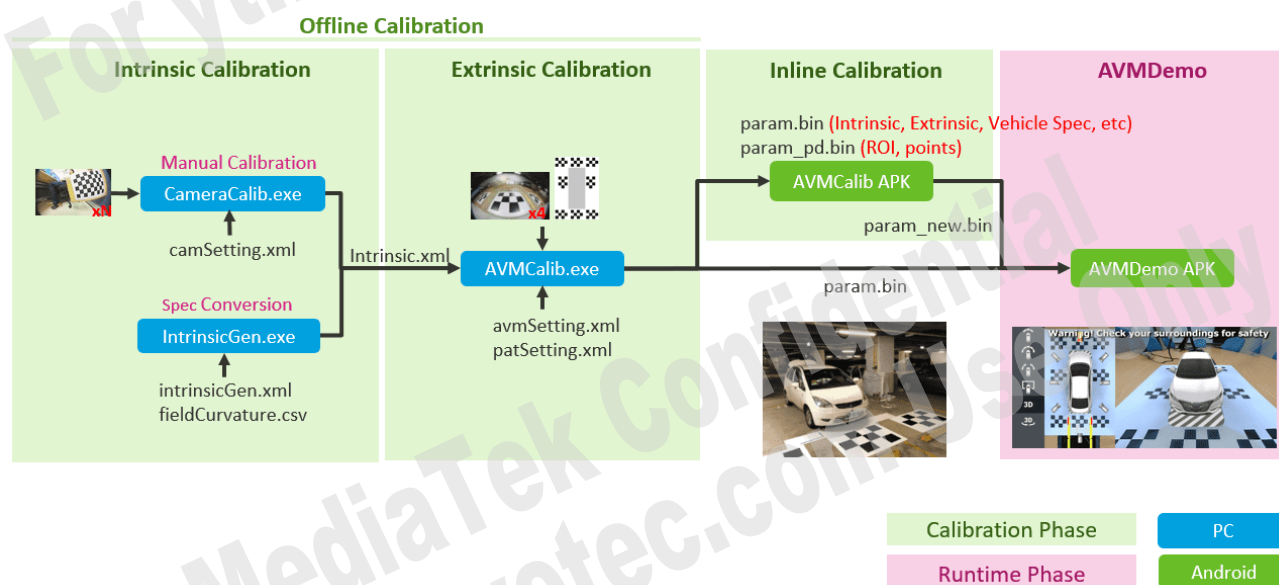
服务

实际的摄像头和显示管道由 AVM Daemon 服务根据标定过程中生成的标定参数文件进行控制。AVM 守护进程负责管理硬件资源，例如摄像头和显示硬件。它还管理所需的所有硬件内存缓冲区。大多数图像处理功能（例如

AVM 图像拼接和扭曲算法) 都封装在名为 AVM 运行时的库中, 但车轮轨迹线除外, 它是 AVM 守护进程的一部分。

有关服务设计的更多信息, 请参阅运行时系统页面。

2.1 标定流量

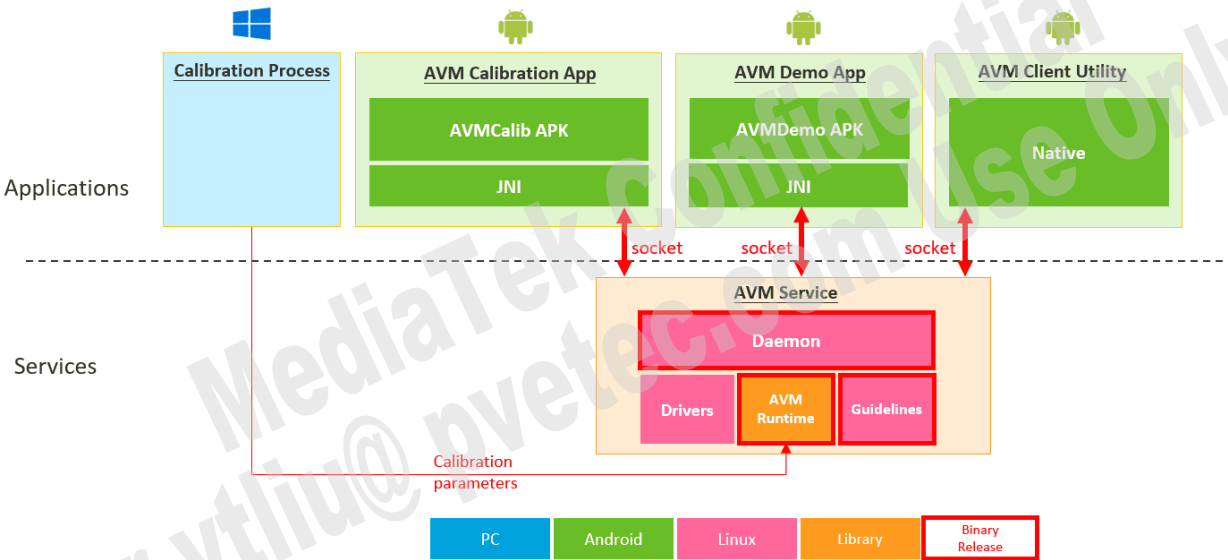


标定流程可分为3个阶段:

- **内在标定:** 我们提供两种方法 (手动标定或曲率表转换) 来生成内参。在我们的定义中, 内参包括鱼眼畸变系数 (k_1 、 k_2 、 k_3 、 k_4) 和针孔相机模型 (c_y 、 c_x 、 f_x 、 f_y)。
- **外部标定:** 我们从目标车辆捕获前/后/左/右图像, 并应用 AVM 标定过程将这些图像与标定图案的地面真实布局进行匹配, 以生成标定参数。这些参数是摄像头的 RT 矩阵 (旋转和变换)。我们的运行时算法将使用这些指标将摄像头图像重新投影到 2D AVM 的拼接鸟瞰图像或 3D AVM 的 3D 场景中。
- **在线标定:** 我们为生产线上的每辆车标定提供参考流程, 以减少因摄像头差异和安装公差差异而导致的质量下降。摄像头差异来自镜头、CMOS 和装配差异, 这意味着每台摄像头都需要一组不同的内部参数才能获得更好的结果。我们先进的标定算法将自动调整每台摄像头的内部和外部参数, 并为车辆生成更好的设置。

2.2 运行时系统

运行时系统设计



如概述部分所述，AVM 运行时系统负责获取标定过程生成的参数文件，然后在 Android 平台上执行整个摄像头到显示器的数据转换。

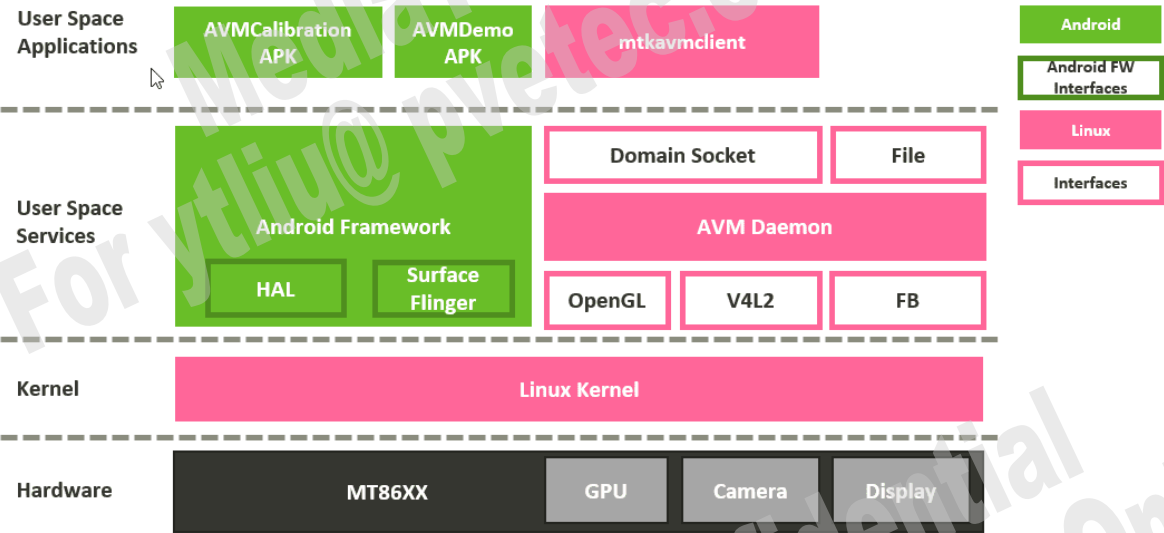
要实现的主要目标：

- 高性能：AVM 运行时必须足够快，以便为车辆驾驶员提供低延迟显示。它还必须为同一平台上的其他任务留出足够的处理能力。例如，GPS 导航或音乐播放组件可能同时运行。

为了减轻 CPU 核心的负担，运行时系统设计为利用基于 OpenGL 的 GPU 硬件。

应用程序和服务

基于这些概念，AVM 运行时组件分为应用程序和服务。应用程序提供用户界面，例如 Android UI 按钮或命令行工具。实际的 AVM 功能由名为 AVM Daemon 的服务实现。该服务访问硬件资源，如下面的框图所示。



应用

提供了三个演示应用程序：

- mtkavmclient 是一个原生命令行程序，可让您通过 Android Debug Bridge (ADB) shell 快速测试 AVM 功能。此程序还可方便地捕获相机图像，这些图像可用作相机标定过程的输入。
- AVMDemo 和 AVMCalibration 是 Android 演示应用程序，展示了重要的 AVM 功能和内联标定过程。

这些演示应用程序附带源代码，可以作为开发 AVM 应用程序时的示例。

服务

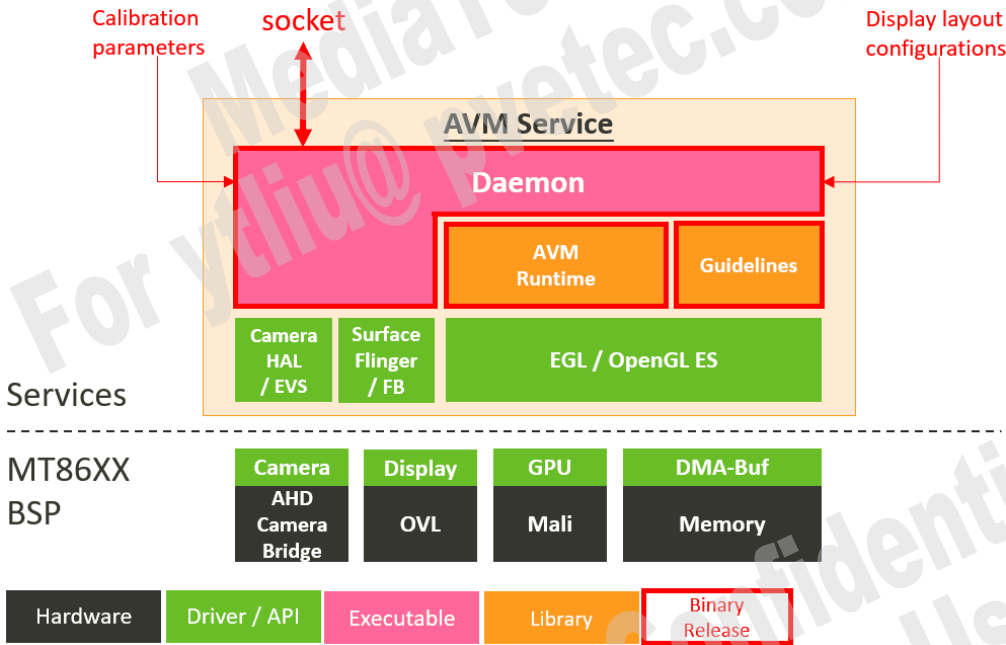
核心 AVM 功能由原生 Linux 用户空间服务 AVM 守护进程提供。该守护进程使用 OpenGL ES 3 访问 GPU 资源。它通过域套接字向 AVM 应用程序提供接口。它充当服务器，向 AVM 应用程序提供 AVM 功能。

客户端和服务器的分离使得 AVM 守护进程能够尽早初始化。有关 AVM 守护进程初始化流程的详细说明，请访问生命周期页面。

标定参数和其他显示布局配置存储在文件系统中，并在初始化期间由守护进程解析。

用户输入（例如触摸事件）由 AVM Android 应用程序（例如 AVMDemo）处理。这些应用程序充当 AVM 守护进程的客户端。服务器和客户端之间的通信通过 Linux 域套接字使用简单协议进行传输。

该服务由以下模块组成：



模块名称	二进制名称和位置	功能
AVM 守护进程	/system/bin/mtkfastavm	生命周期 控制和 硬件管道 管理
AVM 车轮轨迹线 和图库	/system/lib64/libmtkavmproc. so	GPU 初始化和 动态车轮轨迹线
AVM 运行时库	/system/lib64/libmtkavm.so	图像拼接和变换算法实现

在运行时阶段，AVM 服务是执行 mtkfastavm 的单个进程。可执行文件动态链接到图形库 libmtkavmproc.so 和 AVM 运行时库 libmtkavm.so。

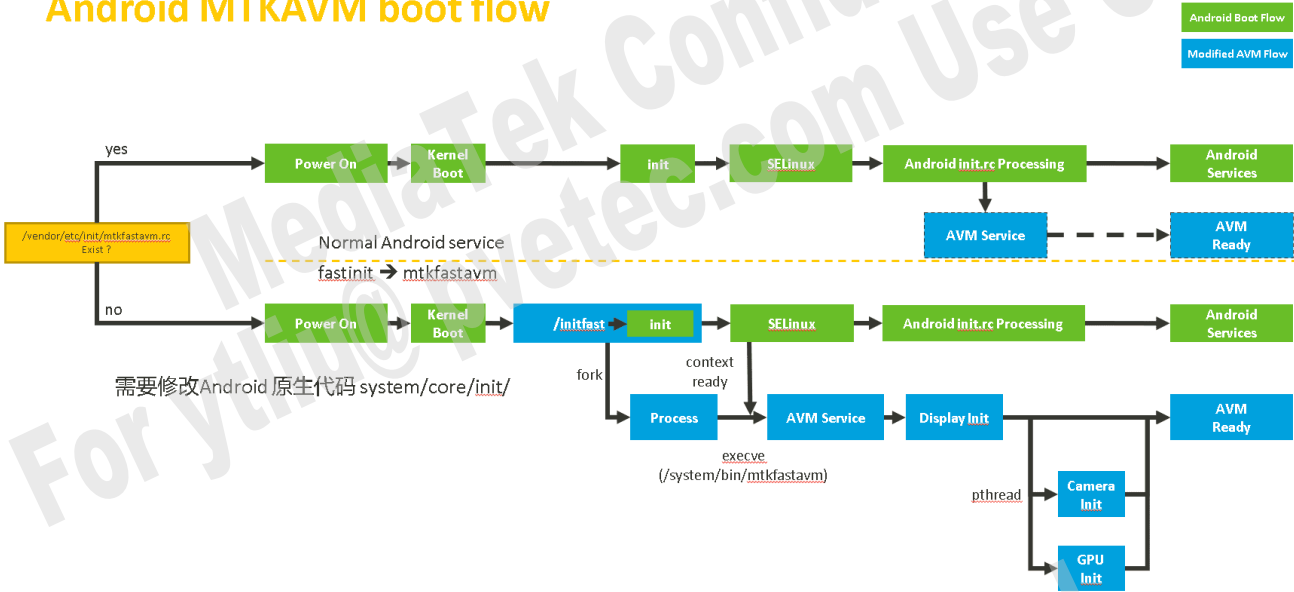
启动流程

Android

启动 mtkfastavm 服务有两种方式：

- 在 android init 中 fork mtkfastavm 进程。这种方式需修改 android init 的源代码：system/core/init
- mtkfastavm 作为 android 服务启动。这种方式需要在 Android.mk 中设置 LOCAL_INIT_RC := mtkfastavm.rc。在 mtkfastavm.rc 中，mtkfastavm 会在 early-init 信号时启动。

Android MTKAVM boot flow



2.3 生命周期

服务生命周期

运行时系统分为应用程序和服务 AVM 守护进程。本节介绍 AVM 守护进程和应用程序的生命周期以及它们之间的交互。

AVM 守护进程生命周期

AVM 守护进程作为 Linux 原生可执行文件，在系统初始化期间初始化并且始终处于活动状态。初始化时，守护进程会加载所有标定参数文件和显示布局配置文件，并相应地分配显示和摄像头资源。然后它通过来自应用程序的套接字命令等待启动显示事件，并且永不终止。由于守护进程永不终止，因此初始化完成后它只有 2 个状态：

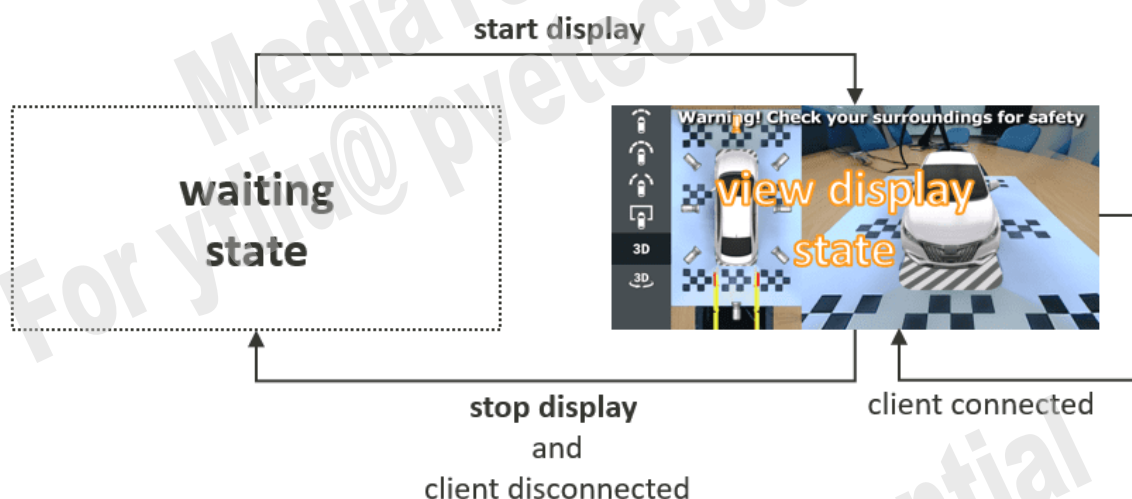
- 等待：在此状态下，AVM 显示屏被隐藏，摄像头流停止。如果出现以下情况，此状态将转换为查看显示状态：
 - 应用程序发送开始显示命令
- 视图显示：在此状态下，AVM 显示屏可见，并且摄像机流连续滚动。

活跃显示模式由内部的模式变量决定。客户端应用程序可以更改活跃显示模式。

- 一个视图中的后置/前置摄像头由倒档状态决定。
- 如果出现以下情况，此查看模式将转换至等待模式：
 - 客户端发送停止显示命令，或者

- 客户端断开连接且倒档状态未设置。

下图说明了这一点：



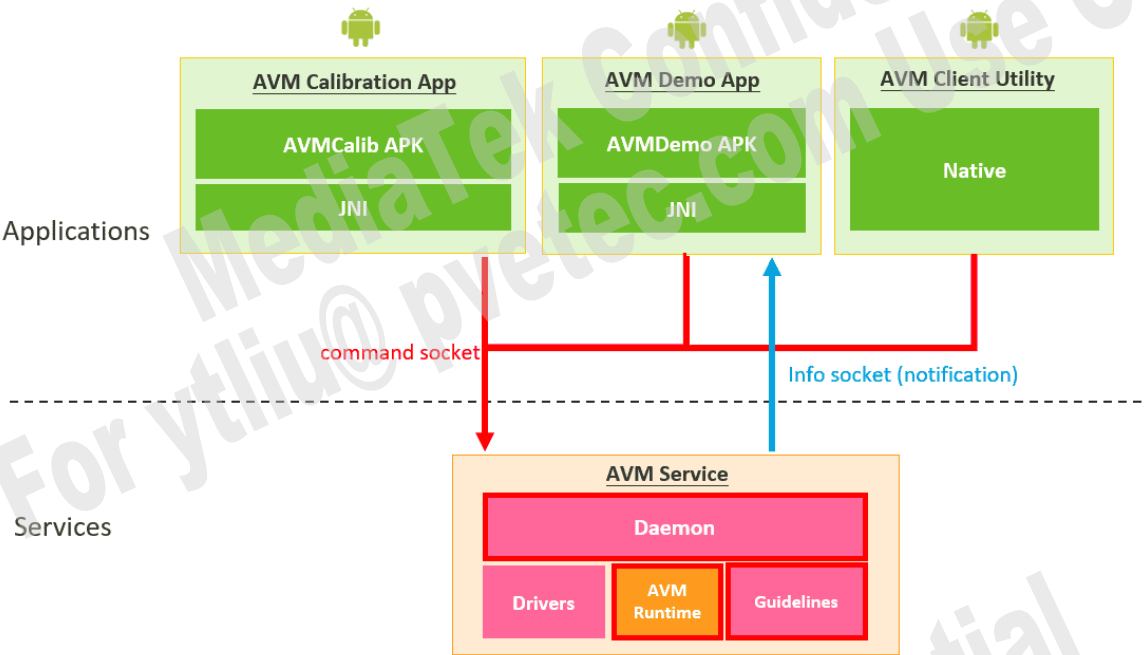
应用程序守护进程连接

AVM 守护进程实际上为应用程序提供了两个套接字端口：

- 应用程序通过命令套接字向守护进程发送命令。应用程序可以通过此套接字控制守护进程的可见性和活动视图模式。
- 信息套接字由守护进程提供。应用程序订阅此套接字并接收来自守护进程的事件通知。

这些套接字是独占访问的，因此应用程序必须协作释放这些套接字。在当前的参考设计中，

- mtkavmclient 实用程序连接到命令套接字，发送命令，然后立即断开连接
- AVMDemo 和 AVMCalibration 应用程序在前台时连接到命令套接字，并在暂停时断开连接。



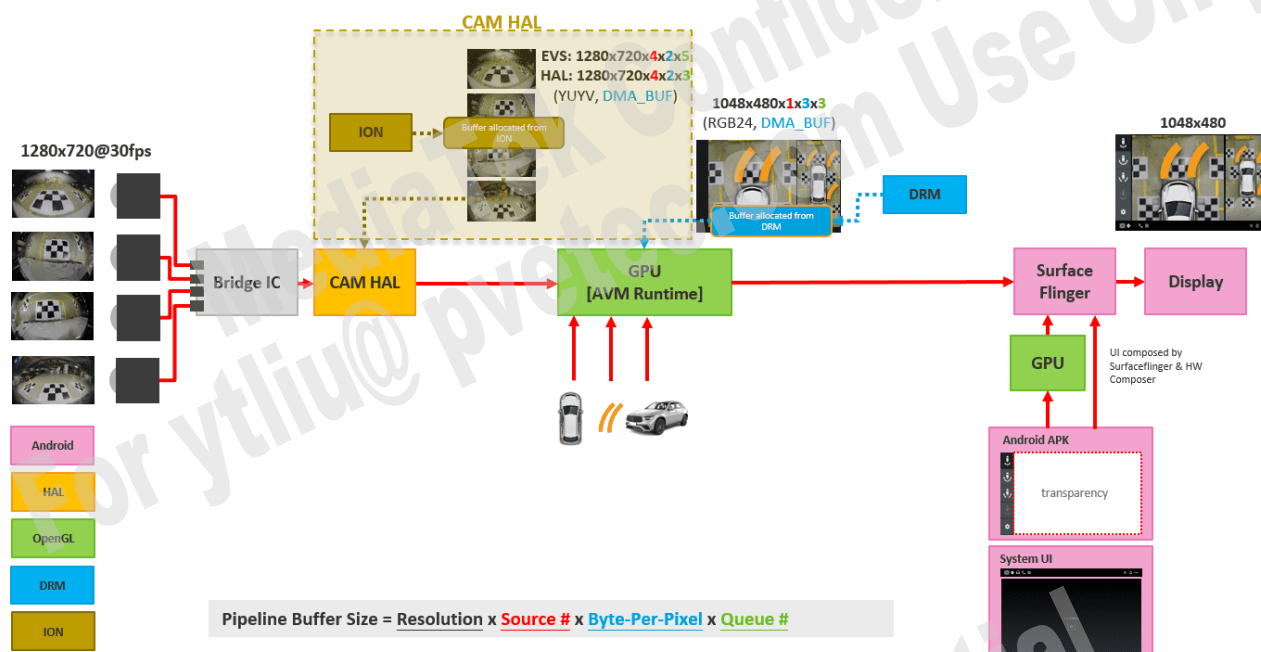
2.4 显示管道

AVM 运行时系统的核心是从摄像头到 GPU 再到显示器的数据处理管道。本节介绍 AVM 守护进程中硬件管道的总体设计以及联发科技 SPM Android 平台中显示路径的配置。

硬件流水线

AVM 运行时系统涉及多个硬件组件。我们从摄像头到显示器开始列出组件：

- 需要外部 AHD 桥接 IC 将多个 AHD 摄像头连接到联发科 SPM 平台。
- CAM HAL（Android 相机硬件抽象层）用于提供相机帧。可配置 EVS 或 HAL 源类型。
- 联发科技 SPM 平台中的 GPU 用于执行实际的图像拼接和扭曲算法。
- Surface Flinger 是连接到联发科 SPM 平台主显示屏的显示合成模块，可合成 Android 应用程序的 UI 和 AVM 显示输出。



AVM守护进程负责构建硬件数据管道并分配所需的资源。

缓冲区分配和使用

由于AVM运行系统本质上是一个数据管道，输入输出缓冲区传递的效率成为首要考虑的问题，实现目标是尽量减少冗余的缓冲区复制和格式转换。

为了最大限度地减少冗余缓冲区复制，所有渲染资源分配均由 AVM 守护进程控制。在初始化期间，AVM 守护进程通过 libDRM 将所有所需的输入和输出缓冲区分配为 Linux DMABuf。

一旦相机缓冲区准备就绪，守护进程就会通过 EGL_KHR_image_base 扩展将 DMABUF 句柄转换为 EGLImage 纹理句柄。由于相机缓冲区是 YUYV 格式，因此使用扩展 EXT_YUV_target 来防止在格式转换期间进行冗余缓冲区复制。然后使用 OpenGL API 生成大多数图形元素（例如车轮轨迹线、2D 汽车图像和 3D 汽车模型），并将其渲染到同一个输出缓冲区上。

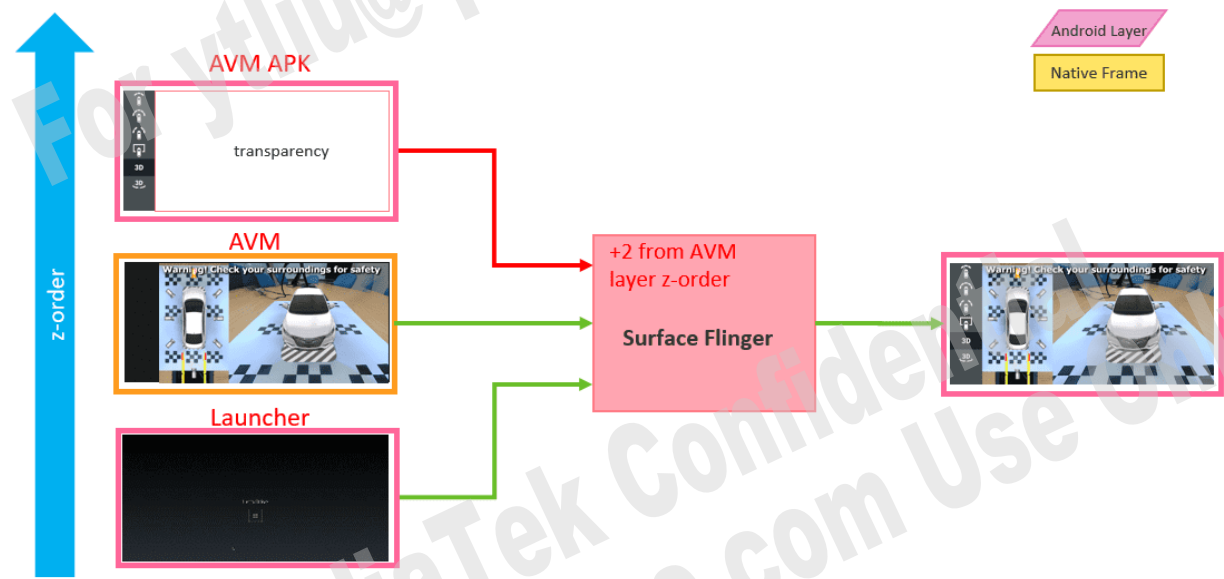
然后将输出缓冲区传递给 SurfaceFlinger。

通过利用 DMABUF 框架和 GPU 驱动程序扩展，运行时系统可防止 CPU 不必要地访问缓冲区，从而最大限度地减少 CPU 负载和内存带宽消耗。

硬件管道的设置和控制 在 AVm 守护进程中实现。GPU 和 OpenGL 相关的实现主要位于 AVm Runtime 库中。

显示组成

由于 AVm 屏幕是始终位于顶部的native layer，因此 Android 框架可能会在 AVm 屏幕顶部合成其他 UI 元素，例如应用程序 UI 组件。AVmCalibration 应用程序就是一个例子，它需要在 AVm 显示屏顶部合成其他 UI 元素，如下图所示：



因此，Surface Flinger 被修改来识别从 AVm Android 应用程序发送的表面。对于那些 Z order应该大于 AVm 本机层（在 AVm 守护进程的输出之上合成）的表面，会分配一组特殊的 Z 顺序值。

Surface Flinger 的修改

- 找到 frameworks/native/services/surfaceflinger，并将您的 APK 列表添加到其中。

```
String8 ActivityWhitelist [WHITE_LIST_LENGTH]= {
android :: String8 ( “ com.mediatek.avmcalibration / com.mediatek.avmcalibration.CaptureActivity” )，
android :: String8 ( “ com.mediatek.avmcalibration / com.mediatek.avmcalibration.InspectActivity” )，
android :: String8 ( “ com.mediatek.avmcalibration / com.mediatek.avmcalibration.VerifyActivity” )，
android :: String8 ( “ com.mediatek.avmcalibration / com.mediatek.avmcalibration.CalibrationActivity” )，
android :: String8 ( “ com.mediatek.avmcalibration / com.mediatek.avmcalibration.SplashActivity” )，
android::String8( “com.mediatek.avm/com.mediatek.avm.MainActivity” )
};
bool traverseWhiteList(String8 strName){
for (size_t i = 0 ; i <WHITE_LIST_LENGTH ; i ++ ) {
if (strName.find(ActivityWhitelist[i]) == 0)
return true ;
}
}
```

```
return false;
}

...

status_t SurfaceFlinger::addClientLayer()
{
    ...
    /// M: for avm function @ {
    if(traverseWhiteList(lbc->getName())) {
        whiteListLayer = lbc;

        if (avmLayer != nullptr)
            whiteListLayer->setLayer(avmLayer->getZ()+2);

        mCurrentState.layersSortedByZ.add ( lbc ) ;
        ALOGI("add whitelist layer : %s ",lbc->getName().string());
    }else {
        parent->addChild(lbc);
    }
    /// @}

    ...
}
```

通过这些修改，AVM 系统允许应用程序和系统 UI 与 AVM 守护进程的显示输出正确复合。

然而，这意味着该系统存在一些局限性：

- 开发人员必须定义好应用程序的层次顺序和系统UI。
- 开发人员必须修改 SurfaceFlinger 才能使用不同的 AVM 应用程序包名称。

MediaTek Confidential
For ytliu@pvetec.com Use Only

MediaTek Confidential
For ytliu@pvetec.com Use Only

MediaTek Confidential
For ytliu@pvetec.com Use Only

3 性能概览

本节概述了 AVM 功能的重要性能指标，包括 CPU/GPU 负载和每秒帧数 (FPS)。

MT86XX 系列

Part #	MT8666	MT8667
CPU	4x CA73 @ 2.2G 4x CA53 @ 2.2G	2x CA75 @ 2.0G 6x CA55 @ 1.7G
GPU	Mali-G72 MP3 @ 800MHz 86G FLOPS	ARM52 2EEMC2 @ 620MHz
DRAM	LPDDR4-1800, 16 bits x 4 ch, Max. 8GB	
Storage	eMMC 5.1, uFS 2.1	eMMC 5.1
Video output	DSI 4lane 1080 + DPI 720x2	DSI 2520x1080
Video input	Multi CAM Max. 6	
VPU	1080p60 decoding (H.264/H.265), JPEG decoder, 1080p30/720p120 encoding (H.264)	2Kp30 decoding (H.264/H.265), JPEG decoder, 2Kp30 / 720p120 encoding (H.264/H.265)
Peripheral	USB2.0/OTG, USB3.0/device, 3x UART, 6x SPI masters, 6x I2C, GPIOs	USB2.0/OTG, 2x UART, 6x SPI masters, 9x I2C, GPIOs

性能指标

本文包含以下性能指标：

- FPS：**AVM 守护进程将 FPS 统计信息输出到 Android 日志。在内部，AVM 守护进程计算发送到 Surface Flinger 的渲染帧数，并计算 5 秒内的平均值。
- 摄像头到显示屏的延迟：**启动过程中，使用 120 FPS 摄像头捕捉 EVB，EVB 旁边放置秒表。延迟的计算方法是将显示屏上的时间值和秒表上的时间值相减。

- **CPU 负载：**我们给出了整体系统 CPU 负载和 AVm 进程 CPU 负载的测量结果。整体系统负载是在以下条件下使用 MET 工具测量的：

- 使能性能模式：

- 锁定CPU核心数及频率：

- 锁定CPU核心数的命令

```
// m 代表 cluster0, n 代表 cluster1; m, n = 0~4, -1 代表免费限制
adb shell "echo mn> /proc/ppm/policy/ut_fix_core_num"
```

- 锁定CPU频率的命令

```
// m 代表 cluster0, n 代表 cluster1; m, n = 0~15; 0 为最高频率; -1 为自由限制
adb shell "echo mn> /proc/ppm/policy/ut_fix_freq_idx"
```

- 例子

```
adb shell "echo 4 4 > /proc/ppm/policy/ut_fix_core_num"
adb shell "echo 0 0 > /proc/ppm/policy/ut_fix_freq_idx"
```

- 锁定GPU频率：

1. 首先，查看GPU频率支持列表

```
adb shell "cat /proc/gpufreq/gpufreq_opp_dump"
```

2. 然后修改GPU频率

```
// f 是您想要设置的频率
adb shell "echo f> /proc/gpufreq/gpufreq_opp_freq"
```

3. 最后，检查当前 GPU 频率

```
adb shell "cat /proc/gpufreq/gpufreq_var_dump"
```

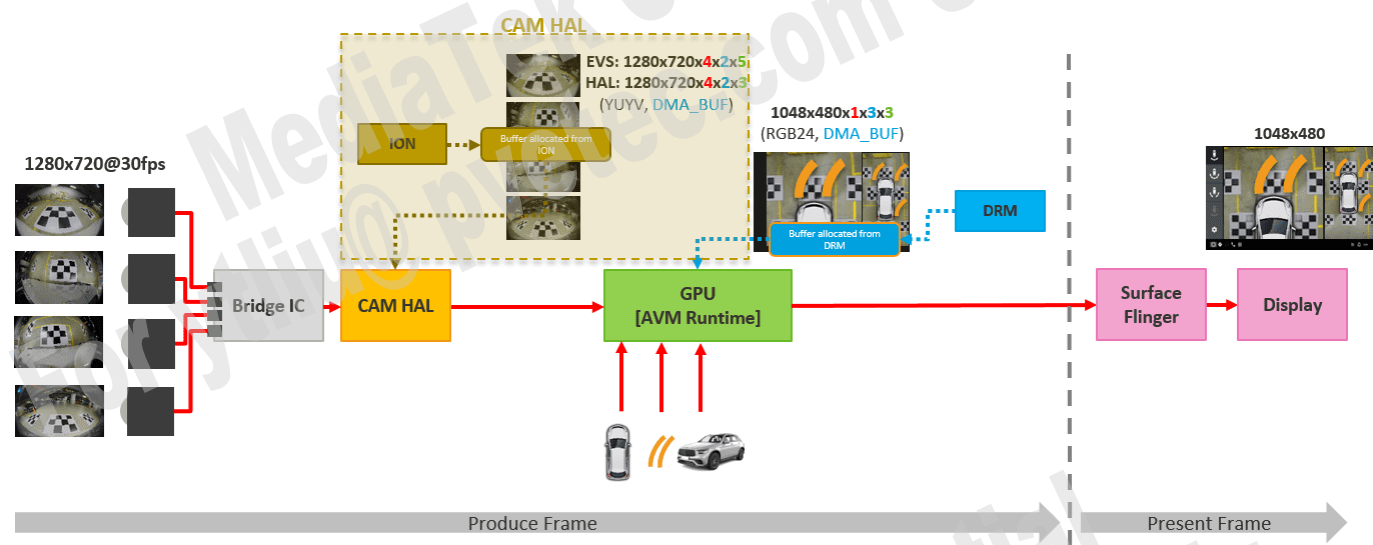
- 不使用 Android UI 运行，因为根据客户的要求，它会进行很大改变

```
adb root
adb shell am force-stop com.mediatek.avm
```

实际上，整个系统负载受许多组件的影响。在 AVm 显示管道中，它包括摄像头流、DRM、图像拼接、2D/3D 渲染算法和 Surface 显示。如有必要，我们可以进一步将管道细分为 2 个阶段进行研究。

- 生成帧：这是 AVm 功能的主要功能，包括将摄像机图像作为输入并方便 GPU 渲染到目标缓冲区。

- **Present Frame**: 在目前的设计中，目标缓冲区是一个 Surface。然后与 Android 系统的其他层合成以产生最终的屏幕显示结果。



- **GPU 负载**: 用 MET 工具测量。
- **内存带宽**: 用 MET 工具测量。

MET 工具需要 Linux 内核检测和 Android 调试桥 (ADB) 的存在。

用于测量FPS和CPU / GPU负载的系统配置如下表所示:

Performance Indict or	Android Configuration	ADB Enabled	MET Enabl ed	Verbose ILog fro m Bootloader & K ernel
CPU/GPU loading m emory bandwidth FP S	userbug	Yes	Yes	Yes

测量环境

- 评估板: MT8667
- DRAM: 2GB LPDDR4-2800, 64 位
- 显示: 1048x480
- 摄像头模块: AHD摄像头 (1280x720)
- Android版本: Android 10

- 软件发布版本：AVM 2.3-spm

重要的提醒

以下性能统计数据是使用 2.3-spm 版软件测得的。新功能可能会影响性能统计数据。

MT8666 性能

关键性能指标

- 摄像头到显示器延迟：~100ms (± 10 ms) (用户构建超过 10 个样本)
- 内存占用：~235MB
- 磁盘存储占用空间：~48MB

CPU/GPU 负载

- 整体系统 CPU 负载：用 MET 工具测量。
- 进程CPU使用率：用top命令测量，进程名称：mtkfastavm。
- 场景：AVM 使用基本视图运行（默认启用光度标定）

View Mode	FPS	Overall System CPU Loading	Process CPU Usage	GPU Loading	Memory BW
Launcher(Idle)	N/A	A73: 100%, 0.4%, 0.3%, 0.6% A53: 2.0%, 0.0%, 0.0%, 0.0%	0%	0%	268 MB/s
Rear	25	A73: 7.0%, 8.2%, 5.7%, 6.5% A53: 2.4%, 0.9%, 0.1%, 0.1%	14%	4%	828 MB/s
Rear Wide	25	A73: 10.0%, 7.8%, 7.6%, 8.4% A53: 3.0%, 0.8%, 1.1%, 0.1%	16%	3%	643 MB/s
Rear Side	25	A73: 8.2%, 8.5%, 7.4%, 6.9% A53: 2.6%, 0.5%, 0.4%, 1.0%	16%	4%	804 MB/s
Rear Top	25	A73: 7.0%, 8.4%, 6.4%, 6.7% A53: 3.0%, 1.0%, 0.3%, 0.4%	15%	6%	939 MB/s
Rear 3D	25	A73: 10.9%, 10.9%, 6.3%, 7.3% A53: 3.3%, 0.8%, 0.7%, 0.1%	20%	18%	1250 MB/s
Free 3D	25	A73: 8.7%, 11.5%, 8.6%, 7.5% A53: 3.6%, 1.0%, 1.2%, 0.7%	19%	21%	1255 MB/s
Free 3D, but photometric alignment disable	25	A73: 8.1%, 7.9%, 8.7%, 7.7% A53: 3.2%, 0.8%, 1.0%, 0.0%	17%	20%	1249 MB/s

MT8667 性能

关键绩效指标

- 摄像头到显示器延迟：~100ms (±10ms) (用户构建超过 10 个样本)
- 内存占用：~235MB
- 磁盘存储占用空间：~48MB

CPU/GPU 负载

- 整体系统 CPU 负载：用 MET 工具测量。
- 进程CPU使用率：用top命令测量，进程名称：mtkfastavm。
- 场景：AVM 使用基本视图运行（默认启用光度标定）

View Mode		FPS	Overall System CPU Loading	Process CPU Usage	GPU Loading	Memory BW
Launcher(Idle)		N/A	A75: 100%, 7.0% A55: 3.7%, 0.6%, 2.1%, 0.1%, 0.6%, 0.1%	0%	0%	340 MB/s
Rear		25	A75: 12.7%, 8.8% A55: 6.2%, 2.3%, 1.3%, 1.3%, 0.6%, 0.4%	12%	5%	876 MB/s
Rear Wide		25	A75: 11.1%, 13.5% A55: 7.3%, 2.9%, 1.5%, 1.0%, 0.6%, 0.5%	12%	4%	680 MB/s
Rear Side		25	A75: 13.8%, 18.0% A55: 8.0%, 3.6%, 2.0%, 2.0%, 0.8%, 0.5%	13%	4%	908 MB/s
Rear Top		25	A75: 18.4%, 16.6% A55: 7.6%, 3.2%, 2.1%, 2.3%, 0.4%, 0.4%	13%	5%	1048 MB/s
Rear 3D		25	A75: 12.5%, 15.2% A55: 9.2%, 3.6%, 2.3%, 0.4%, 0.5%, 0.3%	18%	20%	1278 MB/s
Free 3D		25	A75: 14.9%, 14.3% A55: 8.4%, 3.2%, 1.6%, 1.0%, 1.1%, 0.3%	17%	25%	1259 MB/s
Free 3D, but photometric alignment disable		25	A75: 13.8%, 15.3% A55: 8.4%, 2.9%, 1.3%, 1.7%, 0.5%, 0.4%	15%	23%	1187 MB/s

3.1 内存占用

本章节显示了支持 3D 的 AVM 的测量内存消耗，包括磁盘存储内存消耗和运行时内存消耗。

磁盘存储占用空间

AVM 使用的文件总大小（包括默认 3D 资产文件和内联标定文件）约为 48MB。3D 资产（包括汽车模型和纹理）是主要贡献者，占总文件大小的 50% 左右。另一个主要贡献者是用于内联标定的 OpenCV 库，占总文件大小的 35% 左右。

下表显示了使用 userdebug 构建的 Android 目标上的程序可执行二进制文件和资产文件的磁盘大小摘要。这些数字是通过 Android adb shell 中的 du -h -c 命令测量的。请注意，BSP 库（例如 libEGL.so）未列在表中。

AVM 文件	地点	尺寸
2D 资源	/etc/ automotive /mt6771/avmdata	1MB
3D 资源	/etc/ automotive /mt6771/avmdata/asset_3d	62MB
标定参数	/mnt/mtkdata/avmdata	1.2MB
守护进程可执行文件	/system/bin/mtkfastavm	12 KB
AVM 库	/system/lib64/lib*avm*.so	2.4MB
3D 资源库	/system/lib64/libassimp.so	1.4MB
在线标定 OpenCV 库	/system/lib64/libopencv_java3.so	17MB
AVM 演示版 APK	/system/app/MtkAvmDemo	2.2MB
在线标定 APK	/system/app/AVM_CALIBRATION_DEMO	208 千字节
全部的	—	~88MB

运行时内存占用

参考 3D AVM 系统（摄像头：1280x720，显示器：1920x1080）为 AVM 守护进程服务消耗约 179MB RAM，为 Android APK（AVMDemo）消耗约 77MB RAM。

内存占用量是通过 userdebug build 上的 top 命令测量的。RES（驻留内存）字段用于指示内存占用量。（top -s 7 按 RES 排序）

Administrator: C:\Windows\System32\cmd.exe - adb shell "top -s 7"

Tasks: 443 total, 1 running, 442 sleeping, 0 stopped, 0 zombie										
Mem: 1.7G total, 1.7G used, 15M free, 49M buffers										
Swap: 0.9G total, 353M used, 660M free, 566M cached										
800%cpu 165%user 0%nice 106%sys 526%idle 0%iow 0%irq 2%irq 0%host										
PID	USER	PR	NI	VRT	RES	[SHR]	S	%CPU	%MEM	TIME+ ARGS
426	system	0	-20	3.1G	284M	179M	S	40.6	12.7	11:34.65 mtkfastavm
3045	system	10	-10	5.4G	105M	77M	S	0.0	5.7	0:00.37 com.mediatek.avm
641	cameraserver	20	0	1.9G	71M	71M	S	107	3.8	917:17.21 camerahalserver
1605	u0_a100	20	0	5.6G	97M	69M	S	0.0	5.2	0:02.22 com.android.lau+
1157	system	18	-2	7.6G	49M	49M	S	5.3	2.6	69:38.55 system_server
594	root	20	0	153M	36M	36M	S	7.6	1.9	94:46.02 cameraserver
1978	system	20	0	5.6G	38M	33M	S	0.0	2.0	0:05.12 com.mediatek.ca+
1330	u0_a89	20	0	5.9G	42M	30M	S	0.0	2.2	0:08.60 com.android.sys+
3479	u0_a52	20	0	5.4G	39M	29M	S	0.0	2.1	1:43.91 android.process+
432	root	20	0	5.1G	47M	27M	S	0.0	2.5	0:02.49 zygo64
1445	radio	20	0	5.7G	21M	21M	S	0.0	1.1	0:07.14 com.android.pho+
3766	u0_a90	20	0	5.2G	42M	21M	S	0.0	2.3	0:00.26 com.android.dia+
3241	u0_a54	20	0	5.1G	41M	19M	S	0.0	2.2	0:00.32 com.android.cel+
1422	network_sta+	20	0	5.1G	36M	17M	S	0.0	1.9	0:00.11 com.android.net+
3292	u0_a107	20	0	5.1G	38M	17M	S	0.0	2.0	0:00.15 com.android.cal+
1710	u0_a106	20	0	5.2G	47M	17M	S	0.0	2.5	0:00.23 com.android.inp+
2074	u0_a73	20	0	5.1G	38M	16M	S	0.0	2.0	0:00.10 com.android.pri+
3325	system	20	0	5.2G	36M	16M	S	0.0	1.9	0:00.14 com.android.key+
2270	radio	20	0	5.3G	31M	15M	S	0.0	1.6	0:00.43 com.mediatek.ims
2858	u0_a56	20	0	5.2G	38M	14M	S	0.0	2.0	0:00.19 com.android.mms

Daemon

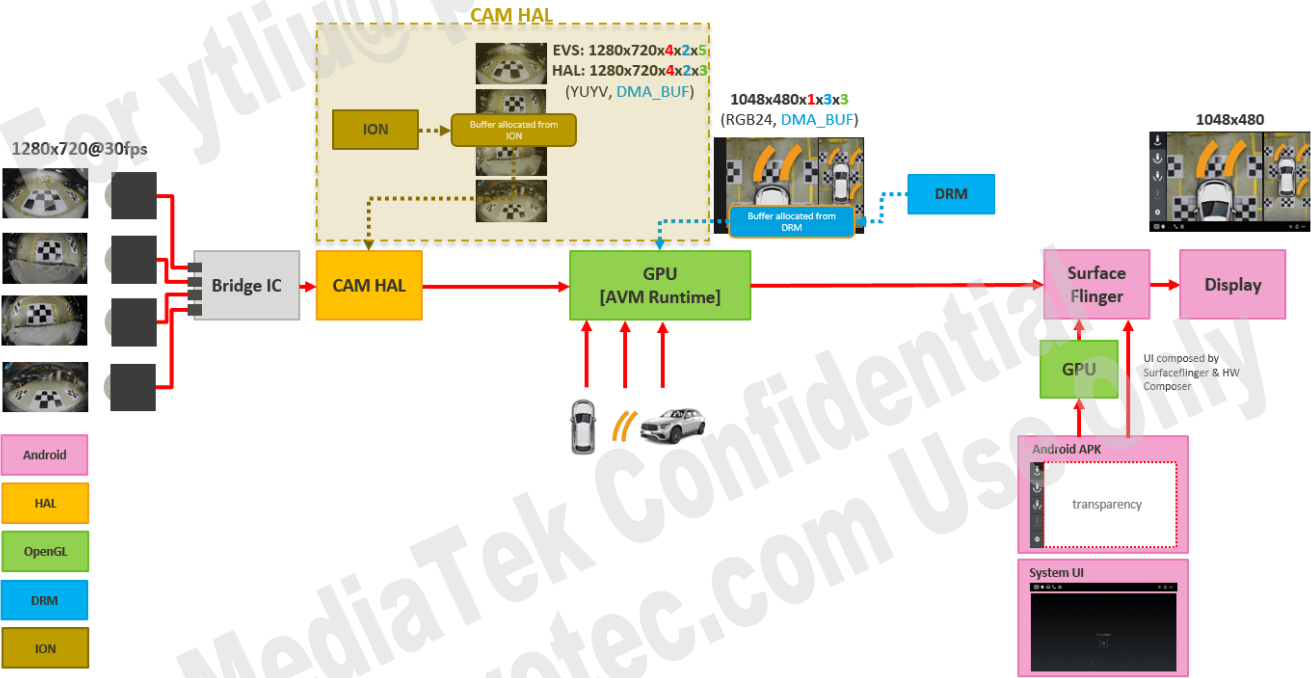
APK

CAM HAL

足迹主要受两个因素影响：

- 3D AVM 资源：当前示例 3D 汽车模型占用约 100MB 的常驻内存。如果没有加载 3D AVM 资源，则常驻内存使用量约为 77MB。
- AVM 渲染管道。缓冲区的大小与相机输入分辨率和输出显示尺寸高度相关。下图给出了一个简单的估算公式：

Pipeline Buffer Size = Resolution x Source # x Byte-Per-Pixel x Queue #



AVM 渲染管道中有两个主要的缓冲队列。您可以通过更改摄像头输入分辨率和显示输出分辨率来粗略计算缓冲队列大小。

	Camera Input Resolution	1280x720						
	Display Output Resolution	1048x480						
	Module	Width	Height	Source #	BPP	Queue #	Byte	MB
A	Camera Queue	1280	720	4	2	3	22118400	21.09
B	DRM Display Queue	1048	480	1	3	3	4527360	4.32
							Total	25.41

3.2 项目配置

当前代码库中有多个项目。只有 spm86xx_avm 项目针对 AVM 进行了良好配置。如果您想在其他项目上正确运行 AVM，以下段落是设置它的说明。

Android 上的 AVM

配置 AVM

找到 device/mediatekprojects/spm86xx_xxx/ProjectConfig.mk，然后修改以下标志

- MTK_SPM_AVM_SUPPORT = 是

编译

1. build system image

```
source build/envsetup.sh
export OUT_DIR=out_sys
lunch sys_mssi_spm_64_cn-userdebug
make -j24 sys_images 2>&1ltee sys.log
```

2. build system image

请将spm86xx_xxx替换为你自己的项目名称

```
export OUT_DIR=out_vnd_spm86xx_xxx
lunch vnd_spm86xx_xxx-userdebug
make -j24 vnd_images krn_images 2>&1ltee vnd.log
```

3. package image

一行内的命令

请将spm86xx_xxx替换为你自己的项目名称

```
python out_sys/target/product/mssi_spm_64_cn/images/split_build.py --system-dir out_sys/target/product/mssi_spm_64_cn/images --vendor-dir out_vnd_spm86xx_xxx/target/product/vnd_spm86xx_xxx/images --kernel-dir out_vnd_spm86xx_xxx/target/product/spm86xx_xxx/images --output-dir output_load_spm86xx
```

下载

使用 flesh_tool 下载 \output_load_spm86xx 下的images。

4 开发套件

AVM 开发套件包括工具、软件和文档。

成套工具	描述
软件开发工具包	1. CameraCalib PC Tool 用于计算相机模块的固有参数 2. AVMCalib PC Tool 用于计算将四个相机源拼接/混合到 BEV（鸟瞰图）图像中的参数 3. AVMCalib Android apk 作为如何使用标定库在生产线上构建标定流程的示例应用程序 4. AVMDemo Android apk 作为如何使用底层服务和库的示例应用程序 5. AVM 标定库 包含一个 pd module 来查找图案上的参考点和一个 calib module 来生成 AVM 拼接/混合标定数据 6. AVM 运行时服务/库 包含一个 守护daemon module，它控制整个生命周期/数据路径（从相机到显示器），并使用高性能拼接/混合运行时算法来生成所有视图和相关车轮轨迹线
	The diagram illustrates the software architecture for AVM calibration and runtime services. It is organized into several components and platforms: PC Tools (Windows icons): Camera Calibration Tool: Contains the Command Line PC Tool. AVM Calibration Tool: Contains the Command Line PC Tool. Android Apps (Android icons): Inline Calibration App: Contains Calibration APK and Calibration JNI. AVM Demo App: Contains AVM APK and AVM JNI. AVM Calibration Libraries (Orange box): Point Detector (avm_pd) Calibration & Stitch Algorithm (avm_calib) AVM Runtime Service /Libraries (Pink box): Service Daemon Drivers AVM Runtime Guidelines Platform Legend: PC: Blue box Android: Green box Linux: Pink box Cross Platform: Orange box Binary Release: Red-bordered box
文档	所有技术支持相关材料均记录在此 Wiki 中。

4.1 AVM SDK

AVM 系统的软件开发套件包括以下项目：

物品	描述	网址
开发车轮轨迹线	本网站	Link
AVM PC Tool	包含工具可执行文件和示例输入文件的压缩包。	—
AVM 软件包	SPM Android 源代码版本的附加 repo 项目。	—

- 本开发车轮轨迹线包含有关使用和修改软件模块的文档和说明
- AVM PC 工具包含
 - 标定工具可帮助您生成相机和车辆设置所需的参数。
 - 3D 相机定制工具，用于定制 3D AVM 中的相机动画路径。
- AVM 源代码包是一个 Android repo 项目。项目名称为 alps/vendor/mediatek/proprietary/hardware/mtkcam_sec_lib。获取的 Android 源代码树包含以下子目录：

```
vendor/mediatek/proprietary/hardware/mtkcam_sec_lib/mtkavm
├── avmalgo
├── avmcalibapk
├── avmdaemon
└── avmdemoapk
```

其中，

- avmdaemon 包含预构建的模块库，例如 AVM 守护进程 (libmtkavmfunction.so) 和 Guideline (libmtkavmproc.so)。
- avmalgo 包含预构建的模块库，例如 AVM 运行时 (libmtkavm.so) 和标定库 (libmtkavmcalib.so 和 libmtkavmpd.so)。
- avmcalibapk 包含 AVMCalibration Android 演示应用程序的源代码。
- avmdemoapk 包含 AVMDemo Android 演示应用程序的源代码。

如果您在下载源代码或工具包时遇到任何问题，请联系您的联发科技帐户窗口。

构建配置

spm86XX_avm 项目默认启用 AVM 功能。若要在其他项目中启用 AVM，请遵循这些配置。

Android 项目配置

Android 项目默认位于 device/mediatekprojects/。请确保在 ProjectConfig.mk 中正确设置以下配置选项：

AVM 功能

MTK_SPM_AVM_SUPPORT = yes

5 相机

AVM 系统需要安装四个摄像头，分别面向前、左、后、右。左、右摄像头区域安装在后视镜下方。

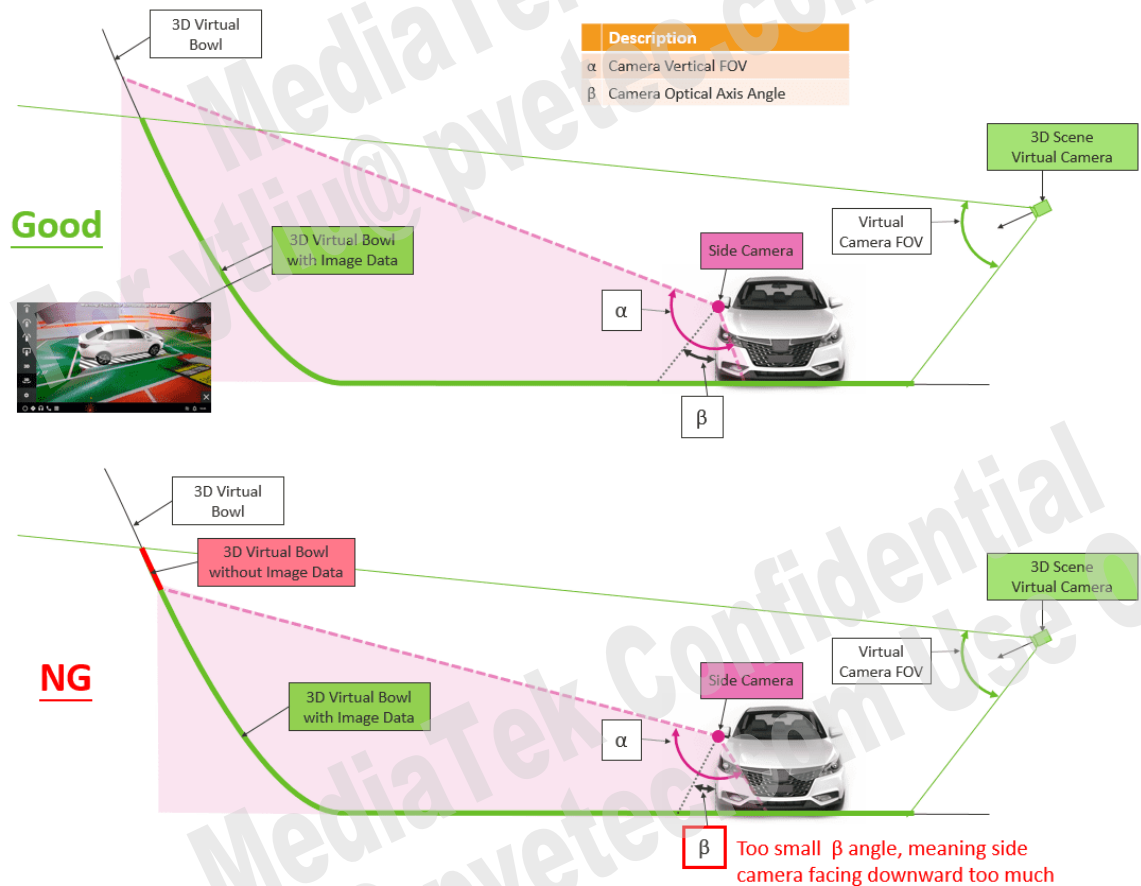
AVM 相机规格要求

AVM 系统需要使用四个相同的鱼眼相机，水平视场 > 180°，垂直视场 > 130°，以生成正确的 2D 顶视图和 3D 场景。

Camera 安装位置和角度

摄像机安装位置和角度有三条经验法则要求

- 1. 对于所有相机，请确保在 AVM 3D 模式下地面上方有足够的相机图像数据来生成 3D 虚拟碗。一般来说，相机的垂直视场角 > 130°，并且保险杠/格栅/门障碍物占据下部图像的 5~15% 左右，可以满足大多数情况。如果相机向下倾斜太多，可能没有足够的相机图像数据来绘制完整的 3D 虚拟环境。



MediaTek Confidential
For ytliu@pvetec.com Use Only

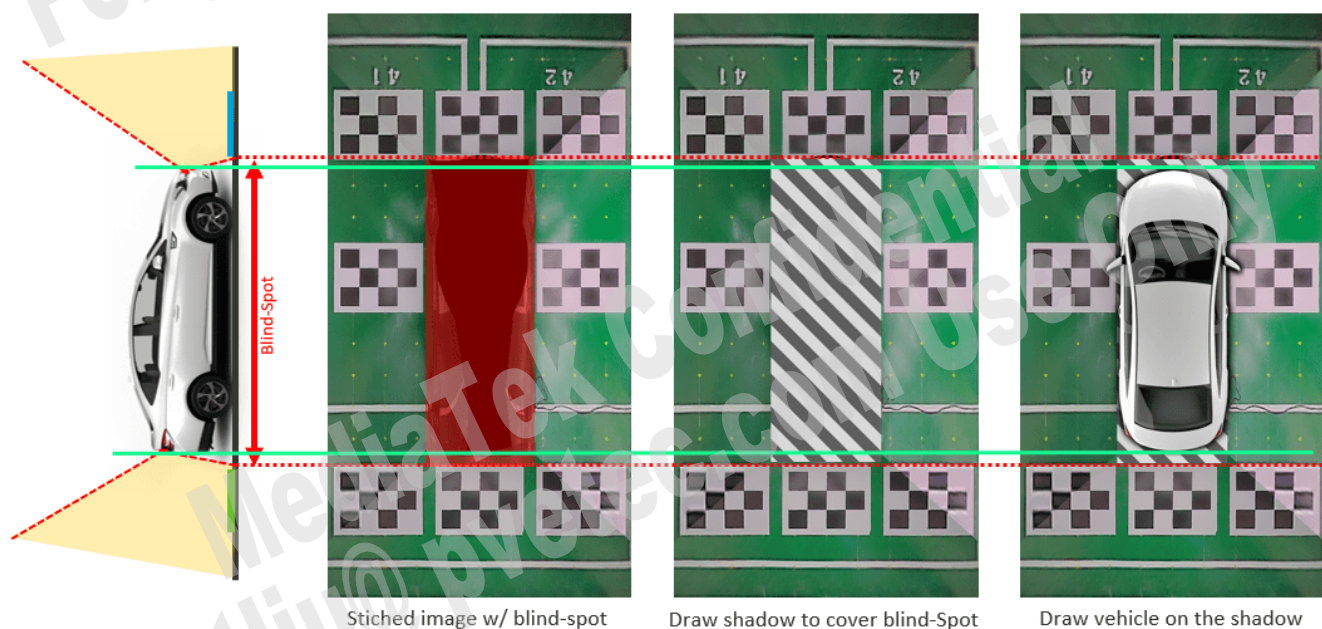
MediaTek Confidential
For ytliu@pvetec.com Use Only

MediaTek Confidential
For ytliu@pvetec.com Use Only

2. 标定过程中所有图案均清晰可见。在大多数情况下，标定图案已在工厂地板上涂漆。当车辆停在指定位置时，请确保所有图案在捕获的图像中均具有良好的可见性。
3. 保留图像中大约 5~15% 的保险杠/格栅/车门。它可以提示驾驶员地板图像与车身的关系以及您可以在摄像机安装车轮轨迹线部分找到有关每个摄像机要求的更多详细示例

盲点区域

由于车辆的配置设计，前/后摄像头大多安装在格栅和后备箱内。摄像头的视线被保险杠/格栅遮挡，可能会产生一些盲点区域。在我们的 AVM 系统中，我们将绘制阴影图案来覆盖盲点区域。

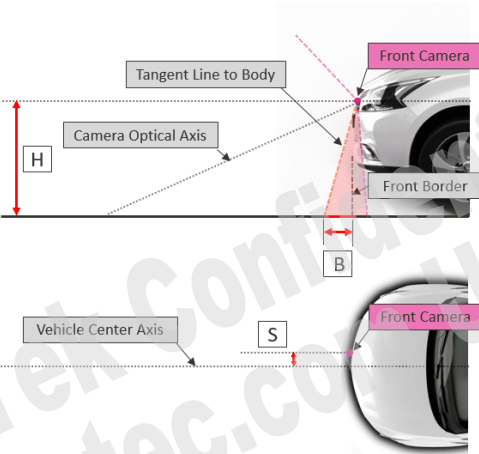
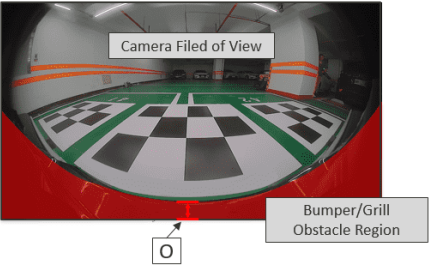
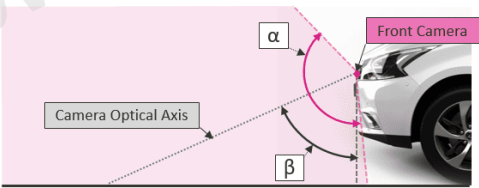


5.1 安装车轮轨迹线

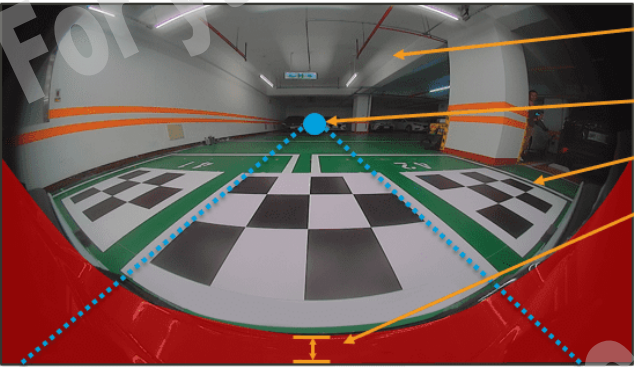
前置摄像头

下表包含一些前置摄像头安装的参考角度和位置：

Description	Reference Value
α Camera Vertical FOV	$> 130^\circ$
β Camera Optical Axis Angle	$50^\circ \sim 75^\circ$
H Camera Installation Height	500 ~ 1000 mm
B Blind-spot region	< 200 mm
O Bumper/Grill Obstacle Percentage	5%~15%
S Camera to Center Axis	0 mm



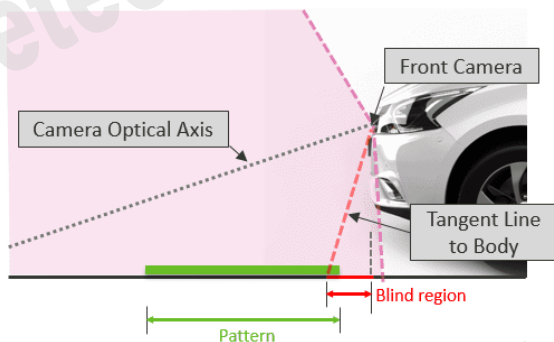
以上数字仅供参考，经验法则是在从车辆捕获的摄像头图像中实现以下四个要求：



- Req 1: Enough image above the ground to generate 3D virtual bowl in AVM 3D mode
- Req 2: Clear visibility of patterns for calibration
- Req 3: 5%~15% of bumper obstacle to avoid blind regions
- Req 4: Desired vanishing point for front camera view

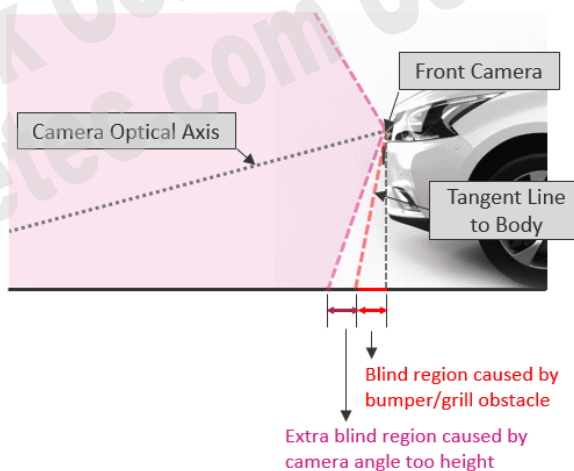
NG 案例研究 1

- 问题：格栅/保险杠障碍物遮挡了图案。这将导致标定失败。
- 解决方案：调整相机位置或将图案移离车辆。



NG 案例研究 2

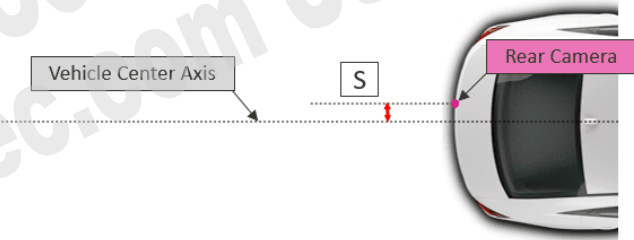
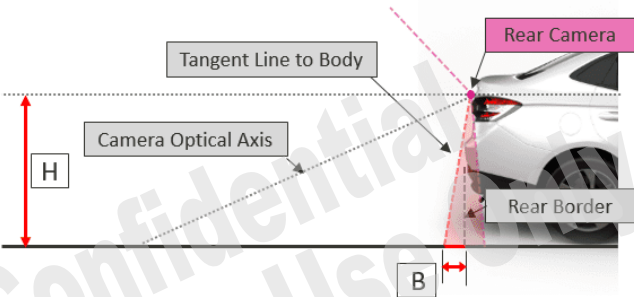
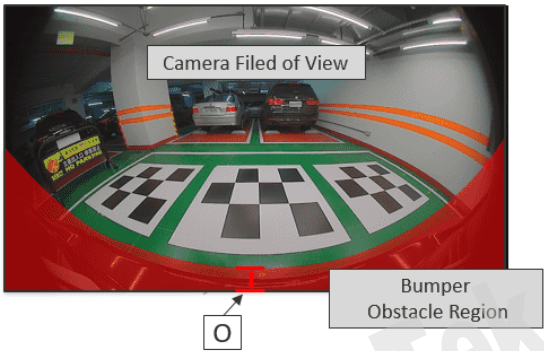
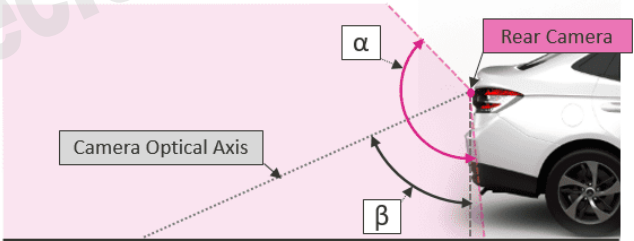
- 问题：相机光轴角度过高可能会造成一些额外的盲区。
- 解决方法：调整摄像头光轴角度，尽量让保险杠/格栅占据5%~15%的图像。



后置摄像头

下表包含一些后置摄像头安装的参考角度和位置：

	Description	Reference Value
α	Camera Vertical FOV	$> 130^\circ$
β	Camera Optical Axis Angle	$50^\circ \sim 75^\circ$
H	Camera Installation Height	600 ~ 1200 mm
B	Blind-spot region	< 200 mm
O	Bumper Obstacle Percentage	5%~15%
S	Camera to Center Axis	± 200 mm



以上数字仅供参考，经验法则是在从车辆捕获的摄像头图像中实现以下四个要求：

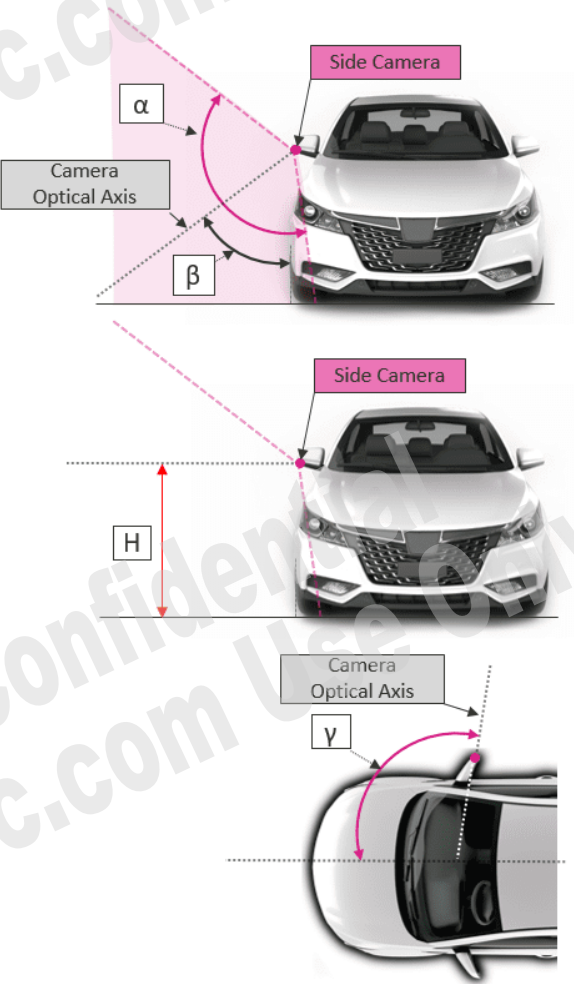
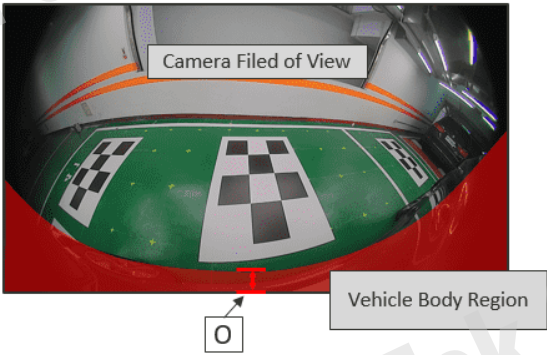


- Req 1: Enough image above the ground to generate 3D virtual bowl in AVM 3D mode
- Req 2: Clear visibility of patterns for calibration
- Req 3: 5%~15% of bumper obstacle to avoid blind regions
- Req 4: Desired vanishing point for rear camera view

侧面摄像头

下表包含一些侧摄像头安装的参考角度和位置：

	Description	Reference Value
α	Camera Vertical FOV	$> 130^\circ$
β	Camera Optical Axis Angle	$40^\circ \sim 65^\circ$
γ	Camera Optical Axis Angle	$90^\circ \sim 95^\circ$
H	Camera Installation Height	900 ~ 1200 mm
O	Vehicle Body Percentage	5%~15%



以上数字仅供参考，经验法则是在从车辆捕获的摄像头图像中实现以下三个要求：



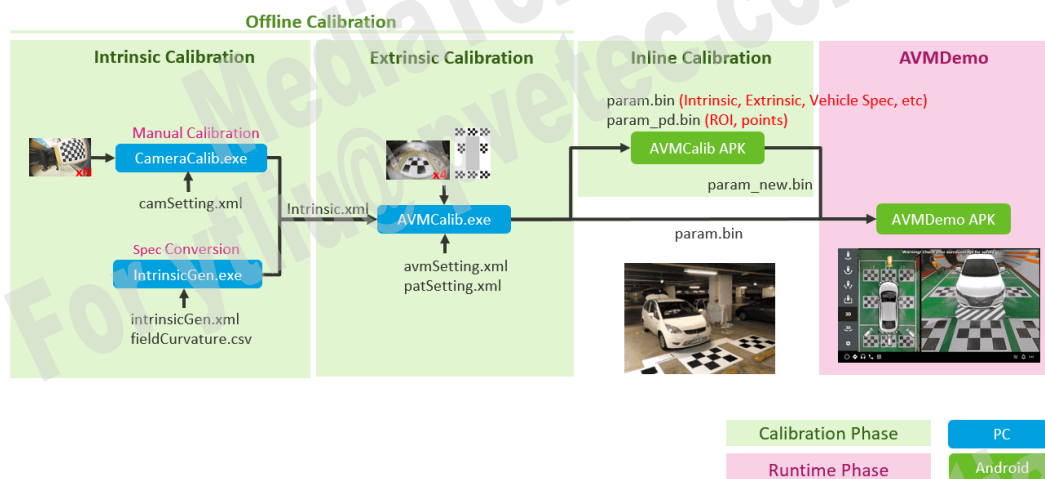
- Req 1:** Enough image above the ground to generate 3D virtual bowl in AVM 3D mode
- Req 2:** Clear visibility of patterns for calibration
- Req 3:** 5%~15% of vehicle door region for side view cropping and make sure no blind-spot region



6 标定过程

我们提供三个 PC 标定工具、一个 Android 标定应用程序、一个 Android AVM 演示应用程序和一个图像捕获工具来完成端到端 AVM 标定过程：

- 相机标定工具 (CameraCalib.exe) 是一个用于计算相机固有参数（相机投影矩阵和去畸变系数）的 PC 工具。
- 固有参数生成工具 (IntrinsicGen.exe) 是一款 PC 工具，用于根据相机模块制造商提供的规范生成相机的固有参数。
- AVM 标定工具 (AVMCalib.exe) 是一款 PC 工具，可使用从真实设备捕获的图像来计算 AVM 拼接/混合/显示参数。
- AVM 标定应用程序 (AVM Calib APK) 是一款 Android 应用程序，用于标定生产线上的每辆车，以补偿安装差异。
- AVM 演示应用程序 (AVM Demo APK) 是 AVM 应用程序的 Android 应用程序。
- 客户端实用程序 (mtkavmclient) 是一个 Linux 原生应用程序，用于从摄像头捕获图像，了解如何捕获图像。

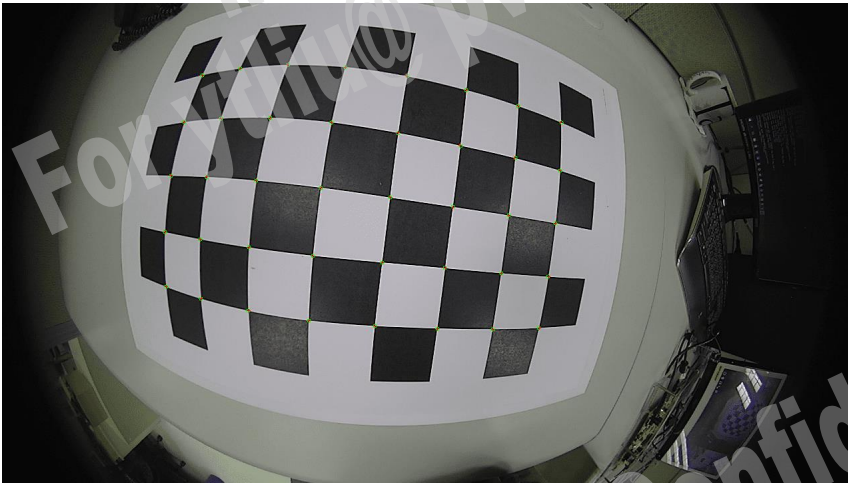


标定原理

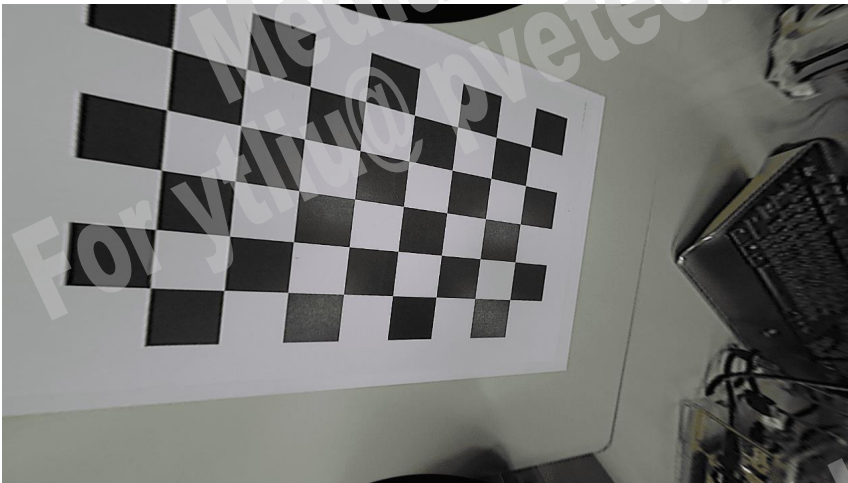
相机标定

该过程将生成内参，并将其用于消除鱼眼图像的失真。

- 检测鱼眼图像上的点。



- 通过生成的固有参数消除图像畸变。

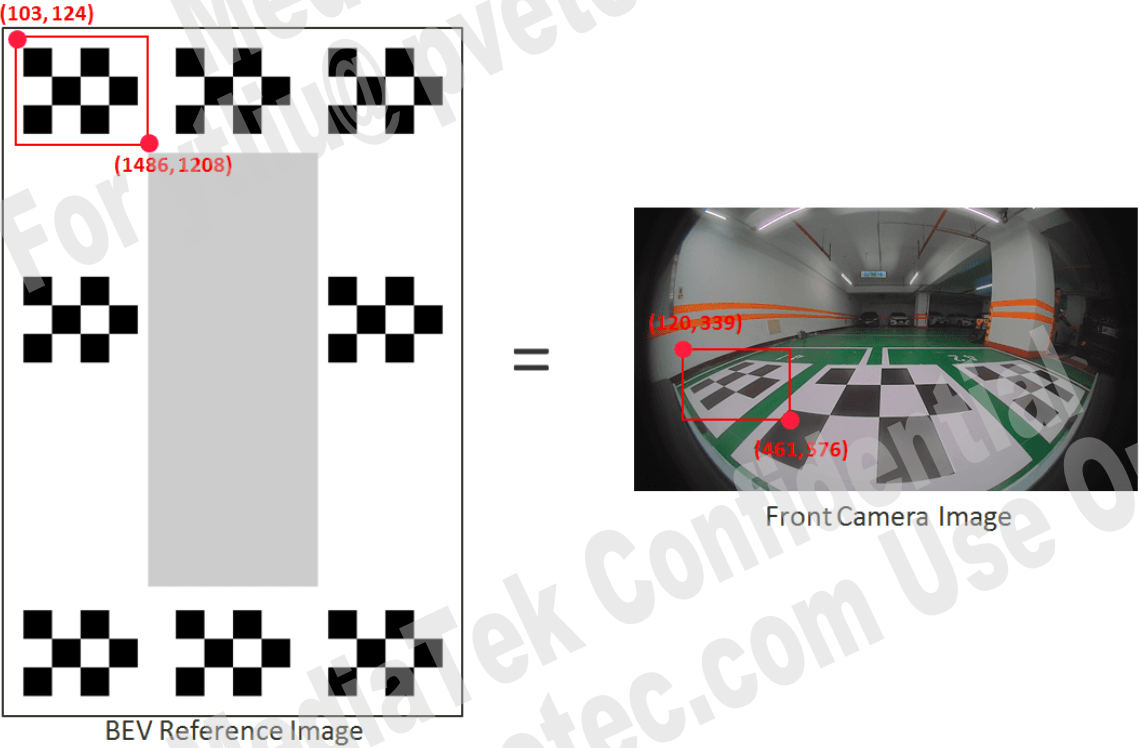


AVM 标定

该过程将尝试匹配相机框架和 BEV 图像（地面实况）之间的点。

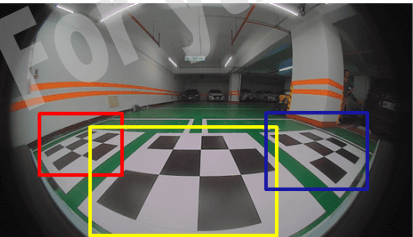
注意：BEV 的尺寸（像素）与真实世界坐标（毫米）相同。

- 相机上的图案和BEV之间的关系。



- 如何将鱼眼图像投影到顶视图的工作流程。

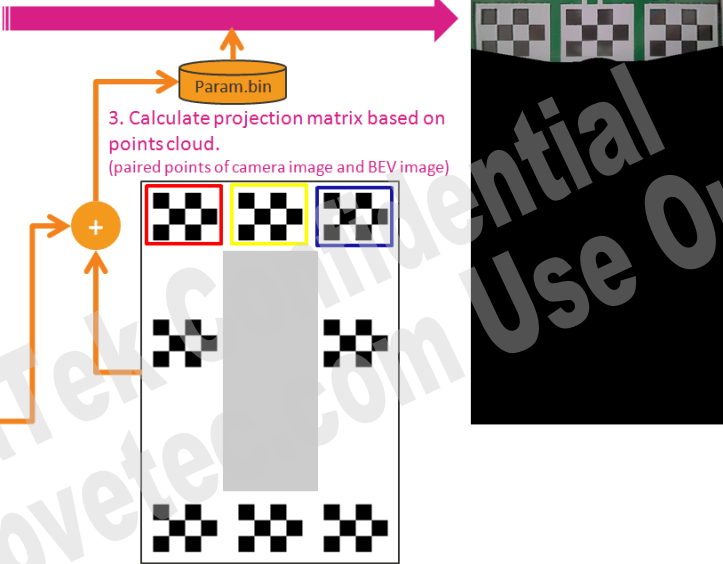
1. Detect points on camera image



2. Undistort points by intrinsic parameters



4. Camera images re-project to AVM top view



6.1 预备知识

以下是您在使用标定过程之前需要了解的一些重要背景知识。

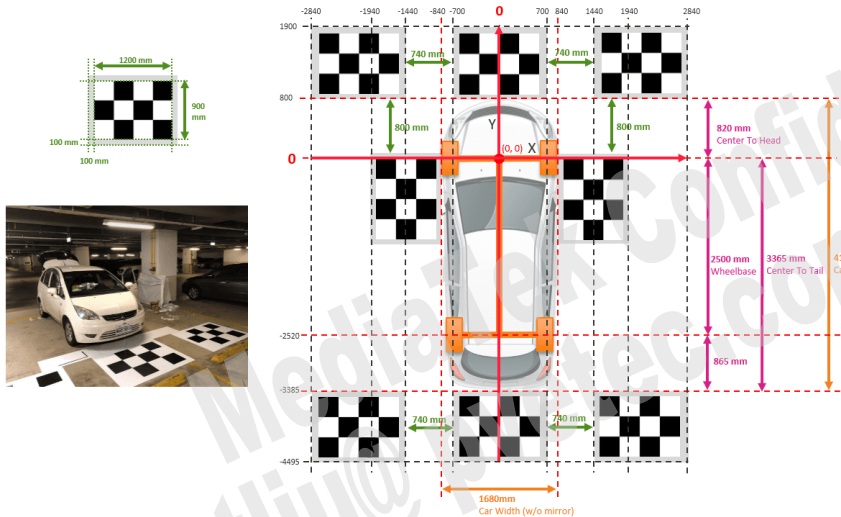
坐标系

测量单元

使用物理坐标毫米（mm）作为共同的测量单位来对齐图案、鸟瞰图（BEV）、车辆、轨迹和指导线。

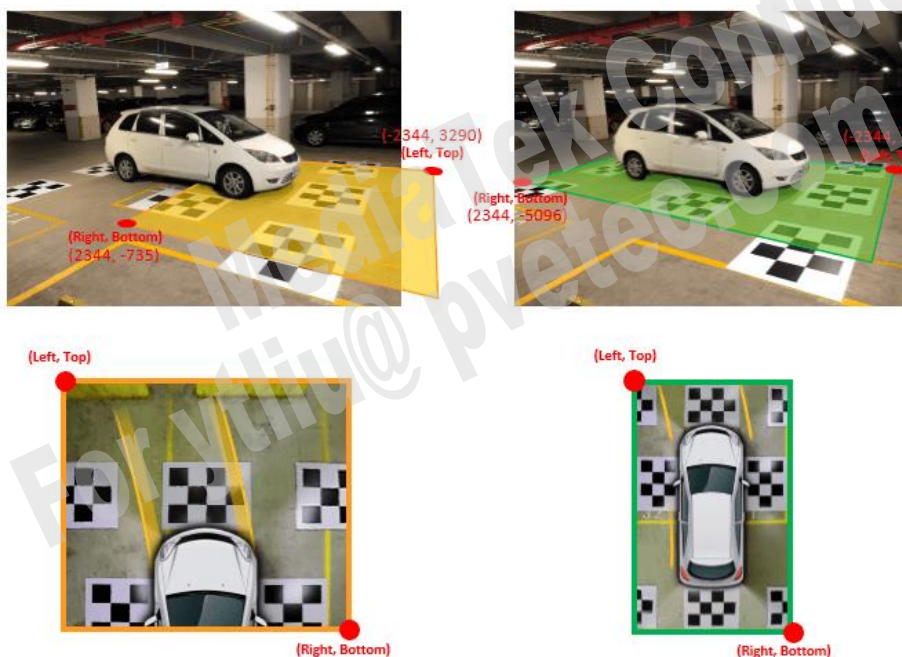
坐标中心

坐标系的中心（ $x=0, y=0$ ）位于前轮轨迹的中心。



视锥体

使用视锥体（左、上、右、下，也称为 LTRB）来描述物理世界中的空间区域。



6.2 相机标定

相机的成像原理是基于针孔几何模型，该模型由本征矩阵和畸变方程组成，相机标定就是计算相机模组本征参数和畸变系数的过程。

内参

内部参数是一个 3×3 的矩阵，是相机坐标系到图像坐标系的变换矩阵。

失真系数

畸变系数用镜头模型的多项式参数来表示，AVM系统采用鱼眼镜头通过鱼眼模型的半径投影来捕捉广角图像，会造成图像严重的光学畸变，该系数可以将鱼眼图像反向投影为无畸变图像。

工具下载与安装

请访问联发科技 DCC 下载 AVM 标定工具包。您可以在该工具包中找到 CameraCalib 和 AVMCalib 工具。使用这些工具之前，请安装 Visual Studio 运行时 (vc_redist_x64.exe)。

文件夹结构

CameraCalib的文件夹结构如下：

相机标定/

├──	CameraCalib.exe
├──	opencv_world341.dll
├──	/AG190C
│	├── /input/*.png
│	├── /output/intrinsic.xml
│	├── /temp/_point_cloud.png, *.reproj.png
│	├── camSetting.xml
└──	chessboard_A3_8x5x40mm.pdf

CameraCalib.exe是相机标定工具的执行文件

其中：

- opencv_world341.dll是执行CameraCalib.exe所需的动态库文件。
- /input是相机标定图像目录。
- inner.xml 是标定结果文件。它将由 AVM 标定过程使用。
- /temp 是调试映像目录。
- camSetting.xml 是标定设置文件。大多数情况下，您不需要修改此文件，除非您想更改图案类型或尺寸。
- chessboard_A3_8x5x40mm.pdf 是 A3 尺寸的棋盘图案。

环境设置

个人电脑

- Windows 7/10 (64 位)
- 需要 Visual Studio 运行时 (安装 vcredist_x64.exe)

MTK EVB (安卓)

- EVB 至少连接一个摄像头
- 已启用 AVM 和 Android 调试桥 (ADB) 的系统映像
- 启动系统并确保 AVM 正在运行

棋盘图案

- 以 A3 尺寸打印棋盘图案 chessboard_A3_8x5x40mm.pdf

运行示例

1. 检查以下文件

- 项目配置: /CameraCalib/AG190C/camSetting.xml
- 相机标定图像: /CameraCalib/AG190C/input/*.png

2. 运行相机标定工具

- 打开命令行控制台并转到 /CameraCalib 文件夹
- 从控制台执行以下命令:

```
CameraCalib.exe AG190C/camSetting.xml
```

3. 检查结果

- 控制台日志
- /CameraCalib/AG190C/output/intrinsic.xml 中的输出
- /CameraCalib/AG190C/temp/*.png 中的调试图像

相机标定步骤

准备棋盘图案

- 您可以在 /CameraCalib/chessboard_A3_8x5x40mm.pdf 中找到图案图像。请打印出棋盘图案并将其粘贴到平坦牢固的表面上。

捕获相机标定图像

- 通过 USB 连接 PC 和 EVB
- 授予 ADB 根访问权限和可写分区

```
adb root
adb remount
```

- 创建新文件夹来存储捕获的图像

```
adb shell mkdir /data/avm/
adb shell mkdir /data/avm/image/
```

- 停止 AVM 应用程序，因为它占用了 AVM 守护进程资源

```
adb shell am force-stop com.mediatek.avm
```

- 终止并重启服务守护进程

```
adb shell killall mtkfastavm
start adb shell mtkfastavm
```

- 切换到相机视图

```
adb shell mtkavmclient 1 5
```

- 切换摄像头 (3 0 = 前、3 1 = 左、3 2 = 右、3 3 = 后)

```
adb shell mtkavmclient 3 1
```

- 捕获图像 (AVM 守护进程将自动附加文件扩展名)

```
adb shell mtkavmclient 4 1 /data/avm/image/image_001
```

- 重复此步骤以使用不同的文件名捕获更多图案图像。请查看常见问题解答部分以获取更多捕获技巧

```
adb shell mtkavmclient 4 1 /data/avm/image/image_002
adb shell mtkavmclient 4 1 /data/avm/image/image_003
...
exit
```

- 将所有捕获的图像复制到主机 PC

```
adb pull /data/avm/image
```

运行标定过程

请按照以下步骤开始您的相机标定项目：

- 克隆 AG190C 文件夹并重命名为 your_project 并清理 /input 文件夹。
- 按照捕获相机标定图像的说明捕获棋盘图案图像。
- 将这些图像放入 /your_project/input 文件夹中。此文件夹中的所有图像都将用于标定过程。
- 打开命令行控制台并执行 CameraCalib.exe /your_project/camSetting.xml。
- CameraCalib 工具将开始运行。它将自动丢弃质量较差的图像并多次运行标定过程，直到达到良好的结果。
- 您可以在 /output/intrinsic.xml 中找到内参。

检查日志

```
D:\AVM_Calibration_Toolkit\CameraCalib>CameraCalib.exe AG190C\camSetting.xml
Searching for PNG files in D:\AVM_Calibration_Toolkit\CameraCalib\AG190C\input\*.png
.....
60 PNG files found.

Start detecting pattern...
.....
59 PNG files detected corner.

Start to calibrate 1 times...
Calibration succeeded. Total avg re-projection error = 113.463

Saving point cloud view into temp directory

Images that cannot detect any points:
input\32.png

Images that re-projection error is high:
input\19.png, 825.407
input\30.png, 5.00631
input\39.png, 279.121

Start to calibrate 2 times...
Calibration succeeded. Total avg re-projection error = 0.223055

Saving point cloud view into temp directory

Images that re-projection error is high:
none

Saving re-projected images into temp directory
.....
Saving un-distorted images into temp directory
.....
Finished.
```

Total avg re-projection error: 该值为实际检测到的角点坐标与重新投影的角点坐标之间的像素位置平均误差值，反映固有参数和畸变系数应用到检测到的角点后效果如何。

无法检测到任何点的图像：在标定过程中，无法检测到棋盘角的图像将被丢弃。

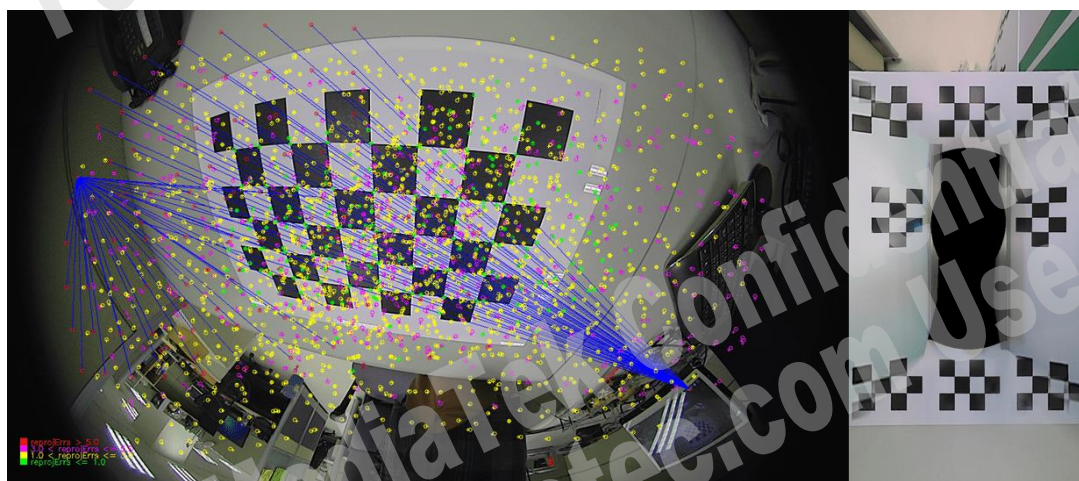
重新投影误差较大的图像：重新投影位置误差过高的图像在标定过程中将被丢弃。

检查点云图像

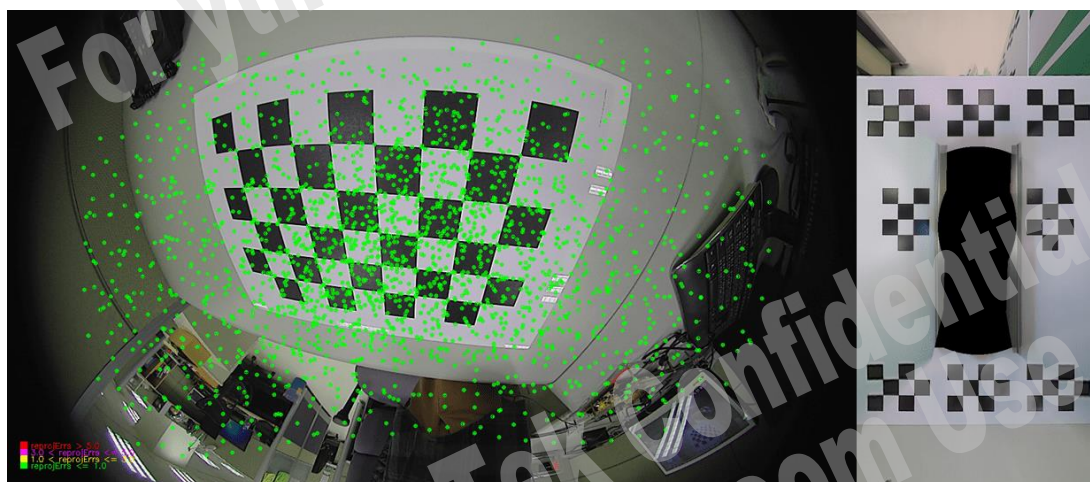
您可以在文件夹“/temp”中找到用于调试的点云图像。所有检测到的角点都聚集并绘制在此图像中，并且所有点都根据其重新投影误差值用不同的颜色标记。您可以观察点的分布以检查标定图像是否完全覆盖相机的视野。

相机标定过程将多次运行标定以提高质量，并仅输出最终结果。下图是三轮不同标定的输出，仅供参考。如您所见，较低的重新投影误差可带来更好的图像拼接质量。

- 第一轮标定 (平均重新投影误差 = 113.463)



- 第二轮标定 (平均重投影误差 = 0.223055)



检查输出

CameraCalib 工具会在输出文件夹中生成一个intrinsic.xml文件，该文件记录了intrinsic参数和畸变系数。

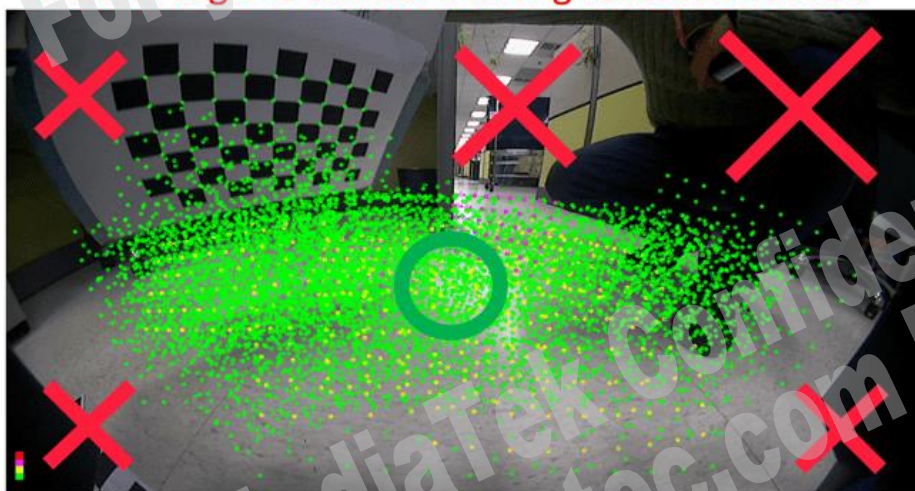
```
<?xml version="1.0"?>
<opencv_storage>
  <calibration_Time>"Mon Dec 9 18:12:07 2019"</calibration_Time>
  <nrOfFrames>56</nrOfFrames>
  <image_Width>1280</image_Width>
  <image_Height>720</image_Height>
  <board_Width>8</board_Width>
  <board_Height>5</board_Height>
  <square_Size>40.</square_Size>
  <FixAspectRatio>1</FixAspectRatio>
  <!-- flags: +use_intrinsic_guess +fix_aspectRatio +zero_tangent_dist +fix_skew +recompute_extrinsic -->
  <flagValue>11</flagValue>
  <Camera_Matrix type_id="opencv-matrix">
    <rows>3</rows>
    <cols>3</cols>
    <dt>d</dt>
    <data>
      2.4360062424267073e+02 0. 6.3633540224359581e+02 0.
      2.4333536472731711e+02 3.4728592135007216e+02 0. 0. 1.</data></Camera_Matrix>
  <Distortion_Coefficients>
    4.0675644806350608e-01 -1.1937902112578469e-01 1.0371211657355463e-02
    4.7947457331419313e-04</Distortion_Coefficients>
  <Avg_Reprojection_Error>2.2305498726208095e-01</Avg_Reprojection_Error>
</opencv_storage>
```

故障排除

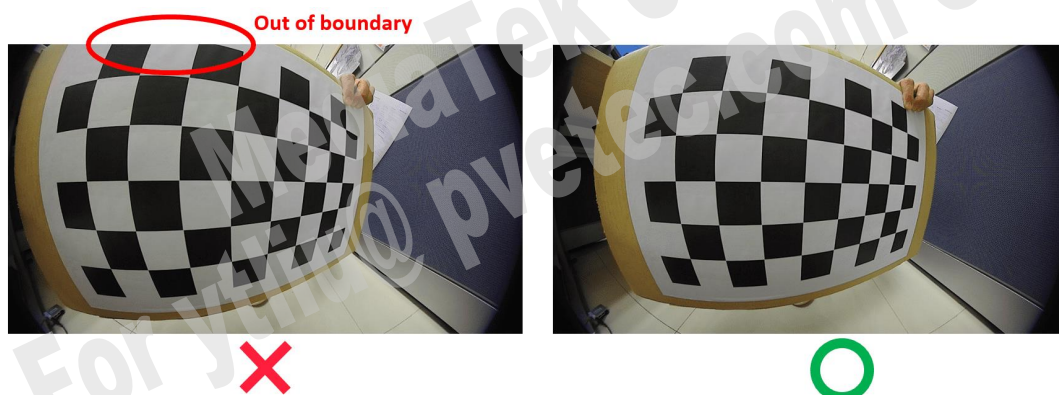
捕获图案图像时，尽量覆盖相机视图的大部分区域。下面显示的示例有许多输入图像，但仅关注中心区域。运行标定过程后，仍会得到一个小的重新投影误差，但应用输出结果后，图像的外部区域将不会具有正确的去扭曲效果。

❖ Region covered by patterns will be accurate

❖ Other regions will have wrong calibration result



- 标定图像中棋盘格图案的位置必须在相机的视野范围内，可以提高棋盘格角点的检测率。



- 请不要使用棋盘图案太细的白边框来捕捉标定图像。



常见问题

棋盘图案的正方形尺寸是多少？

正方形尺寸为40mm x 40mm。

如何观察相机标定结果的质量？

可以观察总平均重投影误差是否较小，根据实验经验，当该值小于1时，会得到很好的标定质量。另外，还必须观察点云图像，确认图像采样点的分布已经完全覆盖相机的视场，这样才能更准确地校正固有参数和畸变系数。

为什么同一台相机使用CameraCalib工具会产生不同的内参和畸变系数？

相机在工厂生产组装时，会因为焦距的调整而产生一定的误差，不过这个误差值会在AVMCalib工具中得到微调和改善。

6.3 内参生成

虽然可以通过CameraCalib工具得到相机的固有参数和畸变系数，但是手动标定有时会得到较差的效果，因为拍摄图像的棋盘格图案位置分布区域并不能覆盖整个相机镜头视场。因此我们提出了另一种非手动标定工具，称为Intrinsic Generation工具。该工具主要根据相机模组厂商提供的规格书和光学镜头厂商提供的镜头场曲率数据，直接计算相机的固有参数和畸变系数。

场曲率数据

场曲率是光束进入镜头的角度与相机传感器上图像相对位置之间的关系。由于广角镜头的曲面是非线性的，因此，从不同角度进入镜头的光线会产生不同的折射角度，这种关系可以用曲线来表示。

工具下载与安装

请访问联发科技DCC 下载 AVM 标定工具包。您可以在此工具包中找到 IntrinsicGen 工具。

文件夹结构

IntrinsicGen的文件夹结构如下：

IntrinsicGen/
├── intrinsicGen.exe
└── /AG190C
├── /input/*.png, fieldCurvature.csv
├── intrinsicGen.xml
├── /output/intrinsic.xml
└── /temp/*.undistor.png

其中：

- intrinsicGen.exe是生成相机固有参数和畸变系数的执行文件。
- /input 是相机标定图像目录。fieldCurvature.csv 是镜头畸变表。您需要从光学镜头制造商处获取它。
- inner.xml 是标定结果文件。它将由 AVM 标定过程使用。
- /temp 是调试不失真图像目录。
- intrinsicGen.xml是此工具设置文件。您需要根据相机模块规格修改CameraModule的内容。

MediaTek Confidential
For ytliu@pvetec.com Use Only

MediaTek Confidential
For ytliu@pvetec.com Use Only

MediaTek Confidential
For ytliu@pvetec.com Use Only



环境设置

个人电脑

- Windows 7/10 (64 位)

MTK EVB (Android)

- EVB 至少连接一个摄像头。
- 已启用 AVM 和 Android 调试桥 (ADB) 的系统映像。
- 启动系统并确保 AVM 正在运行。

棋盘图案

- 以 A3 尺寸打印棋盘图案 chessboard_A3_8x5x40mm.pdf

运行示例

1. 检查以下文件
 - 项目配置: /IntrinsicGen/AG190C/intrinsicGen.xml
 - 镜头场曲率数据: /IntrinsicGen/AG190C/input/fieldCurvature.csv
 - 相机标定图像: /IntrinsicGen/AG190C/input/*.png
2. 运行 IntrinsicGen 工具
 - 打开命令行控制台并转到 /IntrinsicGen 文件夹
 - 从控制台执行以下命令:

```
intrinsicGen.exe AG190C/intrinsicGen.xml
```

3. 检查结果
 - 控制台日志
 - /CameraCalib/AG190C/output/intrinsic.xml 中的输出
 - /CameraCalib/AG190C/temp/*.png 中的调试图像

内在生成步骤

准备镜头场曲率数据

将光学镜头制造商提供的镜头场曲率数据 (SEMI-ANGLE、REAL IMAGE-HEIGHT 和 REF IMAGE-HEIGHT) 复制到 fieldCurvature.csv 表中。

SEMI-ANGLE	REAL IMAGE-HEIGHT	REF IMAGE-HEIGHT	OPTICAL DISTORTION	SEMI-ANGLE	REAL IMAGE-HEIGHT	REF IMAGE-HEIGHT
0.100000	0.001785	0.001785	0.000088	0.1	0.001785	0.001785
0.200000	0.003569	0.003569	0.000088	0.2	0.003569	0.003569
0.300000	0.005354	0.005354	0.000089	0.3	0.005354	0.005354
0.400000	0.007139	0.007138	0.000090	0.4	0.007139	0.007138
0.500000	0.008924	0.008923	0.000091	0.5	0.008924	0.008923
0.600000		0.010708	0.000092	0.6	0.010709	0.010708
0.700000		0.012493	0.000093	0.7		0.012493
0.800000		0.014277	0.000095	0.8		0.014277
0.900000		0.016062	0.000097	0.9		0.016062
1.000000		0.017847	0.000100	1.1		0.017847
1.100000		0.019633	0.000102	1.2		0.019633
1.200000		0.021418	0.000105	1.3		0.021418
1.300000		0.023203	0.000108	1.4		0.023203
1.400000	0.024992	0.024989	0.000111	1.5	0.024992	0.024989
1.500000	0.026778	0.026775	0.000114	1.6	0.026778	0.026775
1.600000	0.028564	0.028560	0.000118	1.7	0.028564	0.02856
1.700000	0.030350	0.030346	0.000122	1.8	0.03035	0.030346
1.800000	0.032137	0.032133	0.000126	1.9	0.032137	0.032133
1.900000	0.033923	0.033919	0.000130	2.0	0.033923	0.033919
2.000000	0.035711	0.035706	0.000135		0.035711	0.035706

准备intrinsicGen.xml

在intrinsicGen.xml中填写相机模组厂商提供的规格，包括FocalLength、SensorImageArea、SensorActiveArraySize、OutputImageSize、PrincipalPoint，其中PrincipalPoint值一般设置在图像坐标平面的中心。

```
<?xml version="1.0"?>
<IntrinsicGen>

  <InputSetting>
    <!-- All debug images(*.png/jpg) will be enumerate within this directory as input for calibration -->
    <ImageDirectory>.\input</ImageDirectory>

    <!-- Field curvature data of the lens -->
    <LensFieldCurvatureCSV>fieldCurvature.csv</LensFieldCurvatureCSV>

    <!-- Camera module specification -->
    <CameraModule>
      <!-- Focal length of the lens (mm)-->
      <FocalLength f="1.02" />

      <!-- Sensor image area (mm)-->
      <SensorImageArea horizontal="5.41" vertical="4.57" />

      <!-- Sensor active array size (pixel)-->
      <SensorActiveArraySize horizontal="1280" vertical="1080" />

      <!-- Image resolution (pixel)-->
      <OutputImageSize width="1280" height="720" />

      <!-- (cx, cy) is a principal point that is usually at the image center-->
      <PrincipalPoint cx="639.5" cy="359.5" />
    </CameraModule>
  </InputSetting>

  <OutputSetting>
    <!-- An intrinsic.xml file will be generated within this directory. -->
    <BaseDirectory>.\output</BaseDirectory>
  </OutputSetting>

  <DebugSetting>
    <!-- All undistort images(*.png/jpg) will be enumerate within this directory -->
    <BaseDirectory>.\temp</BaseDirectory>

    <!-- Horizontal FOV setting (degree) of undistort images-->
    <UndistortImageFOV angle="160" />
  </DebugSetting>
</IntrinsicGen>
```

捕获相机标定图像

- 通过 USB 连接 PC 和 EVB
- 授予 ADB 根访问权限和可写分区

```
adb root
adb remount
```

- 创建新文件夹来存储捕获的图像

```
adb shell mkdir /data/avm/
adb shell mkdir /data/avm/image/
adb shell
```

- 停止 AVM 应用程序，因为它占用了 AVM 守护进程资源

```
am force-stop com.mediatek.avm
```

- 终止并重启服务守护进程

```
killall mtkfastavm; mtkfastavm
```


- 切换到相机视图

```
mtkavmclient 1 5
```

- 切换摄像头 (3 0 = 前、3 1 = 左、3 2 = 右、3 3 = 后)

```
mtkavmclient 3 1
```

- 捕获图像

```
mtkavmclient 4 1 /data/avm/image/image_001.png
```

- 重复此步骤以使用不同的文件名捕获更多图案图像。请查看常见问题解答部分以获取更多捕获技巧

```
mtkavmclient 4 1 /data/avm/image/image_002.png
mtkavmclient 4 1 /data/avm/image/image_003.png
...
exit
```

- 将所有捕获的图像复制到主机 PC

```
adb pull /data/avm/image
```

运行你的生成过程

请按照以下步骤启动您的内在生成项目：

- 克隆 AG190C 文件夹并重命名为 your_project 并清理 /input 文件夹。
- 按照捕获相机标定图像的说明捕获相机鱼眼图像。
- 将这些图像放入 /your_project/input 文件夹中。
- 准备厂家提供的镜头场曲率数据，将镜头场曲率数据复制到intrinsicGen.xml文件中。
- 将intrinsicGen.xml文件放入/your_project/input文件夹中。
- 修改 /your_project/intrinsicGen.xml 中的相机模块规范
- 打开命令行控制台并执行intrinsicGen.exe your_project/intrinsicGen.xml。
- IntrinsicGen 工具将开始运行。您可以在 /output/intrinsic.xml 中找到内部参数。

检查日志

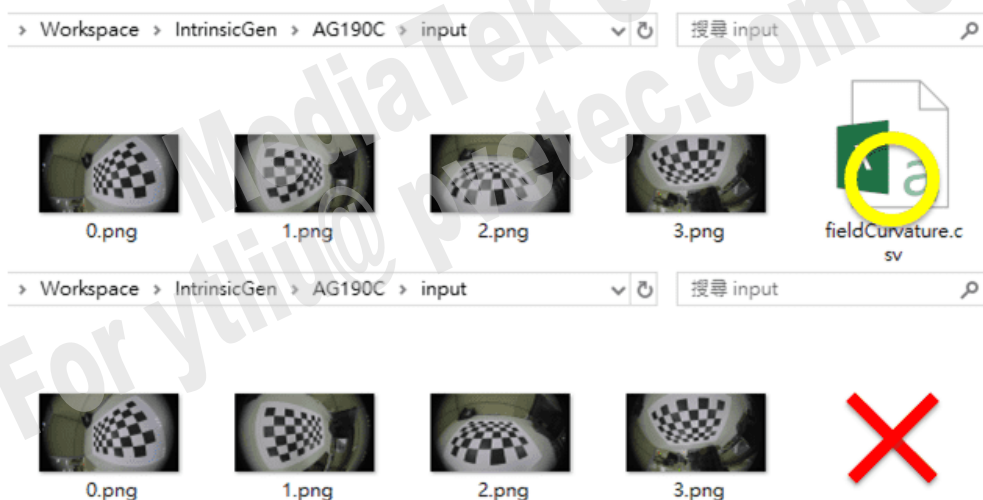
当工具日志显示 “Outputintrinsic.xml done” 和 “Undistort images saved successful” 时，表示Intrinsic Generation工具运行成功。此intrinsic.xml可用于avmCalib拼接图像。此外，您还可以观察undistort图像中的线条特征。如果线条是直的，则表示相机的intrinsic数据是正确的。

```
C:\Workspace\IntrinsicGen>intrinsicGen.exe AG190C/intrinsicGen.xml
The fieldCurvature.csv is found.
Start to calibrate.
Camera intrinsic parameters:
cx: 639.5
cy: 359.5
fx: 241.33086876155267
fy: 241.0503282275711
Lens distortion coefficients:
k1: 0.4005871601858683
k2: -0.11505986223784002
k3: 0.007643918923461579
k4: 0.0012556983682010122
Camera calibration succeeded.
Output intrinsic.xml completed.
Start to output undistort images.
Horizontal FOV (degree): 160.0
Saving undistort images into temp directory....
Undistort images saved successfully.
Finished.
```

该工具的错误/警告信息显示如下。

- 没有字段Curvature.csv:

/your_project/input/ 文件夹中没有 fieldCurvature.csv 文件。因此，该工具无法成功运行。



```
C:\Workspace\IntrinsicGen>intrinsicGen.exe AG190C/intrinsicGen.xml  
[Error] The fieldCurvature.csv is not found.  
[Error] Camera calibration failed.
```

- 没有调试图像：

/your_project/input/ 文件夹中没有相机标定图像，因此该工具无法生成无失真图像，但它仍然可以计算内部参数并输出intrinsic.xml数据。



- 超出范围设置 (FOV):

意思是intrinsicGen.xml的UndistortImageFOV角度设置值无法被工具计算出来，必须修改为消息显示的范围值。



- fieldCurvature.csv 数据错误:

fieldCurvature.csv 内容中有空白列，必须修改为正确，然后工具才能成功执行。



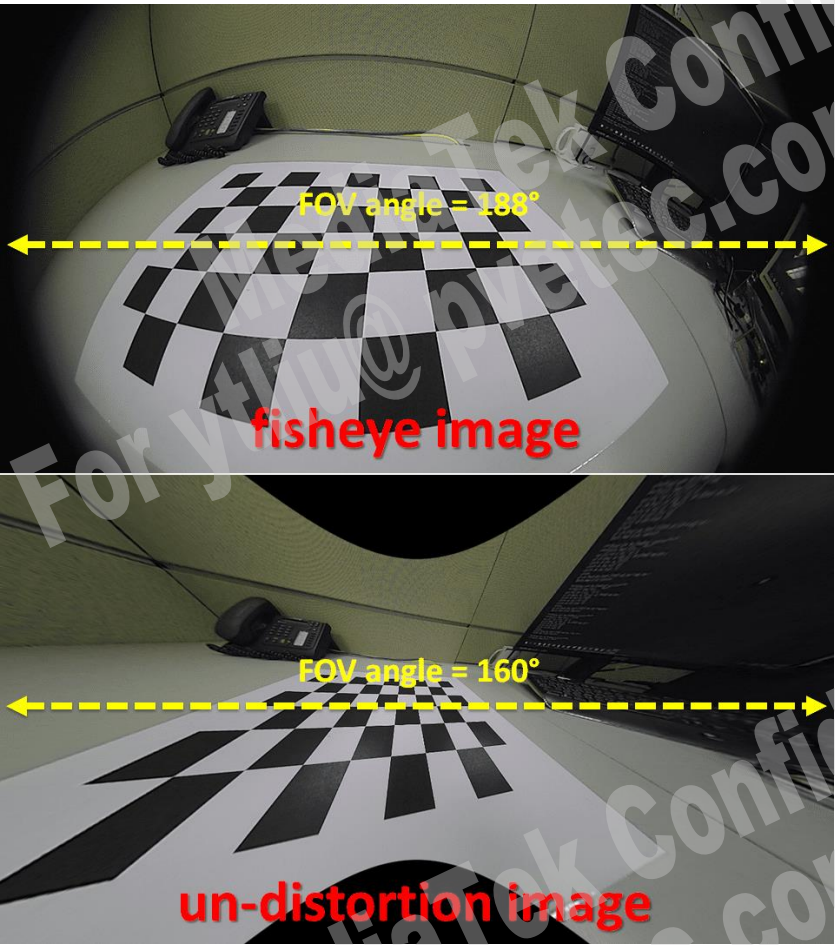
fieldCurvature.csv

SEMI-ANGLE	REAL IMAGE-HEIGHT	REF IMAGE-HEIGHT
0.1	0.001785	0.001785
0.2	0.003569	0.003569
0.3	0.005354	0.005354
0.4	0.007139	0.007138
0.5	0.008924	0.008923
0.6	0.010709	0.010708
0.7	0.012494	0.012493
0.8	0.014279	
0.9	0.016064	?
1	0.017849	
1.1	0.019635	0.019633
1.2	0.02142	0.021418

```
C:\Workspace\IntrinsicGen>intrinsicGen.exe AG190C/intrinsicGen.xml
The fieldCurvature.csv is found.
[Error] Please check fieldCurvature.csv data correct.
[Error] Camera calibration failed.
Finished.
```

检查调试无失真图像

输入文件夹中的Camera Calibration Images（鱼眼图像）的去畸变处理是基于intrinsic.xml数据，其中图像FOV由intrinsicGen.xml设置。另外，相机的固有参数是使用相机模组厂家提供的数据计算的。实际上，在每台相机的生产和组装过程中，手动调焦步骤会导致每台相机的固有参数出现误差。但这个误差会被avmClib中的AutoTuning算法调整。AutoTuning算法的主要作用是将相机的FxFyCxCy调整到最佳值。



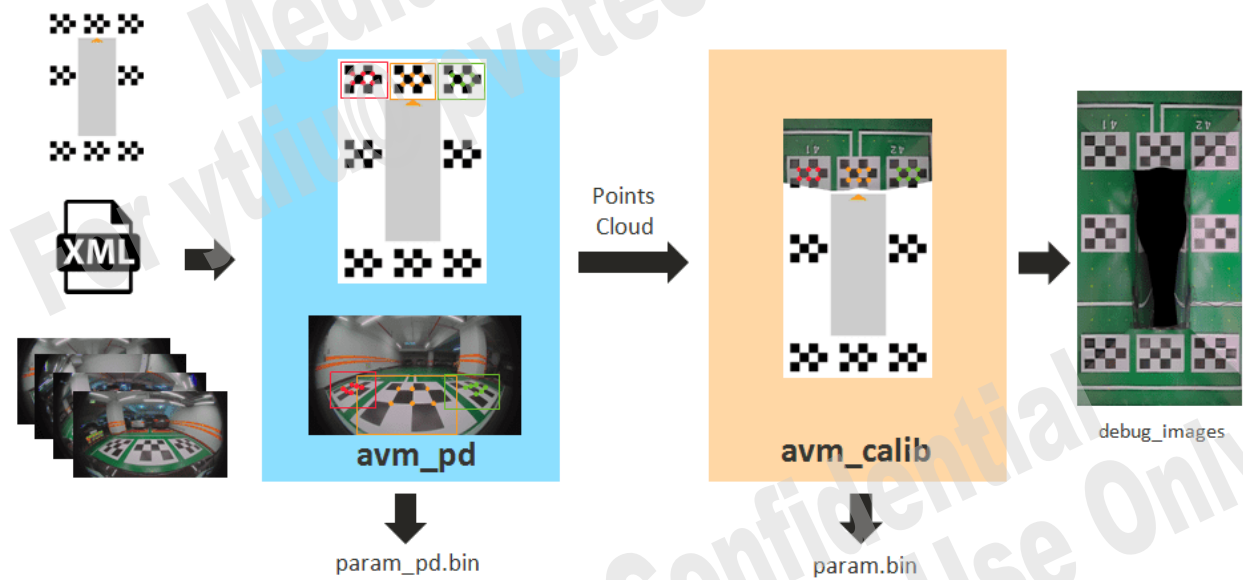
检查输出

Intrinsic Generation 工具会在输出文件夹中生成一个intrinsic.xml文件，该文件记录了固有参数和畸变系数。

```
<opencv_storage>
<calibration_Time>"2019/12/12 14:07"</calibration_Time>
<image_Width>1280</image_Width>
<image_Height>720</image_Height>
<Camera_Matrix type_id="opencv-matrix">
  <rows>3</rows>
  <cols>3</cols>
  <dt>d</dt>
  <data>
    2.4133086876155267e+02 0. 6.3950000000000000e+02 0.
    2.4105032822757110e+02 3.5950000000000000e+02 0. 0. 1.</data></Camera_Matrix>
<Distortion_Coefficients>0.4005871601858683 -0.11505986223784002
0.007643918923461579 0.0012556983682010122</Distortion_Coefficients></
opencv_storage>
```


6.4 AVM 标定

AVM 标定是一个计算参数 (param_pd.bin、param.bin) 的过程，用于将四个鱼眼相机输入转换/拼接为无缝鸟瞰图像。



工具下载与安装

请联系联发科技索取 AVM 标定工具包。您可以在该工具包中找到 CameraCalib 和 AVMCalib 工具。使用这些工具之前，请先安装 Visual Studio 运行时 (vc_redist_x64.exe)。

文件夹结构

安装AVM标定工具包后，AVMCalib结构如下：

AVM标定工具包

├── AVMCalib
│ ├── example/roi
│ │ ├── <examples>
│ │ └── script
│ │ ├── <scripts>
│ │ ├── AVMCalib.exe
│ │ └── <xxx.dll>
└── PatternAnnotationTool
├── index.html
├── mpa.js
└── mpa.css

其中：

- **example/roi** : 该文件夹包含使用各种 ROI 模式的标定案例。
 - 棋盘:
 - **asset**: 包含BEV材料, 包括BEV和可打印的微型车图案。
 - **minicar** : 是微型车的一个例子 (真实汽车的缩小版)。
 - **input**: AVM 标定输入文件, 包括 4 张相机图像、1 张 BEV 图像、相机固有图像和模式设置。详细信息请参阅 AVM 标定输入。
 - **avmSetting.xml**: 项目配置, 详情请见了解avmSetting.xml。
 - **rectborder_thick**:
 - **asset**: 包含BEV材料, 包括BEV和可打印的微型车图案。
 - **minicar** : 是微型车的一个例子 (真实汽车的缩小版)。
 - **input**: AVM 标定输入文件, 包括 4 张相机图像、1 张 BEV 图像、相机固有图像和模式设置。详细信息请参阅 AVM 标定输入。
 - **avmSetting.xml** : 项目配置, 详情请见了解 avmSetting.xml。
 - **rectborder_thin**:
 - **asset**: 包含BEV材料, 包括BEV和可打印的微型车图案。
 - **minicar** : 是微型车的一个例子 (真实汽车的缩小版)。
 - **input**: AVM 标定输入文件, 包括 4 张相机图像、1 张 BEV 图像、相机固有图像和模式设置。详细信息请参阅 AVM 标定输入。
 - **avmSetting.xml** : 项目配置, 详情请见了解 avmSetting.xml。
 - **rectsolid**:
 - **asset**: 包含BEV材料, 包括BEV和可打印的微型车图案。
 - **minicar** : 是微型车的一个例子 (真实汽车的缩小版)。
 - **输入**: AVM 标定输入文件, 包括 4 张相机图像、1 张 BEV 图像、相机固有图像和模式设置。详细信息请参阅 AVM 标定输入。
 - **avmSetting.xml** : 项目配置, 详情请见了解 avmSetting.xml。
- **script** : 包含Android环境中有用的脚本的文件夹。
 - **avm_camera_dump.bat**用于捕获四个摄像机的图像, 用于 AVM 标定输入。
 - **avm_clear_inlinecalib_bin.bat**清理在线标定的结果 (param.bin、param_pd.bin)。
 - **avm_pull_bin.bat**将param.bin和param_pd.bin保存到PC。
 - **avm_pull_inlinecalib_image.bat**保存内联标定应用程序用于当前标定的图像。
 - **avm_push_bin.bat**将 param.bin 和 param_pd.bin 推送至目标并应用结果。
 - **avm_restart_daemon.bat**重新启动守护进程。

- **avm_screencap.bat**用于转储当前 Android 屏幕。
- **xxx.dll**：执行AVMCalib.exe所需的动态库。
- **AVMCalib.exe**：可执行 AVM 标定，更多详细信息请参阅运行示例。
- **PatternAnnotationTool**：一个 Java 脚本工具，用于帮助生成 patSetting.xml。详细信息请参阅 Pattern Annotation Tool。

环境设置

个人电脑

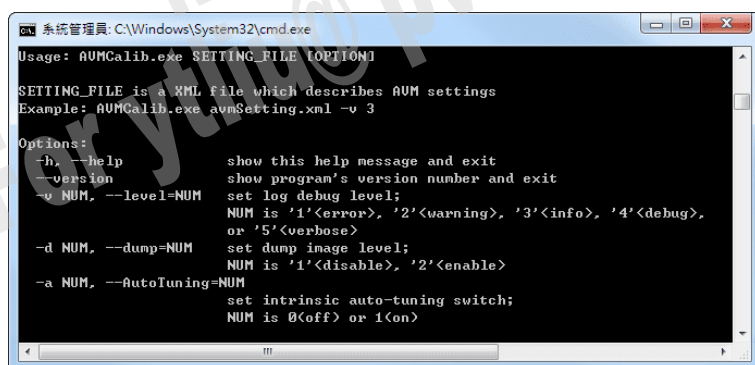
- Windows 7/10 (64 位)
- 需要 Visual Studio Runtime (在 AVM_Calibration_Toolkit 文件夹下安装 vcredist_x64.exe)。

MTK EVB (Android)

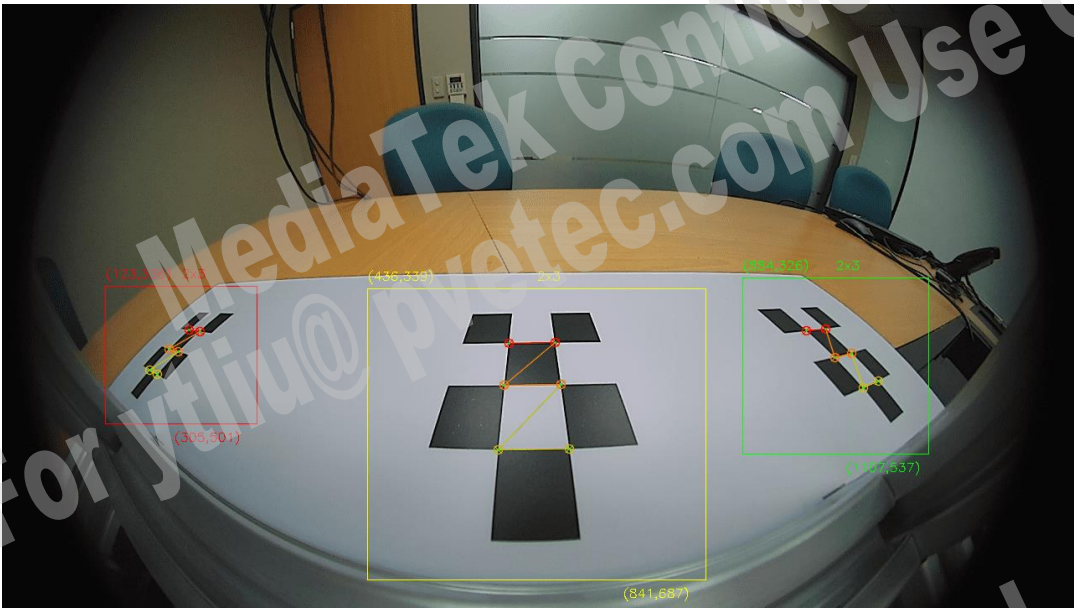
- EVB连接四台摄像头。
- 已启用 AVM 和 Android Android Debug Bridge(ADB) 的系统映像。
- 启动系统并确保 AVM 正在运行。

运行示例

1. 检查 AVM 标定输入。
2. 使用设置文件运行 AVMCalib.exe。
 - 例如 AVMCalib minicar\avmSetting.xml

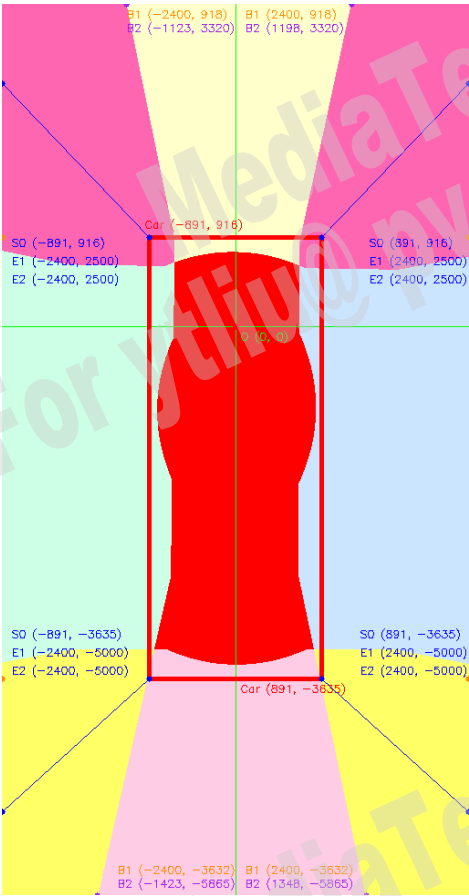


3. 检查输出文件夹中的标定输出。
 - output\param_pd.bin 和 output\param.bin
 - 这些参数箱是 EVB 上的快速预览所必需的。
4. 检查结果和故障排除。
 - 检查扭曲图像的结果。例如左摄像机图像ROI 和参考点。



检查针迹混合结果

- 混合锥设置
- 重叠区域检查



5. EVB 快速预览

- 将 param*.bin 推送至 EVB

```
adb root
adb wait-for-device
adb remount
adb shell "rm /mnt/mtkdata/avmdata/param_pd_new.bin"
adb shell "rm /mnt/mtkdata/avmdata/param_new.bin"
adb push param.bin /mnt/mtkdata/avmdata
adb push param_pd.bin /mnt/mtkdata/avmdata
adb reboot
```

注意：如果您曾经运行过内联标定，请确保 /mnt/mtkdata/avmdata 下没有 param_new.bin 和 param_pd_new.bin。

- 系统重启后，AVM 守护进程将重新加载参数。
- 启动 AVMDemo 应用程序来显示顶视图。

AVM 标定输入

AVM 标定需要以下输入，我们将在以下章节中以微型车示例详细描述。

- avmSetting.xml 项目配置。
- input\BEV.png 参考图样的鸟瞰图。
- input\front(left/right/rear).png 将摄像头安装到指定位置后从车辆上捕获的四个摄像头图像。
- input\intrinsic.xml 相机模块固有参数及畸变系数。
- input\patSetting.xml 配置文件，用于描述鱼眼相机图像与 BEV 参考图之间的关系。

捕捉相机图像

要捕捉相机图像，您需要：

- 首先启动系统并确保 AVM 正在运行
- 授予 ADB 根访问权限和可写分区

```
adb root
adb wait-for-device
adb remount
```

- 停止 AVM 应用程序，因为它占用了 AVM 守护进程资源


```
adb shell "am force-stop com.mediatek.avm"
```

- 切换到相机视图

```
adb shell "mtkavmclient 1 5"
```

- 切换和捕获 (3 0 = 前、3 1 = 左、3 2 = 右、3 3 = 后、4 1 = 捕获)

```
adb shell "mtkavmclient 3 0"
adb shell "mtkavmclient 4 1 /data/front"
adb shell "mtkavmclient 3 1"
adb shell "mtkavmclient 4 1 /data/left"
adb shell "mtkavmclient 3 2"
adb shell "mtkavmclient 4 1 /data/right"
adb shell "mtkavmclient 3 3"
adb shell "mtkavmclient 4 1 /data/rear"
```

- 将图像复制到主机 PC

```
adb pull /data/front.png
adb pull /data/left.png
adb pull /data/right.png
adb pull /data/rear.png
```

了解 avmSetting.xml

您可以在 XML 中找到三大元素组，我们以 \minicar\avmSetting.xml 来详细说明：

- <InputSetting> 描述输入配置。
- <OutSetting> 描述输出的顶视图应该是什么样子。例如顶视图可视区域有多大，如何融合两个相邻图像，如何裁剪侧视图的图像以及如何裁剪未失真的驾驶辅助视图图像。
- <DebugSetting> 描述在哪里找到调试图像。

以下是您可能需要根据您的 AVM 设计修改的一些重要元素。

< InputSetting >

< CamInput >

```
<!-- Multiple Camera Images -->
<CamInput>
  <!-- direction can be "all", "front", "left", "right", and "rear" -->
  <IntrinsicXML direction="all">intrinsic.xml</IntrinsicXML>
```

```
<CamImage direction="front">front.png</CamImage>
<CamImage direction="left">left.png</CamImage>
<CamImage direction="right">right.png</CamImage>
<CamImage direction="rear">rear.png</CamImage>
<!-- input resolution in pixels -->
<CamResoulution width="1280" height="720"/>
</CamInput>
```

此 <CamInput> 元素描述相机输入源。您需要定义图像的文件名、内参的 XML、去畸变系数和相机图像分辨率。当前系统要求所有相机具有相同的分辨率输出。

如果您的系统使用相同的四个摄像头模块，则您只需对这四个摄像头模块使用一个intrinsic.xml即可。

```
<IntrinsicXML direction="all">intrinsic.xml</IntrinsicXML>
```

如果您的系统使用具有不同光学特性和 FOV 的不同相机模块，您可以单独分配这些 XML。

```
<IntrinsicXML direction="front">intrinsic_front.xml</IntrinsicXML>
<IntrinsicXML direction="left">intrinsic_left.xml</IntrinsicXML>
<IntrinsicXML direction="right">intrinsic_right.xml</IntrinsicXML>
<IntrinsicXML direction="rear">intrinsic_rear.xml</IntrinsicXML>
```

内参文件可能来自相机标定过程。

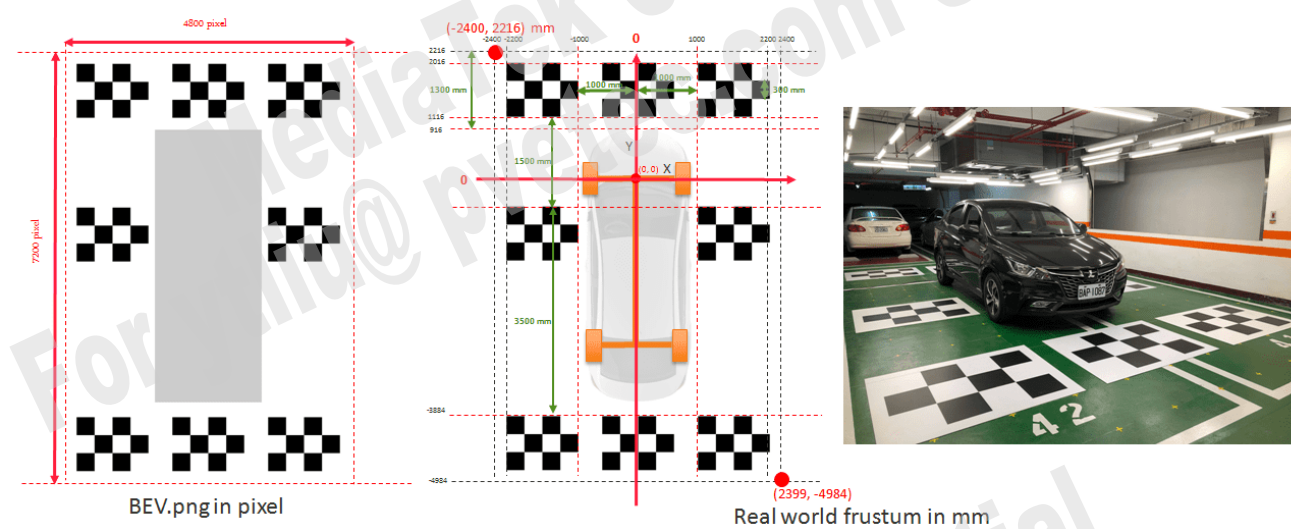
< BEVInput >

```
<!-- BEV(Top View) Calibration Image -->
<BEVInput>
  <Filename>BEV.png</Filename>
  <Frustum left="-2400"
    top="2216"
    right="2399"
    bottom="-4984" />
</BEVInput>
```

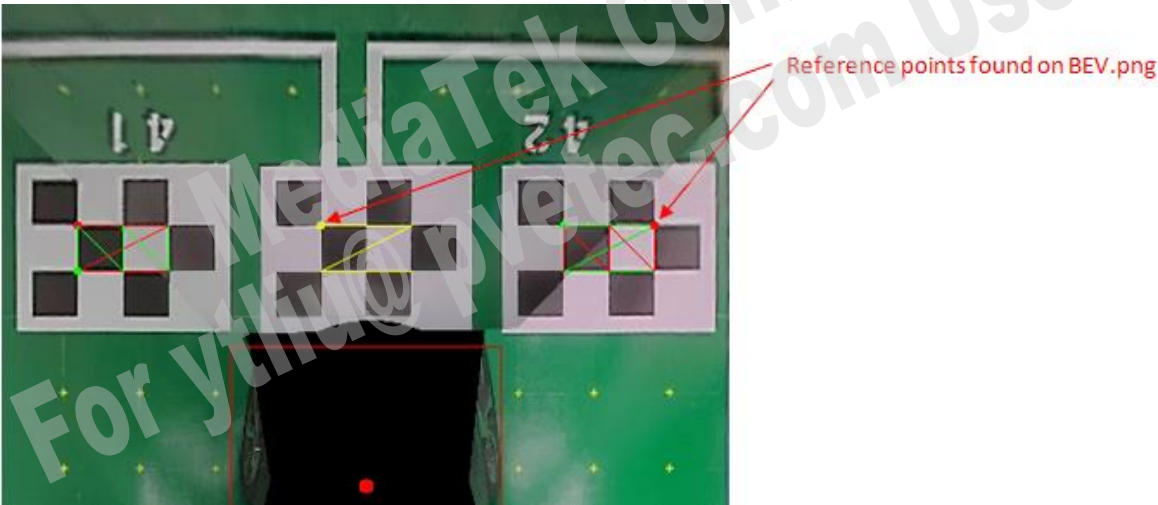
此 <BEVInput> 元素描述 BEV 图像（地面实况）。此图像上的图案大小和布局应与真实环境中的完全相同。avm_pd 算法（“pd”表示“点检测”）将在这些图案上找到参考点。avm_calib 将尝试将相机图像上检测到的点与这些地面实况点进行匹配，并生成投影矩阵。

<Frustum> 元素定义 BEV 中心点，该点应与汽车中心对齐。因此 AVMCalib 将知道 BEV 参考点（地面实况）与汽车之间的相对位置。

以下是真实汽车 BEV 设置和布局的示例：



您可以在调试图像 /temp/bev_preview_rp.png 上检查检测到的 BEV 参考点。

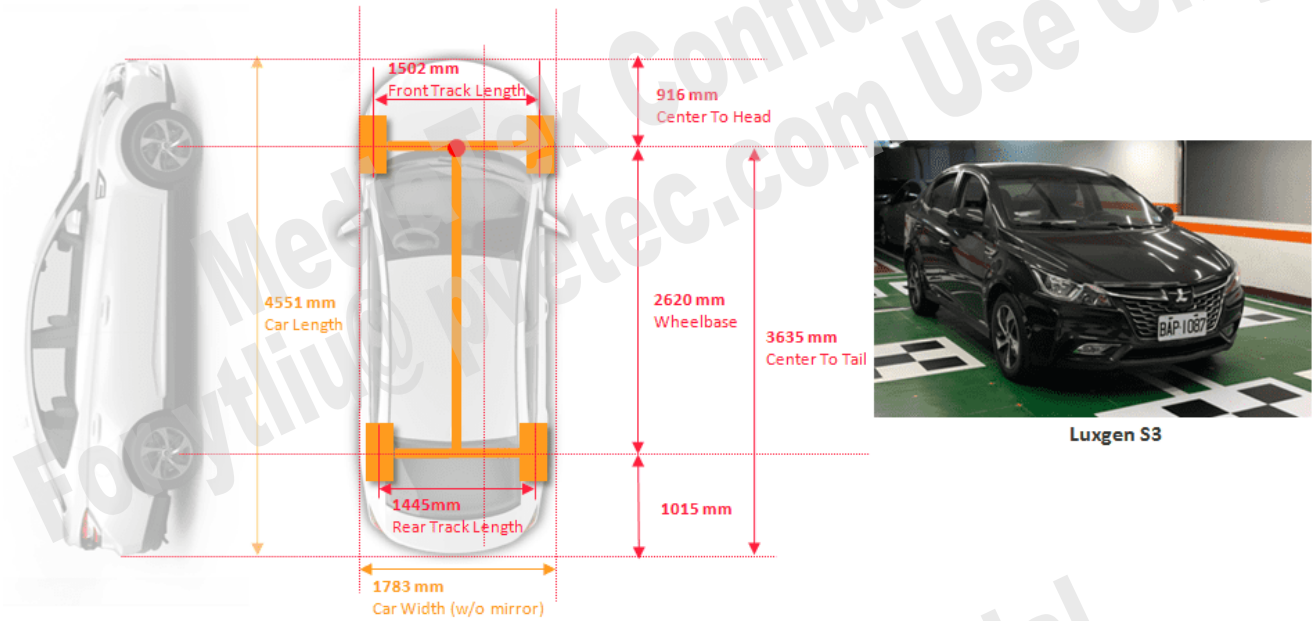


< CarDimension >

<CarDimension

```
width="1783"
length="4551"
wheel_base="2620"
front_track="1502"
rear_track="1514"
center_to_head="916" />
```

此 <CarDimension> 元素描述车辆的规格。请注意，宽度不包括后视镜。



您可以检查具有汽车尺寸的调试图像 /temp/bev_preview_rp.png 以检查配置是否符合预期。



< OutputSetting >

< DrivingAssistanceView >

```
<DrivingAssistanceView>
  <FOV view="front" percent="75"/>
  <FOV view="left" percent="75"/>
  <FOV view="right" percent="75"/>
  <FOV view="rear" percent="75"/>
</DrivingAssistanceView>
```

此 <DrivingAssistanceView> 元素描述了在未失真的鱼眼视图中使用的 FOV。

< AVMPParam & PdParam >

```
<!-- Camera transformation output-->
<AVMPParam>
  <Filename>param.bin</Filename>
</AVMPParam>

<!-- point detector output-->
<PdParam>
  <Filename>param_pd.bin</Filename>
</PdParam>
```

此 <AVMPParam> 元素和 <PdParam> 元素描述输出箱文件名。

< DebugSetting >

< BEVBlendData >

```
<!-- Bird Eye View (Top View) blend output-->
<BEVBlendData>
  <Frustum left="-2400"
            top="3320"
            right="2400"
            bottom="-5865" />
  <StitchRegion>
    ...
  </StitchRegion>
</BEVBlendData>
```

此 <BEVBlendData> 元素描述如何将四个摄像头图像拼接并混合到顶视图图像中。此设置仅适用于 /temp/bev_preview.png 和 /temp/bev_preview_rp.png，用于调试目的。



视锥体表示混合顶视图的最大可视区域。您可以将其设置为真实车辆的顶视图模拟设置。

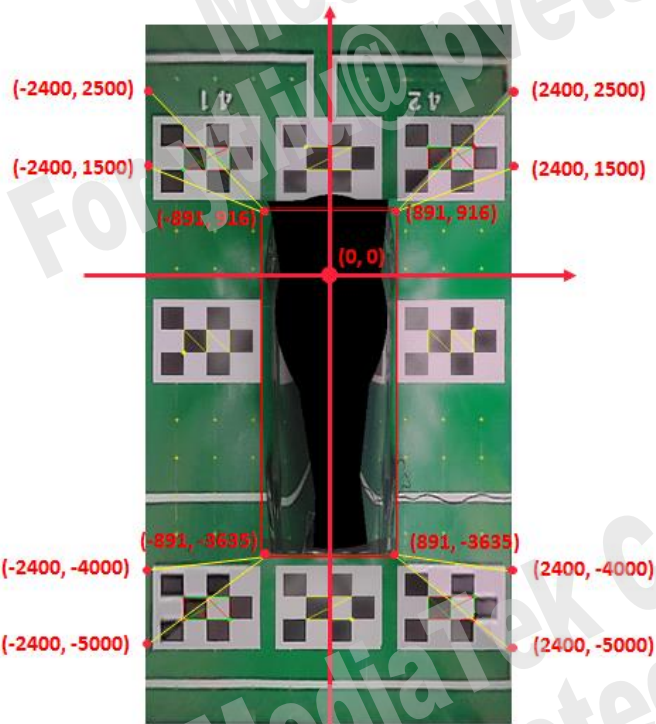
您可以检查 /temp/bev_preview.png 和 /temp/bev_preview_rp.png 以了解查看区域。



< StitchRegion >

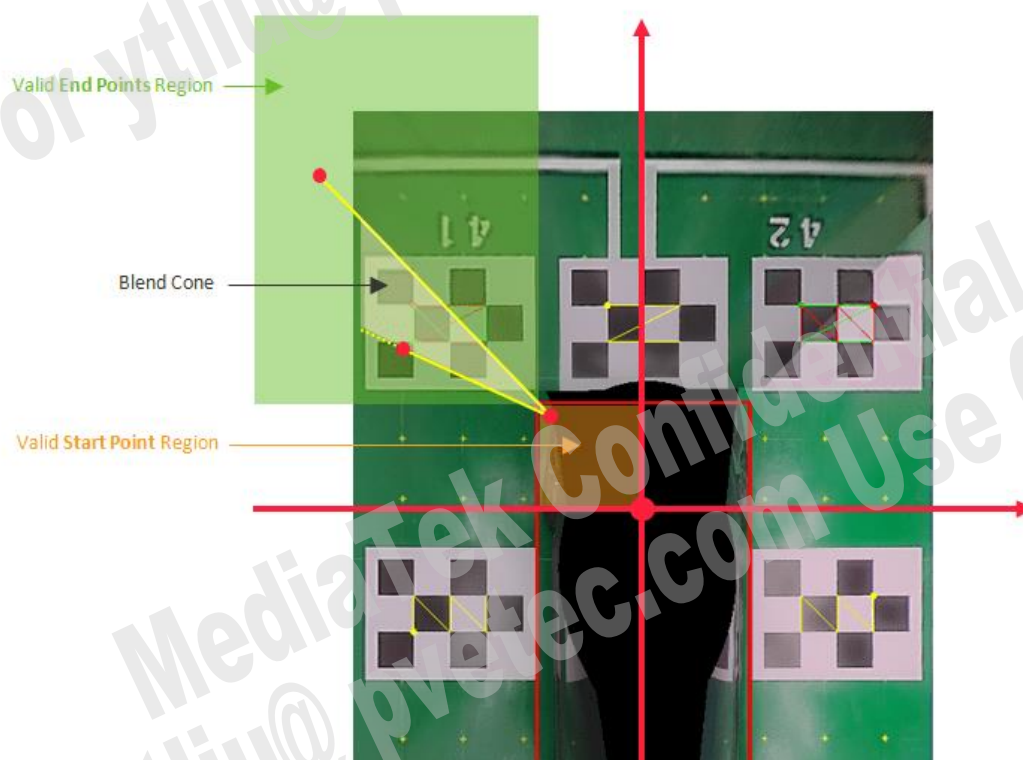
```
<StitchRegion>
  <ConeBlend corner="front-left" start_x="-891" start_y="916" end_x1="-2400" end_y1="1500" end_x2="-2400" end_y2="2500" />
  <ConeBlend corner="front-right" start_x="891" start_y="916" end_x1="2400" end_y1="1500" end_x2="2400" end_y2="2500" />
  <ConeBlend corner="rear-left" start_x="-891" start_y="-3635" end_x1="-2400" end_y1="-4000" end_x2="-2400" end_y2="-5000" />
  <ConeBlend corner="rear-right" start_x="891" start_y="-3635" end_x1="2400" end_y1="-4000" end_x2="2400" end_y2="-5000" />
</StitchRegion>
```

此 <StitchRegion> 元素描述了两个相邻图像的混合锥体。相邻图像将在此锥体内进行混合。



以下是一些 <ConeBlend> 的赋值规则。您可以以左前角为例：

- **起点**应位于车辆区域内
- **端点**应位于左前区域内。如果端点位于此区域之外，我们将从此区域选择最近的点
- **端点**不需要放置在 BEVBlendData 视锥区域的边界上，我们将进行插值以找到与视锥的交点并相应地计算混合数据。我们可以在 (end_x1, end_y1) 和 (end_x2, end_y2) 上设置相同的值以最小化混合区域，以检查拼接边缘的质量。



您可以检查调试图像 /temp/bev_stitch_map.png 和 bev_preview_front.png（左、右、后）以查看相邻图像的重叠区域和锥体混合区域。

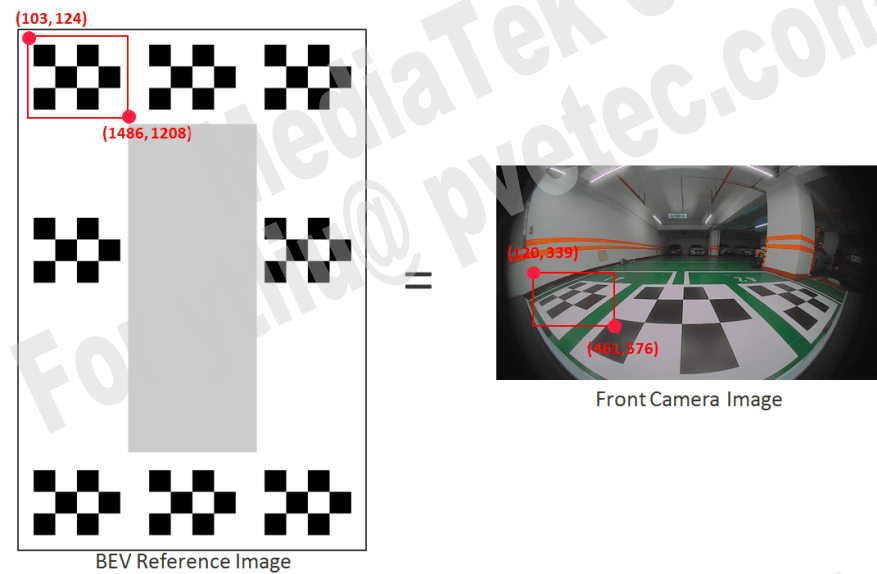


您可以使用 /temp/bev_preview.png 检查最终的混合结果。

了解 patSetting.xml

/input/patSetting.xml 通过使用 ROI 来描述在相机图像上找到的参考点与在 BEV 参考图像上找到的参考点之间的关系。请注意，ROI 单位是像素，图像的左上角是原点 (0,0)。avm_pd 模块将使用自动模式检测算法来查找 ROI 内的参考点。

```
<Pattern type="chessbord">
  <GridSize width="3" height="2"/>
  <BEVROI>
    <AutoFind>
      <ROI left="103" top="124" width="1383" height="1084"/>
    </AutoFind>
  </BEVROI>
  <CamROI>
    <AutoFind>
      <ROI left="120" top="339" width="341" height="237"/>
    </AutoFind>
  </CamROI>
</Pattern>
```



图案注释工具

该工具提供了友好的用户操作界面，用于生成AVM标定过程的patSetting.xml。

工具下载

请访问联发科的 DCC 下载 AVM 标定工具包。您可以在该工具包中找到 PatternAnnotationTool 工具。

运行工具

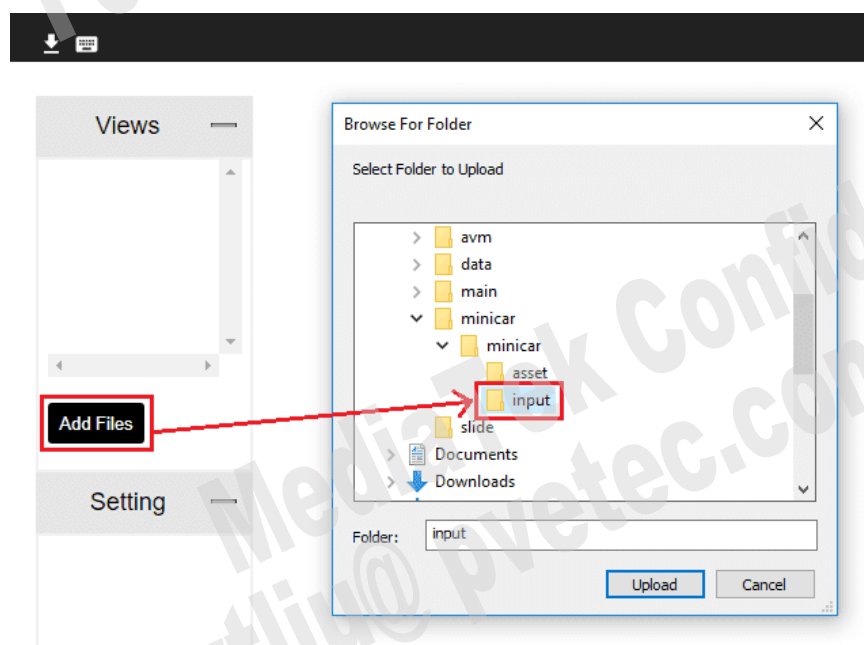
请在 Google Chrome 中运行该工具

- 双击 index.html 执行该工具

注释教程

文件导入器

1. 单击“添加文件”按钮并选择包含 avm 标定图像的目录。



2. 确认图片上传：

Upload 7 files to this site?

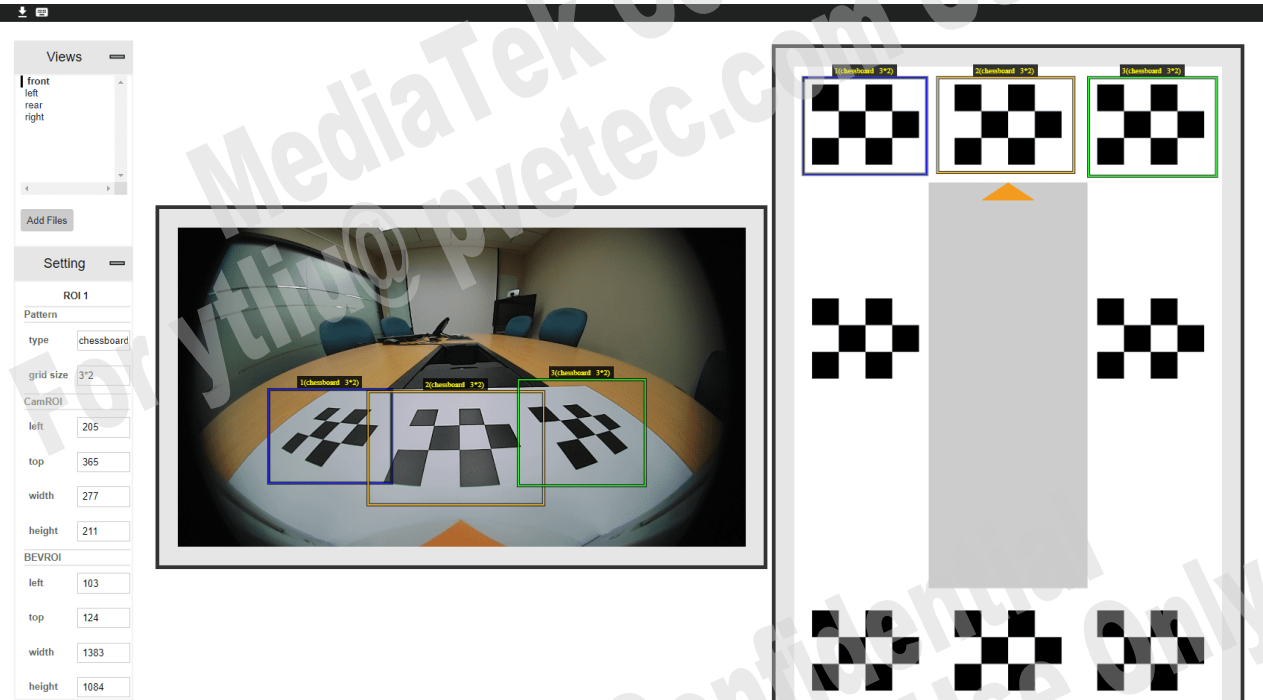
This will upload all files from "input". Only do this if you trust the site.



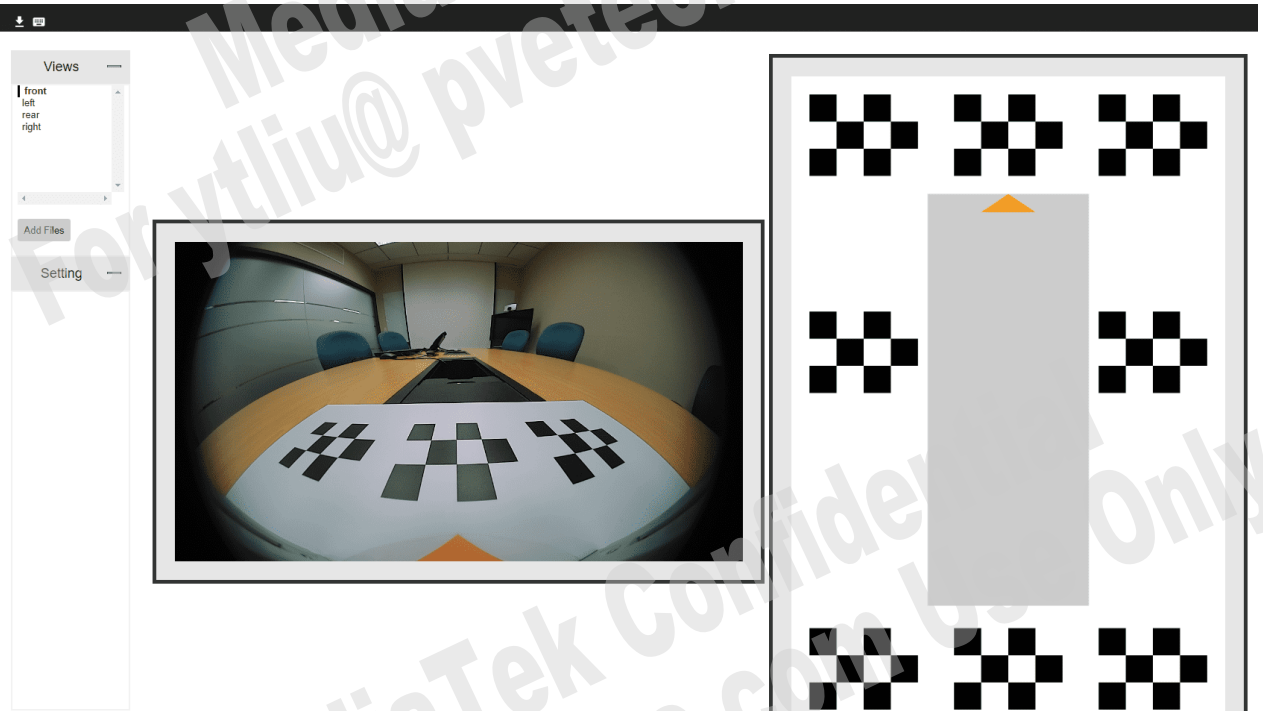
- 文件名区分大小写。请检查所有文件是否存在，并且其名称与示例图像相同。



- 如果您的输入文件夹已经有文件patsetting.xml，我们将自动导入现有的注释设置。



- 如果没有，则需要绘制配对的 ROI。

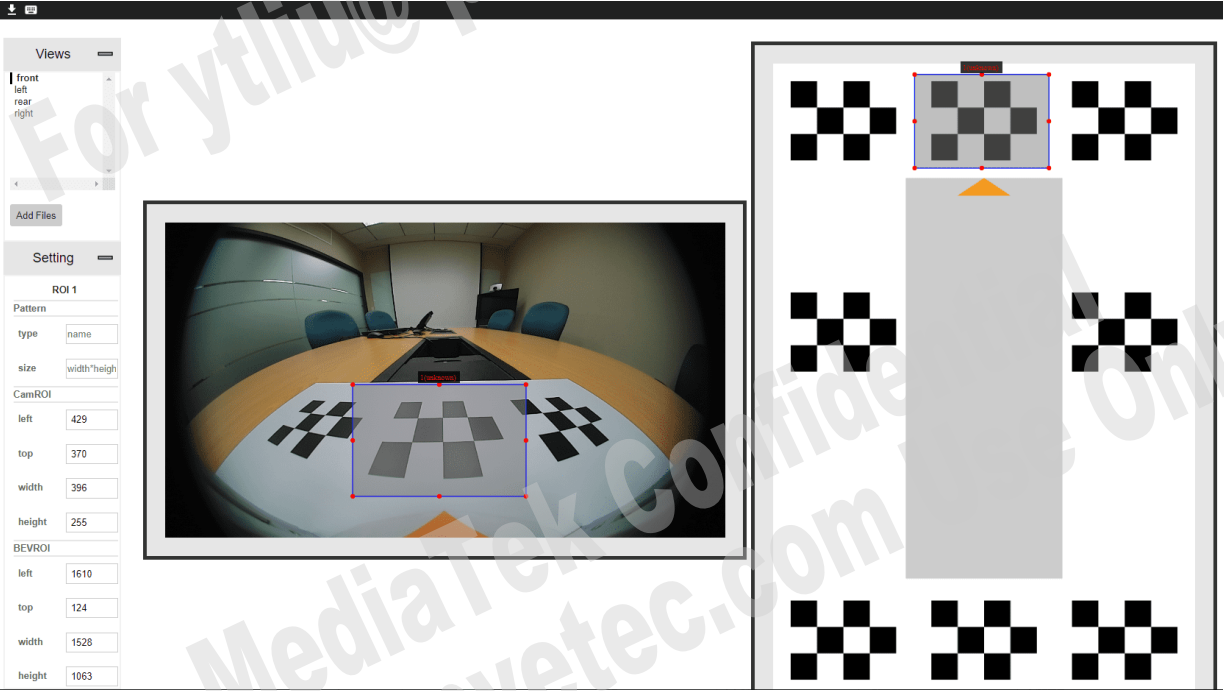


ROI操作

ROI操作的工作区在黑色边界内，请在工作区内完成各项操作。

画

通过拖动鼠标在相机视图和 BEV 视图上绘制配对的 ROI。



选择/取消选择

- 如果您想选择 ROI，请单击 ROI 内部。
- 如果要取消选择 ROI，请单击工作区中的任意位置（调整大小点除外）。

移动

通过拖动鼠标或者按键盘的上/下/左/右键来移动选定的ROI。

调整大小

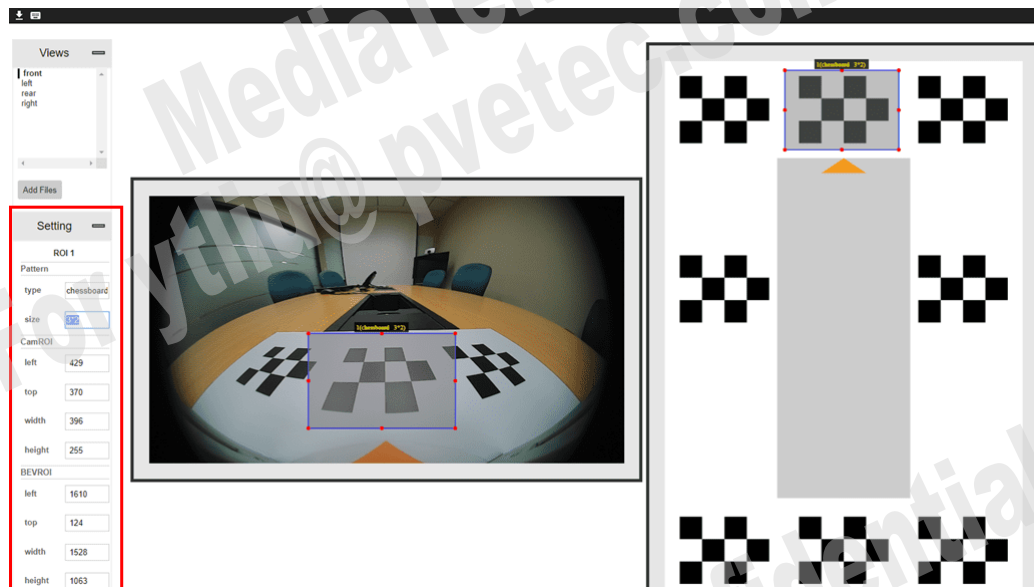
通过拖动 8 个调整大小点来调整所选 ROI 的大小。

删除

按键盘的 Delete 键删除选定的 ROI。

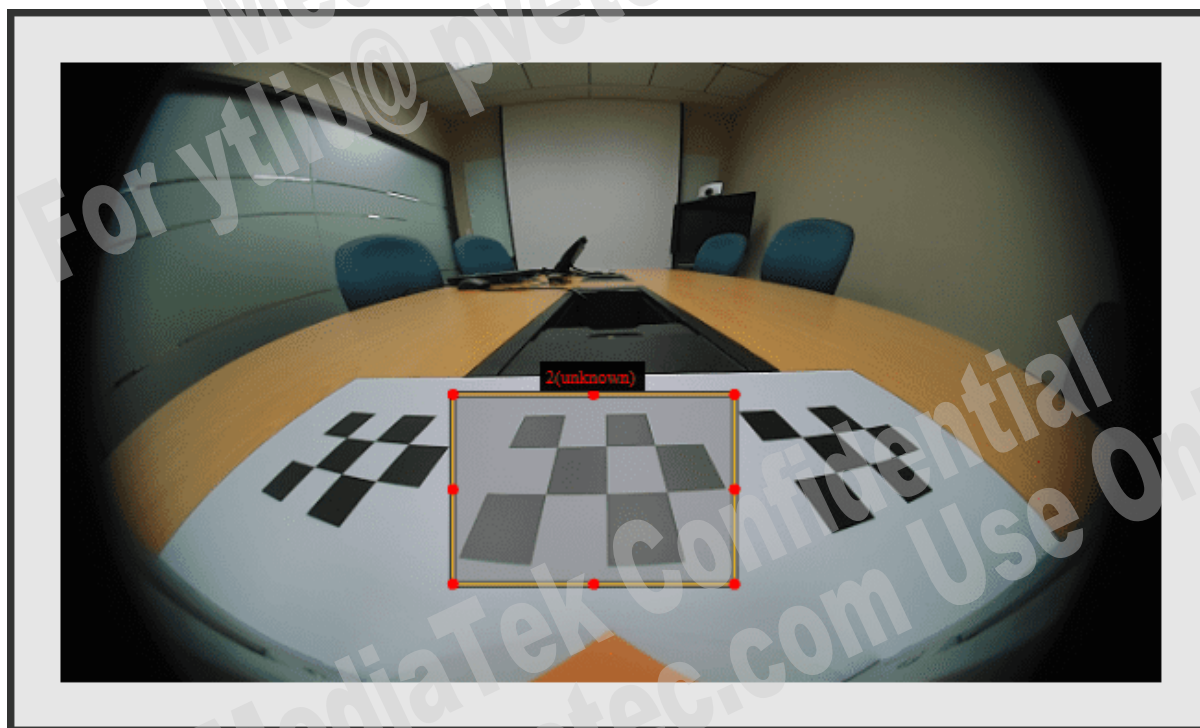
- 属性设定

使用设置面板来微调选定的 ROI 信息。



- 复制和粘贴

- 按 Ctrl + C 复制选定的 ROI。
- 当您按下 Ctrl + V 时，ROI 将粘贴在同一位置。



- 放大和缩小

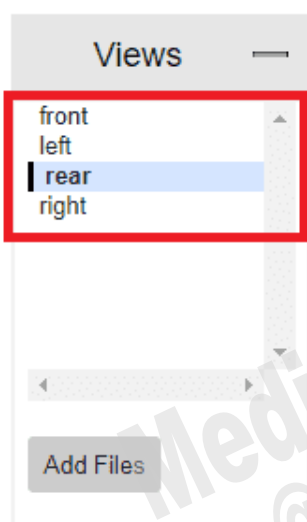
通过按键盘的 + / - / = 键来放大 / 缩小 / 重置选定的 ROI。

- 撤消和重做

通过按 Ctrl + Z / Ctrl + Y 撤消 / 重做 / 执行的操作。

- 切换视图

- 点击查看列表进行切换



- 在区域未选择状态下按键盘的上/下键切换到相对上一个/下一个视图。

文件导出器

- 1. 将 patSetting.xml 保存到下载文件夹:



- 2. 将 patSetting.xml 复制到输入文件夹。

键盘快捷键

如果您对 ROI 操作有任何问题，请参考此页面。

A screenshot of the application's keyboard shortcuts window. The window is titled 'Keyboard Shortcuts' and has a close button (X) in the top right corner. It contains two sections: 'Available only on image focus' and 'Always Available'. The 'Available only on image focus' section lists commands like 'Move selected region by 1 px', 'Copy selected region', 'Paste selected region', 'Delete selected region (pairs if exist)', and 'Zoom in/out with selected region(in current cursor placed image)'. The 'Always Available' section lists commands like 'Move to next/previous image', 'Undo the action', 'Redo the action', and 'Zoom reset'. A red arrow points from the 'Keyboard Shortcuts' button in the top left of the application interface to this window.

Keyboard Shortcuts

Available only on image focus

Command	Shortcut
Move selected region by 1 px	← ↑ → ↓
Copy selected region	Ctrl + c
Paste selected region	Ctrl + v
Delete selected region (pairs if exist)	Delete
Zoom in/out with selected region(in current cursor placed image)	+ -

Always Available

Command	Shortcut
Move to next/previous image	↑ ↓
Undo the action	Ctrl + z
Redo the action	Ctrl + y
Zoom reset	=

调试

日志详细程度

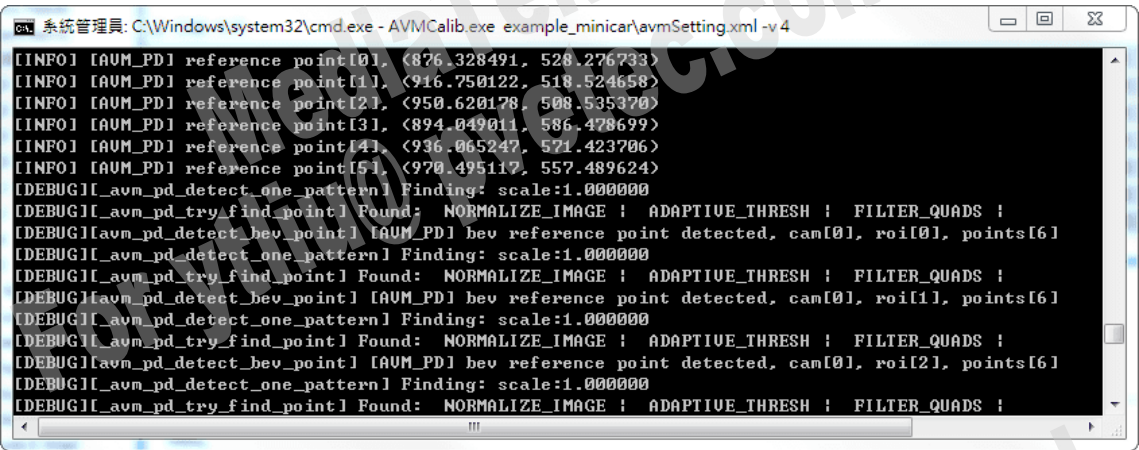
按照严重程度的顺序，有多种记录消息的方法：

- Error (1) - 任何对操作造成致命影响但对应用程序不造成致命影响的错误
- Warn (2) - 任何可能会导致应用程序异常但我们可以自动恢复的情况。
- Info (3) - 通常需要记录的有用信息（流程、数据信息等）
- Debug (4) - 诊断信息不仅对开发人员有帮助，对其他人也有帮助
- Verbose (5) - 仅当你“跟踪”代码并尝试找到函数的某一部分时

如果遇到任何问题，请将调试日志级别设置为调试（4）。建议在向联发科技提交问题时将调试或详细日志附加到 eService。

要调整日志级别，请使用 -v 选项：

AVMCalib.exe avmSetting.xml -v 4



调试图像级别

关于调试映像级别，调试映像有三个级别：

- Disable (1)：关闭调试映像。您还可以通过设置此选项来加快标定过程。
- Enable (2)：我们将输出带有检测到的参考点的 BEV 预览图像、拼接图和 ROI 图像。

要调整日志级别，请使用 -d 选项，例如：

AVMCalib.exe avmSetting.xml -d 2

以下命令启用所有详细日志和所有调试图像：

```
AVMCalib.exe avmSetting.xml -v 4 -d 3
```

检查ROI设置和点检测结果

您可以检查在调试图像 /temp/roi-front.png 中找到的 ROI 和参考点。



如果您正在实现模式检测算法 (avm_pd)，并且在开发期间自动查找的准确性不够高，则可以使用手动分配的点来绕过自动查找。

```
<Pattern type="chessbord">
  <GridSize width="3" height="2"/>
  <BEVROI>
    <AutoFind>
      <ROI left="103" top="124" width="1383" height="1084"/>
    </AutoFind>
  </BEVROI>
  <CamROI>
    <AutoFind>
      <ROI left="120" top="339" width="341" height="237"/>
    </AutoFind>
    <ManualFind>
      <Point x="258" y="416"/>
      <Point x="288" y="423"/>
      <Point x="321" y="429"/>
      <Point x="204" y="441"/>
      <Point x="228" y="454"/>
    </ManualFind>
  </CamROI>
</Pattern>
```

```
<Point x="258" y="468"/>  
</ManualFind>  
</CamROI>  
</Pattern>
```


自动调谐开关

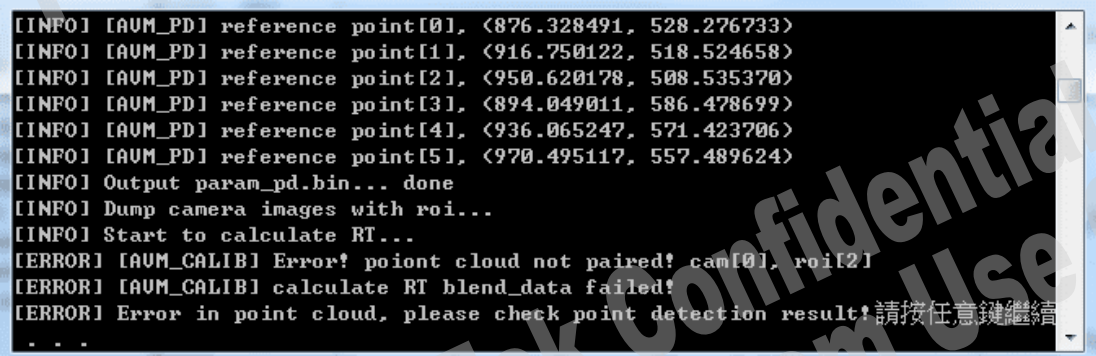
为了克服镜头的组装误差，我们的算法将尝试补偿以获得更好的质量。您可以在开发阶段将其关闭以检查真实相机的内在质量。

```
AVMCalib.exe avmSetting.xml -a 0
```

故障排除

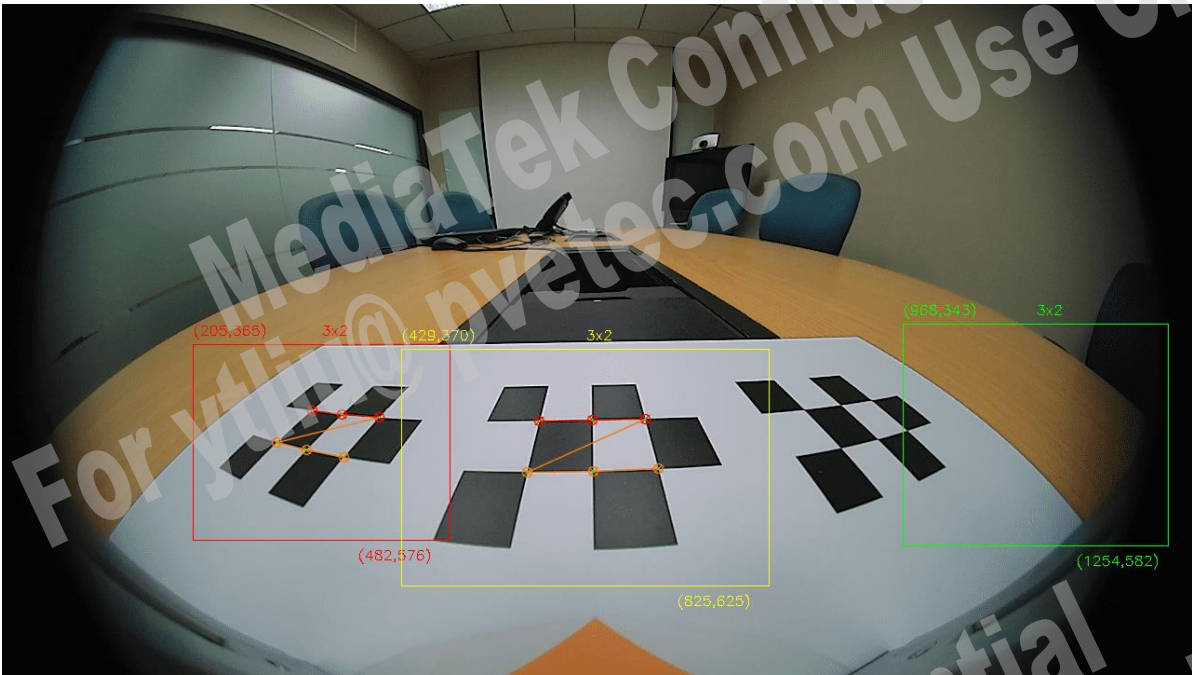
点云信息不匹配

如果 AVMCalib 报告如下错误：



此错误是由于相机参考点和 BEV 参考点之间的点数不匹配造成的。在大多数情况下，这是由于相机参考点检测不正确造成的。请先检查 patSetting.xml 中的 ROI 设置。

下图显示了 ROI 未正确覆盖模式的示例：



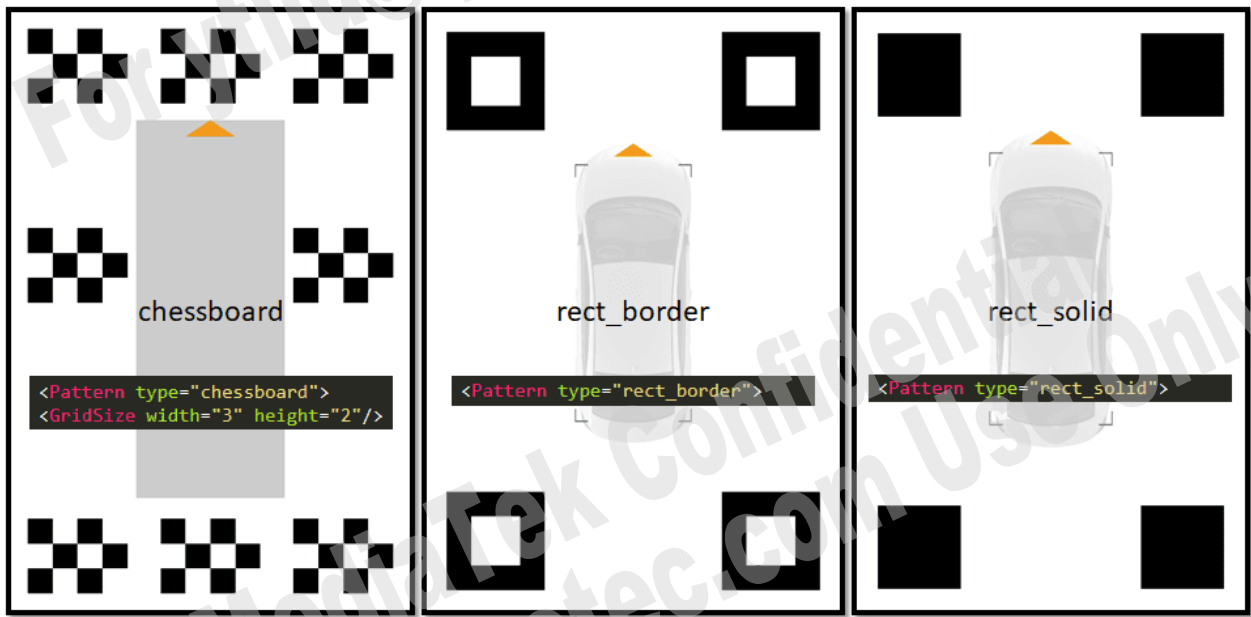
常见问题

可以使用四种模式代替八种模式吗？

是的，我们建议模式应该涵盖所有特殊情况以获得更好的质量。

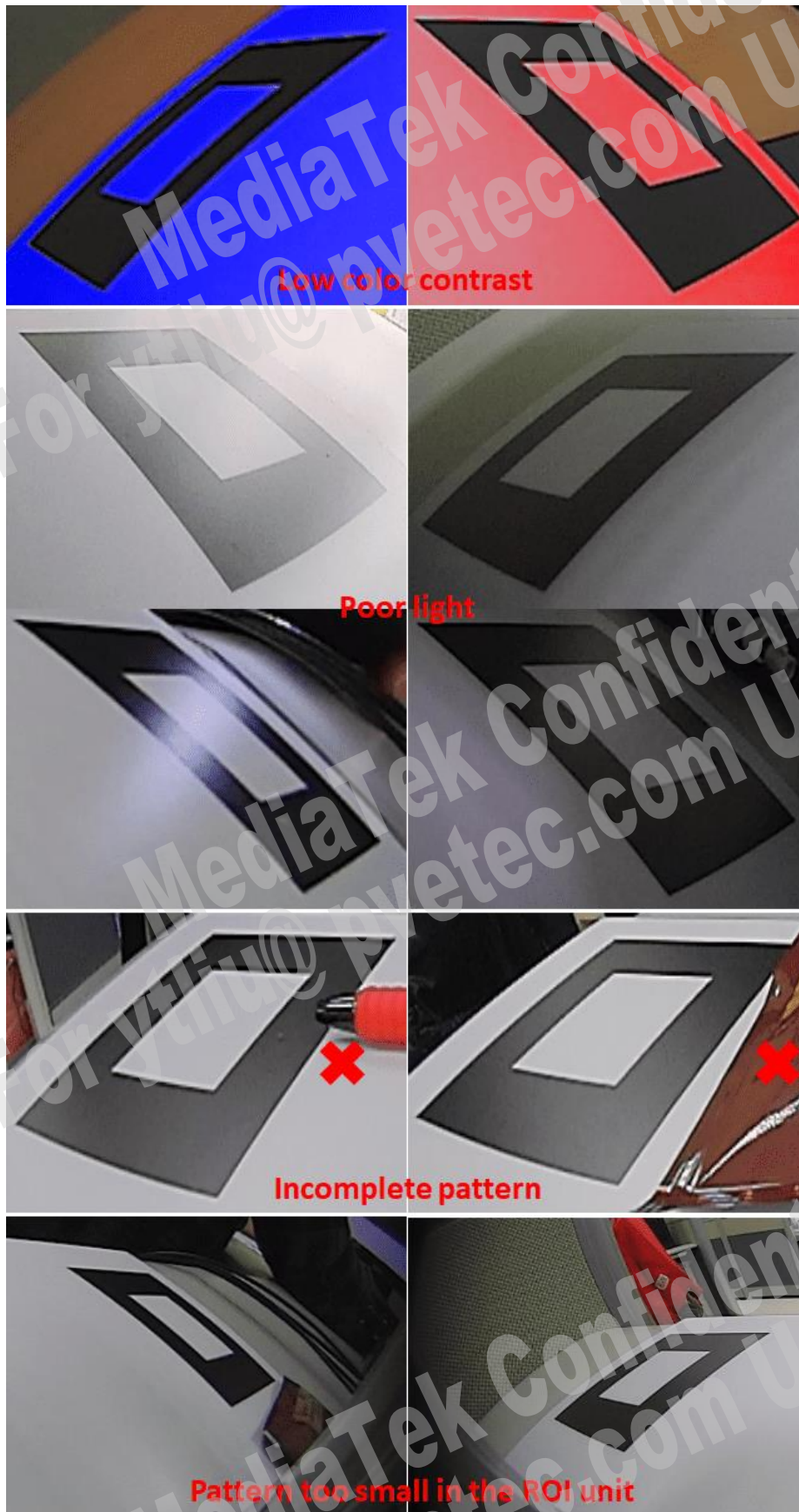
可以使用其他类型的 BEV 参考图像来执行 AVM 标定工具吗？

是的，AVM 标定工具支持三种 BEV 参考图像图案类型。其中包括棋盘、rect_border 和 rect_solid 图案类型。



如何避免BEV参考图的检测点失败？

图案与地板背景颜色对比度要高（例如：黑白）。图案不能受弱光影响（过曝或过暗）。图案主体不能被物体遮挡，否则会导致图案形状特征消失。图案周围环境要干净，去除非必要物体。在patSetting.xml中设置ROI位置时，必须减少非图案主体的其他物体的选取。



6.5 在线标定

AVMCalibration 是一个用于工厂生产线上每辆车标定的 Android 演示应用程序。您必须在固件中存储一个通用版本的标定参数作为默认值，并且您可以在生产中微调这些标定参数以补偿每辆车的安装差异。

车辆停在指定位置后，工人可以使用我们的自动模式检测算法在不到 1 分钟的时间内完成在线标定过程。

在线标定过程包括：

- 检查相机质量，如有必要进行调整/更换。
- 检查摄像机安装公差，如有必要，进行修复/更换。
- 车辆自动模式识别与标定。

注意：该在线标定应用程序是一个演示应用程序，您可以参考此应用程序来设计适合您的系统和工厂生产线的在线标定流程。

环境设置

车辆

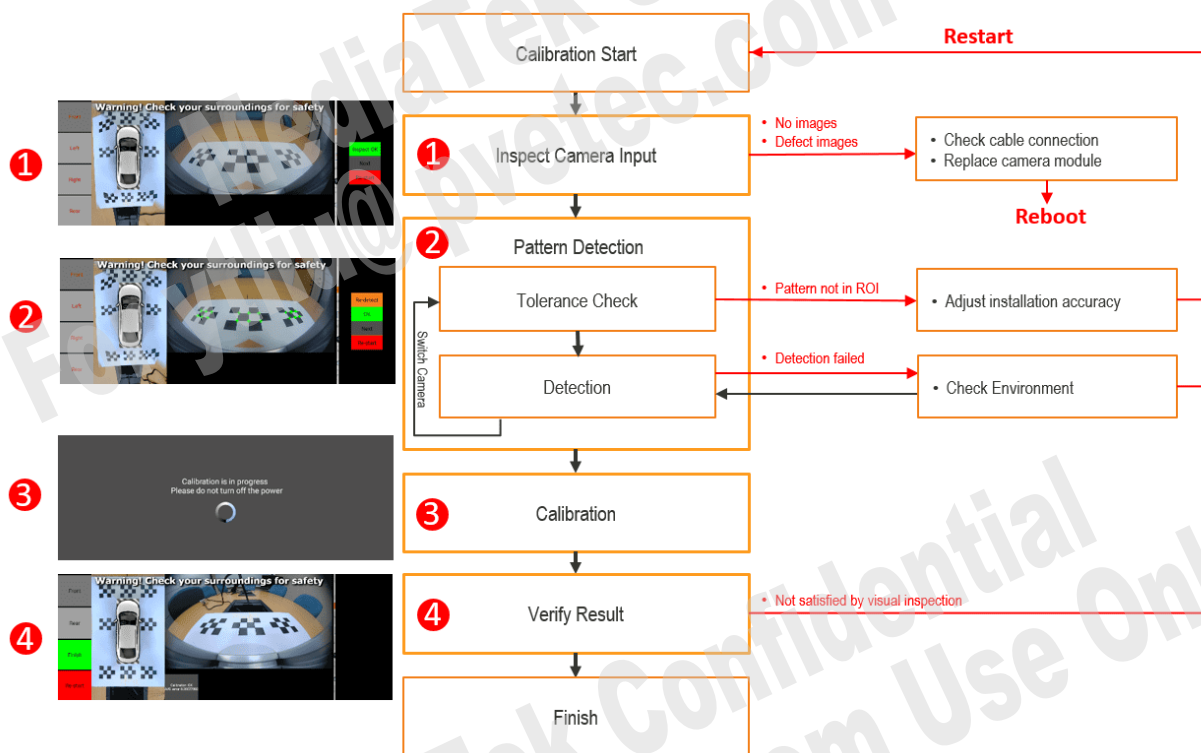
- 我们的 SDK 中的真实车辆或微型车在正确的位置安装了摄像头。

联发科技评估板

- EVB连接四台摄像头。
- 已启用 AVM 的系统映像，其中安装了与车辆匹配的预标定参数 (param.bin、param_pd.bin)。

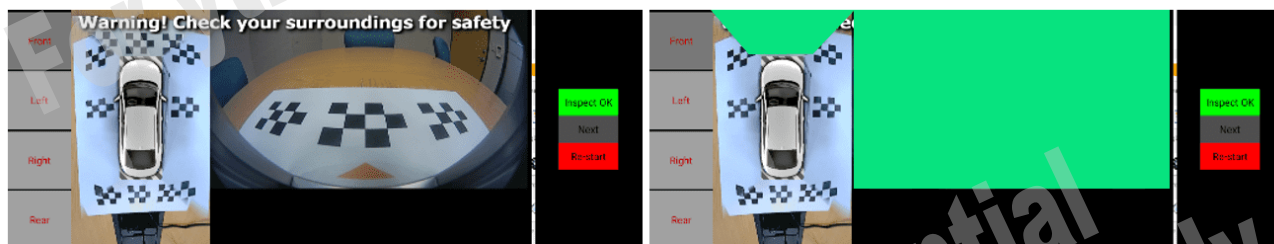
在线标定步骤

在线标定过程包含四个步骤。



1. 检查相机输入

此阶段需检查相机输入图像的质量，若相机图像质量不理想（颜色不对、有杂讯等）或完全无图像输出，则需依生产流程检查线材连接或更换相机模组，并重启系统。请依序按下 Inspect OK 按钮，检查四台相机。

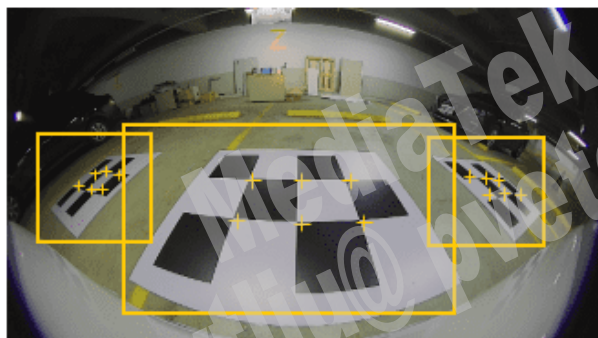


Correct Camera Input

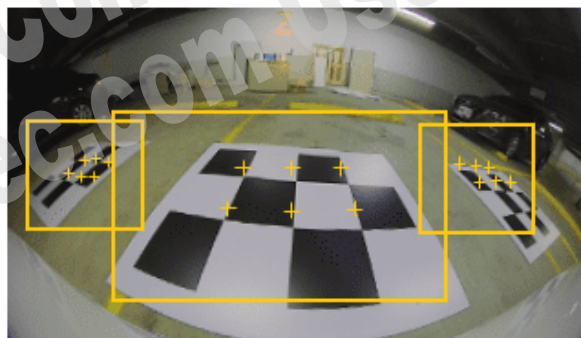
Fault Camera Input

2. 模式检测

在此阶段，您应从相机安装公差检查开始，以检查相机安装角度是否正确。您应在相机图像上检查通用版本标定数据的 ROI 和参考点（用橙色绘制）。如果相机图像中的图案与 ROI 和参考点相距太远，则相机支架可能存在一些安装错误。您应调整相机支架并重新启动在线标定过程。

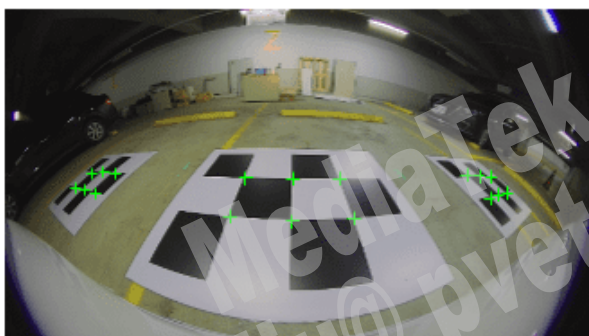


Correct Camera Installation

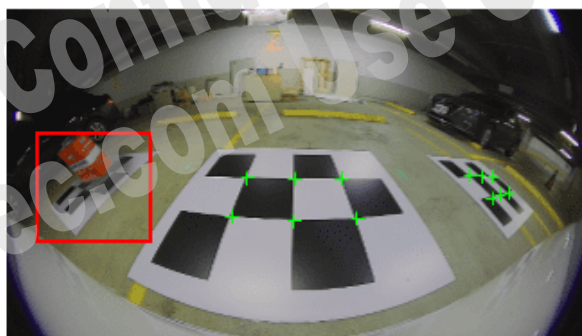


Fault Camera Installation

完成容差检查后，您应按下“检测”按钮。应用程序将自动检测 ROI 内的参考点并将其绘制为绿色。您应目视检查参考点是否被正确检测到。在某些极端环境条件下（极端光源、图案被某些障碍物覆盖等），检测可能会失败，并会显示红色 ROI。您应检查并修复环境问题，然后重新检测参考点。



Detection Succeeded

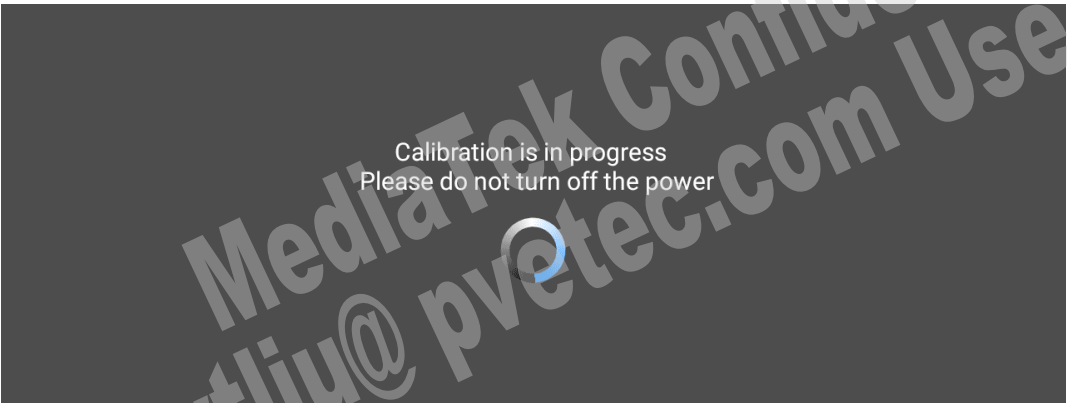


Detection Failed

成功检测到四个摄像机上的参考点后，按下标定按钮开始标定过程。

3. 标定

计算新的 AV



4. 检查结果

AVM 图像已应用于新标定的参数。您应目视检查结果并确定质量是否足够好。如果您接受标定结果，请按“完成”按钮以完成整个在线标定过程。新参数将应用于系统（param_new.bin、param_pd_new.bin）。如果您不接受结果，请按“重新启动”按钮并重新完成整个过程。



7 标定步骤

本章简要介绍如何逐步进行 AVM 标定。

0. 准备

- 以下步骤基于 MTK AVM 守护进程。我们强烈建议首先建立联发科 AVM 环境：
 - AVM 守护进程
 - AVMDemo 应用程序
 - AVM 标定应用程序

1. 相机标定

生成每个相机镜头模型的固有参数，并用于去畸变鱼眼图像。

生成内参 (intrinsic.xml) 有两种方法：

- 可以从制造商处获取相机规格和镜头场曲率数据的内在生成工具。
- 相机标定工具，供普通用户通过手动捕捉棋盘图像进行标定。

注意：我们建议对每个镜头型号进行标定。

(例如，如果车辆上安装了两种镜头型号 (A 和 B)，则应运行标定过程并获取 A 和 B 的 2 组固有数据)

2. AVM 标定

计算每个安装的摄像机的参数，包括如何将摄像机图像投射到地面。

- 创建您自己的项目。(它可能与 AVM 标定示例克隆)

AVMCalib

```
| |—— {your_project}
| | |—— input
| | | |—— BEV.png
| | | |—— front.png
| | | |—— intrinsic.xml
| | | |—— left.png
| | | |—— patSetting.xml
| | | |—— rear.png
```

```
| | | | right.png
| | | | avmSetting.xml
```

- 根据客户要求创建 BEV 图像。请参阅 BEVInput 部分。您可能需要检查以下设置：
 - 车辆规格
 - 图案类型/尺寸
 - 摄像机角度和位置
- 将 avmSetting.xml 替换为您的车辆/相机规格。请参阅了解 avmsetting.xml。您可能需要检查 <InputSetting> 中的这些字段：
 - <CamInput>
 - <CamResolution>
 - <BEVInput>
 - <Filename>
 - <Frustum>
 - <CarDimension>
- 用上一步相机标定的输出替换intrinsic.xml。
- 从车辆收集摄像机图像并将其放入输入文件夹。
 - 如何从 AVM 守护进程捕获。
 - 默认文件命名是front.png、left.png、right.png和rear.png。
- 修改patSetting.xml
 - 注释工具可以帮助您生成/修改 patSetting.xml。注意：当前版本仅支持棋盘图案。
 - 设置可靠的 ROI 矩形和相关模式类型。详细信息请参阅了解 patSetting.xml 部分。
 - 注意：由于内联标定依赖于 ROI 矩形设置来检测每辆车上的模式，因此 ROI 矩形不应设置为“太适合”您的模式，否则内联标定可能会因在错误的 ROI 中未检测到模式而失败。
- 运行AVMCalib.exe并生成bin文件（param.bin和param_pd.bin）。
 - 使用你的项目执行 AVMCalib.exe
 - AVMCalib {你的项目}\avmSetting.xml

3. 在线标定

为了克服相机规格公差和安装误差，每辆车在运输前都应进行在线标定。因此，在生成 AVM bin 文件后，我们需要设置在线标定配置。

- 设置 AVM 标定 bin 文件 (param.bin 和 param_pd.bin) 的两种方法。
 - Build image phase:

- 在构建 Android 映像之前，将 bin 放入 /vendor/mediatek/proprietary/hardware/mtkcam_sec_lib/mtkavm/avmdaemon/spm/calibdata。
- Integration phase (with ADB root):
 - adb root; adb wait-for-device;
 - adb remount;
 - adb push param.bin /mnt/mtkdata/avmdata;
 - adb push param_pd.bin /mnt/mtkdata/avmdata
- 按照在线标定车轮轨迹线为每辆车生成并保存 bin 文件。

常见问题

相机标定

应该拍摄多少张照片才能保证质量？

所需图像总数没有绝对限制。唯一的要求是在捕捉图案图像时尽量覆盖相机视野的大部分区域。在此基础上，越多越好.....

8 演示应用程序



AVM SDK 包含两个 Android 应用程序，用于演示 SDK 的实际用例。

- AVMDemo 允许您在不同的 AVM 视图之间切换，这是 AVM 守护进程的一个用例。
- AVMCalibration 允许您执行基本的在线标定过程。这演示了在线 AVM 标定库和点检测库的实际用例。

还提供了额外的命令行工具 `mtkavmclient`，以便通过简单命令控制 AVM 守护进程。这可用于调试或测试目的。

以下部分解释了这两个 Android 应用程序背后的设计原理，以帮助您将守护进程和库集成到您自己的应用程序设计中，并为命令行实用程序提供使用示例。

8.1 AVM 演示



AVMDemo 是一款 Android 应用，用于演示 Android 平台上的 AVM 集成。这是一款简单的 Android 应用，只有一个活动。它演示实时摄像头视图和指导，并允许用户使用工具栏在不同的 AVM 显示模式之间切换。

应用程序启动

如何启动 Android AVM 应用程序：

- 使用 Android 启动器并点击/单击 AVMDemo 应用程序图标。

视图切换

点击或单击屏幕左侧的应用程序工具栏可在不同的显示模式之间切换。下图说明了工具栏中每个按钮的功能：



显示模式的名称仅供说明之用。下表概述了每种显示模式的用途：

显示模式	内容	目的
Normal	前/后摄像头图像及顶视图	停车辅助
Wide	经过畸变校正的后置摄像头图像	驾驶辅助
Side	左/右摄像头图像及裁剪视图	检查轮辋以进行停车或检查车舱
Top	放大前/后顶视图	停车和拖车停车援助
3D	虚拟车辆的 3D 摄像机视图	停车辅助
Free 3D	不断旋转的 3D 相机	仅演示

要自定义每个显示模式中的视图布局，请参阅布局自定义页面。

除了模式切换按钮之外，还有其他按钮，包括：

- 退出应用程序：点击此按钮可离开 AVMDemo 应用程序。

- 3D 显示模式下的相机图标：点击这些相机图标可切换到每个带有动画的相机位置。相机位置和动画可以自定义，如 3D 相机页面中所述。

设计概述

此演示应用程序与普通的 Android 应用程序不同。AVM 显示不是演示应用程序的一部分。

显示布局

此演示应用不控制 AVM 显示的布局。事实上，它不渲染 AVM 显示。AVM 显示由 AVM 守护进程渲染，如系统概述中所述。演示应用仅定义工具栏，然后将其与 Android 系统 UI 一起合成在 AVM 屏幕的顶部。下图对此进行了说明：



要更改 UI 布局，您必须更改两个布局配置：

1. 工具栏的布局，由应用程序的布局 XML 控制
2. AVM 显示的布局，由布局配置文件 default_config.json 控制。

因此，AVMDemo APK 只是向 AVM 守护进程发送消息来控制 AVM 显示的可见性。

消息协议以 JNI 辅助库的形式实现。Java 应用程序代码是一个薄包装器，可将 UI 输入重定向到辅助库。

有关应用程序和 AVM 守护进程之间的整体组合机制，请参阅显示组合部分。

连接到 AVM 守护程序

此应用程序通过 Linux Abstract Namespace Socket 连接到 AVM Daemon。守护进程创建两个域套接字：

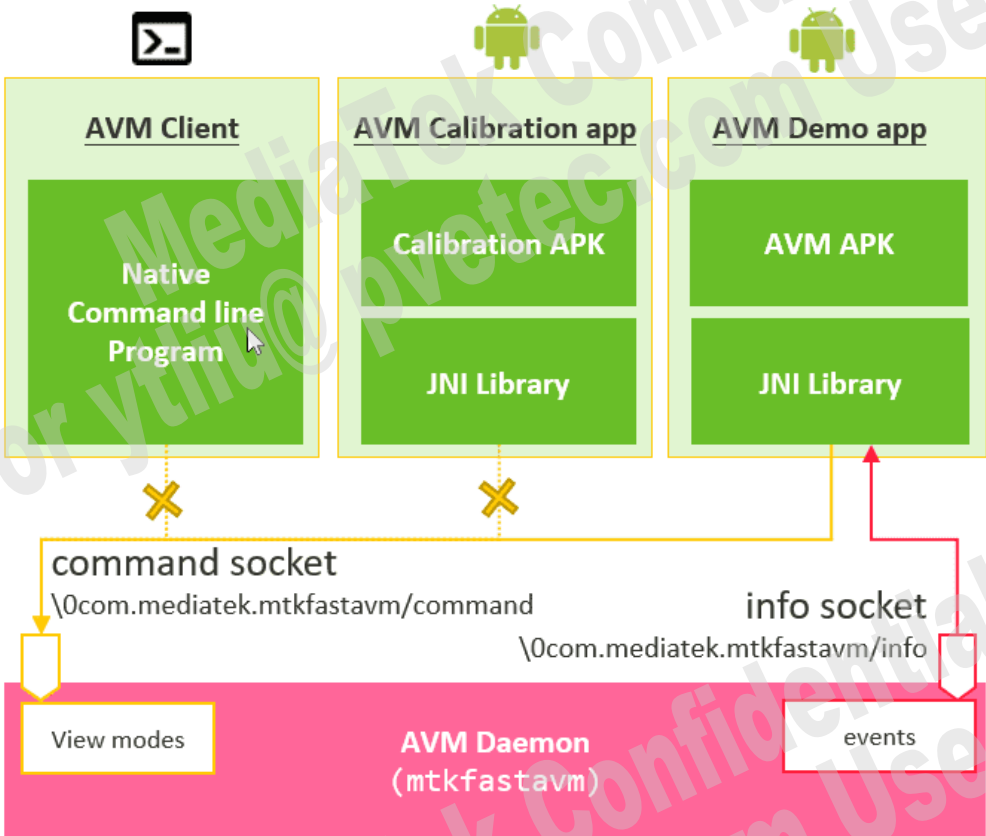
- 允许客户端（例如此演示应用）设置可见性和切换视图模式的命令套接字。套接字路径为 `\0com.mediatek.mtkfastavm/command`。
- 信息套接字，允许客户端从守护进程注册重要事件。例如，当守护进程准备好显示 AVM 帧时，守护进程会通过套接字发送 `AVM_NOTIFY_READY_SHOW` 事件。套接字路径为 `\0com.mediatek.mtkfastavm/info`。

JNI 本机库用于域套接字连接。这些套接字不可共享。因此，客户端必须相互合作以确保 AVM 守护程序可访问。策略是：

- 信息套接字始终被 AVM 演示应用程序占用，因为它必须监视反向开关的状态。
- 命令套接字在所有客户端之间共享。当 AVM 演示应用程序不在 Android 活动堆栈顶部时，它必须断开与命令套接字的连接。只有当命令套接字再次可见时，应用程序才会重新连接到命令套接字。

这导致了一个限制：当 AVM Demo 应用程序在前台可见时，`mtkavmclient` 命令行实用程序不起作用。

下图展示了 AVM 守护进程与其客户端通过套接字连接之间的交互：



源代码

演示应用程序的源树如下表所示:

avmdemoapk
├── AndroidManifest.xml
├── Android.mk
├── jni
├── res
└── src/com/mediatek/avm

其中:

- Android.mk是构建系统配置文件。此 apk 是在 Android 源代码树中构建的，而不是在 Gradle 构建系统中构建。
- AndroidManifest.xml 是标准的 Android 应用程序清单文件。
- jni 是与 AVM 守护进程通信的 JNI 帮助库的源代码。
- res 是标准的Android应用程序资源目录。
- src/com/mediatek/avm 存储所有 java 源文件。

全局事件接收器

应用程序的 AndroidManifest.xml 将全局事件接收器 .AvmGlobalsReceiver 注册到 android.intent.action.BOOT_COMPLETED 事件。这会导致在 Android 框架准备就绪时执行应用程序 AvmApp。然后，AvmApp 类委托给 AvmGlobals 类，该类监视来自 JNI 本机库的事件。

一旦 EVENT_DRIVE_REAR_ENABLE_FROM_AVM_SERVER 事件到达，AvmGlobals 类就会通过启动 MainActivity 类来启动工具栏活动。

```
@Override
public void handleMessage(Message msg) {
    Log.d(TAG, "handleMessage: " + msg.what);
    switch (msg.what) {
        case Constants.EVENT_DRIVE_REAR_ENABLE_FROM_AVM_SERVER:
            if (mContext != null) {
                Log.d(TAG, "handleMessage: start MainActivity");
                Intent intent = new Intent(mContext, MainActivity.class);
                intent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK | Intent.FLAG_ACTIVITY_NO_ANIMATION);
                intent.putExtra(Constants.LAUNCH_REASON,
                    Constants.commandsToString(
                        Constants.EVENT_DRIVE_REAR_ENABLE_FROM_AVM_SERVER).toString());
                mContext.startActivity(intent);
            }
            break;
        default:
            Log.d(TAG, "handleMessage: no handle message");
    }
}
```

工具栏事件处理程序

应用程序工具栏的主要逻辑可以在 src/com/mediatek/avm/MainActivity.java 中找到。它定义了主活动类 com.mediatek.avm.MainActivity。活动类只是使用方法 sendAVMCommandById 将 UI 点击事件重定向到底层 JNI 辅助库：

```
public void onClick(View v) {
    // ...omitted...
    // activate the clicked button
    final int id = v.getId();
    activateChild(mButtonBar, id);

    // ...omitted...

    // Send mode switch command to AVM daemon
    // after UI update.
```

```

mHandler.postDelayed(new Runnable(){
    @Override
    public void run() {
        sendAVMCommandById(id);
    }, 1);
}

```

JNI helper library在jni子目录中实现：

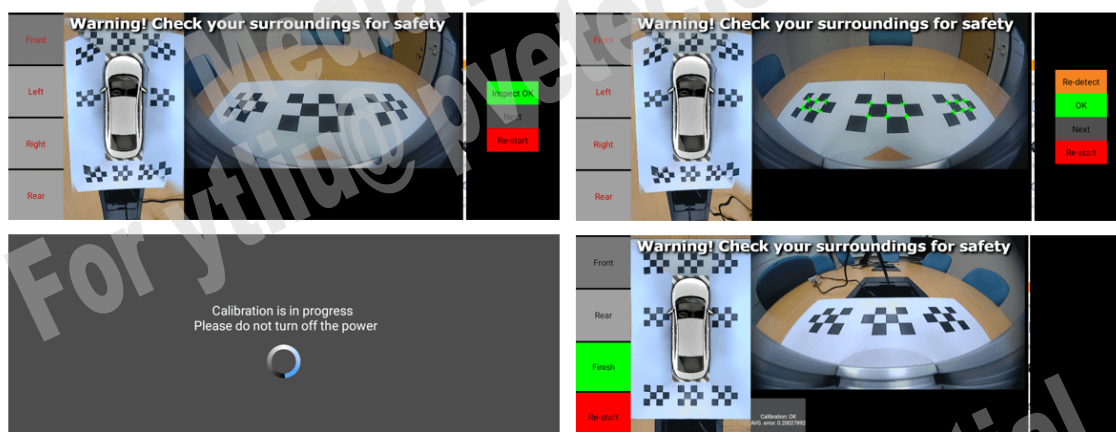
- avm_jni.cpp 管理套接字连接和 Java-C++ 互操作。
- avm_client.cpp 构造消息。消息格式由 AVN 守护进程定义。

常见问题

可以将 AVN 显示视为 Android 应用程序中的 SurfaceView 吗？

否。当前设计在 Android SurfaceFlinger 准备就绪之前分配 AVN 显示缓冲区。目前，无法将缓冲区作为有效表面传回 SurfaceFlinger。

8.2 AVNCalarion 系统

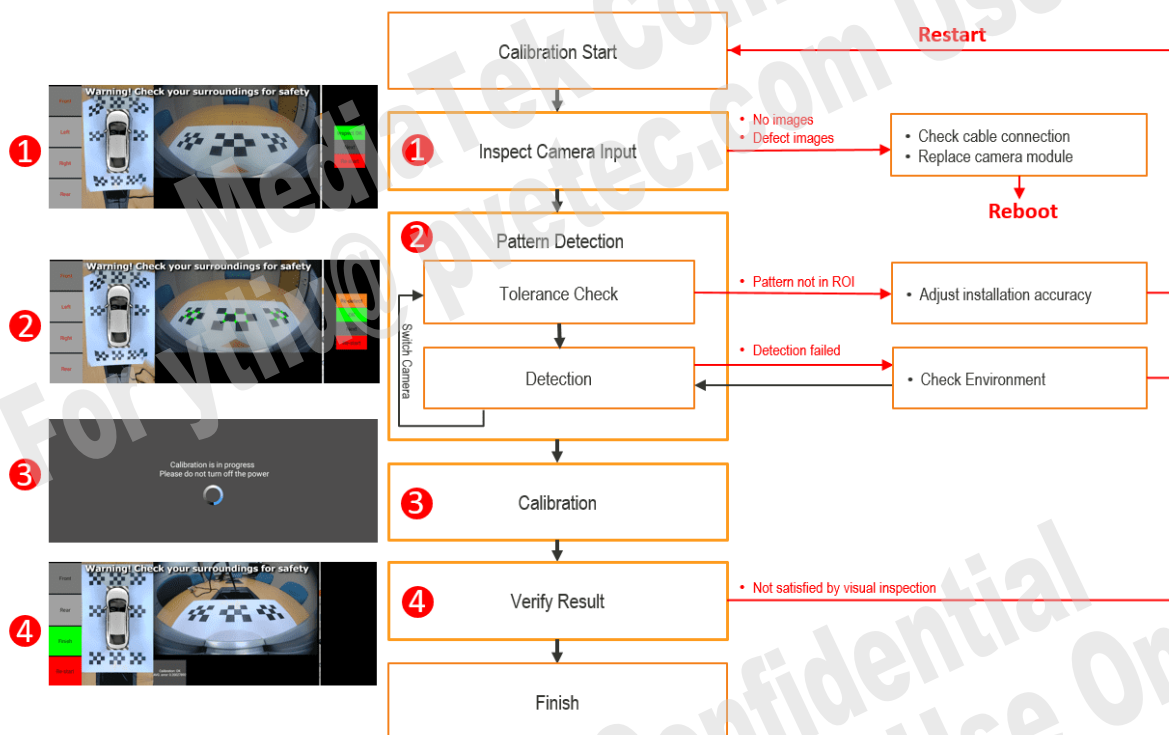


AVNCalibration 是一款 Android 应用，它演示了可用于工厂生产线标定 AVN 参数的在线标定过程。

请参阅在线标定页面以了解如何使用此演示应用程序。

设计概述

AVNCalibration 应用程序的目的是展示在线标定工作流程的示例。下图是对在线标定过程的回顾：

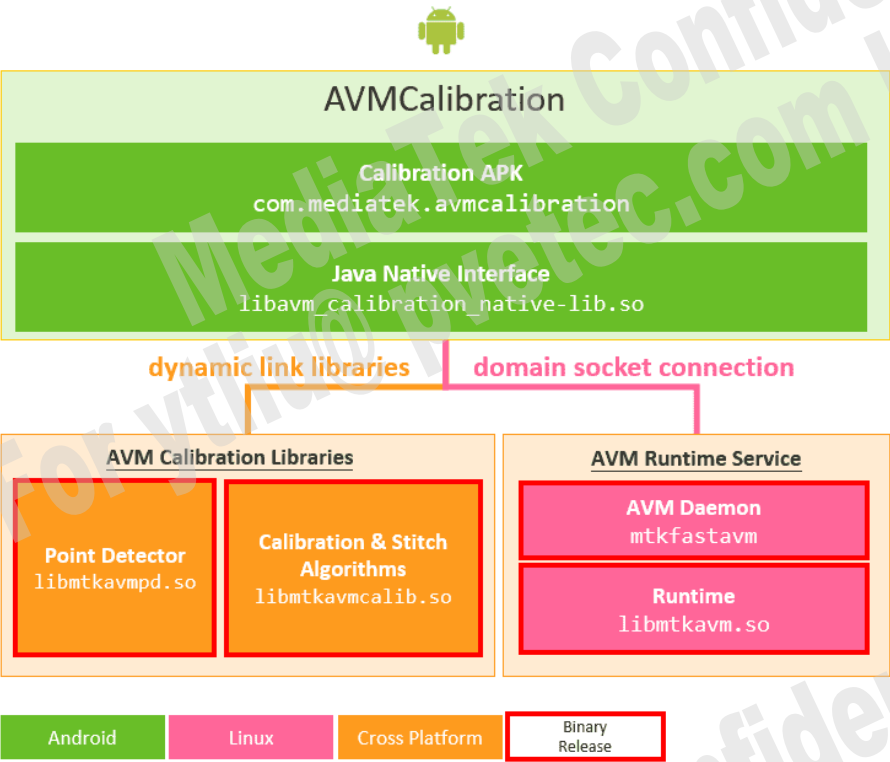


标定过程是 SDK 标定功能的实际用例。我们在 SDK 中提供了 2 个本机库来支持内联标定工作流程。此外，AVM 视图的显示是 AVM 守护程序的职责，与 AVMDemo 应用程序相同。

因此，AVMCalibration 应用程序是这些模块的薄包装：

- 点检测库 (/system/lib64/libmtkavmpd.so) 提供从实时相机图像中检测点几何图形的功能。这是上图中的第 2 步。当前实现检测 4x3 黑白棋盘图案。点检测库的输出是一组不同点之间的映射关系。该组点称为点云。您可以将其替换为您的实现，以检测不同的标定图案，例如圆形图案，并将点云输出到 AVM 标定库（第 3 步）。
- AVM 标定库 (/system/lib64/libmtkavmcalib.so) 根据前面步骤检测到的点云计算标定参数。然后将标定参数存储为不透明缓冲区。这是上图中的第 3 步。此库以二进制格式发布。
- AVM 守护进程 (/system/bin/mtkfastavm) 根据 AVMCalibration 应用程序发送的命令显示不同的摄像头视图。它还提供命令接口，允许预览新标定参数的输出。

可以用下面的框图来总结：



以下文件树显示了 SDK 中的相关文件：

```
├── avmcalibapk
│   ├── AndroidManifest.xml
│   ├── Android.mk
│   ├── jni
│   │   ├── Android.mk
│   │   ├── avm_calibration_client.cpp
│   │   ├── avm_calibration_client.h
│   │   ├── avm_calibration_jni.cpp
│   │   └── avm_calibration_jni.h
│   ├── res
│   └── src
│       └── com
├── avmalgo
│   ├── Android.mk
│   ├── arm
│   │   ├── libmtkavmcalib.so
│   │   ├── libmtkavmpd.so
│   │   └── libmtkavm.so
│   ├── arm64
│   │   ├── libmtkavmcalib.so
│   │   ├── libmtkavmpd.so
│   │   └── libmtkavm.so
```

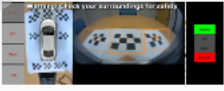
```
|   └── inc  
|       ├── avm_calib.h  
|       ├── avm_param.h  
|       ├── avm_point_detector.h  
|       ├── avm_runtime.h  
|       └── avm_types.h
```

其中：

- avmcalibapk 存储 AVMCalibration 应用程序的 Android 应用程序源。
- avmcalibapk/jni 存储 JNI 库的源代码。此库依赖于 avmalgo 目录中的模块
- avmalgo 目录存储 AVM 标定库和 AVM 运行库的头文件（inc）和预构建二进制文件（arm 和 arm64）。

申请活动

AVMCalibration 演示应用程序包含几个活动:

Activity Name	Display	Description
LaunchActivity		Starting point of the inline calibration process. All the "reset workflow" actions are directed to this activity.
InspectActivity		This activity controls AVM daemon to display camera source images from all 4 cameras.
CaptureActivity		Allows user to capture images from all 4 cameras and detect feature points from the captured images.
CalibrationActivity		This activity indicates that the calibration algorithm is running.
VerifyActivity		This activity allows you to preview and save the result of the calibration, or discard it.

我们在以下章节中简要介绍每个活动:

启动并检查

LaunchActivity 是一个简单的activity, 有 1 个按钮, 用于启动检查活动。此活动是在线标定过程的起点。单击其他活动中的“重新启动”按钮将带您返回此起点。

InspectActivity 通过向 AVM 守护程序发送命令来显示来自不同摄像头的预览图像。这允许您确认摄像头是否正确连接到系统。例如, 如果您单击左按钮, 活动将发送以下命令:

```
mJniHelper.sendAvmCalibrationCommand(Constants.EVENT_CAPTURE_VIDEO_FROM_LEFT_CAMERA);  
mJniHelper.sendAvmCalibrationCommand(Constants.EVENT_VIDEO_DISPLAY_PREVIEW_TO_AVM_SERVER);
```

然后, AVM 守护程序会在屏幕上显示左侧摄像头的预览图像。请注意, 摄像头预览显示的布局是 AVM 守护程序的配置。AVMCalibration 应用程序无法确定其布局。

捕获和点检测

CaptureActivity 允许用户捕获相机图像, 然后检测图像上的特征点, 以便进行后续的标定过程。通过单击“捕获”按钮, 此活动委托给一个简单的 SplashActivity, 该 SplashActivity 在后台 AsyncTask 中执行相机捕获和点检测:

```
// ...omitted...
@Override
protected Boolean doInBackground(Void... voids) {
    mJniHelper.sendAvmCalibrationCommand(Constants.EVENT_CAPTURE_IMAGE_FROM_CURRENT_CAMERA);

    switch (cam_id) {
        case 0:
            mJniHelper.sendAvmCalibrationCommand(Constants.EVENT_FIND_FRONT_IMAGE_POINTS);
            break;
        case 1:
            mJniHelper.sendAvmCalibrationCommand(Constants.EVENT_FIND_LEFT_IMAGE_POINTS);
            break;
        case 2:
            mJniHelper.sendAvmCalibrationCommand(Constants.EVENT_FIND_RIGHT_IMAGE_POINTS);
            break;
        case 3:
            mJniHelper.sendAvmCalibrationCommand(Constants.EVENT_FIND_REAR_IMAGE_POINTS);
            break;
    }
    mGraphicsView.updateParamsFromConfigWithCamId(cam_id);
    mFindRP = mGraphicsView.getMFindresult();
    Log.i(TAG, "mFindRP = " + mFindRP);
    return true;
}
```

JNI 辅助库将摄像头捕获请求绕过 AVm 守护进程。这是由 avm_calibration_jni.cpp 中的 sendAvmCalibration Command() 实现的:

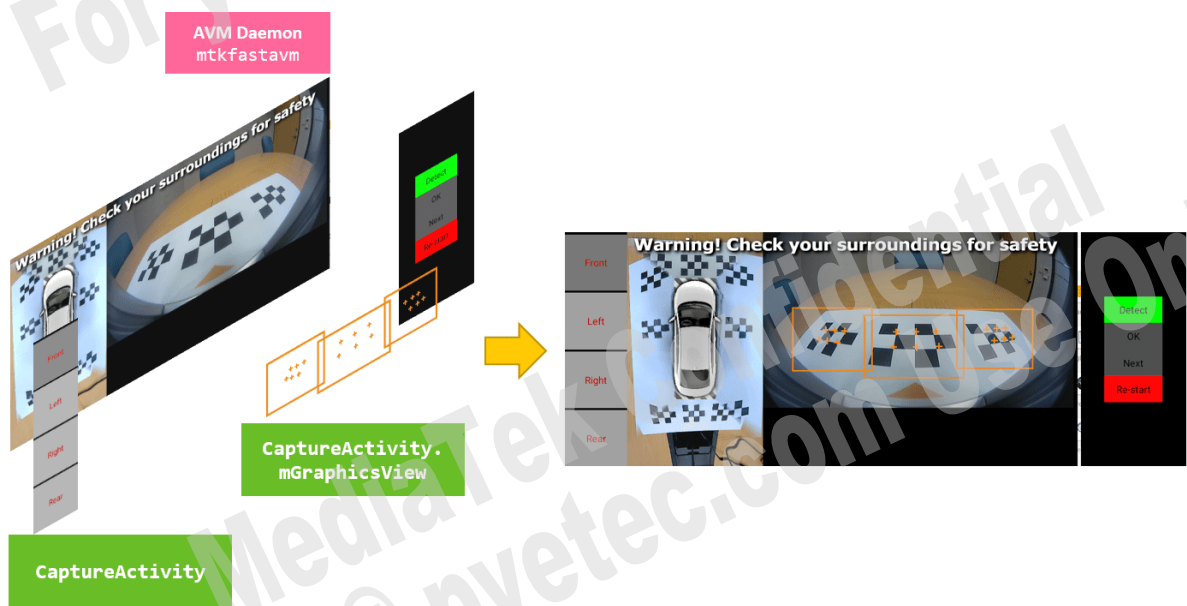
```
case EVENT_CAPTURE_IMAGE_FROM_CURRENT_CAMERA:
    socket_buf = (char *)malloc(1920*1080*4);
    sendMsg(AVM_MSG_DUMP_VIEW, 1, socket_buf);
    save_data_to_png(socket_buf);
    free(socket_buf);
    socket_buf = NULL;
    break;
```

AVm 守护进程不会写入磁盘。相反, 捕获的图像会传回 Android 应用程序的 JNI 帮助程序, 以 PNG 格式压缩并保存到磁盘。出于安全原因, 所有文件创建和修改均由 Android 应用程序完成, 而不是 AVm 守护进程。Android 应用程序具有写入 AVm 配置数据路径 (/mnt/mtkdata/avmdata) 的系统权限。

相机图像被捕获后，会将其传递给本机函数 findPoints()。 findPoints() 函数通过调用点检测库的 API avm_pd_detect_point 来执行特征点检测。 检测到的特征点和相关的感兴趣区域矩形随后会显示在 GraphicsView 视图上。

GraphicsView 是一个自定义视图，位于 AVM 守护进程的输出区域之上。其 onDraw() 方法绘制检测到的特征点，并用红色标记无效的感兴趣区域以通知用户。

下图说明了 AVM 守护进程和 CaptureActivity 之间的 z 顺序：



标定

一旦所有特征点准备就绪，它们将被发送到 CalibrationActivity 以通过以下对 JNI 帮助库的调用来生成新的标定参数：

```
@Override
protected Boolean doInBackground(Void... voids) {
    mJniHelper.sendAvmCalibrationCommand(Constants.EVENT_SAVA_PARAM_PD_BIN_TO_NATIVE_LIB);
    mJniHelper.sendAvmCalibrationCommand(Constants.EVENT_CALIBRATE_TO_NATIVE_LIB);
    return true;
}
```

JNI 辅助库将标定计算委托给 AVM 标定库。这是由 avm_calibration_jni.cpp 中的 calibrate() 函数实现的。有关 AVM 标定库 API 的使用，请参阅标定模块页面。

然后，AVMCalibration 应用程序将不透明参数作为临时文件写入 AVM 数据目录 (/mnt/mtkdata/avmdata)，并继续执行下一个活动。

结果验证

VerifyActivity 将临时标定参数发送到 AVM 守护进程以进行结果预览。

单击“重新启动”按钮后，临时文件将被丢弃，应用程序将返回到启动活动 LaunchActivity。

如果单击“完成”按钮，此活动会将临时参数文件重命名为 AVM 守护程序分配的有效参数文件名。如果存在，AVM 守护程序会加载新的参数文件。

故障排除

深入解析Android P的TapPoint检测

我们在开发过程中发现一个 Google 问题，它存在于多个 Activity 的透明层上。如果我们在 AVM 标定 APP 中双击按钮，它会跳转到汽车启动器或后台菜单（汽车启动器）中的某个应用程序。

/frameworks/base/services/core/java/com/android/server/wm/TaskPositioningController.java

Android P:

```
void handleTapOutsideTask(DisplayContent displayContent, int x, int y) {
    mHandler.post(() -> {
        int taskId = -1;
        synchronized (mService.mWindowMap) {
            final Task task = displayContent.findTaskForResizePoint(x, y);
            if (task != null) {
                if (!startPositioningLocked(task.getTopVisibleAppMainWindow(), true /*resize*/,
                    task.preserveOrientationOnResize(), x, y)) {
                    return;
                }
                taskId = task.mTaskId;
            } else {
                taskId = displayContent.taskIdFromPoint(x, y);
            }
        }
        if (taskId >= 0) {
            try {
                mActivityManager.setFocusedTask(taskId);
            } catch (RemoteException e) {
            }
        }
    });
}
```

```

    }
    });
}

```

该问题已在 Android Q 中修复:

```

void handleTapOutsideTask(DisplayContent displayContent, int x, int y) {
    mHandler.post(() -> {
        synchronized (mService.mGlobalLock) {
            final Task task = displayContent.findTaskForResizePoint(x, y);
            if (task != null) {
                if (!startPositioningLocked(task.getTopVisibleAppMainWindow(), true /*resize*/,
                    task.preserveOrientationOnResize(), x, y)) {
                    return;
                }
                try {
                    mActivityManager.setFocusedTask(task.mTaskId);
                } catch (RemoteException e) {
                }
            }
        }
    });
}

```

我们尝试基于 Android Q 进行修复: (我们的 Android P 补丁)

```

void handleTapOutsideTask(DisplayContent displayContent, int x, int y) {
    mHandler.post(() -> {
        //int taskId = -1;
        synchronized (mService.mWindowMap) {
            final Task task = displayContent.findTaskForResizePoint(x, y);
            if (task != null) {
                if (!startPositioningLocked(task.getTopVisibleAppMainWindow(), true /*resize*/,
                    task.preserveOrientationOnResize(), x, y)) {
                    return;
                }

                try {
                    mActivityManager.setFocusedTask(task.mTaskId);
                } catch (RemoteException e) {
                }
                //taskId = task.mTaskId;
            } /* else {
                taskId = displayContent.taskIdFromPoint(x, y);
            } */
        }
        /*if (taskId >= 0) {

```

```
try {  
    mActivityManager.setFocusedTask(taskId);  
} catch (RemoteException e) {  
}  
}  
}*/  
});  
}
```

请注意透明层问题，您可以根据需要尝试此修补程序。

8.3 AVM 客户端实用程序

AVM 客户端实用程序

mtkavmclient 命令行实用程序可用于测试来自 AVM Daemon 的所有控制和响应消息。程序源代码位于 <Base>/vendor/mediatek/mt2712/mtkavm/avmdaemon/avmclientdemo，它会生成一个可执行文件 mtkavmclient。

您可以使用此程序来测试控制消息以进行调试。例如，您可以：

- 启动/停止 AVM 守护进程的显示
- 在 AVM 显示模式之间切换
- 在 3D AVM 模式下更改 3D 相机位置
- 模拟反向开关信号
- 显示原始相机输入图像
- 将相机输入图像转储到 PNG 文件
- 将 AVM 显示转储到 PNG 文件
- 改变 3D 车辆的颜色和透明度 (v2.2 中的新功能)
- 改变 3D 车辆的门和灯的状态 (v2.2 中的新功能)

请注意，客户端应用程序与 AVM Daemon 之间的连接是互斥的。客户端应用程序包括 mtkavmclient、AVM Demo 和 AVM Calibration。

因此，在使用此命令行工具之前，您必须关闭 AVMDemo 或 AVMClibration。或者，您可以使用以下 shell 命令强制停止这些应用程序：

```
am force-stop com.mediatek.avm
am force-stop com.mediatek.avmcalibration
```

用法

您可以使用以下命令来测试所有控制消息。[输出路径] 是仅适用于某些消息类型的可选参数：

```
mtkavmclient <message type> <message info> [output path]
```

例如，您可以使用以下命令启动/停止 AVM 守护程序：

```
mtkavmclient 0 1 //start avm daemon
```

```
mtkavmclient 0 0 //stop avm daemon
```

您可以使用以下命令转储每四个摄像机的图像：

```
mtkavmclient 1 5 //switch to camera preview mode
mtkavmclient 3 0 //switch preview camera to front camera
mtkavmclient 4 1 /data/front_camera //dump front camera view to `/data/front_camera.png`
mtkavmclient 3 1 //switch preview camera to left camera
mtkavmclient 4 1 /data/left_camera //dump left camera view to `/data/left_camera.png`
mtkavmclient 3 2 //switch preview camera to right camera
mtkavmclient 4 1 /data/right_camera //dump right camera view to `/data/right_camera.png`
mtkavmclient 3 3 //switch preview camera to rear camera
mtkavmclient 4 1 /data/rear_camera //dump front camera view to `/data/rear_camera.png`
```

或者你可以使用以下命令一次转储所有四个摄像头：

```
mtkavmclient 1 5 //switch to camera preview mode
mtkavmclient 4 0 /data/cam //dump all 4 cameras to /data/cam0.png ~ cam4.png
```

请注意，.png 后缀是自动添加的。

有关控制和响应消息的规范，请参阅AVM守护进程模块。

下表列出了您可以使用的所有消息类型和相应的参数。

展示

类型	争论	输出路径	描述
0	0	不适用	停止 AVM Daemon 显示
0	1	不适用	启动AVM Daemon显示

例如，要启动 AVM 显示，请使用以下命令：

```
mtkavmclient 0 1
```

显示模式

显示模式由 default_config.json 定义，如布局页面中所述。每种显示模式承载不同的视图类型。使用 mtkavmclient 1 [mode_index] 在不同显示模式之间切换。请注意，值为 0 的 mode_index 是特殊的：它是 AVM 守护进程的默认显示模式。

参考设计中的显示模式列表

下表描述了参考设计中不同模式的含义。有些模式是为演示应用程序设计的，而其他模式是为在线标定应用程序或用于调试目的设计的。

类型	模式索引	输出路径	由应用程序使用	描述
1	0	不适用	AVM 演示	切换到模式0，这是参考设计中的正常模式。
1	1	不适用	AVM 演示	切换到模式1，参考顶部显示模式。
1	2	不适用	AVM 演示	切换到模式2，参考侧显示模式。
1	3	不适用	AVM 演示	切换到模式3，参考宽显示模式。
1	4	不适用	AVM 演示	切换到模式 4，即参考第二显示模式，外部显示屏上显示两个侧视图。
1	5	不适用	-	切换到模式 5。此模式显示原始相机输入以用于调试目的。
1	6	不适用	AVM 标定	切换到模式 6。AVM 标定应用程序使用此视图预览单个摄像头图像。
1	7	不适用	AVM 演示	切换到模式7，固定摄像机位置的3D显示模式。
1	8	不适用	AVM 演示	切换到模式 8，具有动态摄像头位置和触摸输入支持的自由 3D 显示模式。
1	9	不适用	AVM 演示	切换到模式 9，比较光度标定的开启/关闭效果。AVM Demo 将此用作附加模式之一。
1	10	不适用	AVM 演示	切换到模式 10，这是布局动画的演示，是 AVM Demo 中的额外模式之一。
1	11	不适用	AVM 标定	切换到模式 11，由 AVM 标定应用程序在验证页面中使用，以显示顶视图和放大的正面顶视图。
1	12	不适用	AVM 标定	切换到模式 12，由 AVM 标定应用程序在验证页面中使用，以显示顶视图和放大的后侧顶视图。
1	十三	不适用	-	切换到模式 13，以 TOPVIEW_3D 视图模式实现的顶视图，具有较小的相机 FOV。
1	14	不适用	-	切换到模式 14，以 TOPVIEW_3D 视图模式实现的顶视图，具有更大的相机 FOV。
1	15	不适用	-	切换到模式15，这种调试显示模式和模式7类似，响应3D相机位置，但是相机动画路径的计算方式不同。
1	16	不适用	AVM 演示	切换到模式16，显示4个摄像头输入图像。
1	17	不适用	AVM 演示	切换到模式 17，以 SIDE_3DVIEW 视图模式实现的侧视图，具有内置摄像头视角。
1	18	不适用	AVM 演示	切换到模式 18，以 SIDE_3DVIEW 视图模式实现的侧视图，具有内置摄像机视角和用户定义的摄像机视角。
1	19	不适用	AVM 演示	切换到模式 19，以 USER_CAM_3DVIEW 视图模式实现的 3D 视图，具有多个用户定义的摄像机视角。

切换显示视图模式会立即隐式启动 AVM 显示。

例如，要检查参考设计中的“顶视图”显示模式，请使用以下命令：

```
mtkavmclient 1 1
```

要添加、删除或修改每种显示模式的布局，请参阅显示布局自定义页面。

更改 3D 相机位置

您可以使用客户端实用程序在不同的预定义摄像机位置之间切换。

类型	争论	输出路径	描述
7	3D camera index	不适用	在相机索引之间切换。有效的相机索引范围由 3D 资产文件定义。

请注意，您必须处于模式 7（带固定摄像头的 3D 显示）才能获得可见的结果。

模拟反向开关

Type	Argument	Output Path	Description
2	0	N/A	gear "R" signal quit
2	1	N/A	gear "R" signal enter

这模拟了 EVB 上的倒车开关。主要用于调试。这可在前档和后档模式之间切换 AVM 显示屏。

模拟车门状态

您可以使用以下命令模拟 3D 车辆的门打开/关闭：

Type	Door Index	Door Status	Description
10	0 (left driver door)	0 / 1	0 for closed status, 1 for opened status.
10	1 (right driver door)	0 / 1	0 for closed status, 1 for opened status.
10	2 (left passenger door)	0 / 1	0 for closed status, 1 for opened status.
10	3 (right passenger door)	0 / 1	0 for closed status, 1 for opened status.
10	4 (engine hood)	0 / 1	0 for closed status, 1 for opened status.
10	5 (trunk or hatch door)	0 / 1	0 for closed status, 1 for opened status.

以下命令将切换到“Free 3D view”模式，并打开车辆的右侧车门。默认的 3D 车辆模型支持车门动画，因此发送这些命令后您应该能够看到车门打开。此外，车门打开后，AVM 摄像头图像会变成灰色块。

```
mtkavmclient 1 8
mtkavmclient 10 0 1
mtkavmclient 10 1 1
```

模拟车辆灯光状态

您可以使用以下命令模拟 3D 车辆的灯光开/关状态：

Type	Door Index	Door Status	Description
11	0 (rear breaking light)	0 / 1	0 for off status, 1 for on status.
11	1 (head lamp)	0 / 1	0 for off status, 1 for on status.
11	2 (hazard warning signal)	0 / 1	0 for off status, 1 for on status.
11	5 (break light)	0 / 1	0 for off status, 1 for on status.
11	6 (left turn signal)	0 / 1	0 for off status, 1 for on status.
11	7 (right turn signal)	0 / 1	0 for off status, 1 for on status.

以下命令切换到 “Free 3D view” 模式，并打开前照灯和左转向灯。

```
mtkavmclient 1 8
mtkavmclient 11 1 1
mtkavmclient 11 6 1
```

模拟车辆后视镜状态

由于大多数侧视摄像头都安装在后视镜上，因此当后视镜关闭时，AVM 服务会自动将左/右摄像头图像变为灰色块。请注意，当前的 3D 车辆模型未实现后视镜状态变化的动画。

Type	Door Index	Door Status	Description
16	0 (left rear view mirror)	0 / 1	0 for closed status, 1 for open status.
16	1 (right rear view mirror)	0 / 1	0 for closed status, 1 for open status.

以下命令切换到 “Free 3D view” 模式，并打开前照灯和左转向灯。

```
mtkavmclient 1 8
mtkavmclient 16 0 1
```

改变 3D 车辆颜色

内置的演示 3D 车辆模型支持 2 种不同的颜色。您可以使用以下命令在这些颜色之间切换：

Type	Color Index	Parameter	Description
21	Color index available in the 3D vehicle model	N/A	The built-in demo 3D model supports index 0 ~ 1.

以下命令切换到 “Free 3D view” 模式，并通过将颜色索引设置为 1 将内置 3D 演示模型更改为不同的颜色。

```
mtkavmclient 1 8
mtkavmclient 21 1
```

更改 3D 车辆透明度

内置的演示 3D 车辆模型支持 10 级透明度。

Type	Transparency Index	Parameter	Description
22	0 ~ 9	N/A	0: Fully opaque 3D vehicle 9: Almost transparent 3D vehicle

以下命令切换到 “Free 3D view” 模式，并通过将颜色索引设置为 3 将内置 3D 演示模型变为半透明。请注意，当车辆的透明度索引设置为非 0 时，不透明地板图像将设置为不可见。

```
mtkavmclient 1 8
mtkavmclient 22 3
```

原始相机图像

Type	Argument	Output Path	Description
1	5	N/A	switch to the raw camera view, and start AVM daemon display
3	0	N/A	set raw camera view to front camera
3	1	N/A	set raw camera view to left camera
3	2	N/A	set raw camera view to right camera
3	3	N/A	set raw camera view to rear camera

默认参考实现在 AVM 显示屏上显示原始相机输入。这对于检查相机是否正常工作很有用。请注意，您必须先切换到原始相机视图模式（模式 5）。

以下命令显示后置摄像头的原始输入：

```
mtkavmclient 1 5
mtkavmclient 3 3
```

保存 PNG 文件

Type	Argument	Output Path	Description
4	0	path	dump the all 4 camera images at once. _0.png, _1.png, _2.png, and _3.png are automatically appended to each dumped image.
4	1	path	dump the current active camera image to PNG file. The current active camera is set by mtkavmclient 3 [active_cam_index].
4	4	path	dump entire AVM display to the file path

请注意，.png 会自动附加到输出路径。例如，要检查前置摄像头的原始摄像头输入并将其转储到 PNG 文件，请使用以下命令：

```
mtkavmclient 1 5
mtkavmclient 3 0
mtkavmclient 4 1 /data/avm/front
```

结果是 /data/avm/front.png 文件被保存。

在运行时修改视角

AVM 演示了多种 2D 和 3D 显示模式，其中大多数允许您在运行时微调视角。下表显示了哪些视图类型支持视角修改。

在 default_config.json 中输入名称	如何/在哪里定义默认视角	可以在运行时修改视角吗？
"RAWCAMERA_VIEW"	在 default_config.json 中定义rect	是的。
"FISHEYE_VIEW"	在 default_config.json 中定义rect	是的。
"UNDISTORT_VIEW"	在 default_config.json 中定义rect	是的。
"FREEFORM_VIEW"	在 default_config.json 中定义src_quad	不可以。src_quad是任意四边形，用户不可以修改它。
"TOPVIEW"	在 default_config.json 中定义视锥体	是的。
"FIX_3DVIEW"	AVMViewFix3DView中定义	不可以。 此视图演示了相机动画路径。不允许用户在运行时修改任何相机姿势。
"INTRO_3DVIEW"	AVMViewIntro3DView中定义	不可以。 此视图是一个“开场动画”演示，带有程序控制的一次性相机动画和布局动画，不允许用户在运行时修改任何相机姿势。

"FREE_3DVIEW"	AVMViewFree3DView中定义	不可以。 此视图演示了相机围绕固定中心点旋转一圈。不允许用户在运行时修改任何相机姿势。
"TOPVIEW_3D"	AVMViewTop3DView中定义	否。 此视图演示了模拟 2D AVM 结果的固定相机位置。不允许用户在运行时修改任何相机姿势。
"ORBIT_3DVIEW"	AVMViewOrbit3DView中定义	不可以。 此视图演示了圆形相机动画路径。不允许用户在运行时修改任何相机姿势。
"USER_CAM_3DVIEW"	在 default_config.json 中定义摄像头	是的。
SIDE_3DVIEW	在default_config.json中定义摄像头	是的。

• 平移视角

类型	查看 ID	价值	描述
34	指定要翻译的视图	对于 2D 显示, 值为视图宽度的 %。对于 3D 显示, 值为 cm 。	沿 X 轴平移视图
35	指定要翻译的视图	对于 2D 显示, 值为视图高度的 %。对于 3D 显示, 值为 cm 。	沿 Y 轴平移视图
36	指定要翻译的视图	对于 2D 显示, 不支持。对于 3D 显示, 值为 cm 。	沿 Z 轴平移视图

例子:

- 在 2D 视图中移动视角 (id: 200)。移动距离为沿 X 轴的视图宽度的 50%。

mtkavmclient 34 200 50

- 在 3D 视图中移动相机 (id:335)。沿 Z 轴移动 100 厘米

mtkavmclient 36 335 100

旋转视角

Type	View ID	Value	Description
37	Specify which view to rotate	Only work for 3D display, value is degree	Rotate view around X axis
38	Specify which view to rotate	Only work for 3D display, value is degree	Rotate view around Y axis
39	Specify which view to rotate	Only work for 3D display, value is degree	Rotate view around Z axis

例子:

- 在 3D 视图中旋转相机 (id:335)。绕 X 轴旋转 45 度

mtkavmclient 37 335 45

缩放视角

Type	View ID	Value	Description
40	Specify which view to scale	For 2D/3D display, value is % of view width.	Scale view in horizontal direction
41	Specify which view to scale	For 2D/3D display, value is % of view height.	Scale view in vertical direction
42	Specify which view to scale	For 2D/3D display, value is % of view size.	Uniform scale view

例子:

- 在 2D 视图 (id:200) 中均匀缩放视角, 缩放视图尺寸的 25%。

mtkavmclient 42 200 25

- 在 3D 视图 (id:335) 中均匀缩放相机视角, 缩放视图尺寸的 25%。

mtkavmclient 42 335 25

将视角重置为默认值

类型	查看 ID	价值	描述
43	指定要重置的视图	-	将视角重置为默认值。

mtkavmclient 43 200

转储 AVM 运行时修改

如果您在运行时修改了任何 2D/3D 视角或更改了 AV

- 发出 cmd 以要求 AV

- 应用程序有责任将 JSON 字符串保存为名为 post_config.json 的 JSON 文件。

但是post_config.json应该保存在哪里呢？

该路径是预定义的。根据 AV

- 确保以上步骤完成，应用程序可以停止AVM守护进程并退出。

下次启动 AV

命令列表

注意：“参数” 代表该命令接受的参数，从 P0 开始，然后是 P1、P2，依此类推。

命令名称	价值	参数	描述
AVM_MSG_MODE	1	引用default_config.js on文件中的display_m odes, 以及mode_i d。	mtkavmclient 0 1 启动/显示 avm display_ mode 1
AVM_MSG_GET_ALGO_PATH	2	-	-
AVM_MSG_CAM_IDX	3	-	-
AVM_MSG_DUMP_VIEW	4	-	-
AVM_MSG_RESET_ALGO	5	-	-

命令名称	价值	参数	描述
AVM_MSG_R_SIGNAL_SIMULATION	6	P0: 1(后退), 0(前进)	-
AVM_MSG_3D_CAM_IDX	7	-	-
AVM_MSG_DEBUG_PHOTOMETRIC_FLAG	8	-	-
AVM_MSG_VEHICLE_SPEED	9	P0 公里/小时	命令 "mtkavmclient 9 10" 将车速设置为10公里/小时。
AVM_MSG_VEHICLE_DOOR_STATUS	10	-	-
AVM_MSG_VEHICLE_LIGHT_STATUS	11	-	-
AVM_MSG_VEHICLE_WHEEL_RPS	12	P0: 浮点数	表示车轮每秒旋转的圈数。 请参考 mtk_avm_client_demo.cpp 中的 AVM_MSG_VEHICLE_WHEEL_RPS 发送浮点值。
AVM_MSG_VEHICLE_WHEEL_STEER_ANGLE	十三	P0: int -35(左) ~ 35(右)	使用命令 "mtkavmclient 13 x" 调整前轮方向。
AVM_MSG_VEHICLE_GEAR_STATUS	14	P0: 2 (右) , 3 (右)	-
3DCAM移动CORD	15	-	-
AVM_MSG_VEHICLE_MIRROR_STATUS	16	-	-
AVM_MSG_VEHICLE_RADAR_STATUS	17	-	-
AVM_MSG_DEBUG_REG_RUNTIME_EVENT_FLAG	18	-	-
AVM_MSG_CAM_PQ_BRIGHTNESS_LEVEL	19	-	-
AVM_MSG_CAM_PQ_CONTRAST_LEVEL	20	-	-
3D_CAR_颜色	21	-	-
AVM_MSG_3D_CAR_TRANSPARENCY	22	P0 int 0(0%)~10(100%, 不透明)	命令 "mtkavmclient 22 1" 将车身透明度设置为10%。
AVM_MSG_CAMERA_ENABLE_FLAG	23	P0 int (相机 ID 0 ~ 3) , P1 int (0 ~ 1)	命令 "mtkavmclient 0 0" 表示禁用摄像头 0, 此时来自摄像头 0 的反馈将填充 default_config.json 中配置的摄像头错误颜色。
AVM_MSG_PIP_WIN_MOVE_POS	24	P0: 1(动作向下), -1(动作移动), 0(动作向上); P1 x;P2 y;	测试命令 (不包括括号和其中的字符) : mtkavmclient 1 20mtkavmclient 24 1 xy (操作向下) mtkavmclient 24 -1 xy (操作移动) mtkavmclient 24 0 xy (操作向上) AVM 客户端捕获触摸事件并将其发送到 AVM 服务器。AVM 服务器确定触摸点是否在 PIP (画中画) 窗口范围内。拖动只能在其范围内进行, 并且还设置了边界处理逻辑以确保 PIP 窗口不会移出屏幕。

命令名称	价值	参数	描述
----	25	–	–
----	26	–	–
----	27	–	–
----	28	–	–
----	29	–	–
记录	30	–	–
许可证信息_IMG_DIRTY	31	–	–
AVM_MSG_FLOOR_IMG_IDX	32	–	–
AVM_MSG_RESET_3D_VEHICLE	33	–	–
AVM_MSG_VIEW_ANGLE_TRANSLATE_X	34	–	–
AVM_MSG_VIEW_ANGLE_TRANSLATE_Y	35	–	–
AVM_MSG_VIEW_ANGLE_TRANSLATE_Z	36	–	–
AVM_MSG_VIEW_ANGLE_ROTATE_X	37	–	–
AVM_MSG_VIEW_ANGLE_ROTATE_Y	38	–	–
AVM_MSG_VIEW_ANGLE_ROTATE_Z	39	–	–
AVM_MSG_VIEW_ANGLE_SCALE_X	40	–	–
AVM_MSG_VIEW_ANGLE_SCALE_Y	41	–	–
AVM_MSG_VIEW_ANGLE_SCALE_UNIFORM	42	–	–
AVM_MSG_VIEW_RESET	43	–	–
AVM_MSG_DUMP_USER_MODIFY_PARAM_2_JS ON_STR	44	–	–
AVM_MSG_SET_ENV_HIGHT_OFFSET	45	–	–
AVM_MSG_3D_OBJECT_TRANSLATE_X	46	–	–
AVM_MSG_3D_OBJECT_TRANSLATE_Y	47	–	–
AVM_MSG_3D_OBJECT_TRANSLATE_Z	48	–	–
AVM_MSG_3D_OBJECT_SCALE_UNIFORM	49	–	–
AVM_MSG_3D_OBJECT_RESET	50	–	–
AVM_MSG_DYNAMIC_GUIDELINE_STYLE_IDX	51	–	–
AVM_MSG_STATIC_GUIDELINE_STYLE_IDX	52	–	–
AVM_MSG_3D_CAR_REFLECTION	53	–	–
AVM_MSG_WHEEL_MARKER_STYLE_IDX	54	–	–
AVM_MSG_GET_VERSION_STR	55	–	–
AVM_MSG_GUIDELINE_STYLE_IDX	56	–	–
AVM_MSG_SET_LOG_LEVEL	57	–	–
AVM_MSG_MUTE	58	P0: int 1 (整个 avm	此命令用于使 AVM (全景监控) 屏幕变黑。

命令名称	价值	参数	描述
		窗口将填充黑色) int 0 (avm 将显示以 前显示的内容)	AVM 绘制的所有视觉效果都将填充黑色。应用程序可以根据需要调用此命令。
AVM_MSG_SET_CAMERA_MUTE_COLOR	59	P0 int(相机id, 0 ~ 3), P1(Red) , P2(Green), P2(Blue) , P3(Alpha), P1到P3为浮点型, 取值范围为0.0~1.0。	命令 mtkavmclient 59 0 1.0 0.0 0.0 1.0 将用红色填充相机 0。请注意：3D 模式和 2D 模式下 alpha 的处理方式有所不同。如果要显示相同的颜色，请将 alpha 设置为 1.0。此处设置的颜色将覆盖 default_config.json 中定义的当前 camera_error_color。重新启动后，它将恢复为 camera_error_color 指定的颜色。如果您希望使用 default_config.json 中的 camera_error_color 来填充相机视图，请使用 AVM_MSG_CAMERA_ENABLE_FLAG。
AVMSG_DUMP_CAMERA_IMAGES	60	P0:int 1(开始转储)int 0(停止转储)	-

9 模块

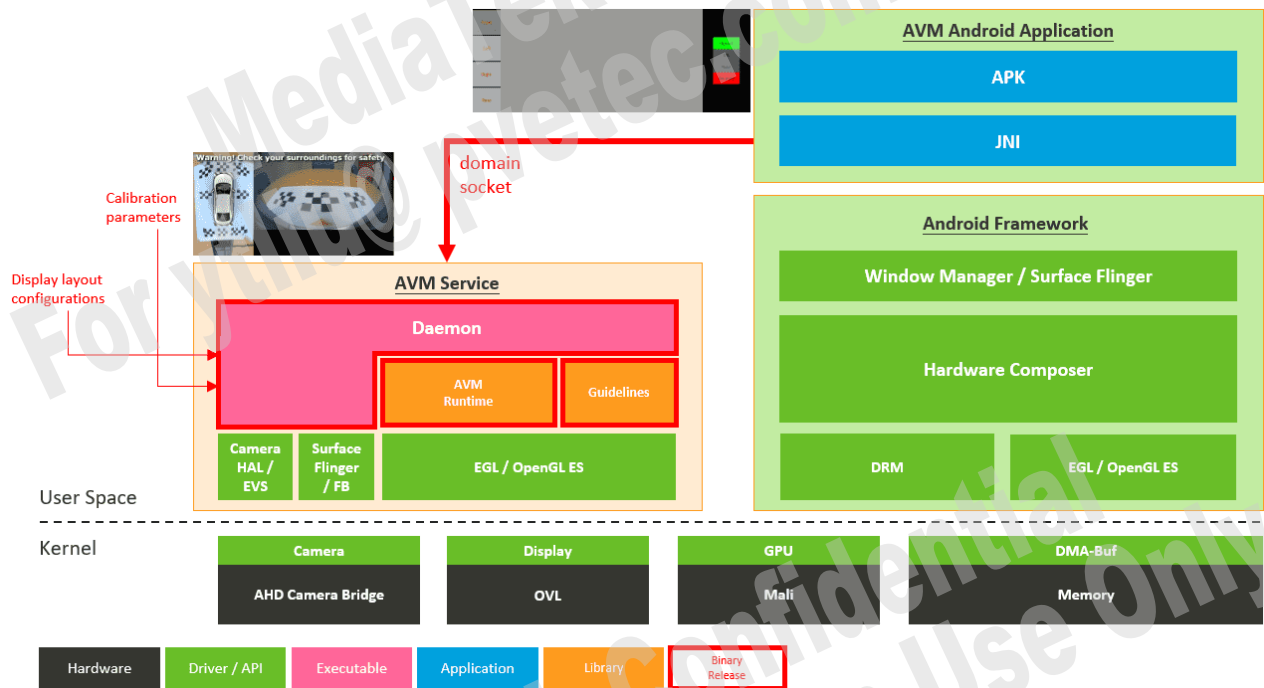
本节介绍 AVM SDK 的重要组件。这些是涵盖的主题：

- AVM 守护进程
- AVM 客户端实用程序
- AVM 点检测库
- AVM 标定库

9.1 AVM 守护进程

AVM 守护进程

AVM 守护进程（AVM 服务）是一个独立于 Android 系统运行的本机程序，并通过套接字与 AVM 应用程序通信，如系统概述中所述。以下框图说明了它与 Android 应用程序和底层硬件驱动程序的交互。



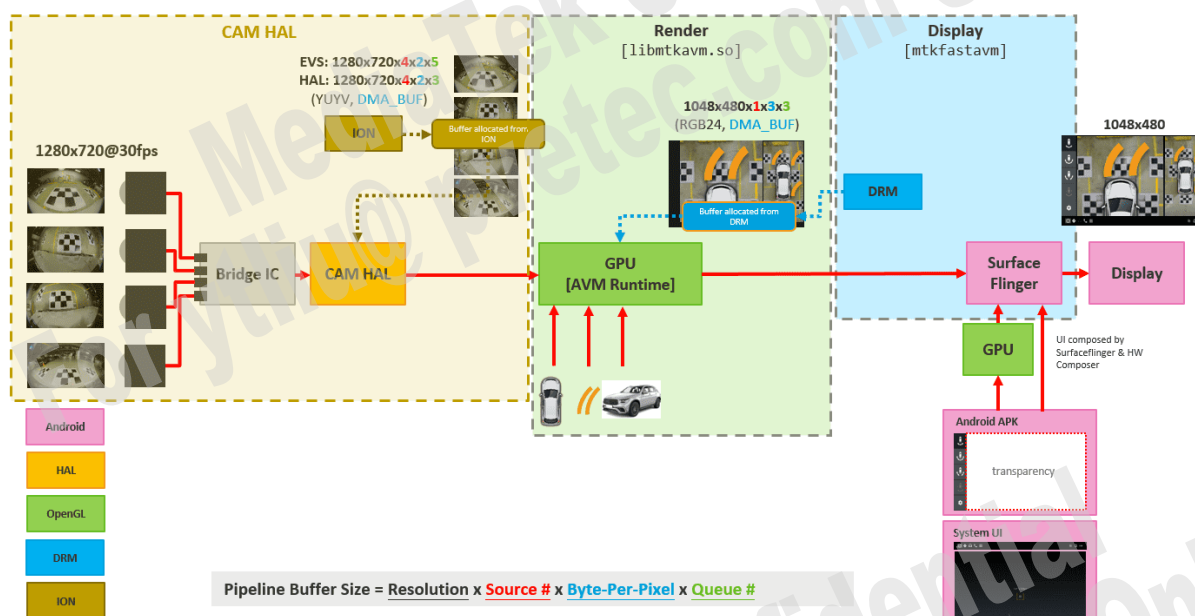
AVM Daemon 中的组件

AVM 守护进程的职责是：

- 管理显示管道
- 通过域套接字接口处理来自应用程序的请求
- 通过域套接字接口监控车辆状态变化

显示管道可以分为 4 个主要组件：相机、渲染、引导和显示。

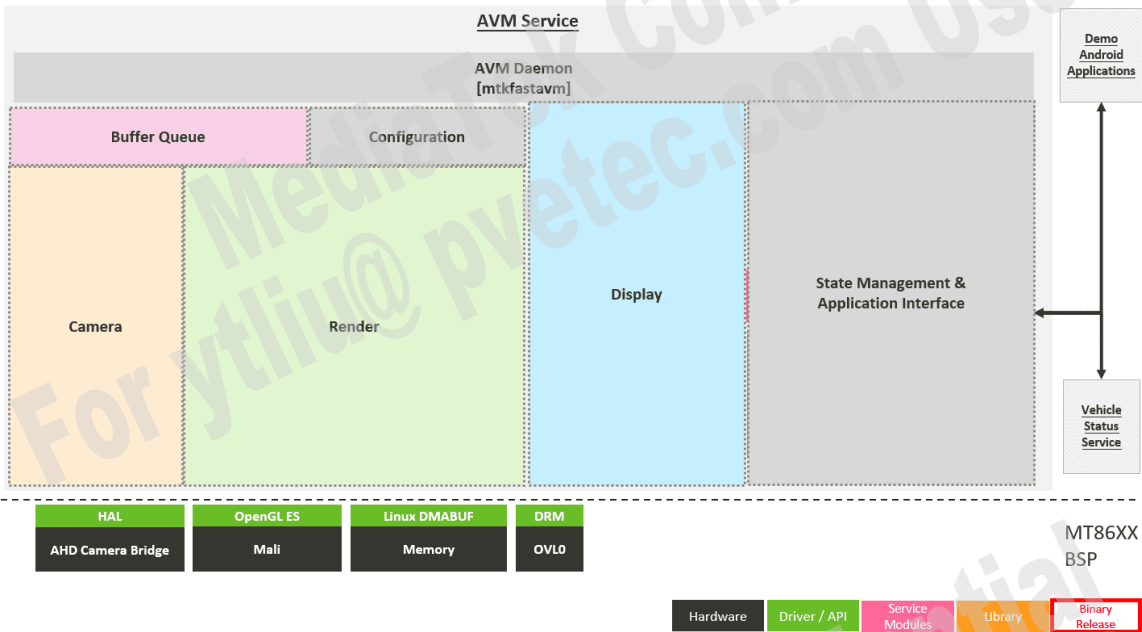
- Camera: 从摄像头引擎管理器接收摄像头输入。摄像头组件与渲染组件分离，以允许适配不同的摄像头。
- Render: 渲染组件控制不同视图的整体布局，并从摄像头和车辆状态获取输入。然后，它将图像扭曲和拼接委托给 AVM 运行时库，以生成摄像头、2D 或 3D AVM 图像。
- Guideline: 该模块使用 OpenGL 将车辆行驶引导线叠加在渲染组件的输出上。引导线与渲染组件分离，以允许不同的车辆模型实现和不同的引导线视觉样式。
- Display: 将合成的 AVM 屏幕图像发送到显示设备。它从渲染和指导组件获取输出并将它们传递给 Surface Flinger。



其他组件包括：

- Configuration: 允许系统供应商使用静态配置文件定制守护进程配置。
- Buffer Management: DMA 缓冲区管理在显示管道中的组件之间传递图像缓冲区。
- Life cycle and state management: 控制 AVM 守护进程的生命周期和可见性状态。

AVM Daemon 的主要组件如下图所示：



二进制文件和文件路径

在运行时阶段，AVM 服务是执行 mtkfastavm 的单个进程。可执行文件动态链接到图形库 libmtkavmproc.so 和 AVM 运行时库 libmtkavm.so。

模块名称	二进制名称和位置	功能
AVM daemon	/system/bin/mtkfastavm	生命周期 控制和 显示管道 管理。
AVM guideline and graphics library	/system/lib64/libmtkavmproc.so	GPU 初始化和动态车轮轨迹线
AVM runtime	/system/lib64/libmtkavm.so	图像拼接和变换算法实现

在初始化期间，AVM 守护进程将解析以下资产和配置文件，并将部分信息绕过指导模块和 AVM 运行时库：

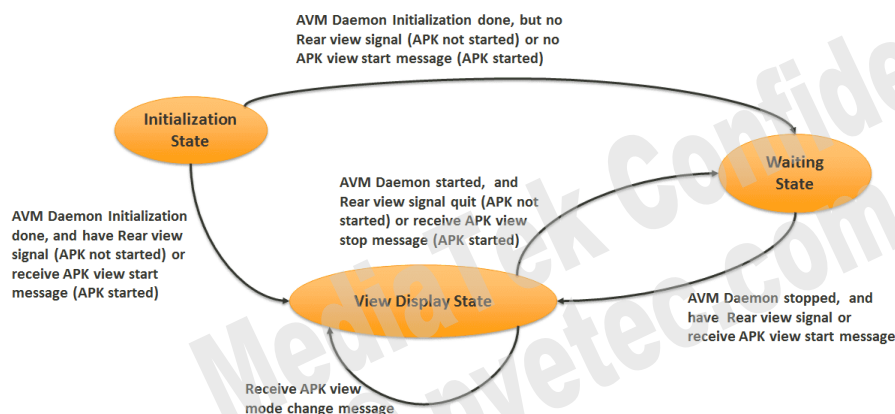
文件	设备上的路径	目的
Calibration Parameters	/mnt/mtkdata/avmdata/*.bin	存储来自 AVM 标定过程 的标定数据。这些数据被绕过 AVM 运行时库 作为 AVM 算法的输入。
Layout Configuration	/etc/automotive/mt6771/avmdata/default_config.json	各显示模式的布局信息
2D Asset Files	/etc/automotive/mt6771/avmdata/	汽车图像文件和警告文字
3D Asset Files	/etc/automotive/mt6771/avmdata/asset_3d	3D AVM 资源

源代码

AVM Daemon 源文件夹路径为 <Base>/vendor/mediatek/proprietary/hardware/mtkcam_sec_lib/avmdaemon，其包含：

- <Base>/vendor/mediatek/proprietary/hardware/mtkcam_sec_lib/avmdaemon/fastavmservice
 - AVM 守护进程程序，它包含 AVM 守护进程的入口点。
- <Base>/vendor/mediatek/proprietary/hardware/mtkcam_sec_lib/avmdaemon/spm/avmdata
 - AVM 配置数据，包含 AVM 标定数据、AVM UI 图像和 AVM UI 配置文件。
- <Base>/vendor/mediatek/proprietary/hardware/mtkcam_sec_lib/avmdaemon/avmclientdemo
 - AVM 客户端实用程序，其中包括一个用于测试所有控制消息和接收相应响应消息的工具。

生命周期



上图展示了 AVM Daemon 的生命周期，它包括：

- Initialization state: 当处理完成后，AVM 进入下一个状态。
- View Display state: AVM Daemon 在屏幕上显示 AVM 实时视图并等待控制消息。
- Waiting state: AVM Daemon 停止显示 AVM 实时视图并等待控制消息。

AVM Daemon 在系统启动后不会终止，因此它没有退出状态。

套接字接口

AVM 定义了一个套接字协议，用于 AVM Daemon 与应用程序进行通信，有两种单独的协议类型：

- 被动消息套接字：应用程序通过此套接字向 AVM Daemon 发送控制消息，AVM Daemon 则向应用程序发送响应消息。当应用程序发送一条消息时，它必须等待 AVM 响应后才能发送下一条消息。AVM Daemon

仅允许一个被动消息套接字连接。如果一个应用程序想要使用被动消息套接字控制 AVM Daemon，则必须断开与 AVM Daemon 连接的其他应用程序。

– 从应用程序到 AVM 守护进程的控制消息协议

Control Message protocol			Description	Response
Msg Len (uint32)	Msg Type (uint32)	Msg Info (uint32)		
12	0	0	stop AVM daemon display	OK/Fail
		1	start AVM daemon display	
12	1	0	switch to wide view, and start AVM daemon display	OK/Fail
		1	switch to top view, and start AVM daemon display	
		2	switch to side view, and start AVM daemon display	
		3	switch to rear assistant view, and start AVM daemon display	
		4	switch to BVM view, and start AVM daemon display	
		5	switch to camera preview, and start AVM daemon display	
		6	Switch to camera preview only, and start AVM daemon display	
12	2	0/1	Get default table/param path	Path string
		2/3	Get new table/param path	
		4/5	Get temp table/param path	
12	3	0	Switch to front camera on camera preview mode	OK/Fail
		1	Switch to left camera on camera preview mode	
		2	Switch to right camera on camera preview mode	
		3	Switch to rear camera on camera preview mode	
12	4	0	Reserved	Image raw data
		1	dump camera preview image	
		4	dump avm screen image to client app	
12	5	0	Reset avm to new table/param (if no new table, reset to default table)	OK/Fail
		1	Reset avm to temp table/param	
12	6	0	Stop R signal simulation	OK/Fail
		1	Start R signal simulation	
		2	Send R signal off	
		3	Send R signal on	

– 从 AVM 守护进程到应用程序的响应消息协议。

Response Message protocol			Description						
Msg Length (uint32)	Ctrl Msg Type (uint32)	Msg type (uint32)	Message Information						
16	Control Message Type	0	0(uint32)					Response OK	
			1(uint32)					Response Fail	
Raw data Length	Control Message Type	1	Raw data format RGB888 YUYV YV12 (uint32)	Width (uint32)	height (uint32)	Raw data length (uint32)	Raw data (uint32)	Next raw data	RAW data
Msg Length	Control Message Type	2	String data (String length)					Response string	

• 主动消息套接字：AVM Daemon 会自动通过此套接字向应用程序发送一些通知消息。应用程序不能通过此套接字向 AVM Daemon 发送消息。AVM Daemon 只允许一个主动消息套接字连接。AVM Demo APK 将在系统启动后将 AVM Daemon 与主动消息套接字连接起来。

– 从 AVM 守护程序到应用程序的通知消息协议。

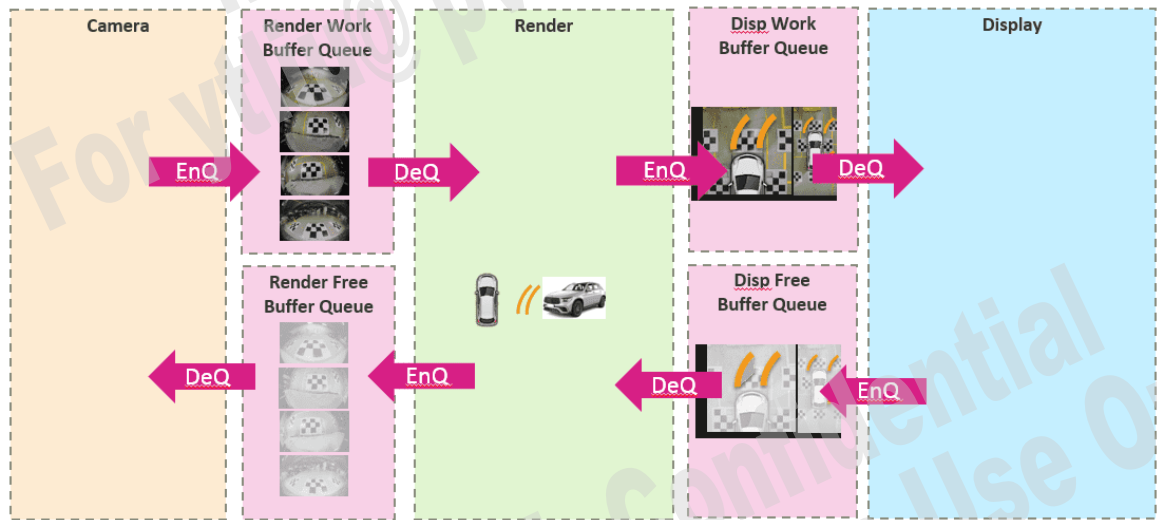
Notification Message protocol			Description
Message Length (uint32)	Message type (uint32)	Message Information (uint32)	
12	0	0	Gear “R” signal quit
		1	Gear “R” signal enter

显示管理

如显示管道所述，AVM 守护进程通过 Surface Flinger 输出到 OVL 硬件。

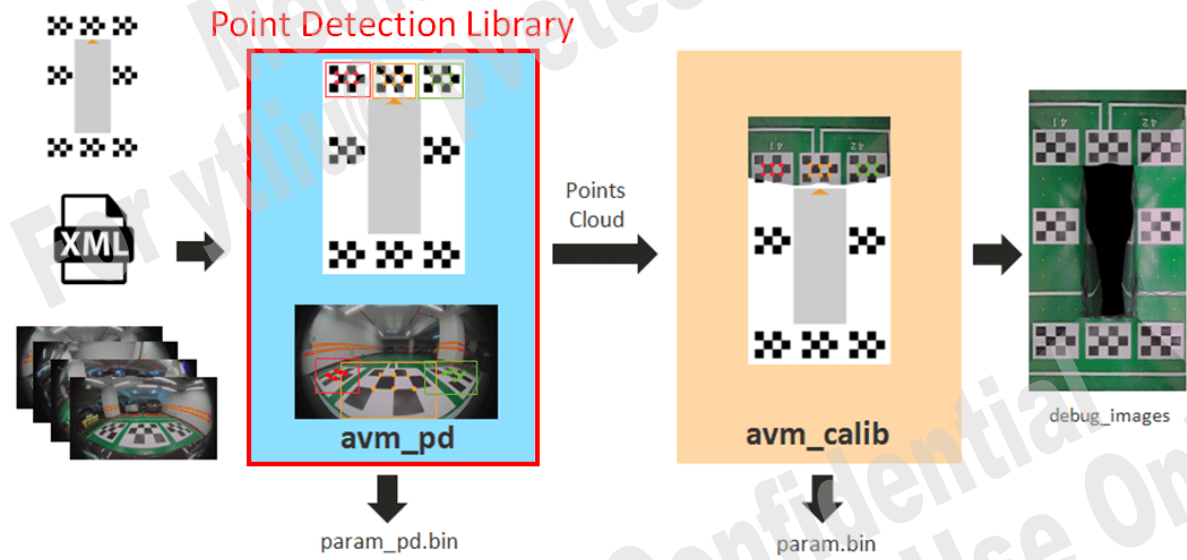
显示管道缓冲区管理

相机、渲染、显示组件被分配到3个不同的线程，各个线程之间通过缓冲队列来交换输入输出缓冲区，如下图所示：



9.2 点检测库

AVM 点检测库是一个本机库，用于检测预定义模式（例如棋盘）上的参考点。



上图显示了 AVM 标定基本流程的概览。

AVM 点检测库以相机图像帧作为输入，检测棋盘图案，并输出点云进行标定过程。

注意：点云包含 BEV 和相机图像帧之间的一对点。

文件夹结构

如果您获得许可源代码，其结构应如下：

```
/vendor/mediatek/mt2712/mtkavm/avmalgo
├── inc
│   ├── avm_point_detector.h
│   └── avm_types.h
├── src
│   ├── avm_pd
│   └── avm_point_detector.cpp
├── test
│   ├── data
│   │   └── <test image/data>
│   └── test.cpp
```

其中：

- avm_point_detector.h 是 AVM 点检测器的标题。
- avm_types.h 定义常见的枚举、数据结构和定义。
- avm_point_detector.cpp 包含 AVM 点检测器的实现。
- test文件夹是Google Test(gtest)框架实现的UT测试用例。

API 概述

点检测库提供了一个 C++ 头文件 avm_point_detector.h，它通过纯 C 函数和结构公开 API。它提供了 6 个基本 API：

- Constructor: construct_avm_pd
- 初始化: avm_pd_init
- 模式配置: avm_pd_add_pattern_info
- 检测点: avm_pd_detect_point/avm_pd_detect_bev_point
- 获取点云: avm_pd_get_point_cloud
- Destructor: destroy_avm_pd

典型的用例可以用下面的伪代码来描述：

- 调用construct_avm_pd获取avm_pd句柄。
- 调用 avm_pd_init 初始化点检测器。
- 对于每个模式：
 - 调用 avm_pd_add_pattern_info 设置模式信息和点检测器。
 - 将返回值 “roi_id” 存储为标识。
- 对于每一帧：
 - 对于每个投资回报率：
 - 调用 avm_pd_detect_point(cam_id, roi_id, image frame...) 来检测相机图像上当前 ROI 的点。
- 对于每一帧：
 - 对于每个投资回报率：
 - 调用 avm_pd_detect_bev_point(cam_id, roi_id, bev image frame...) 来检测 bev 图像上当前 ROI 的点。
- 调用avm_pd_get_point_cloud获取结果，点云。
- 当不再需要该库时，调用 destroy_avm_pd 来释放资源。

请注意，相机图像的点必须按顺序与 BEV 图像的点对齐。

构造 API

```
void* DLL_PUBLIC construct_avm_pd();
```

该API创建点检测器上下文并返回模块句柄。

参数

不适用

返回值

- 返回一个 “void*” 指针，用它作为以下步骤的句柄。

初始化 API

```
int32_t DLL_PUBLIC avm_pd_init(void* hdlr, avm_pd_config* p_cfg);
```

此 API 初始化点检测器并分配资源。客户端代码必须传递包含 avm_pd_config 结构中的基本数据。

参数

- hdlr 是点检测器的处理程序。
- p_cfg 指向 avm_pd_config 结构。

```
struct avm_pd_config {  
    uint32_t camera_count;  
};
```

- camera_count 是全景摄像头的总数。请注意，我们用 4 个摄像头来实现它。

返回值

- 返回正值表示点检测库初始化成功，否则返回负值表示初始化失败。

模式配置 API

```
avm_roi_id DLL_PUBLIC avm_pd_add_pattern_info(void* hdlr, avm_cam_id cam_id, avm_pattern_type type, int32_t grid_height, int32_t grid_width, avm_rect bev_rect, avm_rect cam_rect);
```

此API为对应的摄像头和矩形（ROI）设置一个模式配置。

参数

- hdlr 是点检测器的处理程序。
- cam_id 是摄像机的标识。
- type 是模式类型。我们目前仅支持 AVM_PD_PAT_TYPE_CHESSBOARD。
- grid_height 是棋盘网格高度。
- grid_width 是棋盘网格宽度。
- bev_rect 是 BEV 图像上的 ROI 矩形。
- cam_rect 是摄像机图像上的 ROI 矩形。

返回值

- 如果模式设置成功，则返回正的 roi_id，否则返回负值表示发生错误，请查看错误日志进行调试。

检测点 API

```
void DLL_PUBLIC avm_pd_detect_point(void* hdlr, avm_cam_id cam_id, avm_roi_id roi_id, avm_image * p_image);
```

此 API 可检测图像特定 ROI 上的模式。

参数

- hdlr 是点检测器的处理程序。
- cam_id 是摄像机的标识。
- roi_id 是返回的ROI的标识。
- p_image 应该指向相机图像缓冲区。

返回值

- 此 API 无返回值。您可以通过调用 `avm_pd_get_detected_point(void* hdlr,avm_cam_id cam_id, avm_roi_id roi_id, avm_point_list* p_point_list);` 来检查点列表的数量。

```
struct avm_point_list {
    uint32_t count;
    avm_point2f points[AVM_MAX_POINTS_OF_ROI];
};
```

获取点云 API

```
void* DLL_PUBLIC avm_pd_get_point_cloud(void* hdlr);
```

此 API 返回检测到的点云的指针。

参数

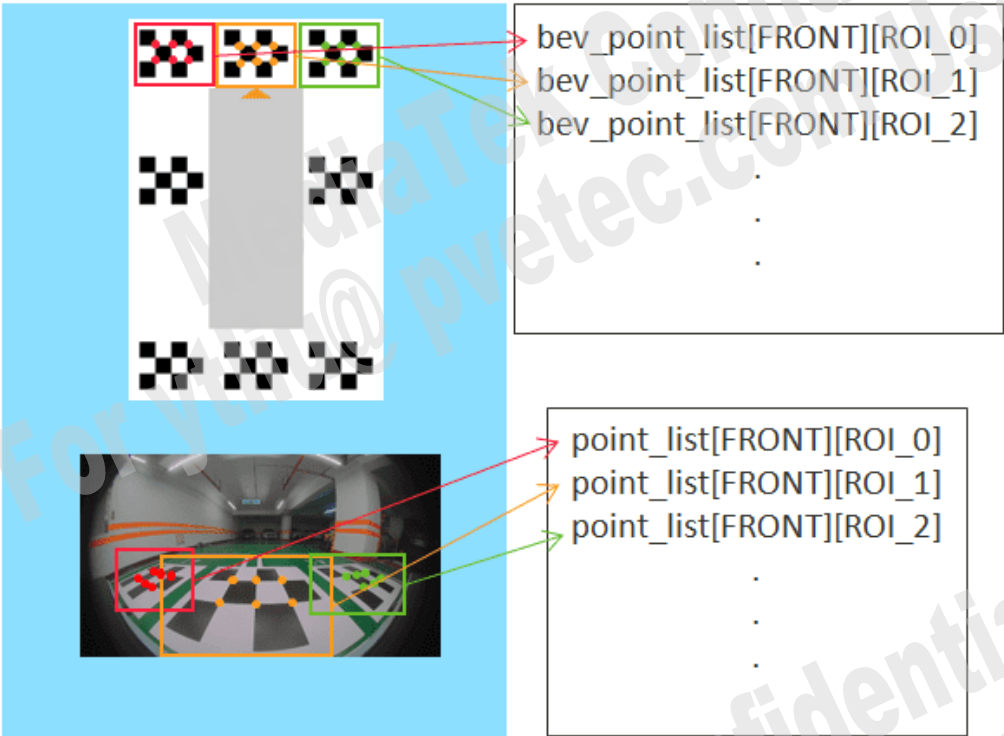
- `hdlr` 是点检测器的处理程序。

返回值

- `void *` 指向点云结构。我们将此指针传递给 AVM 标定模块进行标定过程。

```
struct avm_point_cloud {
    avm_point_list point_list[AVM_MAX_CAM_NUM][AVM_MAX_ROI_OF_CAM];
    avm_point_list bev_point_list[AVM_MAX_CAM_NUM][AVM_MAX_ROI_OF_CAM];
};
```

- `point_list[][]` 用于存储每个 ROI、每个摄像机的点。
- `bev_point_list[][]` 用于存储 BEV (ground truth) 上每个 ROI 的点及其对应位置。



- 确保两点列表配对以进行真实世界的点匹配。

析构函数 API

void DLL_PUBLIC destroy_avm_pd(void* hdlr);

该API清除资源并释放点检测器句柄。

参数

- hdlr 是点检测器的处理程序。

返回值

不适用

UT 测试平台

基于Google Test (gtest) 框架创建了3个测试用例。

- 检查初始化参数-TEST_F(PdTest, InitTest)

- 检查点检测算法在生成图像上的准确性 - TEST_F(PdTestInit, DetectPoint)
- 检查真实图像上的点检测算法准确性 - TEST_F(PdTestInit, DetectPointReallImage)

构建 AVM 标定项目后，测试程序会在 build\x64\{configuration} 下生成。您可以运行此测试程序，通过运行这些测试用例来检查您的修改是否正确。

常见问题

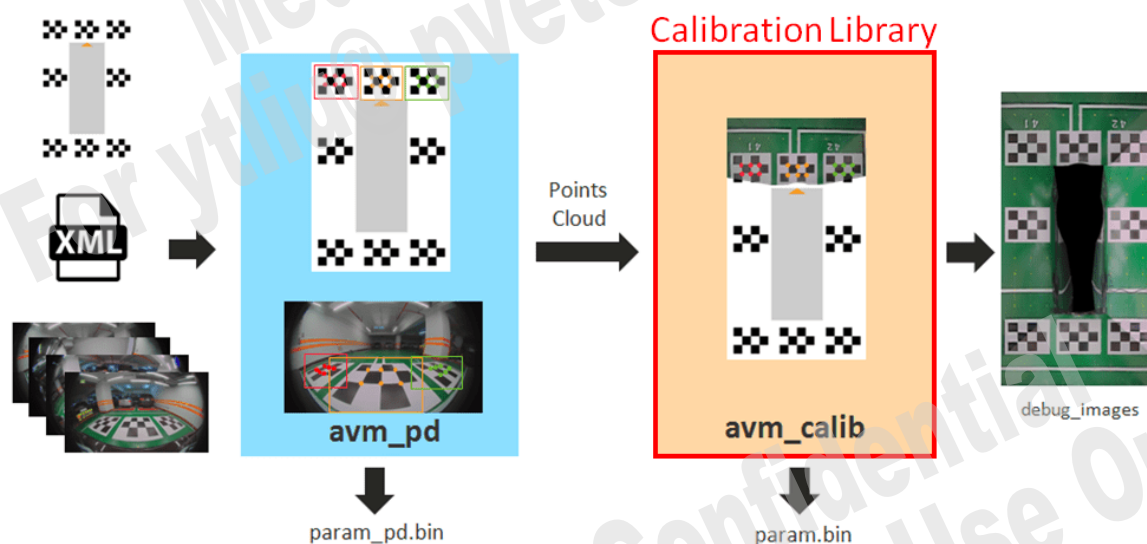
AVM 点检测器是否线程安全？

不可以。由于当前设计中使用全局上下文，因此客户端代码必须在同一线程内调用 avm_pd_init、avm_pd_add_pattern_info 和 avm_pd_detect_point。

可以改变模式和点检测算法吗？

是的。正如 AVM 标定库所述，从 AVM 点检测库到 AVM 标定库的唯一输入是点云。确保您发送到 AVM 标定库的是与 BEV（地面实况）和相机图像点配对的点云。

9.3 标定库



上图显示了 AVM 标定基本流程的概览。

AVM 标定库 (libmtkavmcalib.so) 是计算鸟瞰视图 (BEV) 旋转和平移参数的原生库。计算完成后，AVM 标定库将外部参数保存到 param.bin 中，供 AVM 运行库显示。

API 概述

AVM 标定库提供了一个 C++ 头文件 `avm_calib.h` (位于 `/vendor/mediatek/mt2712/mtkavm/avmalgo/inc`), 该文件通过纯 C 函数和结构公开 API。它提供了 8 个基本 API:

- Constructor: `construct_avm_calib`
- 初始化: `avm_calib_init`
- 相机配置: `avm_calib_set_intrinsic_param`
- 点云设置: `avm_calib_set_point_cloud`
- RT参数计算: `avm_calib_calc_rt`
- 混合配置: `avm_calib_set_beveffect`
- 混合数据计算: `avm_calib_calc_beveblend_data`
- Destructor: `destroy_avm_calib`

典型的用例可以用下面的伪代码来描述:

- 调用 `construct_avm_calib` 获取 `avm_calib` 句柄
- 调用 `avm_calib_init` 初始化标定库
- 调用 `avm_calib_set_point_cloud` 将配对点云传递到标定库。
- 对于每个摄像头:
- 调用 `avm_calib_set_intrinsic_param` 设置相机内参。
- 调用 `avm_calib_calc_rt` 计算 RT 参数。
- 调用 `avm_calib_set_beveffect` 来配置混合区域。
- 调用 `avm_calib_calc_beveblend_data` 计算针迹混合数据。
- 当不再需要该库时, 调用 `destroy_avm_calib` 来释放资源。

构造 API

```
void* DLL_PUBLIC construct_avm_calib();
```

该 API 创建 AVM 标定上下文并返回模块句柄。

参数

不适用

返回值

- 返回一个 “void*” 指针，用它作为以下步骤的句柄。

初始化 API

int32_t DLL_PUBLIC avm_calib_init(void* hdlr, avm_calib_config* p_cfg);

此 API 初始化 AVM 标定库并分配资源。客户端代码必须传递包含 avm_calib_config 结构中基本数据的内存缓冲区。

参数

- hdlr 是 AVM 标定库的处理程序。

```
avm_calib_config:
struct avm_calib_config {
    // number of camera input sources
    // the current implementation supports only 4 cameras
    uint32_t camera_count;

    // This is used in the calibration process as
    // the maximum viewing angle of each virtual camera in BEV(top) view.
    //
    // This value is not related to the actual FOV of the input cameras.
    //
    // The unit is degree and must be in range from 110 to 170.
    // The suggested default value is 160.
    uint32_t camera_fov;

    // Camera input image size in pixels.
    avm_size cam_img_size[AVM_MAX_CAM_NUM];

    // Birdeye view setting
    avm_rect bev_view_frustum; // in millimeters (physical coordinate)
    avm_rect bev_input_frustum; // in millimeters (physical coordinate)

    // bev setting
    int32_t bev_blend_data_w; // in pixels
    int32_t bev_blend_data_h; // in pixels
```

```

// Fisheye Undistortion view setting
int32_t fisheye_undistort_view_w; // in pixels
int32_t fisheye_undistort_view_h; // in pixels
uint32_t fisheye_undistort_view_fov[AVM_MAX_CAM_NUM]; // in degrees

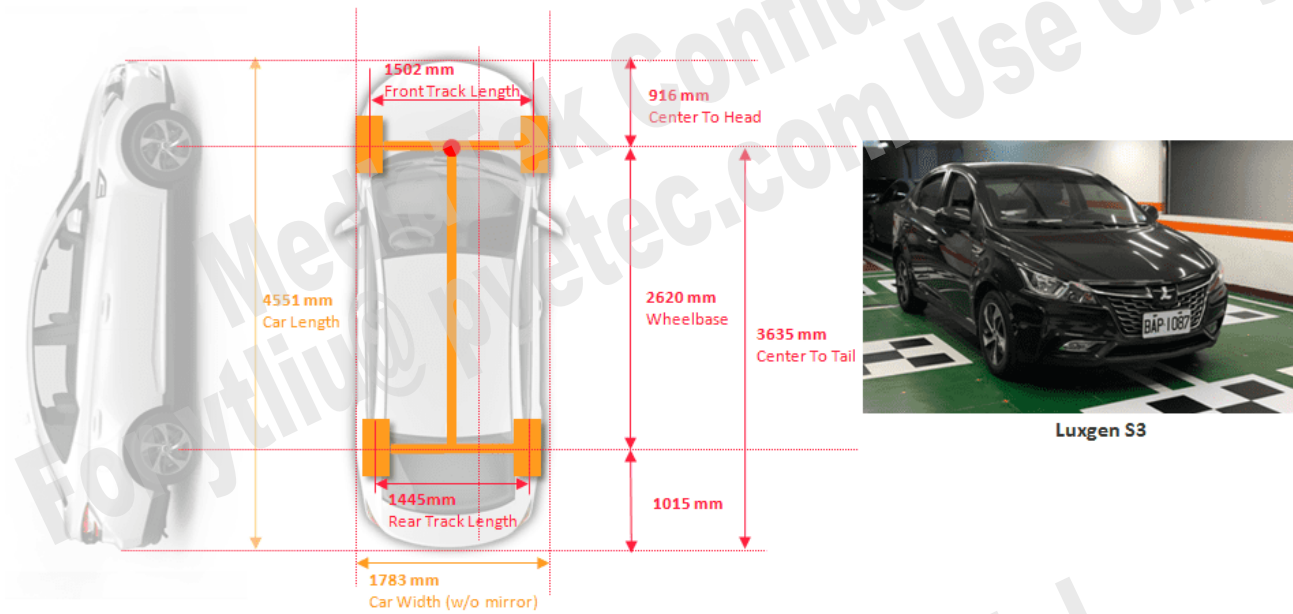
// Guideline and Car Dimension setting
avm_car_setting car_setting;

// AutoTuning switch: 0 (off), 1 (on)
int32_t intrin_refine_flag;
};

struct avm_calib_car_setting {
    int32_t car_width;
    int32_t car_length;
    int32_t wheel_base;
    int32_t front_track_width;
    int32_t rear_track_width;
    int32_t center_to_head;
};

```

- **p_cfg** 指向 **avm_calib_config** 结构。
 - **camera_count** 是全景摄像头的总数。请注意，我们用 4 个摄像头来实现它。
 - **camera_fov** 在标定过程中用作 BEV（顶部）视图中每个虚拟相机的最大视角。建议默认值为 160。
 - **cam_img_size** 是以像素为单位的相机输入图像大小。
 - **bev_view_frustum** 是最终的 BEV（顶部）视图截头体，以毫米（物理坐标）为单位。
 - **bev_input_frustum** 是 BEV 标定模式（地面实况）的视锥体，以毫米（物理坐标）为单位。
 - **bev_blend_data_w** 是拼接混合数据的宽度（以像素为单位）。
 - **bev_blend_data_h** 是拼接混合数据的高度（以像素为单位）。
 - **fisheye_undistort_view_w** 是运行时库的鱼眼不失真视图宽度。
 - **fisheye_undistort_view_h** 是运行时库的鱼眼不失真视图高度。
 - **fisheye_undistort_view_fov** 是运行时库中每个相机的鱼眼不失真视图 fov。
 - **car_setting** 指向结构 **avm_calib_car_setting**，该结构描述了汽车信息。



- `intrin_refine_flag` 是用于启用/禁用相机指令自动调谐的开关。请注意，自动调谐的值将存储在 `param.b` in 中，因此在线标定将使用与离线标定相同的设置。

返回值

- 标定库初始化成功则返回正值，否则返回负值表示初始化失败。

相机配置 API

```
struct avm_intrinsic_param {
    double cam_matrix[3][3];
    double dist_coeff[AVM_MAX_CAM_NUM];
    int32_t img_width;
    int32_t img_height;
};

void DLL_PUBLIC avm_calib_set_intrinsic_param(void* hdlr, avm_cam_id cam_id, avm_intrinsic_param* p_param);
```

该API设置相应相机的固有参数。

参数

- hdlr 是 AVM 标定库的处理程序。
- cam_id 是摄像机的标识。
- p_param指向avm_intrinsic_param结构，其中包含相机矩阵和畸变系数。

返回值

不适用

点云设置API

```
void DLL_PUBLIC avm_calib_set_point_cloud(void* hdlr, void* point_cloud);
```

该API设置相应相机的固有参数。

参数

- hdlr 是 AVM 标定库的处理程序。
- point_cloud 指向由点检测库生成的点云缓冲区。

返回值

不适用

RT 参数计算 API

```
int32_t DLL_PUBLIC avm_calib_calc_rt(void* hdlr, float* average_error);
```

该API比较点云中的点并计算旋转平移参数。

参数

- hdlr 是 AVM 标定库的处理程序。
- average_error指向一个float型数据，旋转平移计算的平均误差输出到这个float型数据中。

返回值

- 如果此过程运行成功，则返回正值。否则，需要检查日志，因为返回负值表示失败。

输出日志

- avm_calib 将输出计算出的相机参数和投影后的平均误差（与地面实况比较）
- 注意：根据我们的经验，平均误差小于0.5意味着投影质量良好。否则，您应该检查点检测结果和相机固有参数。

```
[INFO] Start to calculate RT...
[INFO] can[0] final cx[627.700], cy[348.400], fx[242.27087], fy[240.01033]
[INFO] can[0] Avg Error: 0.436827 in 18 points
[INFO] can[1] final cx[633.500], cy[349.600], fx[244.77087], fy[243.39033]
[INFO] can[1] Avg Error: 0.255372 in 18 points
[INFO] can[2] final cx[633.700], cy[350.100], fx[242.02087], fy[241.11033]
[INFO] can[2] Avg Error: 0.283047 in 18 points
[INFO] can[3] final cx[632.100], cy[350.900], fx[243.85087], fy[246.92033]
[INFO] can[3] Avg Error: 0.202138 in 18 points
[INFO] [Total] Avg Error: 0.294346 in 72 points
```

混合配置 API

```
void DLL_PUBLIC avm_calib_set_bev_effect(void* hdlr, avm_calib_bev_effect* p_effect);
```

该API用于设置相机图像之间的拼接混合效果参数。

参数

```
struct avm_blend_data {
    // we only support 'cone' blending type.
    avm_calib_blend_type type;
    union {
        avm_blend_cone cone;
    };
};

struct avm_calib_bev_effect {
    uint32_t count; // region number
    avm_blend_data blend_regions[MAX_STITCH_REGIONS];
};
```

- hdlr 是 AVM 标定库的处理程序。
- p_effect 指向混合效果缓冲区。
 - count 是混合区域数量
 - blend_regions 包含“锥体”效果设置。

返回值

不适用

MediaTek Confidential
For ytliu@pvetec.com Use Only

MediaTek Confidential
For ytliu@pvetec.com Use Only

MediaTek Confidential
For ytliu@pvetec.com Use Only

混合数据计算 API

```
void DLL_PUBLIC avm_calib_calc_bev_blend_data(void* hdlr, void* p_blend_data, uint32_t buffer_size,
uint32_t* data_size);
```

该API用于计算运行库的针迹混合数据。

参数

- hdlr 是 AVM 标定库的处理程序。
- p_blend_data 指向将写入针迹混合数据的缓冲区。
- buffer_size 是 AVM 标定库参考的数据缓冲区大小。
- data_size 是 p_blend_data 中最终输出数据的大小。

返回值

不适用

析构函数 API

```
void DLL_PUBLIC destroy_avm_calib(void* hdlr);
```

此API清除资源并释放AVM标定库处理程序。

参数

- hdlr 是 AVM 标定库的处理程序。

返回值

不适用

自动调节

虽然相机镜头之间的公差必须在标准范围内，但这些误差将直接影响顶视图的质量。我们的算法将尝试对其进行补偿。



UT 测试平台

基于 Google Test (gtest) 框架创建了 2 个测试用例。

- 检查初始化参数-TEST_F(CalibTest, InitTest)
- 检查预定义点云上的标定算法精度-TEST_F (CalibTestInitied, CalcRtTest)

您可以通过运行这些测试用例来检查修改的正确性。

常见问题

AVM 标定库是线程安全的吗？

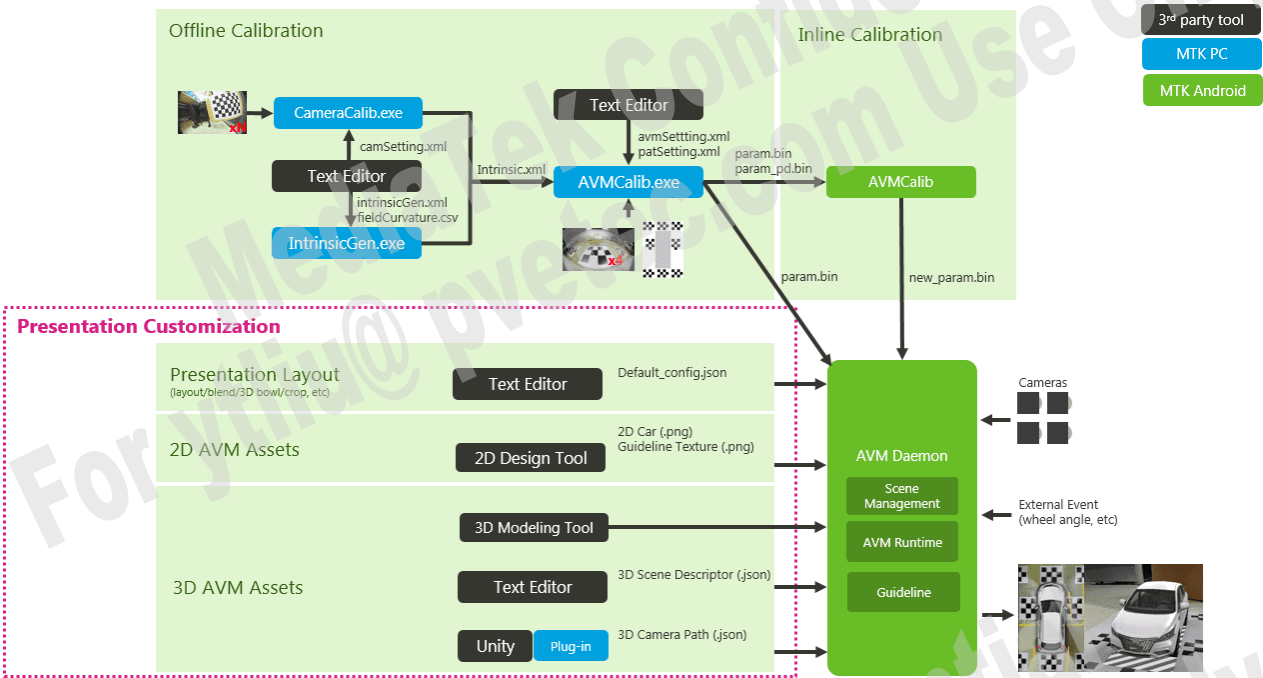
不可以。由于当前设计中使用全局上下文，客户端代码必须在同一线程内调用 avm_calib_init、avm_calib_set_intrinsic_param、avm_calib_calc_rt 和 avm_calib_calc_bev_blend_data。

10 演示客制化

AVM SDK 提供了演示应用程序和显示模式，作为不同 AVM 产品的参考。在不修改源代码的情况下，有一些定制空间。本节介绍可能的定制方法，包括：

- 更改 AVM 显示模式的布局尺寸。
- 修改 2D 车辆图像和“盲点”区域。
- 采用您自己的 3D 汽车模型。
- 在 3D AVM 视图中分配 3D 相机动画。

这些呈现定制涉及修改配置文件和 UI 资源文件，如下面的概览图所示：



10.1 布局

本节介绍 AVm SDK 中显示布局的设计和定制。

大多数 AVm 用例要求 AVm 系统支持不同的显示布局，并允许最终用户在布局之间切换。此外，这些显示布局应该可由系统供应商定制。

AVm SDK 支持以下布局功能：

- 用于切换显示模式的套接字命令
- 布局配置文件可用于设计不同的布局尺寸

显示模式概述

为了在不同的布局之间切换，AVm 守护进程定义了一组显示模式。每个模式都分配有一个模式 ID，即一个整数索引。默认模式 ID 为 0。显示模式的最大数量由 mtk_avm_config.h 中的宏 AVm_DISP_MODE_MAX 静态定义。

在运行时，AVm Demo APK 或 AVm Calibration APK 等应用程序会发送套接字命令 AVm_MSG_MODE 以及模式 ID，以指示 AVm 守护进程切换到所需模式。

下图显示了参考设计中定义的大多数显示模式：



请注意，AVm 演示应用程序和 AVm 标定应用程序共享同一套布局配置，但使用不同的显示模式 ID。参考设计中定义了一些其他显示模式，这些模式未被应用程序用于调试目的。

在大多数情况下，前后齿轮共享相同的显示模式。各个视图类型的实现负责在前后齿轮状态改变时选择相机输入源。

显示模式只是每个屏幕上视图的排列方式。实际的渲染逻辑和相机拼接行为由不同的视图类型定义。例如，参考设计中的默认显示模式 0 是屏幕 0 中 2 个视图的组合，如下所示：



每种视图类型都实现自己的渲染逻辑，并且可能需要不同的参数。例如，上图中的两种视图类型是：

- FISHEYE_VIEW 显示前置或后置摄像头的中心裁剪图像。齿轮状态决定使用哪个摄像头。
- TOPVIEW 采用额外的视锥参数来确定鸟瞰图般的顶视图中的可见区域。

请参阅本页的“视图”部分以获取所有视图类型的列表。

配置和资产

AVM_MSG_MODE 和 mode id 仅表示使用哪种显示模式。它不包含实际的布局信息。布局信息以 JSON 格式静态存储在配置文件中。AVM 守护进程的配置文件位置为：

- 在目标上：/etc/automotive/mt6771/avmdata/default_config.json
- 在源代码仓库中：{avm_repo_root}/avmdaemon/spm/avmdata/default_config.json

通过修改JSON配置文件，系统厂商可以自定义各个显示模式的布局。

此配置文件在 AVM 守护进程的初始化阶段进行解析。开发人员必须在更新配置文件后终止并重新启动 AVM 守护进程可执行文件 (mtkfastavm)，才能观察到更新后的布局。

其他资产资源也存储在相同的位置，如下表所列：

文件/文件夹名称	格式	目的
default_config.json	JSON	布局和 2D 资源配置
warning.png	32 位 PNG	警告文字图像
car.png	32 位 PNG	2D汽车图像
floor.png	32 位 PNG	2D 车身地板覆盖 盲区
pattern.argb	原始 RGBA	动态车轮轨迹线模块 使用
asset_3d	目录	3D AVM 资源文件

配置文件分为几个基本属性：

- version：配置文件的文件格式版本
- camera_input：相机输入格式和后处理配置
- display_modes：布局和每个视图的配置
- view_type_parameter：全局视图配置
- warning_image_info：警告图片位置

因此示例 JSON 配置文件的整体结构如下所示：

```
"mtkfastavm_config": {
  "version": "2.3-spm", // AVM v2.3 configuration file format
  "camera_input": {
    //...
  },
  "view_type_parameter": {
    //...global view parameters
  },
  "display_modes": {
    //...layout and view parameters
  },
}
```

```
"warning_image_info": {  
    //...on-screen position of the warning text  
}  
}
```

每个属性的内容将在以下章节中解释。

相机输入格式

`camera_input` 属性配置相机的输入格式和图像分辨率。可选择添加额外的后处理阶段，包括去隔行或图片质量控制。

`size` 和 `format_fourcc` 属性是必需的，如下所示：

```
"camera_input": {  
  "size": [1280, 720],  
  "format_fourcc": "YUYV",  
}
```

- 大小是相机图像维度的[宽度，高度]数组。
- `format_fourcc` 是 DRM fourcc 代码，支持以下源颜色格式：
 - “YUYV”
 - “UYVY”

您可以选择将 `deinterlace` 或 `mdp_pq` 设置为 `true` 以启用后处理。但是，这些后处理器是互斥的，因此不能同时设置为 `true`。

例如，要使用 720p YUYV 相机启用图片质量控制，请使用以下设置：

```
"camera_input": {  
  "size": [1280, 720],  
  "format_fourcc": "YUYV",  
  
  "deinterlace" : false,  
  "mdp_pq" : true,  
},
```

要使用具有去隔行功能的 480i UYVY 相机，请使用以下设置：

```
"camera_input": {  
  "size": [720, 480],  
  "format_fourcc": "UYVY",  
  
  "deinterlace" : true,  
  "mdp_pq" : false,
```


},

布局配置

配置 JSON 文件包含布局配置设置的键值层次结构。本节介绍 display_modes、view_type_parameter 和 warning_image_info 属性的详细信息。

显示模式

布局信息存储在 display_modes 中，它定义为 JSON 对象数组。以下代码片段说明了文件中单个显示模式的定义：

```
"display_modes": [
  {
    "mode_id" : 0,
    "_name": "normal_mode",
    "screens": [
      {
        "screen_id": 0,
        "rect": [0, 0, 1920, 1080],
        "views": [
          {
            "view_type": "TOPVIEW",
            "rect": [1328, 0, 592, 968]
          },
          {
            "view_type": "FISHEYE_VIEW",
            "rect": [193, 0, 1120, 968]
          }
        ]
      }
      // ... 2nd screen if exists
    ]
  },
  // ... other mode definitions ...
]
```

模式 ID

在上面的代码片段中，对象 display_modes 被定义为 JSON 对象数组。每个对象都有一个名为 mode_id 的属性。在上面的示例中，mode_id 定义为 0。如果 AVM 演示传递模式 ID 为 0 的套接字命令 AVM_MSG_MODE，则 AVM 守护进程会将显示布局更新为此 JSON 对象定义的内容。可以定义的模式数量有一个上限，即 AVM_DISP_MODE_MAX，根据 mtk_avm_config.h 中的定义，它静态设置为 30。

屏幕

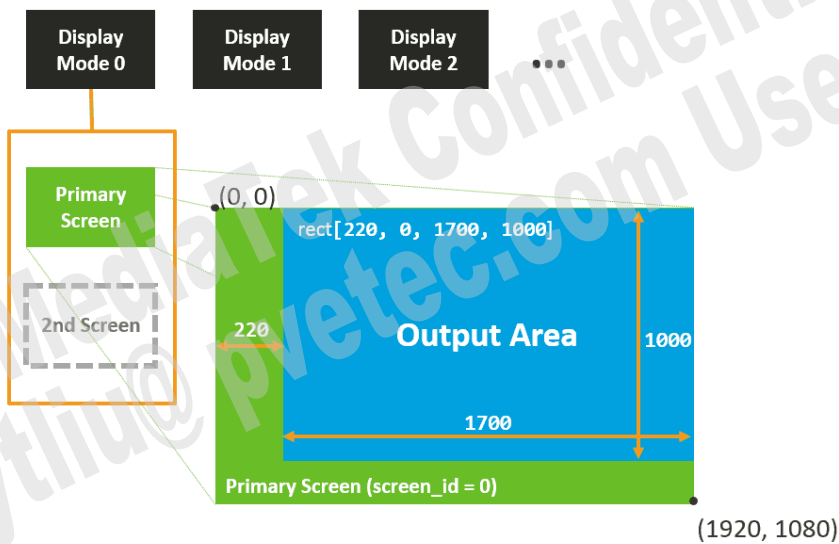
请注意，_name 属性仅用于文档目的。它不会影响布局的实际内容。相反，显示模式包含一个或多个屏幕的布局定义。这存储在名为 screens 的数组中。屏幕是实际的显示设备，例如主显示器 LCD 或辅助外部显示器。

screen_id 在当前版本中只能为 0 或 1，其中 0 为主屏幕，1 为外接屏幕。这些整数 screen_id 共用 libDRM API 定义的同值。

未被任何屏幕或视图覆盖的区域将被绘为黑色。

屏幕的矩形

对于每个屏幕对象，一个 rect 属性定义了 AVM 守护进程的输出矩形。AVM 守护进程仅更新由 rect 属性定义的区域。rect 属性定义为 [x, y, width, height] 的数组，其中 (0, 0) 是屏幕的左上角，如下图所示：



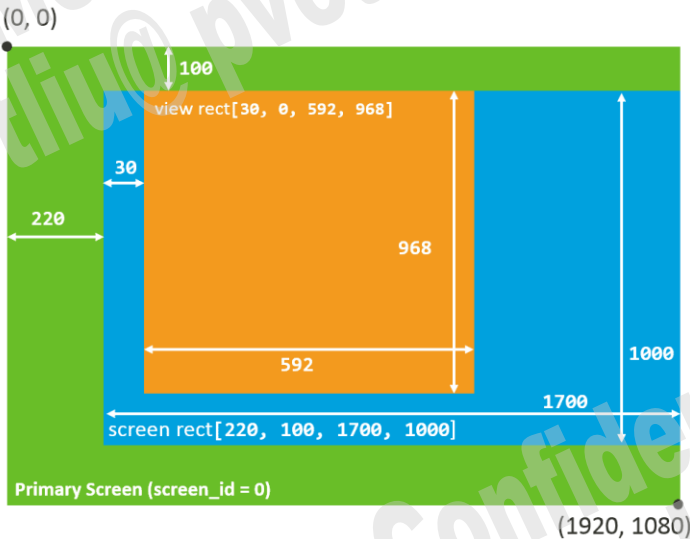
视图

在 AVM 输出区域内，可以在 views 数组中定义一个或多个视图对象。与屏幕类似，每个视图对象还定义一个相对于 AVM 输出区域的 rect 属性。例如，以下 JSON 代码片段：

```
"screen_id": 0,
"rect": [220, 100, 1700, 1000],
"views": [
  {
    "rect": [30, 0, 592, 968],
    "view_type": "UNDISTORT_VIEW"
  },
]
```

```
// ...  
]
```

定义这样的视图布局：



ID

为当前视图分配唯一 ID

视图类型

每个视图对象的实际内容由其视图类型决定。视图类型在 AVM 守护进程中定义和实现。JSON 文件接受每个视图类型的类型名称，例如

```
"views": [  
  {  
    "rect": [30, 0, 300, 720],  
    "view_type": "TOPVIEW",  
    "frustum": [-2814, 2820, 2815, -6365]  
  },  
  {  
    "rect": [340, 0, 300, 720],  
    "view_type": "FISHEYE_VIEW"  
  },  
  // .. other view types  
]
```

其中 TOPVIEW 和 FISHEYE_VIEW 是类型名称。所有可用的 view_type 名称总结在下表中：

在 default_config.json 中输入名称	显示内容	行为
"RAWCAMERA_VIEW"	Single camera	在信箱中显示摄像机图像并适合视图矩形。
"FISHEYE_VIEW"	Single camera	显示摄像机图像，中心裁剪的内部视图矩形。
"UNDISTORT_VIEW"	Single camera	显示未失真的摄像机图像。
"FREEFORM_VIEW"	Single camera	从源摄像机图像裁剪并拉伸四边形区域至显示视图矩形。
"TOPVIEW"	2D AVM	将四个摄像头的输入拼接成类似鸟瞰图的 2D AVM 顶视图图像。
"FIX_3DVIEW"	3D AVM	使用 PC 工具编辑相机动画路径的 3D AVM。
"INTRO_3DVIEW"	3D AVM	具有程序控制的一次性相机动画的 3D AVM。它与布局动画一起使用来实现“开场动画”演示。
"FREE_3DVIEW"	3D AVM	3D AVM，摄像头围绕固定中心点旋转一圈。支持触摸事件输入。
"TOPVIEW_3D"	3D AVM	具有固定摄像机位置的 3D AVM，可模拟 2D AVM 的结果。
"ORBIT_3DVIEW"	3D AVM	3D AVM 具有圆形相机动画路径并支持 3D 相机位置改变。
"USER_CAM_3DVIEW"	3D AVM	具有多个用户定义摄像头视角的 3D AVM。此视图允许用户在 default_config.json 中定义摄像头并在运行时修改摄像头视角。
SIDE_3DVIEW	3D AVM	3D AVM 模拟后视镜视图，帮助驾驶员看到车辆后方和侧面的区域，超出驾驶员的周边视野。此视图提供 3 个内置视角，还允许用户在 default_config.json 中定义摄像头视角，并在运行时修改摄像头视角。

查看矩形动画

可以扩展 rect 值以在进入显示模式时显示一次性布局动画。为此

- 在屏幕中将动画属性设置为 true。
- 通过设置屏幕上的 anim_duration 属性来定义动画持续时间（以毫秒为单位）
- 可选择使用 anim_delay 在屏幕上设置动画的起始延迟
- 为屏幕内的每个视图设置一个附加参数 rect_anim_from，指定视图矩形的“起始位置”。然后，AVM 守护进程将在切换到显示模式时执行布局动画。

例如，以下 JSON 配置会创建一个动画，使 FISHEYE_VIEW 从屏幕左侧移动，同时缩小 3D AVM 视图：

```
"screens": [  
  {
```

```
"screen_id": 0,
"rect": [0, 0, 1920, 1080],

"animated": true,           // set to true to enable animation
"anim_duration" : 750, // animation duration in MS
"anim_delay": 3000,         // animation starting delay in MS

"views": [
  {
    "view_type": "INTRO_3DVIEW",
    "rect": [193, 0, 592, 968],
    "rect_anim_from": [193, 0, 1727, 968]           // start full-screen and shrink
  },
  {
    "view_type": "FISHEYE_VIEW",
    "rect": [800, 0, 1120, 968],
    "rect_anim_from": [1920, 0, 1120, 968]         // slide from the right
  }
]
}
```

查看参数

某些视图类型需要除矩形之外的其他参数来确定最终渲染结果。例如，AVM_VIEW_TYPE_TOPVIEW 需要定义视图可见边界的视锥体信息。这些附加参数在 JSON 中定义为视图对象的一部分：

```
"views": [
  {
    "view_type": "TOPVIEW",
    "rect": [1328, 0, 592, 968],
    "frustum": [-2814, 2820, 2815, -6365]
  }
]
```

其中 TOPVIEW 需要 frustum 属性。

以下部分描述了每种视图类型所需的附加参数。

TOPVIEW 参数

```
"topview": {
  "frustum": [-2814, 2820, 2815, -6365],
  "rear_gear_frustum": [-2814, 2820, 2815, -6365], // optional
  "disable_guideline": false, // optional, default to false.
```



```
},
```

可以将 `disable_guideline` 参数设置为 `true` 以隐藏视图内显示的车轮轨迹线。

`frustum` 参数定义 `AVM_VIEW_TYPE_TOPVIEW` 视图类型的可见区域。在上面的示例中，我们将 `AVM_VIEW_TYPE_TOPVIEW` 视图的可见视锥体设置为

- 左：-2814 毫米
- 顶部：2820 毫米
- 右：2815 毫米
- 底部：-6365 毫米

`rear_gear_frustum` 是可选参数。定义后，它将控制应用倒档时 2D AVM 算法要使用的截锥体。默认情况下，它与 `frustum` 参数的值相同。

关于视锥体坐标系的详细描述，请参考标定流程。

通过 `frustum` 参数，我们可以实现“缩放”或“放大”的顶视图，如下所示：

```
"views": [
  {
    "view_type": "TOPVIEW",
    "rect": [1328, 0, 592, 968],
    "frustum": [-2814, 2820, 2815, -6365] // "full" top view show entire car
  },
  {
    "view_type": "TOPVIEW",
    "rect": [193, 0, 1120, 968],
    "frustum": [-2029, 3320, 2029, -180], // "enlarged" top view show front side
    "rear_gear_frustum": [-2029, -2365, 2029, -5865] // "enlarged" top view show rear side
  }
]
```

在第二个视图中，视锥体定义的边界比顶视图小，从而实现了“放大”效果，如下图所示：

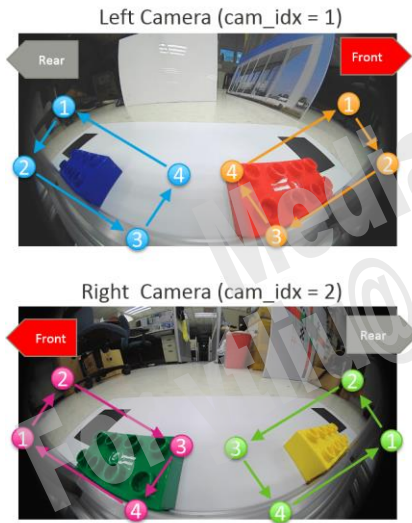


FREEFORM_VIEW 参数

FREEFORM_VIEW 允许您将源相机图像上的任何凸四边形区域裁剪到矩形区域。除了矩形参数外，以下参数也是必需的：

- cam_idx 定义源摄像机 ID，
 - 0：前置摄像头
 - 1：左摄像头
 - 2：右摄像头
 - 3：后置摄像头
- src_quad 是源图像上四个位置的数组。
 - 每个位置都是一个按 [x, y] 顺序排列的两个整数的数组。
 - 位置的顺序很重要。第一个位置始终映射到输出矩形区域的右上角，后面的位置按顺时针方向映射。
 - 要“旋转”输出图像，只需将 src_quad 的位置反转即可。
 - 可以定义可选的 rear_gear_src_quad 来提供倒档状态下的源四边形区域。

下图直观地展示了 rect、cam_idx 和 src_quad 之间的映射：



```

"mode_id": 2,
"name": "side_mode",
"screens": [
  {
    "screen_id": 0,
    "rect": [0, 0, 1080, 1080],
    "views": [
      {
        "view_type": "TOPVIEW",
        "rect": [760, 8, 592, 968],
        "frustum": [-2814, 2820, 2815, -6365]
      },
      {
        "comment": "left side view",
        "view_type": "FREEFORM_VIEW",
        "rect": [193, 0, 553, 968],
        "cam_idx": 1,
        "src_quad": [
          [1130, 220],
          [1270, 430],
          [804, 719],
          [732, 511]
        ],
        "rear_gear_src_quad": [
          [170, 220],
          [10, 430],
          [386, 719],
          [548, 511]
        ]
      },
      {
        "comment": "right side view",
        "view_type": "FREEFORM_VIEW",
        "rect": [1367, 0, 553, 968],
        "cam_idx": 2,
        "src_quad": [
          [10, 430],
          [1270, 430],
          [386, 719],
          [548, 511]
        ]
      }
    ]
  }
]

```

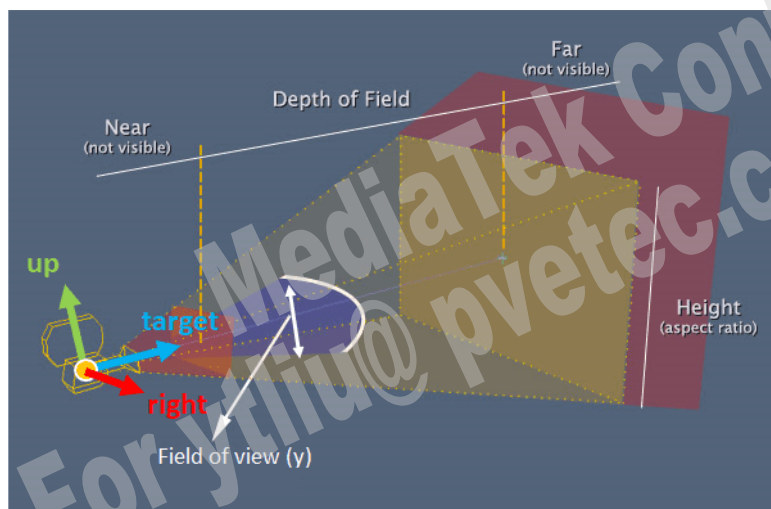


USER_CAM_3DVIEW 参数

USER_CAM_3DVIEW 允许用户直接在 default_config.json 中定义多个相机视角。除 rect 参数外，以下参数也是必需的：

- 相机是一系列定制相机
 - 每个相机都有以下参数。它们构成视锥体。

Parameter	Value	Description
pos target up_dir	vec3	这些参数表示相机的姿势。 - pos：相机的位置 - target：相机注视的目标点 - up_dir：相机的向上方向 坐标空间是右手坐标（OpenGL），Y 向上，-z 向前，单位为cm。
fovy_degree	float	视野（y 方向）度数
clip_plane_near clip_plane_far	float	近裁剪平面和远裁剪平面属性决定了相机视图的开始和结束位置。这些平面垂直于相机方向，并从其位置进行测量。近裁剪平面是将要渲染的最近位置，而远裁剪平面是最远的位置。



```
{
  "id": 335,
  "view_type": "USER_CAM_3DVIEW",
  "rect": [0, 0, 1048, 480],
  "cameras": [
    {
      "pos": [0.000000, 232.980011, 314.011383],
      "target": [0.000000, 0.000000, -135.949997],
      "up_dir": [0.000000, 0.888023, -0.459799],
      "fovy_degree": 60.000000,
      "clip_plane_near": 25.0,
      "clip_plane_far": 5000.0
    },
    {
      "pos": [301.082611, 232.980011, -470.336731],
      "target": [0.000000, 0.000000, -135.949997],
      "up_dir": [-0.307665, 0.888023, 0.341697],
      "fovy_degree": 60.000000,
      "clip_plane_near": 25.0,
      "clip_plane_far": 5000.0
    },
    {
      "pos": [-13.192088, 232.980011, -585.717957],
      "target": [0.000000, 0.000000, -135.949997],
      "up_dir": [0.013481, 0.888023, 0.459601],
      "fovy_degree": 60.000000,
      "clip_plane_near": 25.0,
      "clip_plane_far": 5000.0
    },
    {
      "pos": [-318.836517, 232.980011, -453.453522],
      "target": [0.000000, 0.000000, -135.949997],

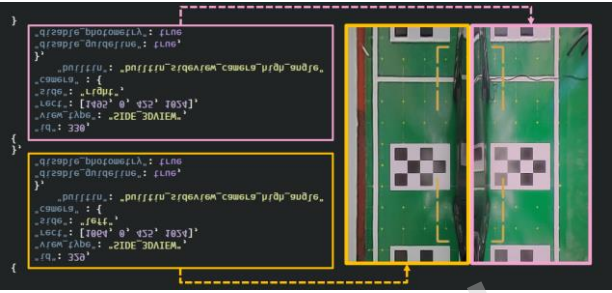
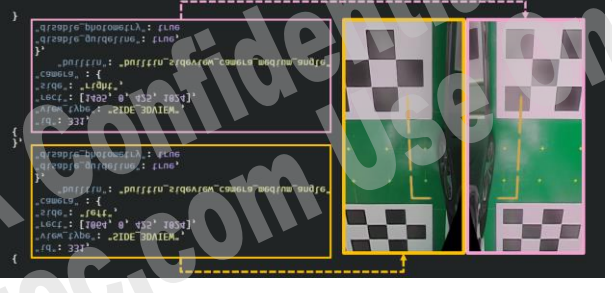
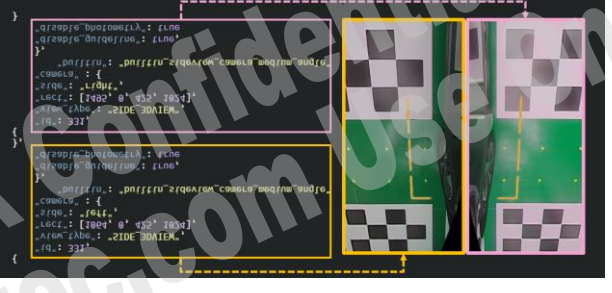
```

```
        "up_dir": [0.307666, 0.888023, 0.341696],
        "fovy_degree": 60.000000,
        "clip_plane_near": 25.0,
        "clip_plane_far": 5000.0
    }
}
}
```

SIDE_3DVIEW 参数

SIDE_3DVIEW 模拟后视镜视图，帮助驾驶员看到车辆后方和侧面的区域，超出驾驶员的周边视野。此视图提供 3 个内置视角，还允许用户在 default_config.json 中定义摄像头视角。除了 rect 参数外，以下参数也是必需的：

- side: 表示预览车辆的哪一侧
 - 右: 要预览的车辆右侧。
 - 左: 要预览的车辆左侧。
- 相机: 设置相机。您可以使用内置视角或定义自定义相机视角。
 - 使用内置视角: 从以下 3 个视角中选择一个。

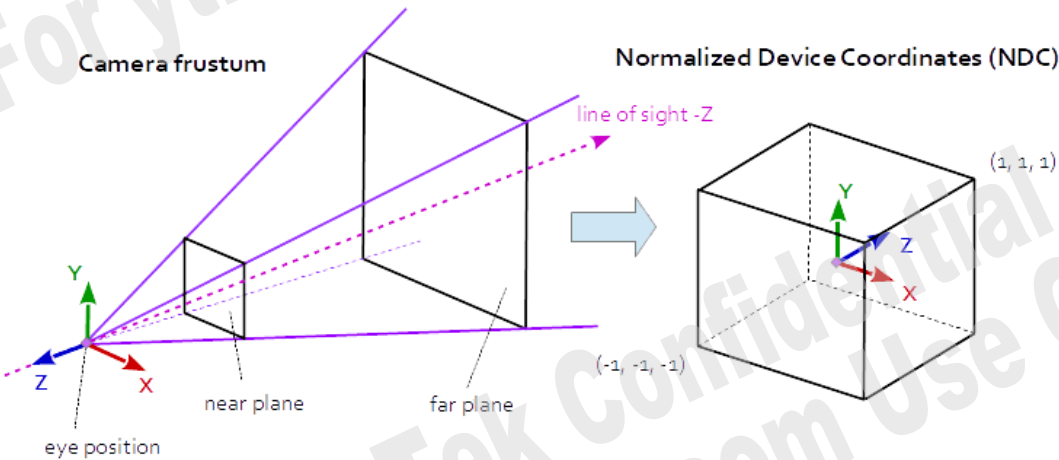
Parameter	Value	Description
builtin	builtin_sideview_camera_high_angle	鸟瞰视图，查看车辆的侧面。 
	builtin_sideview_camera_medium_angle	查看车辆的前侧/后侧。 
	builtin_sideview_camera_low_angle	查看车辆的前侧/后侧。 



- 自定义相机视角：通过以下参数设置相机。它们构成视锥体。但如果你已经指定了内置视角，以下参数将被忽略

Parameters	Type	Description
pos target up_dir	vec3	这些参数表示相机的姿势。 - pos: 相机的位置 - target: 相机注视的目标点 - up_dir: 相机的向上方向 坐标空间是右手坐标（OpenGL），Y 向上，-z 向前，单位为cm。
fovy_degree	float	视野（y 方向）度数
clip_plane_near clip_plane_far	float	近裁剪平面和远裁剪平面属性决定了相机视图的开始和结束位置。这些平面垂直于相机方向，并从其位置进行测量。近裁剪平面是将要渲染的最近位置，而远裁剪平面是最远的位置。

- axis_offset: 创建非对称视锥体的参数，在NDC空间中偏移视锥体。
在3D图形中，上述相机参数可以构建视锥体，它是透视虚拟相机系统的视野。最终视锥体将被映射到规范化设备坐标（NDC）。在NDC空间中，范围在所有维度上都是-1~+1。axis_offset尝试在NDC空间中移动视锥体。



下图显示了 axis_offset 如何影响最终的视角。



其他配置参数

有些跨视图配置不适合在单独的视图参数中定义。例如，汽车图像在毫米坐标空间中的位置和警告图像在屏幕上的位置应该全局定义。这些配置在 JSON 文件中的 view_type_parameter 属性中定义。下表是这些配置属性的摘要：

可能的视图类型参数

view_type_parameter property	Description
2d_car_image	在毫米坐标空间中定义的二维汽车图像的位置
2d_floor_image	在毫米坐标空间中定义的底层图像的位置
topview_blend	2D AVm 算法的混合参数
3dview_blend	3D AVm 算法的混合参数
transparent_car_road_paving	3D AVm 算法中透明 3D 汽车特征的铺设区域和混合参数

以下代码片段是view_type_parameter的示例配置：

```
"view_type_parameter" : {
  "_comment": "global settings for 2D AVm and 3D AVm",

  "2d_car_image" : {
```

```
        "frustum" : [-1140, 1016, 1139, -3735]
    },

    "2d_floor_image" : {
        "frustum" : [-1140, 1016, 1139, -3735]
    },

    "topview_blend" : {
        "blind_region": [-740, 916, 740, -3635],
        "blend_theta": [30, 30,
                                30, 30],
        "blend_phi" : 15.0
    },

    "3dview_blend": {
        "blind_region": [-740, 916, 740, -3635],
        "blend_theta": [30, 30,
                                30, 30],
        "blend_phi" : 15.0
    },

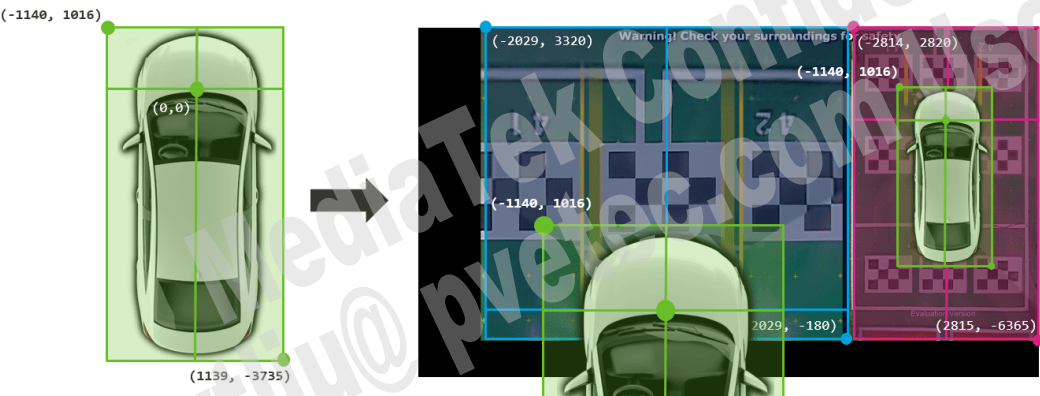
    "transparent_car_road_paving": {
        "paving_region" : [-1040, 1040, 1040, -3735],
        "blind_region": [-740, 916, 740, -3635],
        "blend_theta": [30, 30,
                                30, 30],
        "blend_phi" : 15.0
    }
},
```

2d_car_image

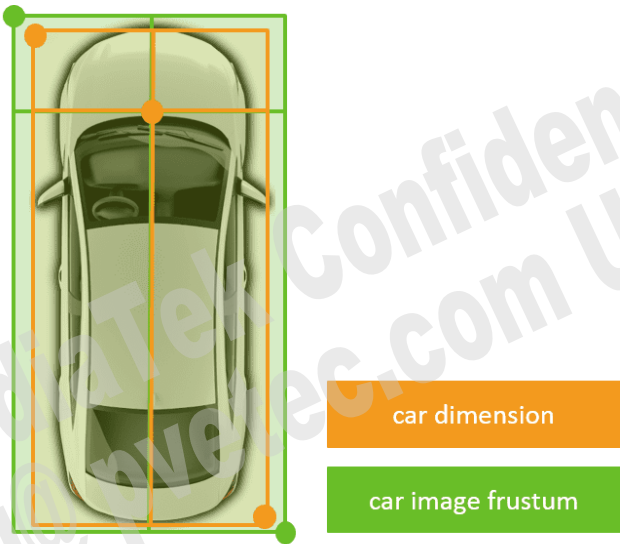
在大多数情况下，顶视图还需要 2D 汽车图像来协助用户。此图像存储在 /etc/automotive/mt6771/avmdata/car.png 中。AVM 守护程序加载此 PNG 文件并将其绘制在 AVM 顶视图上。

汽车图像的位置由视锥体属性定义，该属性与顶视图使用的坐标系相同。然后，AVM 守护程序会自动转换 2D 汽车图像在屏幕上的位置，以匹配每个顶视图配置的视锥体，如下所示：

```
"2d_car_image" : {
    "frustum" : [-1140, 1016, 1139, -3735]
},
```



请注意，此视锥体可能与实际汽车尺寸不同，因为 PNG 文件中可能会有阴影和图像边框等半透明区域需要考虑。因此，如果图像的阴影或填充区域发生变化，则必须重新调整 2d_car_image 视锥体。



2d_floor_image

此配置与 2d_car_image 类似，不同之处在于它定义的不是汽车图像，而是地板图像的位置，在大多数情况下用于覆盖摄像机看不到的盲区。

```
"2d_floor_image" : {  
  "frustum" : [-1140, 1016, 1139, -3735]  
},
```

AVM 视图和 2D 资源的绘制顺序由 AVM 守护进程中的渲染逻辑定义。默认实现是：

1. AVM 视图
2. 2D 地板图像
3. 车轮轨迹线
4. 2D汽车图像

要更改此顺序，您必须修改 mtk avm service.c 中 avm win render 函数中定义的渲染逻辑

topview blend

[illegible]

此参数定义所有AVM VIEW TYPE TOPVIEW视图的混合参数和盲区。

根据相机位置和标定结果，鸟瞰图可能存在以下区域：

- **重叠区域**: 不同相机可能相互重叠的区域
- **盲区**: 对每个摄像头都无效的区域, 例如车辆本身

如下图所示:



topview_blend 节点提供用于定制这些区域外观的参数。

混合锥

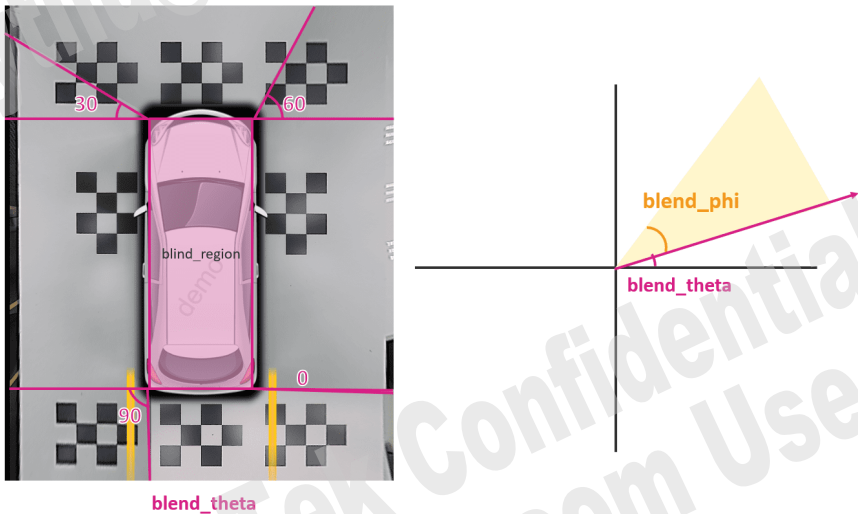
在 AVM SDK 中，重叠区域中不同摄像机图像之间的混合接缝表示为锥形混合区域。使用 `topview_blend` 参数，可以控制 JSON 中圆锥体的角度 (`blend theta`) 和光圈 (`blend phi`):

```
"topview_blend" : {
    "blind_region": [-740, 720, 740, -3265],
    "blend_theta": [30, 30,
                     45, 45],
```

```
"blend_phi" : 5.0
},
```

blend_theta 数组控制每个混合角的方向。顺序为 [左上、右上、左下、右下]，如下图所示。角度定义为圆锥体主轴与水平 X 轴之间的角度。

blend_phi 角度决定每个混合锥体形状的光圈或开度角。设置为 0 时，混合锥体将变成接缝，如下图右侧所示。



盲区

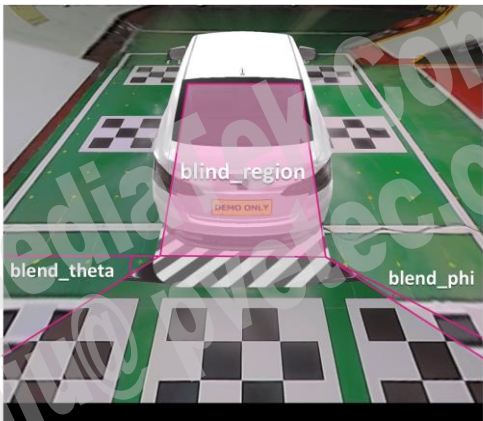
盲区是由 blind_region frustum 定义的矩形。它遵循与顶视图参数相同的坐标。它有两个用途：

- 通过定义此区域，AVM 运行时可以跳过更新盲区内的缓冲区内容以获得更好的性能。
- blind_region 的角也标记了混合锥的起点，如上图所示。

3dview_blend

```
"3dview_blend": {
    "blind_region": [-740, 720, 740, -3265],
    "blend_theta": [30, 30,
                    45, 45],
    "blend_phi" : 5.0
},
```

3D 视图混合设置与 topview_blend 参数共享相同的坐标系和语义，但它确定了 3D AVM 环境中的混合区域。这会影响所有 3D AVM 视图类型。其他 3D 配置由 3D 资产文件定义，详见 3D 模型部分。



透明底盘路面铺设

```
"transparent_car_road_paving": {
  "paving_region" : [-1040, 1040, 1040, -3735],
  "blind_region": [-740, 916, 740, -3635],
  "blend_theta": [30, 30,
30, 30],
  "blend_phi" : 15.0
}
```

此配置专为 3D AVM 环境中的透明汽车功能而设计。它与 2dview_blend 和 3dview_blend 共享相同的坐标系和语义，但它确定了 3D AVM 环境中的道路铺设区域。paving_region 的定义和用途在道路铺设设置中介绍。

warning_image_info

default_config.json 中定义的另一个值是 AVM 警告图像的位置。

```
"warning_image_info" : {
  "rect": [606, 10, 900, 40]
}
```




此节点描述覆盖纹理在屏幕矩形中的位置，该纹理在 AV 输出显示中显示“警告！请检查周围环境以确保安全”。格式为 [x, y, width, height]。

要修改警告图像本身，请修改 {repo_root}/avmdaemon/spm/avmdata/warning.png 中的 PNG 文件。警告图像的目标图像路径在 AV 守护程序中是硬编码的，默认为 /etc/automotive/mt6771/avmdata/warning.png

10.2 3D 模型

AV 支持 3D 显示视图，使驾驶员拥有更多自由和立体视角来检查车辆周围环境以确保安全。它还提供有用的驾驶辅助功能，如 3D 视图中的指导线（停车辅助线）和障碍物标记，以便在停车时提供适当的车辆操纵辅助信息，帮助驾驶员更顺利地停车。

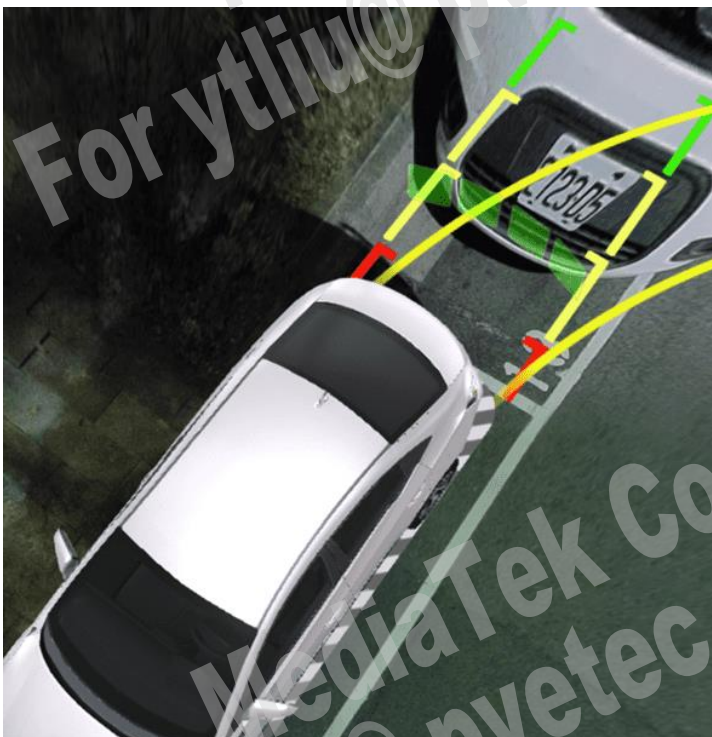
AV 允许您用定制模型替换汽车，实施您自己的风格车轮轨迹线，并轻松构建障碍物地图以便与停车雷达传感器配合使用。

- 汽车：通过渲染 3D 模型来呈现，通过制作 3D 汽车模型来定制它。
- 车轮轨迹线：以代码绘图的方式呈现，发布源代码。
- 障碍物地图：通过渲染 3D 模型来呈现，通过制作 3D 模型来构建。

如上所述，车辆和障碍物地图都涉及 3D 建模，有一些建模规则需要遵循。我们将文档分为四个部分：

- 汽车建模要求
 - 对于 3D 艺术家，谈论制作 3D 汽车模型，介绍模型要求的细节。
- 在 AV 中使用 3D 汽车模型

- 为工程师介绍如何在AVM中正确导入3D汽车模型。
- 障碍物地图 3D 模型要求
 - 对于3D艺术家，谈论制作障碍物地图模型，介绍模型要求的细节。
- 在 AVM 中使用障碍物地图 3D 模型
 - 为工程师介绍如何在AVM中正确导入障碍物地图模型。



MediaTek Confidential
For ytliu@pvetec.com Use Only

MediaTek Confidential
For ytliu@pvetec.com Use Only

MediaTek Confidential
For ytliu@pvetec.com Use Only

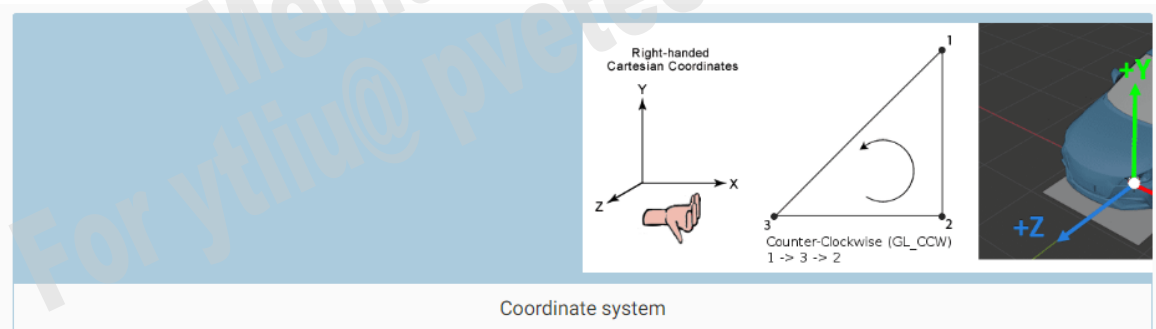
3D 汽车建模要求

交换 3D 文件格式

- OBJ (https://en.wikipedia.org/wiki/Wavefront_.obj_file)
 - 所有模型均由三角面构成。
 - 每个 obj 文件的三角形数量：20000 (65535/3)。
 - 顶点数据包括位置、法线和 UV 坐标。
 - 注意：OBJ通常以MTL(.mtl)文件输出，但对于3D汽车模型，我们不会使用它来渲染。渲染结果和质量主要由纹理直接控制。
- 纹理：漫反射纹理、高光贴图、法线贴图、光纹理和阴影纹理。有关纹理的详细信息，请参阅纹理要求。

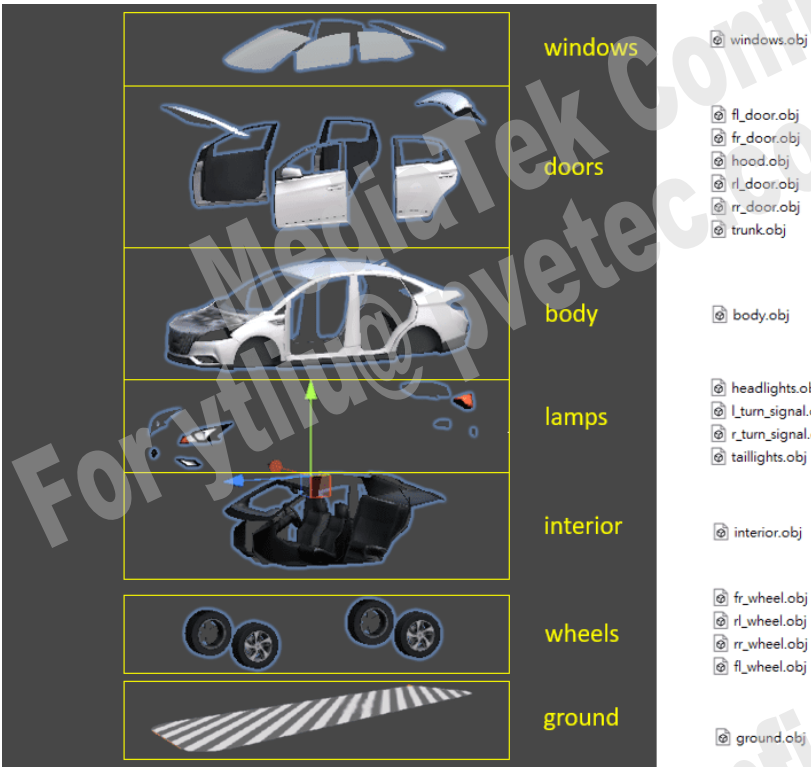
坐标系

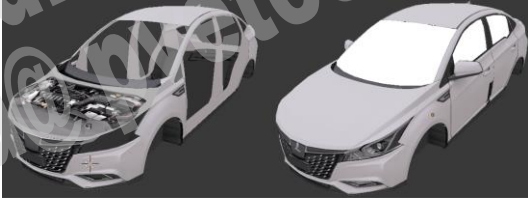
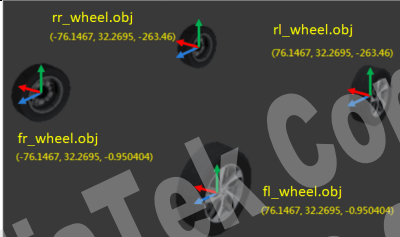
- 坐标系：右手坐标系，与OpenGL相同。
- 模型方向：车辆前部朝 +Z 和 +Y 向上。
- 模型原点：原点 (0,0,0) 位于前轮轨迹投影到地板的中点 (Y=0)。
 - 注意：原点不在车辆的中心。
- 模型比例：3D世界中的1个单位：现实世界中的1厘米。

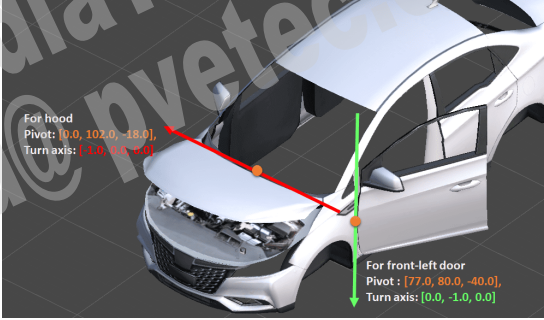


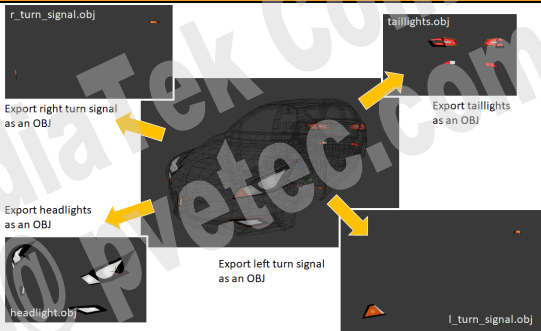
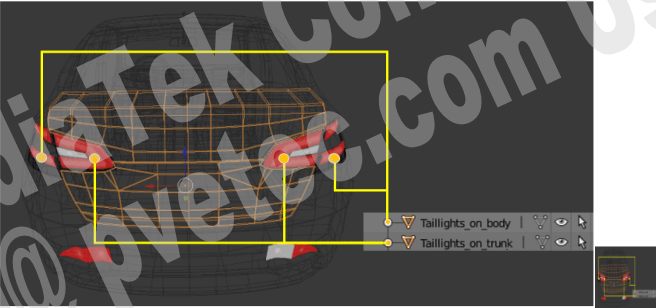
3D 模型分割

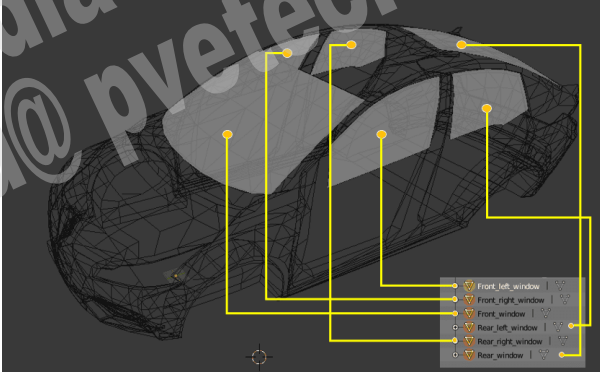
将汽车模型拆分成7个部分。这些部分分别是车身，内饰，车轮，车灯，车门，车窗和地面。下图为示例，车辆分为7个部分，导出18个OBJ文件。



部分	描述	笔记
Body	将车辆外部输出为 OBJ 文件并命名为 body.obj	如果您想要透明窗口，请不要将窗口包含在 body.obj 中，而应将窗口作为独立的 OBJ。否则，您应该将窗口嵌入 body 中。  Doors and windows are separated from body Doors and windows are embedded in body
Interior	将车辆内部输出为 OBJ 文件并命名为 interior.obj	这是可选的。 如果你想有透明的窗户，那么制作内饰是有意义的，因为司机可以透过窗户看到车内的情况。否则，没有必要有内饰模型。
Wheels	将每个车轮输出为 OBJ 文件。 例如： fr_wheel.obj 、 fl_wheel.obj 、 rr_wheel.obj 、 rl_wheel.obj	 为了让车轮像汽车行驶一样移动，您必须提供每个车轮的枢轴点，然后 AVM 才能正确旋转车

部分	描述	笔记
	el.obj、rl_wheel.obj	轮。
Doors	将每扇门输出为 OBJ 文件。 例如： fr_door.obj、fl_door.obj、rr_door.obj、rl_door.obj、hood.obj、trunk.obj	 For hood Pivot: [0.0, 102.0, -18.0], Turn axis: [-1.0, 0.0, 0.0] For front-left door Pivot: [77.0, 80.0, -40.0], Turn axis: [0.0, -1.0, 0.0] 为了在 AVM 中正确打开/关闭门，您必须提供每个门的枢轴点和旋转轴，然后 AVM 引擎才能正确打开/关闭门。

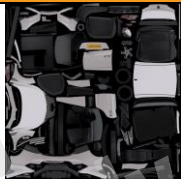
部分	描述	笔记
Lamps	将您想要在 AVM 中打开/关闭的每个灯输出为 OBJ 文件。 例如： headlights.obj、r_turn_signal.obj、l_turn_signal.obj、taillights.obj	 为了在 AVM 中正确打开/关闭特定的灯，您必须将这些灯导出为不同的 OBJ 文件，然后 AVM 可以单独控制它们的状态。对于 AVM 演示车，我们将前灯、右/左转向灯和尾灯制作成不同的 OBJ 文件，因此 AVM 可以切换这些灯。关于尾灯有一件事必须注意：- 如果尾灯的一部分在后备箱上并且后备箱是可打开的。您应该分开它们的网格并赋予它们唯一的名称。以 AVM 演示车为例，taillights.obj 中有 2 个网格。Taillights_on_trunk（网格）在后备箱打开时会在后备箱上，但Taillights_on_body（网格）不会。 

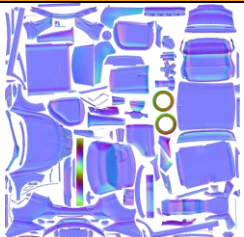
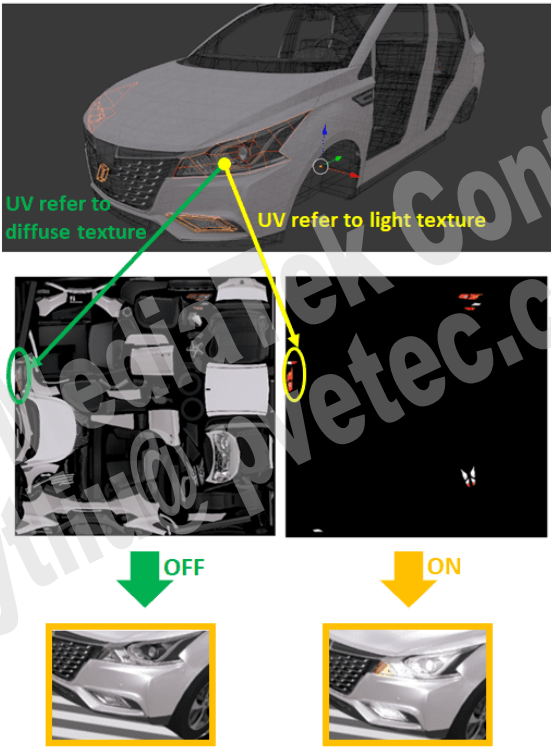
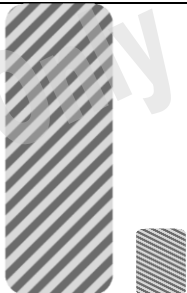
部分	描述	笔记
Windows	将汽车的窗户输出为 OBJ 文件并命名为window.obj	这是可选的。 如果您想要透明的窗口，请将窗口作为独立的 OBJ，并确保每个窗口都是OBJ 中的网格并具有唯一的命名。请参见下图。 
Ground	将地面输出为 OBJ 文件并命名为ground.obj。	地面是一个四边形。四边形的尺寸比汽车的边界框大，但实际尺寸将取决于摄像头安装位置和角度，如下所示，这是在真实汽车上测试时的微调过程。 

纹理要求

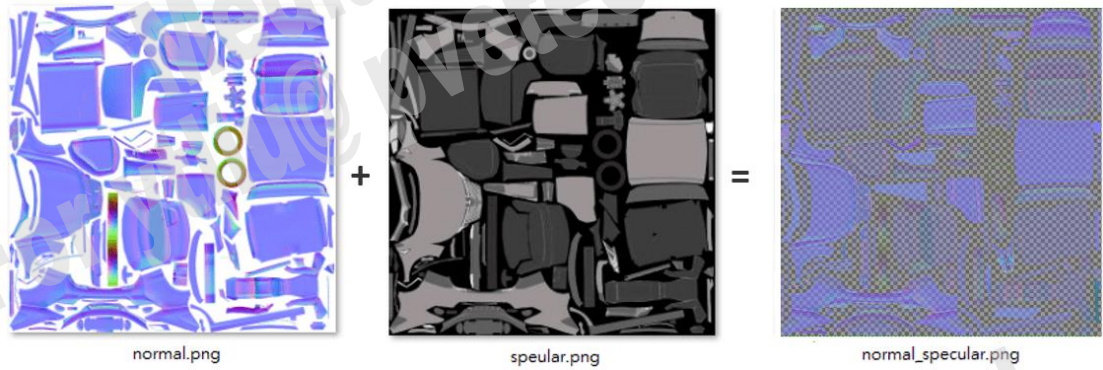
为了优化 3D 渲染，AVM 使用压缩纹理来渲染汽车。压缩纹理可以减少内存占用并提高图形性能。以下是生成纹理的流程：

1. 首先制作5种纹理，分别是漫反射、镜面反射贴图、法线贴图、灯光和地面纹理：

质地	描述	样本
漫反射纹理	1. 用于定义表面的主色调 2. 真实感，光照烘焙纹理 3. 一张纹理覆盖整辆车，包括内饰/外部 4. 分辨率为2048x2048	
镜面反射贴图	1. 用于定义表面的光泽度和高光颜色。像素值越高（从黑到白），表面在场景中看起来就越有光泽 2. 整辆车 一个纹理，包括外部/内部3. 分辨率为2048x2048	

质地	描述	样本
法线贴图	1. 用于增强模型的外观和细节 2.切线空间法线 3.整个汽车的一个纹理，包括外部/内部 4. 分辨率为2048x2048	
光	1. 灯光纹理呈现打开的灯的颜色 2. 在灯光纹理中，大多数像素为零（RGBA 全部为 0），只有灯的像素有颜色 3. 在灯光纹理中，灯像素的位置与漫反射纹理相同。AVM引擎将使用灯网格的 uv 从漫反射和灯光纹理中获取颜色，以呈现灯的关闭和打开 4. 灯光纹理的尺寸可以更小。在 AVM 演示车中，灯光纹理大小为512x512。 5 . 下图显示了灯光纹理的工作原理。 	
地面纹理	纹理用于填充相机的盲区。	

2. 然后将法线贴图和镜面贴图组合成名为 “normal_specular” 的 RGBA 纹理。
- RGB：来自法线贴图。
 - Alpha：来自镜面反射图的值。



3. 将漫反射、法线镜面反射、光和地面纹理转换为压缩纹理格式
- 输出文件：PKM。
 - 压缩格式：RGBA8_ETC2_EAC。
 - 请下载 Mali 纹理压缩工具来转换纹理。(<https://developer.arm.com/tools-and-software/graphics-and-gaming/mali-texture-compression-tool>)。您还可以从附录：纹理压缩中找到压缩纹理的分步过程。
4. 最后输出4张纹理，它们是
- 弥散型
 - 法线镜面反射.pkm
 - 光.pkm
 - 地面.pkm

最终输出文件

所有 OBJ 文件	
body.obj fl_door.obj fl_wheel.obj fr_door.obj fr_wheel.obj ground.obj headlights.obj hood.obj interior.obj l_turn_signal.obj r_turn_signal.obj rl_dpor.obj rl_wheel.obj rr_door.obj rr_wheel.obj taillights.obj trunk.obj windows.obj 3D Object 3D Object 3D Object 3D Object 3D Object 3D Object 3D Object 3D Object 3D Object 3D Object 3D Object 3D Object 3D Object 3D Object 3D Object 3D Object 3D Object 3D Object	
所有纹理文件	



如何使用定制车型

完成模型和纹理后，您可以按照以下步骤在 AV

- 1. 将所有 OBJ 文件和纹理推送到路径：/etc/automotive/mt6771/avmdata/asset_3d/vehicle/

```
$adb root
$adb remount

// push all OBJ into 3D asset path
$adb push body.obj /etc/automotive/mt6771/avmdata/asset_3d/vehicle/
$adb push interior.obj /etc/automotive/mt6771/avmdata/asset_3d/vehicle/
$adb push fr_door.obj /etc/automotive/mt6771/avmdata/asset_3d/vehicle/
$adb push fl_door.obj /etc/automotive/mt6771/avmdata/asset_3d/vehicle/
$adb push rr_door.obj /etc/automotive/mt6771/avmdata/asset_3d/vehicle/
$adb push rl_door.obj /etc/automotive/mt6771/avmdata/asset_3d/vehicle/
$adb push hood.obj /etc/automotive/mt6771/avmdata/asset_3d/vehicle/
$adb push trunk.obj /etc/automotive/mt6771/avmdata/asset_3d/vehicle/
$adb push headlights.obj /etc/automotive/mt6771/avmdata/asset_3d/vehicle/
$adb push taillights.obj /etc/automotive/mt6771/avmdata/asset_3d/vehicle/
$adb push l_turn_signal.obj /etc/automotive/mt6771/avmdata/asset_3d/vehicle/
$adb push r_turn_signal.obj /etc/automotive/mt6771/avmdata/asset_3d/vehicle/
$adb push windows.obj /etc/automotive/mt6771/avmdata/asset_3d/vehicle/
$adb push fr_wheel.obj /etc/automotive/mt6771/avmdata/asset_3d/vehicle/
$adb push fl_wheel.obj /etc/automotive/mt6771/avmdata/asset_3d/vehicle/
$adb push rr_wheel.obj /etc/automotive/mt6771/avmdata/asset_3d/vehicle/
$adb push rl_wheel.obj /etc/automotive/mt6771/avmdata/asset_3d/vehicle/
$adb push ground.obj /etc/automotive/mt6771/avmdata/asset_3d/vehicle/

// push all textures into 3D asset path
$adb push diffusive.pkm /etc/automotive/mt6771/avmdata/asset_3d/vehicle/
$adb push normal_specular.pkm /etc/automotive/mt6771/avmdata/asset_3d/vehicle/
$adb push light.pkm /etc/automotive/mt6771/avmdata/asset_3d/vehicle/
$adb push floor.pkm /etc/automotive/mt6771/avmdata/asset_3d/vehicle/
```

- 2. 编辑/etc/automotive/mt6771/avmdata/asset_3d/vehicle/中的vehicle.json，完成自定义车型设置。

- 1) 找到“纹理”对象，填写“漫反射”、“法线”、“光”和“阴影”的纹理文件名：

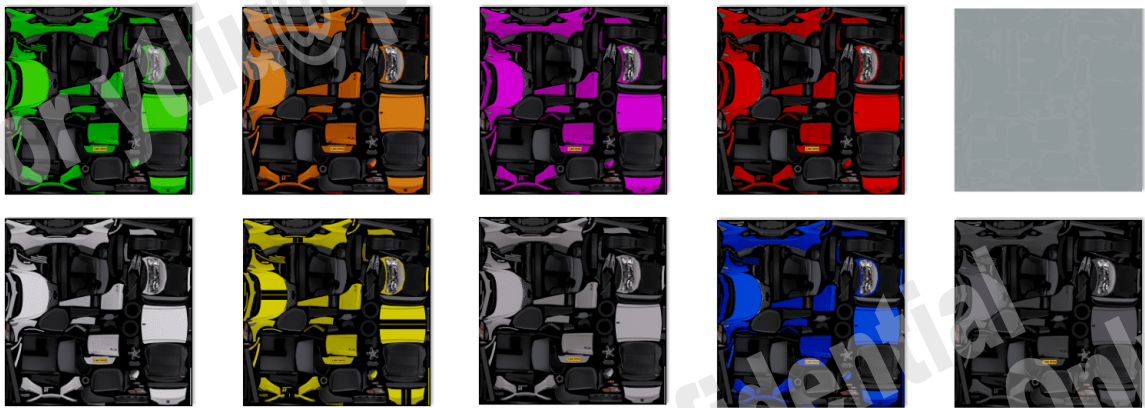
```
"textures" : {
  "diffuse" : "diffuse.pkm",
```



```
"normal" : "normal_specular.pkm",
"light" : "light.pkm",
"shadow" : "floor.pkm"
}
```

对于漫反射纹理，我们允许您分配 1~10 个漫反射纹理。它们的分辨率必须相同（2048x2048）。如果分配了多个漫反射纹理，您可以在运行时选择其中一个作为汽车的当前活动漫反射纹理。以下是 json 中的多漫反射纹理声明。

```
"textures" : {
  "diffuse" : [
    "diffuse.pkm",
    "diffuse_blue.pkm",
    "diffuse_darksilver.pkm",
    "diffuse_green.pkm",
    "diffuse_orange.pkm",
    "diffuse_purple.pkm",
    "diffuse_red.pkm",
    "diffuse_silver.pkm",
    "diffuse_yellow_stripe.pkm",
    "diffuse_transparent.pkm"],
  "normal" : "normal_specular.pkm",
  "light" : "light.pkm",
  "shadow" : "floor.pkm"
},
```



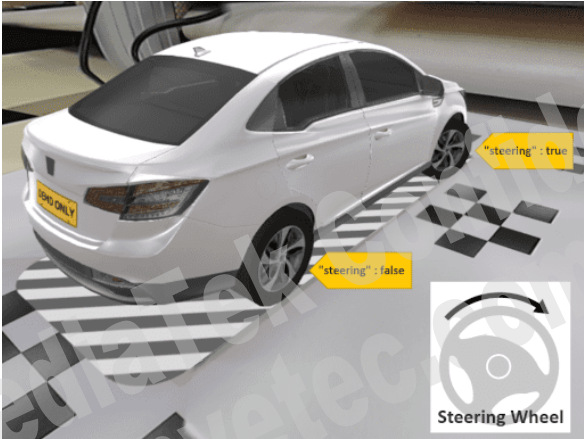
2) 找到 “body”、“interior” 和 “ground” 对象，填写你的OBJ文件名：

```
"body": {
  "filename": "body.obj"
},
"interior": {
  "filename": "interior.obj"
```



```
},  
  
"ground": {  
  "filename": "ground.obj"  
}
```

3) 找到“车轮”对象，它是一组描述汽车有多少个车轮以及如何在行驶时为它们设置动画的组。除了填写正确的 OBJ 文件名外，还有一些重要属性需要编辑：

属性	类型	描述
"pivot"	向量3	此车轮的枢轴点。这是 AVM 发动机在行驶时正确旋转车轮的重要信息。
"steering"	布尔值	如果此轮子是由方向盘转动的，则将标志设置为 true。否则将标志设置为 false。 

```
"wheels" : {  
  // front right wheel  
  "FR_wheel" : {  
    "filename": "fr_wheel.obj",      // fill in corresponding OBJ filename  
    "motion":{  
      "type":"wheel",  
      "pivot": [-76.1467, 32.2695, -0.950404],      // fill in wheel's pivot point  
      "steering" : true // Is turned by steering wheel or not  
    }  
  },  
  // front left wheel  
  "FL_wheel" : {  
    "filename": "fl_wheel.obj",  
    "motion":{  
      "type":"wheel",  
      "pivot": [76.1467, 32.2695, -0.950404],  
      "steering" : true  
    }  
  },  
}
```

```
// Rear right wheel
"RR_wheel" : {
  "filename": "rr_wheel.obj",
  "motion":{
    "type":"wheel",
    "pivot": [-76.1467, 32.2695, -263.46],
    "steering" : false
  }
},
// Rear left wheel
"RL_wheel" : {
  "filename": "rl_wheel.obj",
  "motion":{
    "type":"wheel",
    "pivot": [76.1467, 32.2695, -263.46],
    "steering" : false
  }
}
}
```

4) 找到“门”对象，它是一组描述汽车有多少个门以及如何门打开和关闭时它们设置动画的组。除了填写正确的 OBJ 文件名外，还有一些重要属性需要编辑：

属性	类型	描述
"pivot"	向量3	此门的枢轴点。这是 AVM 引擎正确旋转门的重要信息。
"turn_axis"	向量3	旋转门轴。这是 AVM 引擎正确旋转门的重要信息。

```
"doors" : {
  // Front left door
  "Front_left_door" : {
    "filename": "fl_door.obj", // fill in corresponding OBJ filename
    "motion":{
      "pivot": [77.0, 80.0, -40.0], // fill in door's pivot point
      "turn_axis": [0.0, -1.0, 0.0] // fill in door's rotate axis
    }
  }
}
```

```
    }
  },
  "Front_right_door" : {
    "filename": "fr_door.obj",
    "motion":{
      "pivot": [-77.0, 80.0, -40.0],
      "turn_axis": [0.0, 1.0, 0.0]
    }
  },
  "Rear_left_door" : {
    "filename": "rl_door.obj",
    "motion":{
      "pivot": [77.0, 80.0, -160.0],
      "turn_axis": [0.0, -1.0, 0.0]
    }
  },
  "Rear_right_door" : {
    "filename": "rr_door.obj",
    "motion":{
      "pivot": [-77.0, 80.0, -160.0],
      "turn_axis": [0.0, 1.0, 0.0]
    }
  },
  "Hood" : {
    "filename": "hood.obj",
    "motion":{
      "pivot": [0.0, 102.0, -18.0],
      "turn_axis": [-1.0, 0.0, 0.0]
    }
  },
  "Trunk" : {
    "filename": "trunk.obj",
    "motion":{
      "pivot": [0.0, 115.0, -313.0],
      "turn_axis": [1.0, 0.0, 0.0]
    }
  }
},
```

5) 找到 “lamps” 对象，它是一组描述汽车有哪些类型的灯，然后 AVm 可以单独控制它们：

属性	类型	描述
"loose_parts"	一系列对（“网格名称”，“组件名称”）	1. 建立网格和组件之间的关系 2. 在 AVm 演示车中，尾灯的一部分属于后备箱。当后备箱打开时，它应该仍然在后备箱上。所以我们指定： - Taillights_on_trunk（网格）属于Trunk（组件）



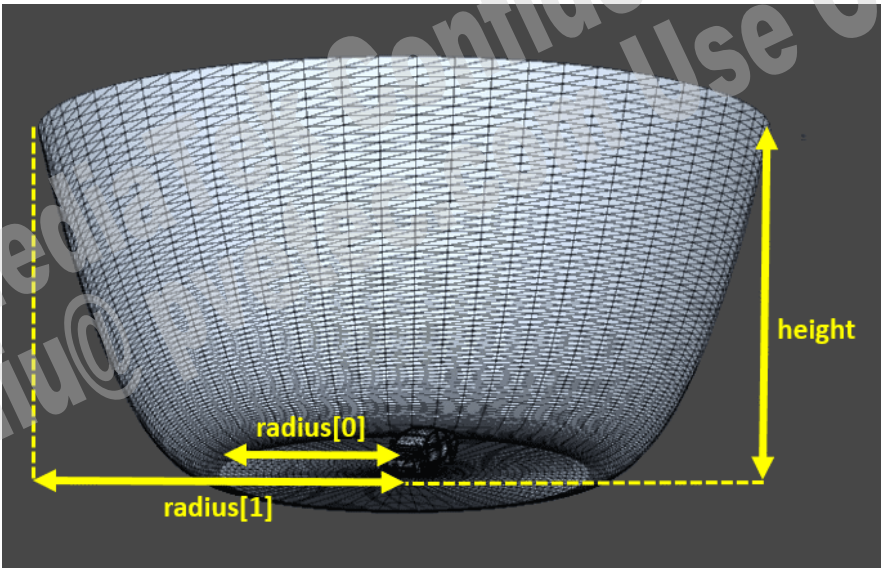
```
"lamps" : {
  /*
  Now we only support these lamps, please set corresponding object name in the *.obj file
  [ASSET_LIGHT_HEADLIGHT] = "Headlights",
  [ASSET_LIGHT_TAILLIGHT_ON_TRUNK] = "Taillights_on_trunk",
  [ASSET_LIGHT_TAILLIGHT_ON_BODY] = "Taillights_on_body",
  [ASSET_LIGHT_TURN_L] = "Left_turn_signal",
  [ASSET_LIGHT_TURN_L_ON_BODY] = "Left_turn_signal_on_body",
  [ASSET_LIGHT_TURN_L_ON_DOOR] = "Left_turn_signal_on_door",
  [ASSET_LIGHT_TURN_L_ON_TRUNK] = "Left_turn_signal_on_trunk",
  [ASSET_LIGHT_TURN_R] = "Right_turn_signal",
  [ASSET_LIGHT_TURN_R_ON_BODY] = "Right_turn_signal_on_body",
  [ASSET_LIGHT_TURN_R_ON_DOOR] = "Right_turn_signal_on_door",
  [ASSET_LIGHT_TURN_R_ON_TRUNK] = "Right_turn_signal_on_trunk",

  Example:
  "Right_turn_signal" : {
    "filename": "r_turn_signal.obj",
    "loose_parts" : [
      "Right_turn_signal_on_trunk", // this is the object name in r_turn_signal.obj
      "Trunk",
      "Right_turn_signal_on_body",
      "body",
      "Right_turn_signal_on_door",
      "Front_right_door"
    ]
  },
  */
  // Headlights
```

```
"Headlights" : {
    "filename": "headlights.obj" // fill in corresponding OBJ filename
},
// Turn signal
"Right_turn_signal" : {
    "filename": "r_turn_signal.obj"
},
// Turn signal
"Left_turn_signal" : {
    "filename": "l_turn_signal.obj"
},
// Taillights, it includes 2 parts. One is on the trunk, another is on the body
"Taillights" : {
    "filename": "taillights.obj",
    "loose_parts" : [
        "Taillights_on_trunk", "Trunk",          // One part is on the trunk
        "Taillights_on_body", "body" // One part is on the body
    ]
}
},
```


- 7) 其他: vehicle.json是AVM 3D的配置文件，除了设置读取的OBJ文件和纹理外，还有其他重要的配置可以设置。
- version
 - 这是 JSON 的版本号，请勿编辑它。
 - “版本”：“1.0.5”
 - “name”
 - 车辆名称，它不会显示在 AVM 屏幕上，只显示在日志文件中。
 - “名称”：“演示”
 - “camera_path”
 - 指定要读取的 JSON 文件。
 - “camera_path”：“path.json”
 - 它用于前/后 3D 显示模式（见下图）。驾驶员可以改变不同的视角来检查汽车周围的安全。当视角改变时，虚拟摄像头将平稳过渡到当前视角。整个虚拟摄像头系统在 JSON 文件中定义。
 - 前置后置 3D 显示模式
 - JSON 由 UnityCameraPathEditor 生成。它是一种 GUI 工具，可让用户轻松高效地定制虚拟相机系统。如何使用 UnityCameraPathEditor 设计虚拟相机系统，请参阅 3D 虚拟相机系统。
 - “env_mesh”
 - 指定碗的内半径、外半径和高度。
 - 内半径必须小于或等于外半径。
 - 在3D世界中，默认的公制单位是厘米。

```
"env_mesh" : {  
  "radius" : [750.0, 1500.0],    // inner radius=750 cm, outer radius=1500 cm  
  "height" : 1500.0             // height = 1500 cm  
},
```



- "fxaa"
 - 快速近似抗锯齿，它是渲染时是否应用抗锯齿的标志。启用它将提高整个帧的视觉质量，使汽车具有平滑的边缘，但也会增加 CPU 和 GPU 的负载。

```
"fxaa" : true,  
  
...  
  
![fxaa_diff](../images/presentation-3d-model/fxaa_diff.png "FXAA compared")
```

障碍物地图 3D 模型要求

除了3D汽车模型定制之外，AVM系统中通常还允许定制一些驾驶辅助功能，即辅助线（停车辅助线）和障碍物标记。这些功能可以为停车时的转向提供适当的车辆操纵辅助信息，帮助驾驶员更顺利地停车。



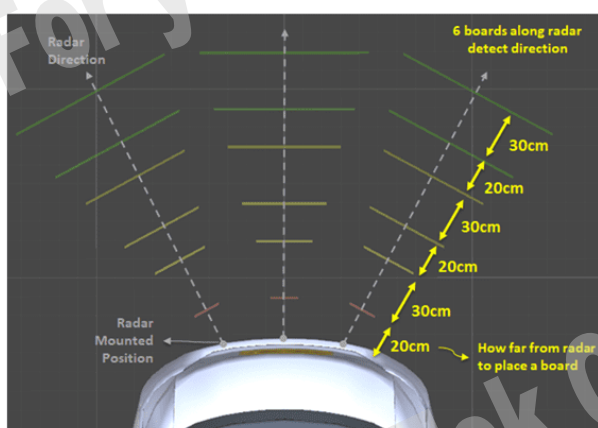
- 车轮轨迹线：

线是动态的，随着转向角度的不同而变化，风格各异。在 AVM 系统中，指导线是通过编程绘制的，您可以在 AVM 代码库中找到完整的源代码和实现细节，随意修改它以满足您的需求。我们不会在接下来的章节中讨论它，因为它不属于建模要求的范围。

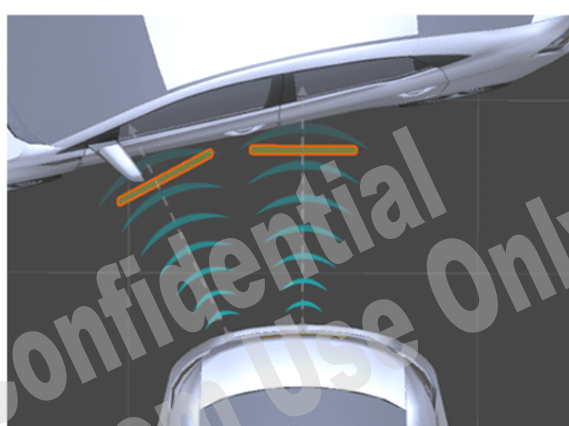
- 障碍物标记:

与指引不同，障碍物标记更为通用且固定。它通常与雷达传感器配合使用，当雷达检测到车辆周围的障碍物时，AVM 会在屏幕上显示最近的板以提醒驾驶员。上图说明了障碍物标记的样子。为了方便自定义板的外观和布局，我们将所有板视为 3D 模型，我们称之为“障碍物地图”。

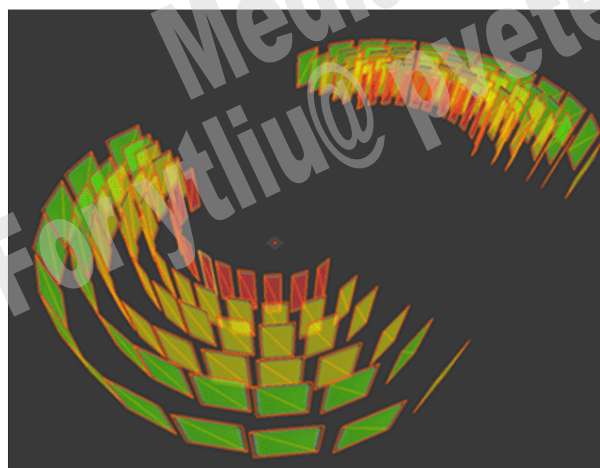
1. 在设计阶段，艺术家会制作所有电路板并将它们放置在预先定义的位置。电路板的放置位置由设计决定，通常与安装在汽车上的雷达传感器的位置和方向密切相关。
2. 运行时，AVM 根据雷达传感器检测的结果显示某些电路板。



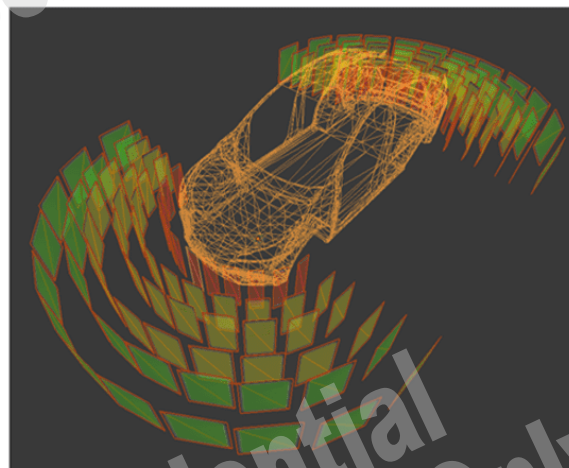
Obstacle map design: 6 boards along radar sensor detect direction



In runtime, boards shown up according to radar detection result



Obstacles Map 3D model in AVM demo.
There are 144 boards around the car
(obstacles_map.obj, obstacles_map.mtl)



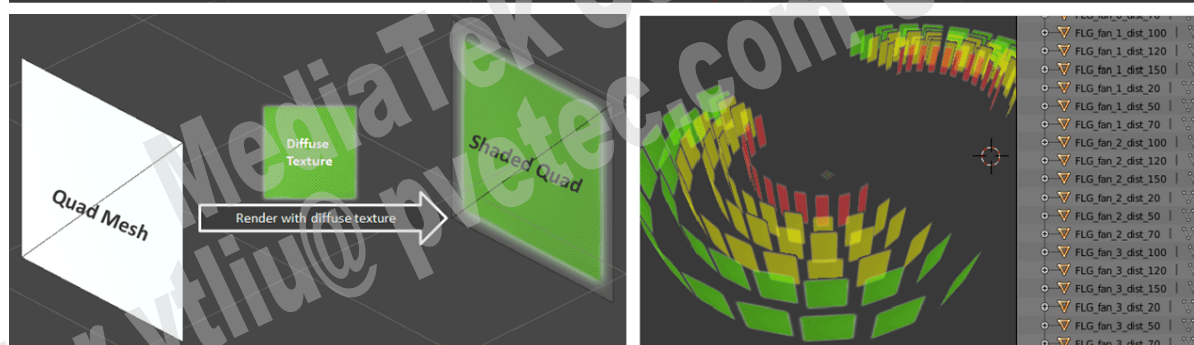
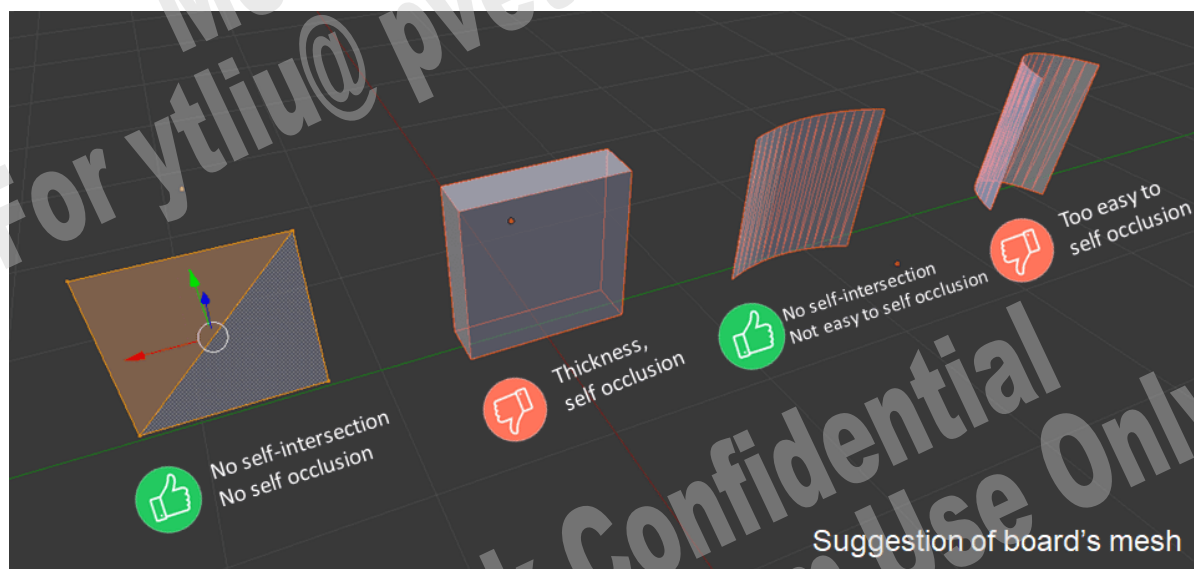
Relative position between car and all boards

建模要求

障碍物地图包含所有板子，导出为OBJ文件和MTL文件。模型坐标系与3D汽车模型相同。

关于每块板，为了获得更好的视觉质量和性能，必须遵循一些规则：

- Board 是一个网格，切片网格，避免自相交、自遮挡和网格与网格相交。四边形是更好的选择。
- 每个板块必须有唯一的名称。名称是访问特定板块并切换其可见性的 ID。
- 使用漫反射纹理控制板的外观，包括板的颜色和透明度。导出时纹理信息将存储在 MTL 文件中。



Diffuse texture to control appearance of board

Each board has unique naming

纹理要求

仅使用漫反射纹理来呈现棋盘。

- 分辨率：512x512 或 256x256。宽度和高度均为 2 的幂。
- 内部格式：RGBA
- 导出文件格式：PNG

为了优化 3D 渲染，还必须将所有板的漫反射纹理转换为压缩纹理格式。您还可以从附录：纹理压缩中找到压缩纹理的分步过程。

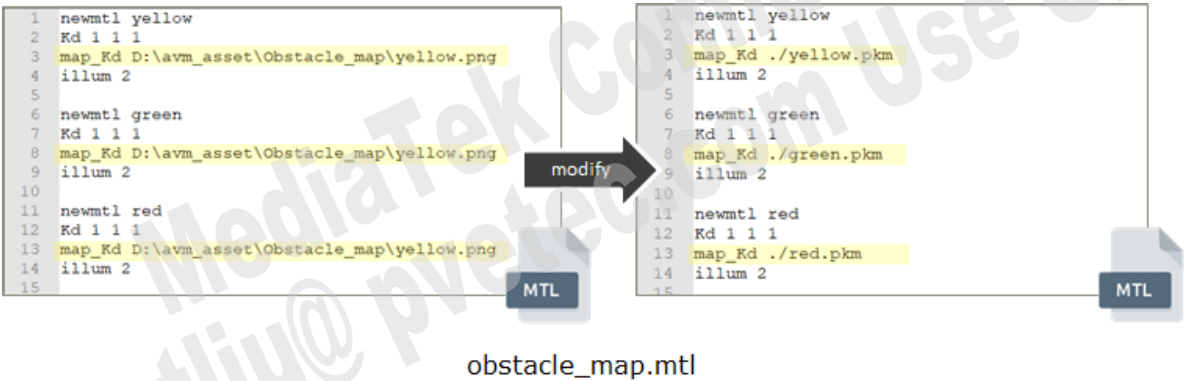
最终输出文件

以下是 AVM 演示的障碍物地图的输出文件，包括一个 OBJ、一个 MTL 和 3 个 PKM 文件。



如何使用自定义障碍地图模型

- 1. 打开Obstacles Map的MTL文件，修改漫反射纹理字段以引用压缩纹理。
 - 找到map_Kd，将纹理路径从绝对路径修改为相对路径，因为漫反射纹理和MTL文件在设备上会放在同一路径下。
 - 将文件扩展名从 PNG 更改为 PKM



- 2. 将 OBJ、MTL 和纹理推送到路径: /etc/automotive/mt6771/avmdata/asset_3d/vehicle/

```
$adb root
$adb remount

// push all OBJ into 3D asset path
$adb push obstacles_map.obj /etc/automotive/mt6771/avmdata/asset_3d/vehicle/
$adb push obstacles_map.mtl /etc/automotive/mt6771/avmdata/asset_3d/vehicle/
$adb push red.pkm /etc/automotive/mt6771/avmdata/asset_3d/vehicle/
$adb push yellow.pkm /etc/automotive/mt6771/avmdata/asset_3d/vehicle/
$adb push green.pkm /etc/automotive/mt6771/avmdata/asset_3d/vehicle/
```

- 3. 编辑 /etc/automotive/mt6771/avmdata/asset_3d/vehicle/ 中的 vehicle.json 以完成自定义障碍物地图设置。

找到 “obstacles” 对象，为障碍物地图填写您的模型文件名。

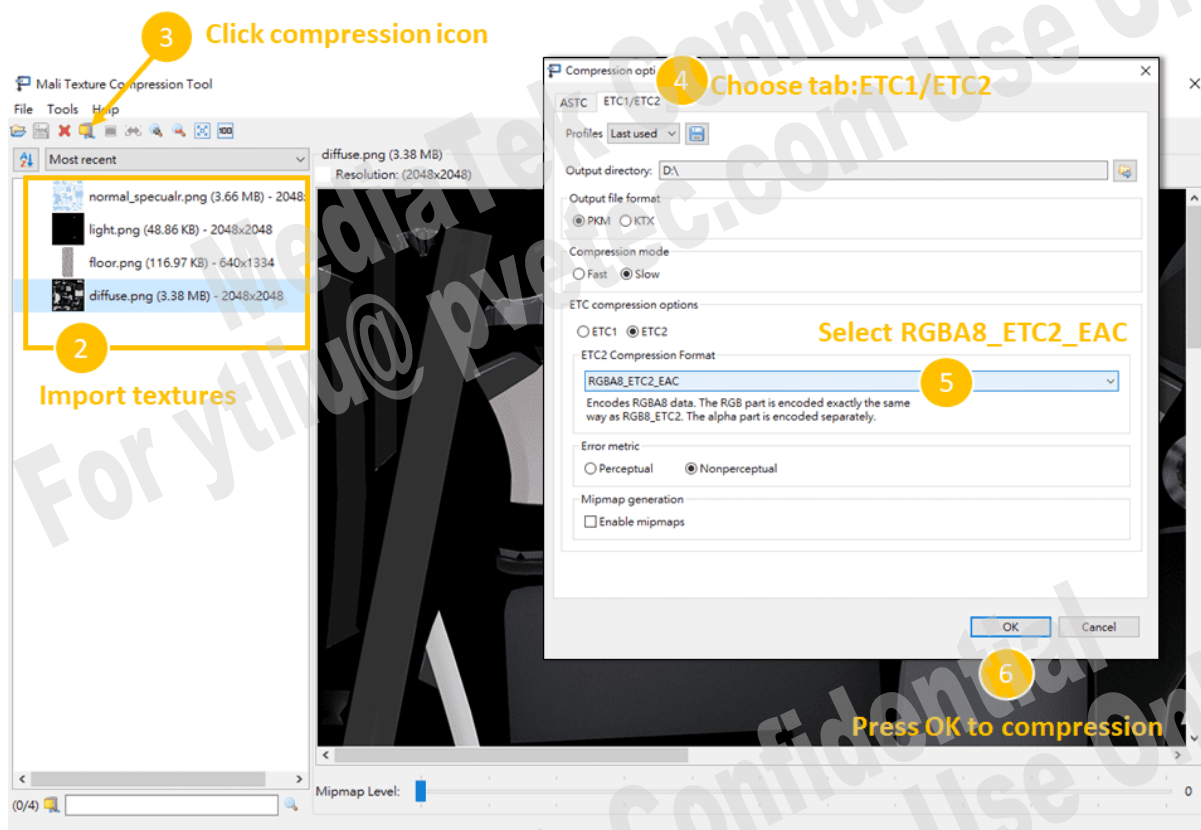
```
"obstacles": {
```

```
"filename": "obstacles_map.obj"  
}
```

附录

纹理压缩

1. 下载 Mali 纹理压缩工具来转换您的纹理。 (<https://developer.arm.com/tools-and-software/graphics-and-gaming/mali-texture-compression-tool>)
2. 打开Mali纹理压缩工具导入纹理
3. 点击压缩图标
4. 在压缩选项菜单上选择选项卡：ETC1/ETC2
5. 选择压缩格式：RGBA8_ETC2_EAC
6. 按OK按钮开始压缩
7. 最后输出压缩纹理（PKM）：



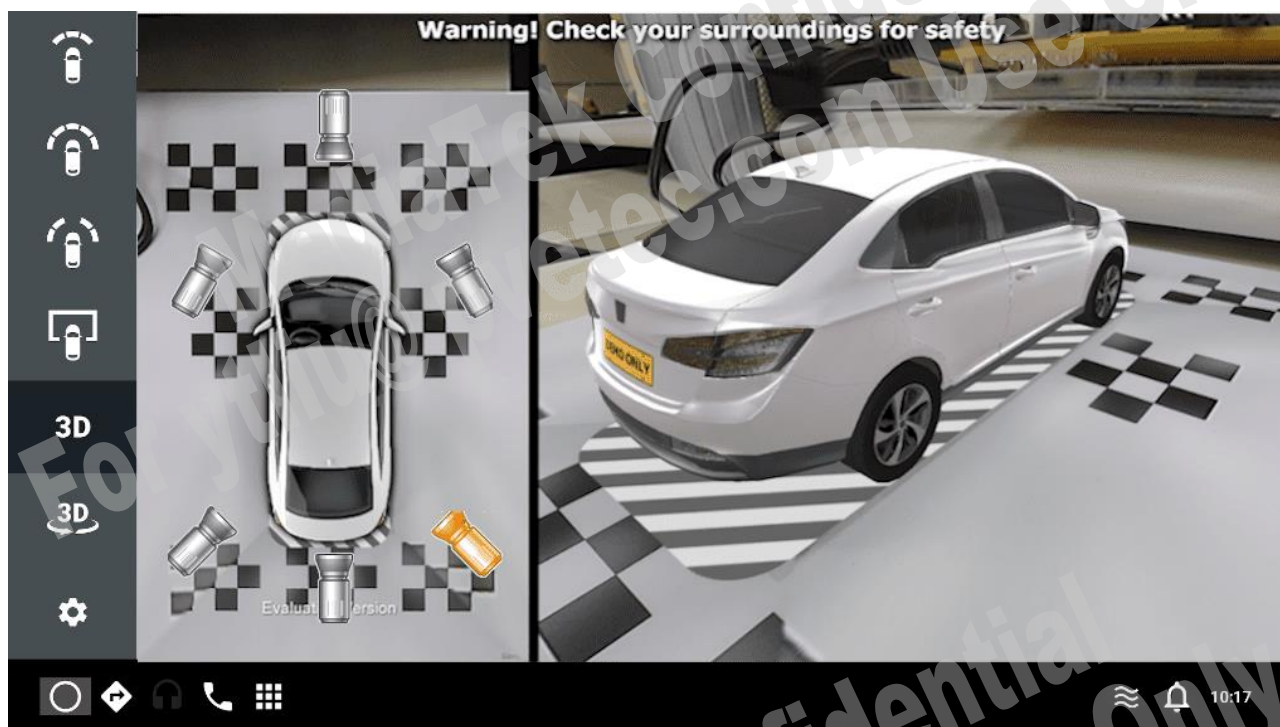
10.3 3D 相机

3D虚拟摄像系统

在 3D 显示模式下，车辆周围有多个预定义的虚拟摄像头，为驾驶员提供不同的视角（见下图）。当驾驶员按下 AVM 上的不同摄像头图标时，我们可以看到从之前的虚拟摄像头视图到当前虚拟摄像头视图的平滑过渡。在 AVM 中，我们默认有两组虚拟摄像头：

- 在前方3D显示模式下，车辆周围有6个虚拟摄像头。
- 在后部3D显示模式下，车辆周围有4个虚拟摄像头。

这些虚拟摄像机的位置和朝向在很大程度上取决于所用车辆的类型。您可以将虚拟摄像机放置在轿车、房车、面包车或跑车的不同位置。AVM 具有允许您自定义自己的虚拟摄像机并在它们之间进行转换的功能。本章我们有一个教程来介绍如何实现这一目标。



{# 测试}

JSON

所有虚拟相机都是预定义的，而不是动态的。所以我们必须将这些虚拟相机系统存储在一个媒体中，然后你可以替换这个媒体来让 AVM 加载不同的虚拟相机系统。最后我们使用 JSON 作为媒体。

3D AVM 中使用了 2 个 JSON 文件，它们位于以下路径：`/etc/automotive/mt6771/avmdata/asset_3d/vehicle/`

- `vehicle.json` 是描述 3D 车辆模型的文件。您可以在 3D 模型中找到有关它的更多详细信息
- `path.json` 是存储虚拟相机设置的文件。所有虚拟相机的信息都写在这里。
 - 与可以轻松打开和修改内容的 `vehicle.json` 不同，`path.json` 不是设计用来由用户直接编辑的，而是由我们称为“UnityCameraPathEditor”的工具生成的。

什么是 UnityCameraPathEditor?

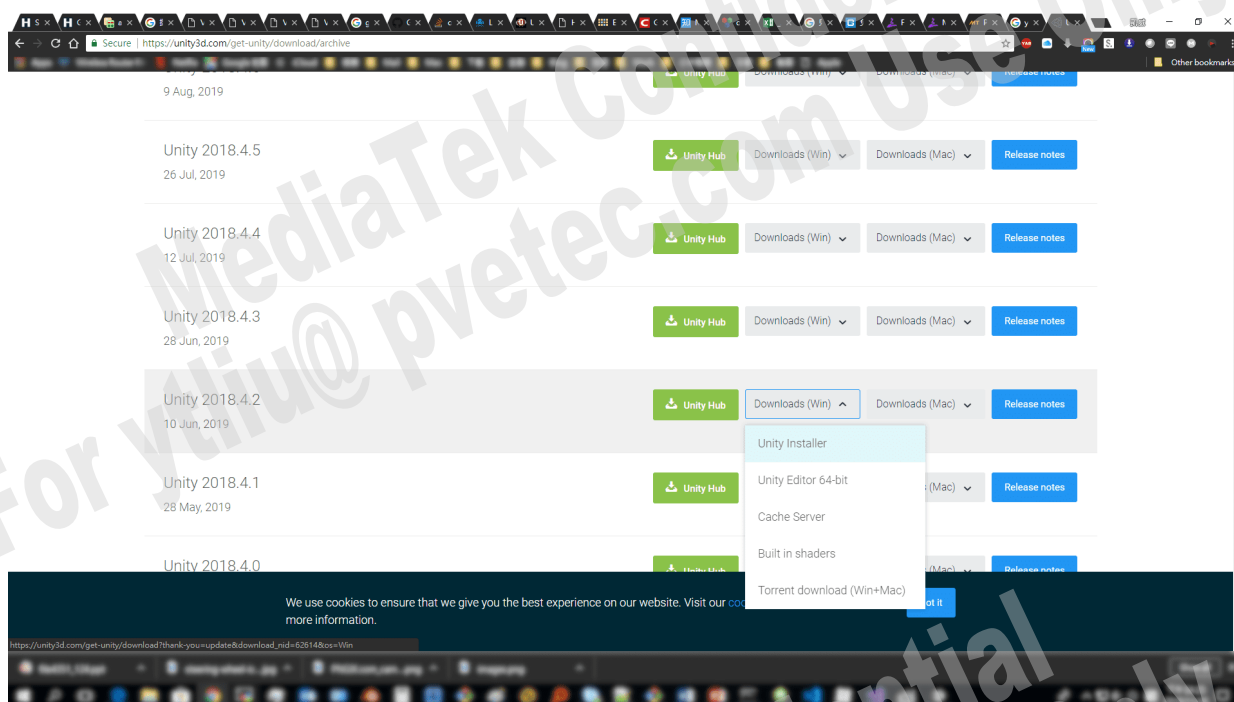
UnityCameraPathEditor 是基于 Unity Engine (<https://unity.com/>) 的插件，该工具功能强大，使用简单，有以下几个优点：

- **3D 环境**
 - 您可以将 3D 车辆模型和虚拟相机放置在 3D 场景中，可以轻松微调虚拟相机的位置，面向 3D 世界。
- **实时预览**
 - 放置虚拟相机后，您可以直接预览视角。它可以方便地确认和锁定您想要在 3D AVM 中显示的视图。
- **自动路径生成**
 - 放置完所有虚拟相机后，工具会自动生成一条路径将所有虚拟相机连接成一条闭合路径。当您在任意两个虚拟相机之间切换不同视图时，虚拟相机将在它们之间进行平滑过渡。

UnityCameraPathEditor 的快速演示

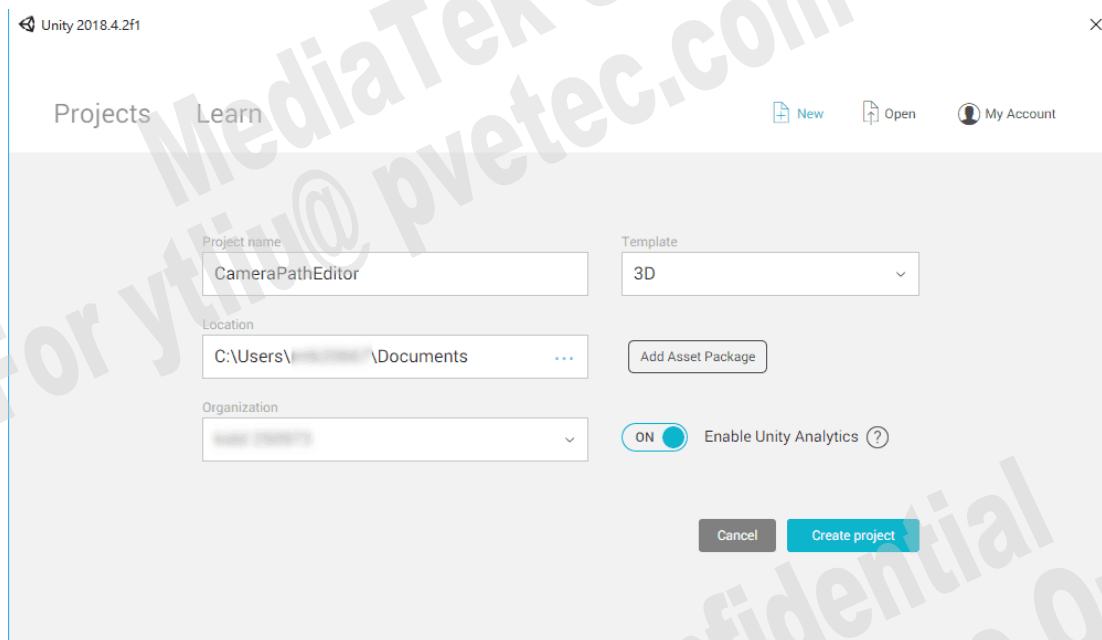
安装 Unity Engine

- 由于 UnityCameraPathEditor 是基于 Unity 引擎的插件，所以如果你之前使用过 Unity 引擎，那么上手会比较容易。如果你从未使用过 Unity，可以参考本教程来熟悉 Unity Editor 的界面。
- 从 Unity 下载档案下载 Unity 2018.4.2 并安装。



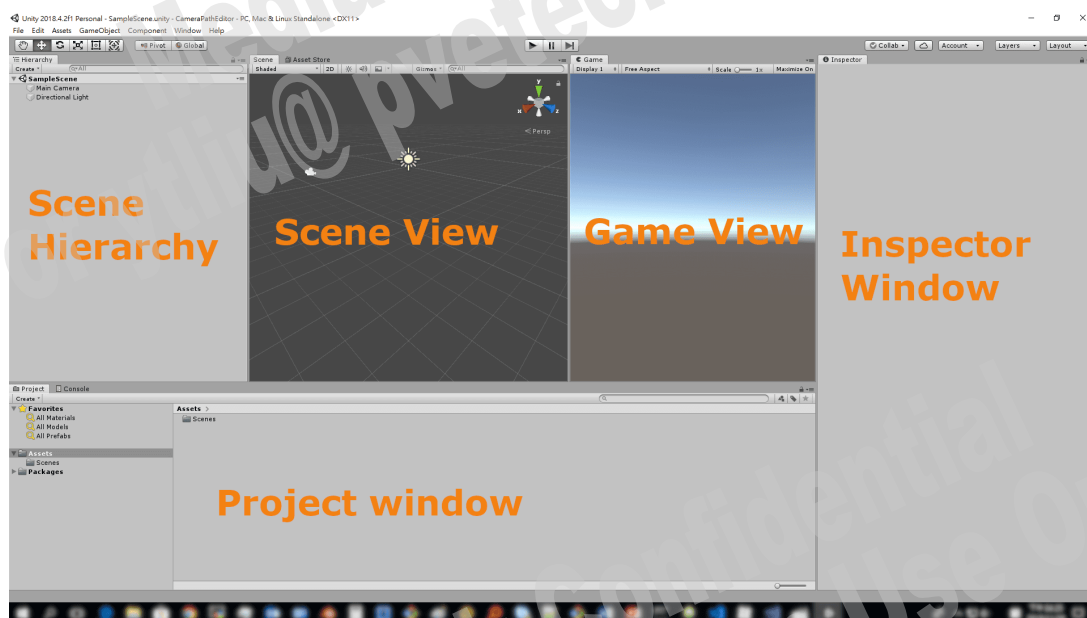
创建Unity项目并导入UnityCameraPathEditor插件

- 安装Unity后，打开Unity并创建一个新的空项目

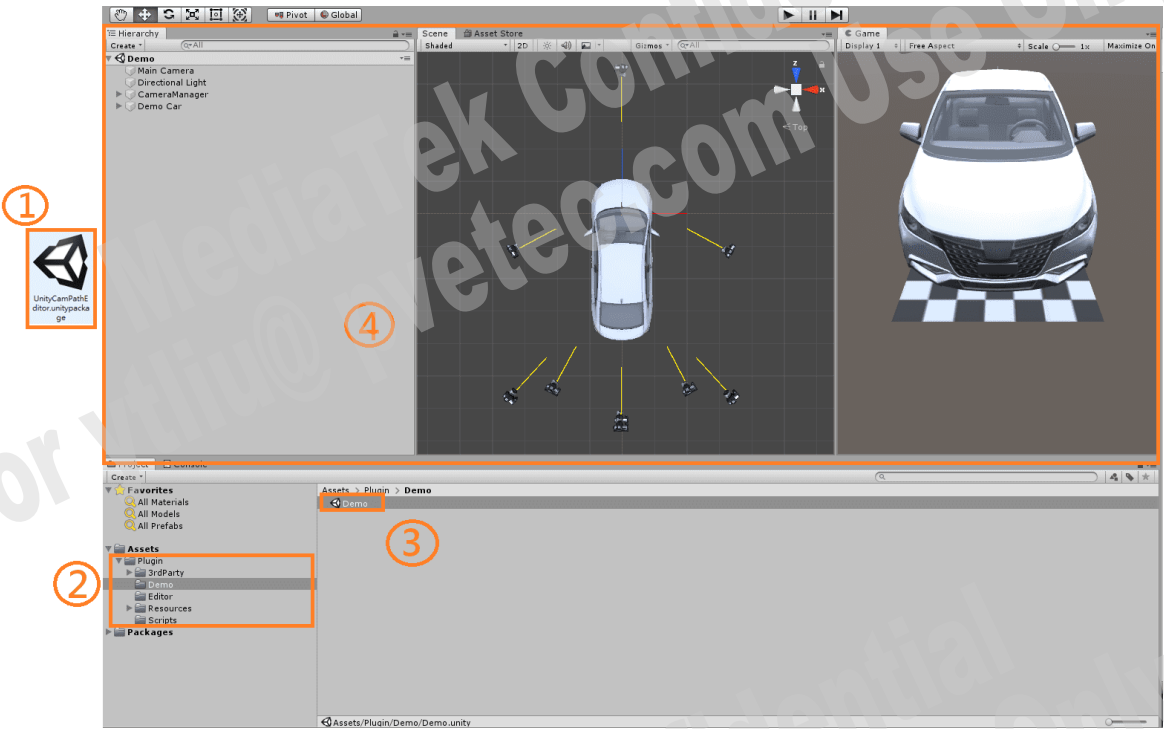


- 当项目打开后，您将看到 Unity 编辑器，其中有几个窗口：
 - 场景层级结构**：它包含当前场景中每个对象的列表。当您在场景中添加或移除对象时，它们也会在层级结构中出现或消失。
 - 场景视图**：这是您对所创建的世界的交互式视图，您可以选择场景中的任何对象，使其平移、旋转和缩放。

- **游戏视图**：它由场景中的相机渲染而成。您可以使用一个或多个相机来控制实际看到的帧。
- **检查器窗口**：它显示有关场景中当前选定对象的详细信息，包括所有附加组件及其属性，并允许您修改场景中对象的功能。
- **项目窗口**：在此窗口中，您可以访问和管理属于您的项目的资产。

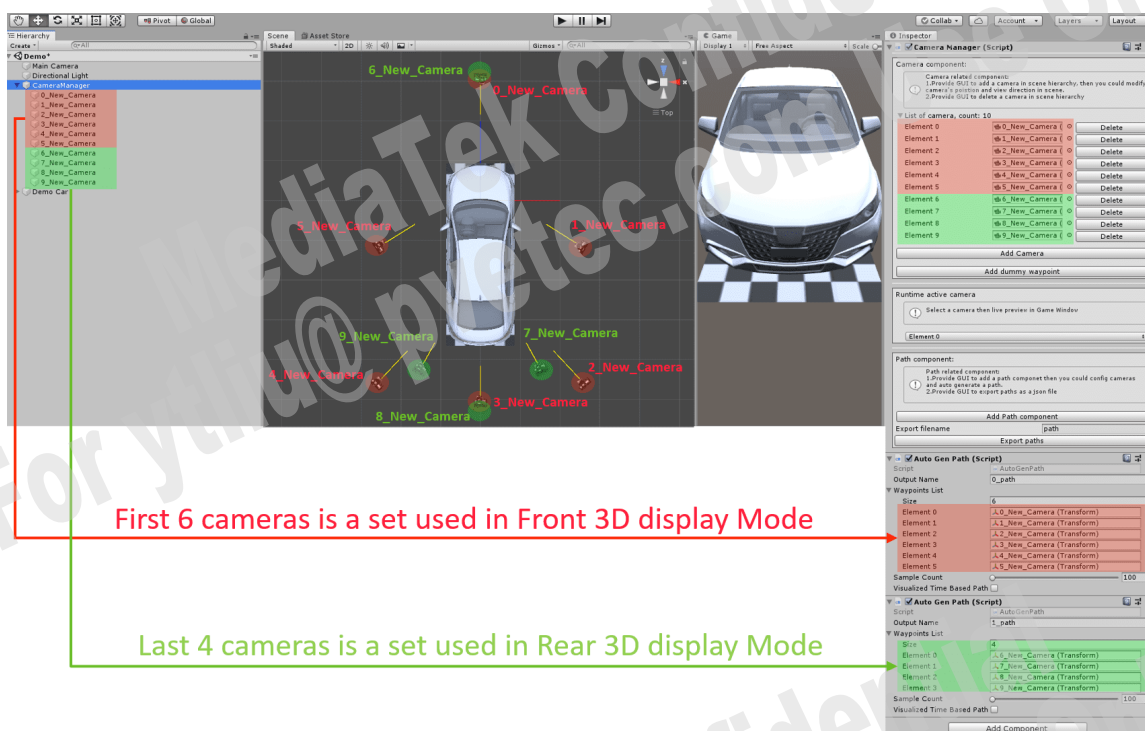


- 导入插件：UnityCamPathEditor.unitypackage到项目。
 1. 点击UnityCamPathEditor.unitypackage导入
 2. 导入后，有一个“Plugin”文件夹，编辑器用到的资源，脚本等都在这里。
 3. 点击“Plugin/Demo/Demo.unity”启动演示场景。
 4. 您会发现演示场景包括一辆演示车和周围的几个虚拟摄像机。



查看演示场景

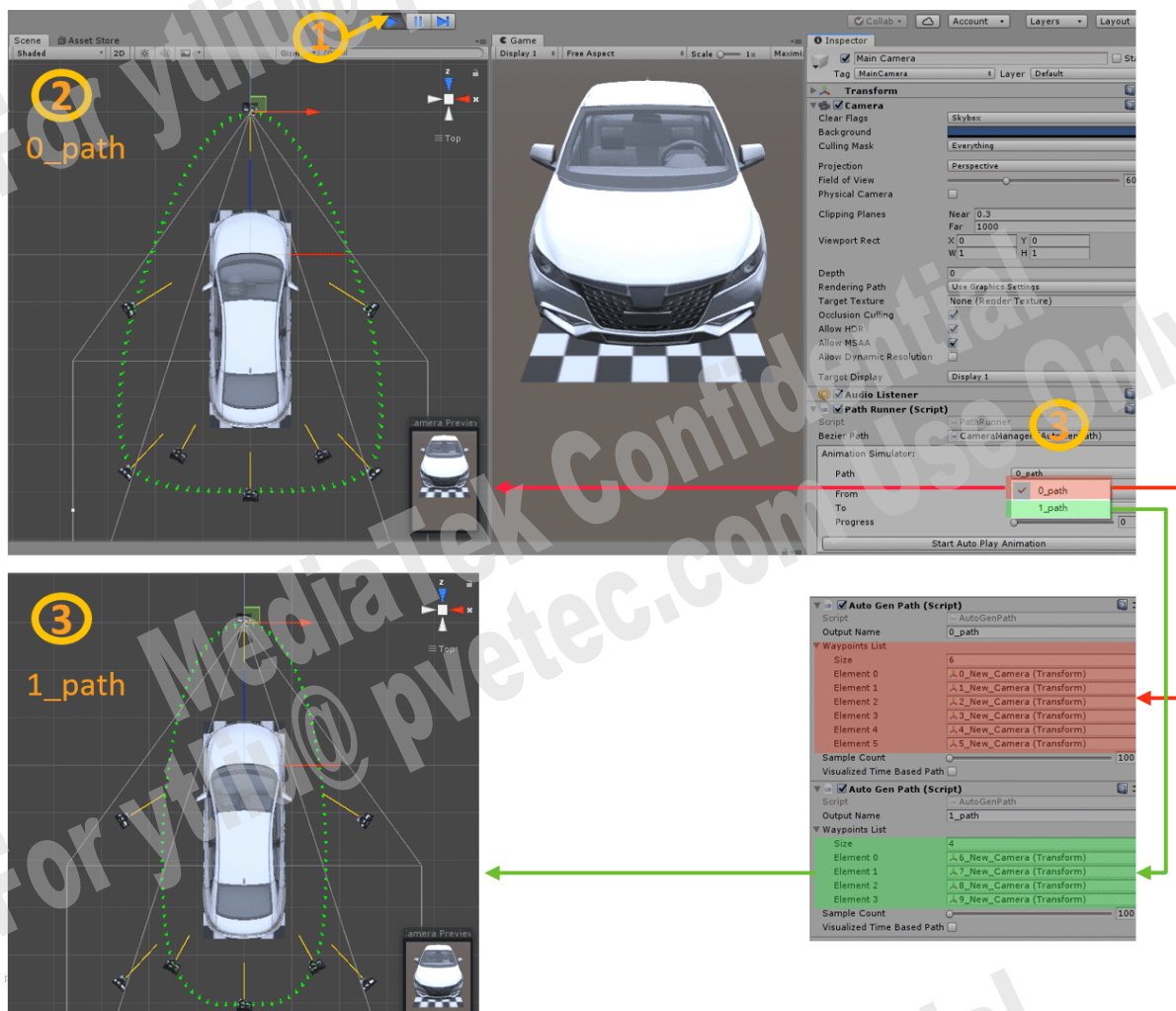
- **CameraManager 对象**
 - 它位于场景层次结构中。它包含所有添加的虚拟摄像机。演示场景中已经有 10 个虚拟摄像机，每个虚拟摄像机都有其位置和视角。
- **CameraManager (脚本)**: 附加在 CameraManager 对象上的主要组件。
 - 这是管理虚拟摄像机的入口（添加或删除）。
 - 它是制作 AutoGenPath（脚本）组件来对选定的虚拟相机进行分组的入口。
 - 负责导出JSON文件。
- **AutoGenPath (脚本)**: 用于对选定的虚拟相机进行分组并为其生成路径的组件。在演示场景中，Camera Manager 对象上附加了 2 个 AutoGenPath 组件：
 - 前 6 个虚拟相机按 AutoGenPath 分组并命名为 “0_path”。这些虚拟相机用于正面 3D 显示模式。
 - 最后 4 个虚拟摄像头由 AutoGenPath 分组并命名为 “1_path”。这些虚拟摄像头用于后置 3D 显示模式。



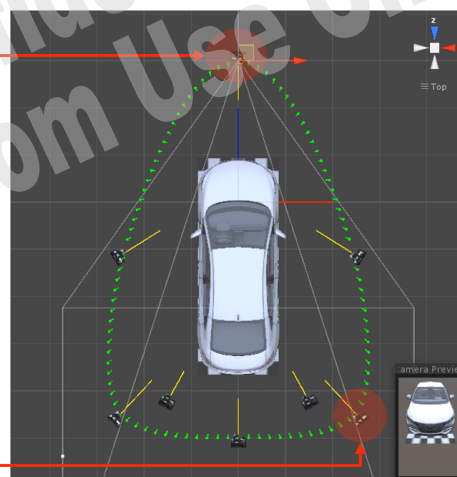
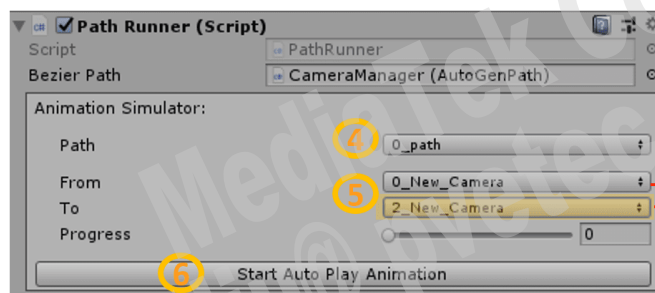
- **主相机对象**
 - 这是 Unity 场景中真正的活动相机。当您按下 “播放” 按钮时（玩）进入PLAY模式，游戏视图显示的是主摄像头看到的画面。
- **PathRunner (脚本)**: 用于模拟虚拟相机如何在不同的虚拟相机视图之间转换。

示范

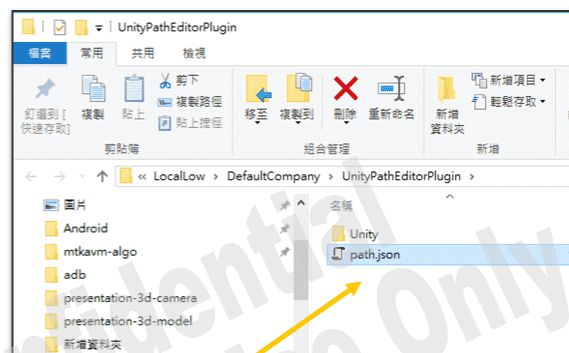
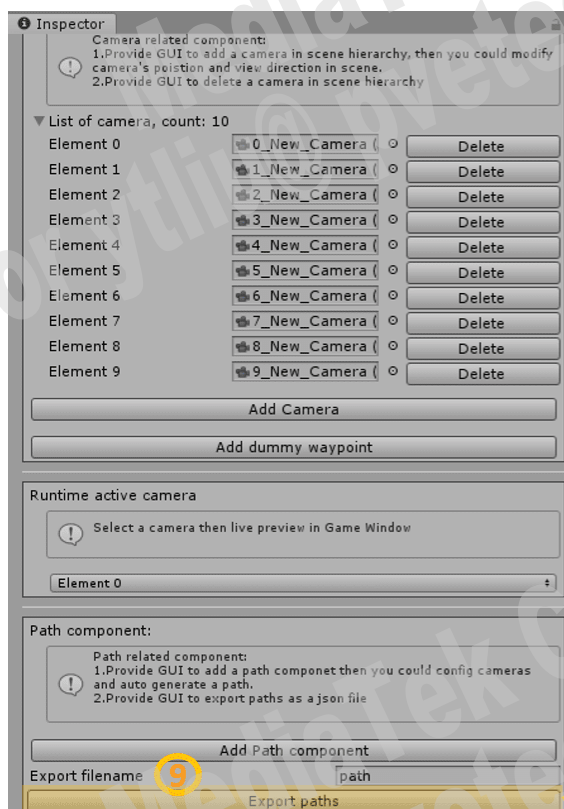
1. 点击“播放”按钮（玩）使编辑器进入PLAY模式。
2. 在场景视图中，你会发现一条绿色路径连接前 6 个虚拟摄像机，该路径由 AutoGenPath（脚本）自动生成。
3. 如上所述，演示场景中有 2 条路径，您可以在 PathRunner（脚本）中选择从“0_path”到“1_path”的“路径”来更改场景中当前的活动路径。



4. 现在我们使用0_path来演示虚拟相机如何在不同视图之间转换。
5. 仍然在 PathRunner(Script) 中，我们设置
 - 'From' = 0_New_Camera
 - 'To' = 2_new_camera
6. 单击“开始自动播放动画”来模拟过渡。



7. 在游戏视图中，你会看到从“0_New_Camera”到“2_new_camera”的平滑过渡
8. 现在，您可以将“From”、“To”更改为 PathRunner（脚本）中的任意不同的虚拟相机。您还可以更改“Path”以检查另一个相机组中的过渡。
9. 最后一步，按“导出路径”将整个虚拟摄像机系统导出为 JSON 文件。
 - 在演示场景中，CameraManager对象上附加了2个AutoGenPath（脚本），因此JSON中会有2组虚拟相机和路径设置。



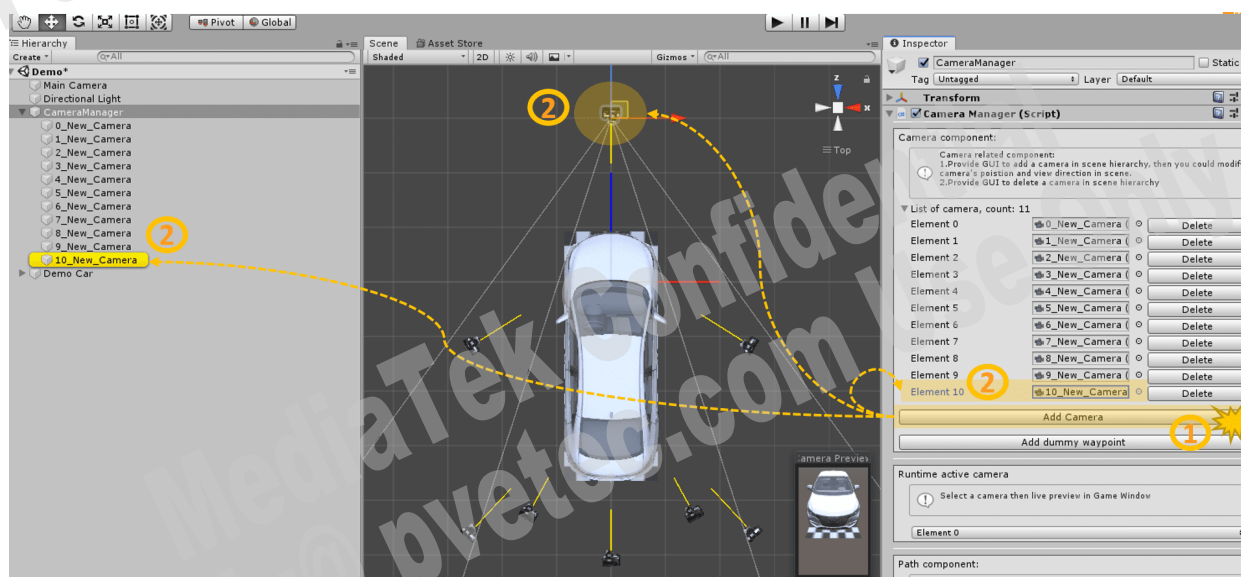
定制虚拟摄像系统

上一章我们通过Demo Scene，了解了UnityCameraPathEditor的功能。本章我们将介绍如何定制自己的虚拟相机系统。

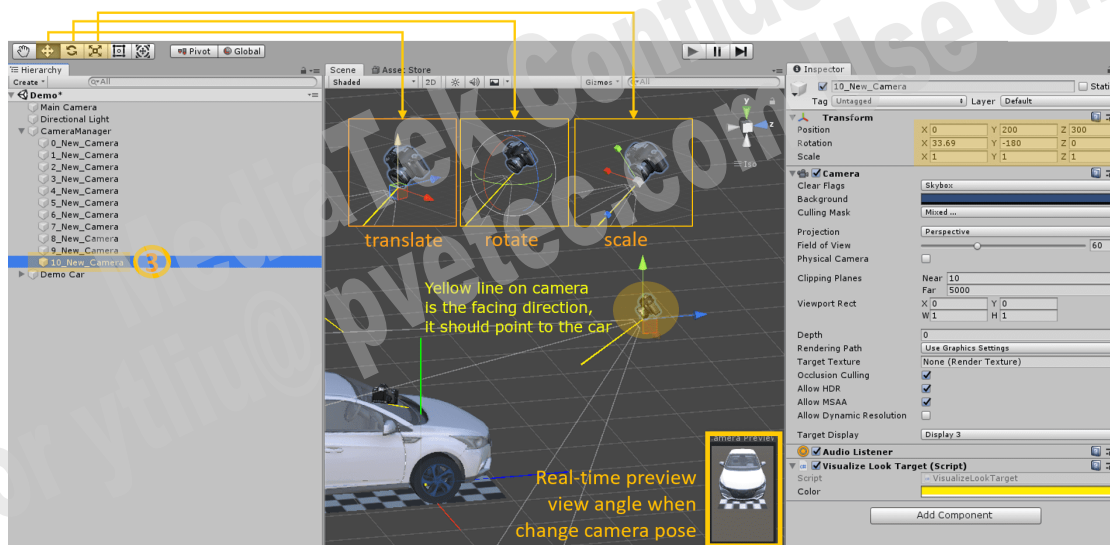
如何添加虚拟摄像头

在编辑之前，请确保编辑器未处于播放模式，“播放”按钮（玩）未按下。在PLAY模式下的任何更改、修改都不会被保存。

1. 在CameraManager（脚本）中单击“添加相机”。
2. 列表、场景层次结构和场景视图中有一个新的虚拟相机。



3. 在场景层次结构中选择新的虚拟相机，您可以通过两种方式改变其姿势：
 - 在变换组件 (Transform Component) 的位置、旋转和比例字段中输入值。
 - 一种有效的方法是使用鼠标在场景视图中手动调整相机的姿势。
 - 调整虚拟相机姿态时可以实时预览视角

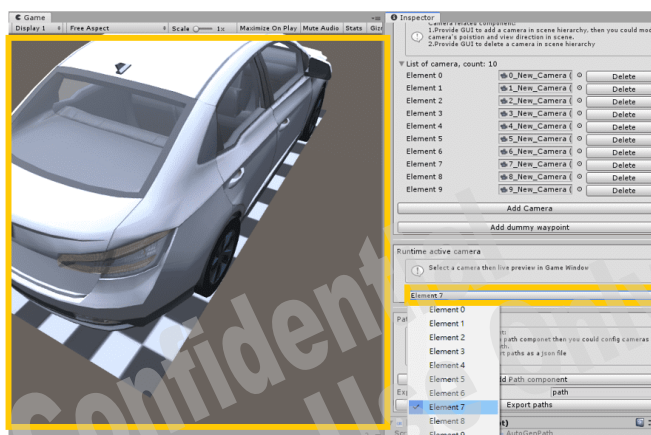
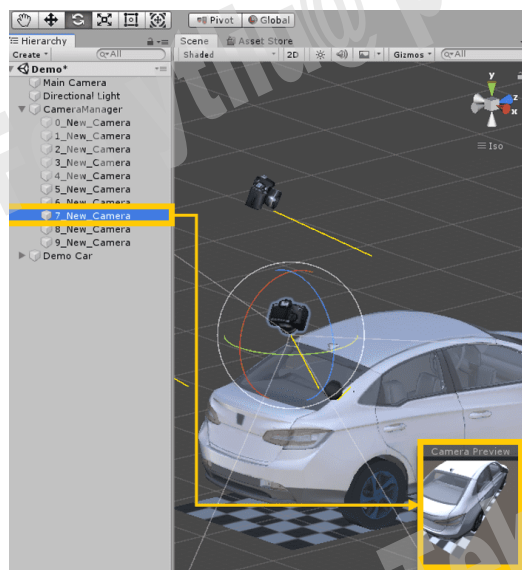


4. 如果您想删除某个虚拟相机，只需按下旁边的“删除”按钮，该相机也将从场景层次结构和场景视图中删除。

预览虚拟相机视角

编辑时预览的方式有两种：

- 在场景层次结构中选择您想要预览的虚拟相机，您将在场景视图的右下角看到一个预览窗口。
- 在CameraManager（脚本）中找到“Runtime active camera”字段，选择要预览的虚拟相机，预览帧将显示在游戏窗口中



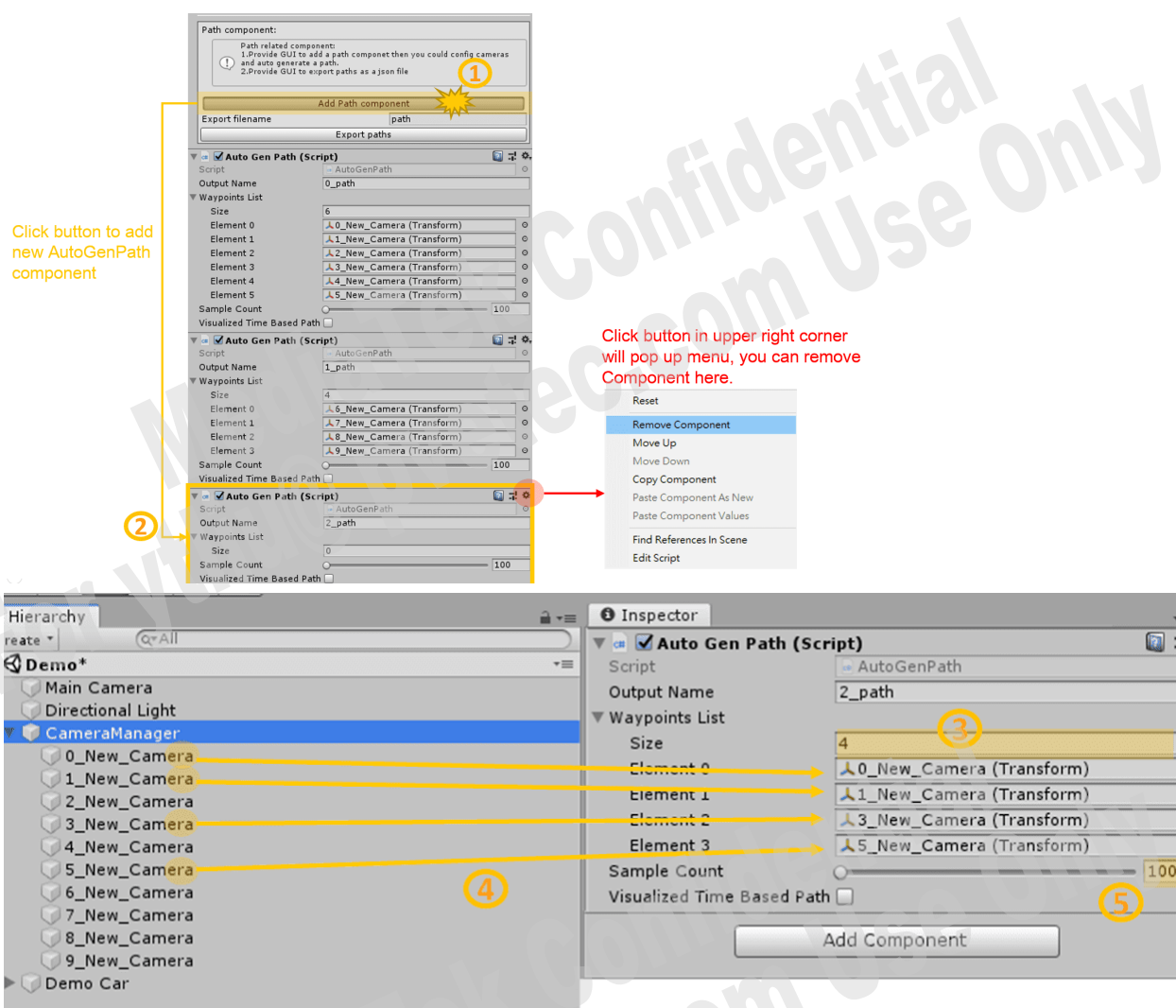
如何生成路径

AutoGenPath（脚本） 负责为选定的虚拟相机生成路径。

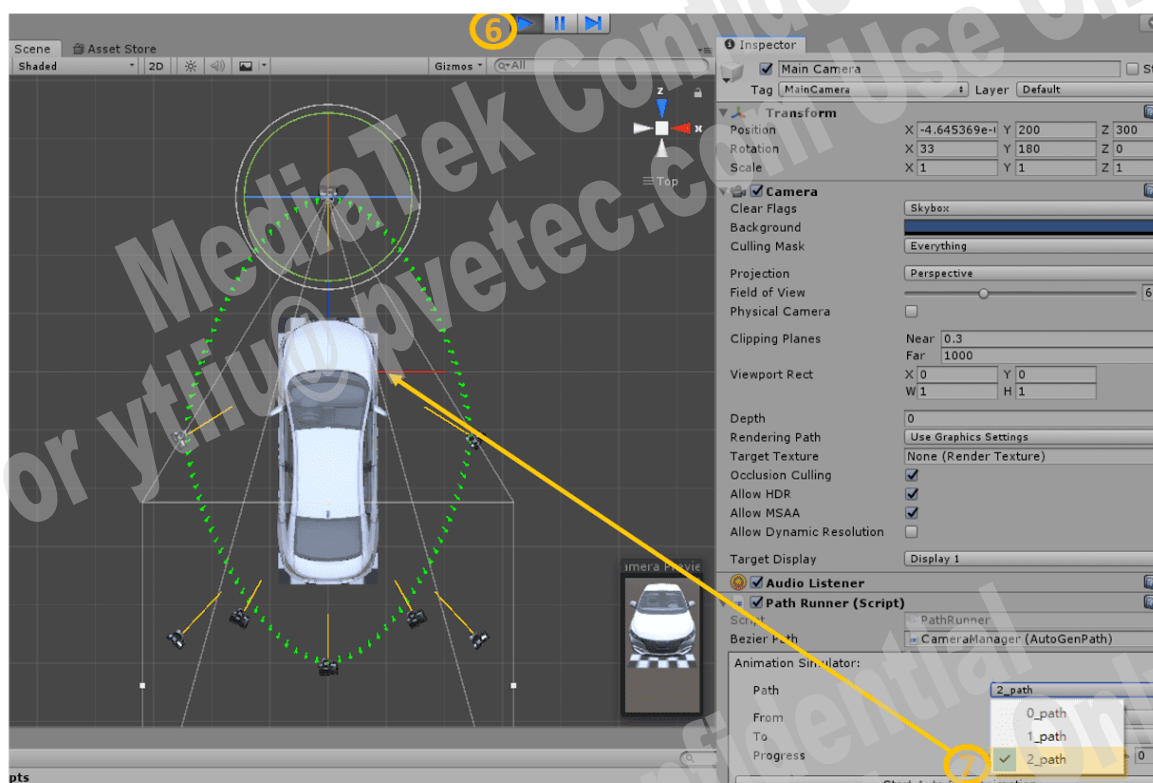
- CameraManager 对象上附加了多少个 AutoGenPath (脚本), 就生成了多少条路径。
- 路径按航点列表的顺序连接虚拟摄像机,
- 路径只在编辑器处于播放模式时生成。在编辑时, 您不会看到任何路径。

按照以下步骤生成路径:

1. 单击 CameraManager (脚本) 中的“添加路径组件”按钮
2. 附加在 CameraManager 对象上的新的 AutoGenPath (脚本)。
3. 输入航点列表的大小。这表示您要将多少个虚拟摄像机连接为一条路径。
4. 从场景层级中拖拽虚拟相机到航点列表的每个字段中。Path 会按照航点列表的顺序依次连接虚拟相机。
5. 必要时修改样本数量。采样点越多, 路径越平滑, 但路径数据的大小也会增加。



6. 点击“播放”按钮(玩)进入PLAY模式查看新添加的路径。
7. 选择主摄像头查看PathRunner (Script), 你会发现有一条新路径生成。

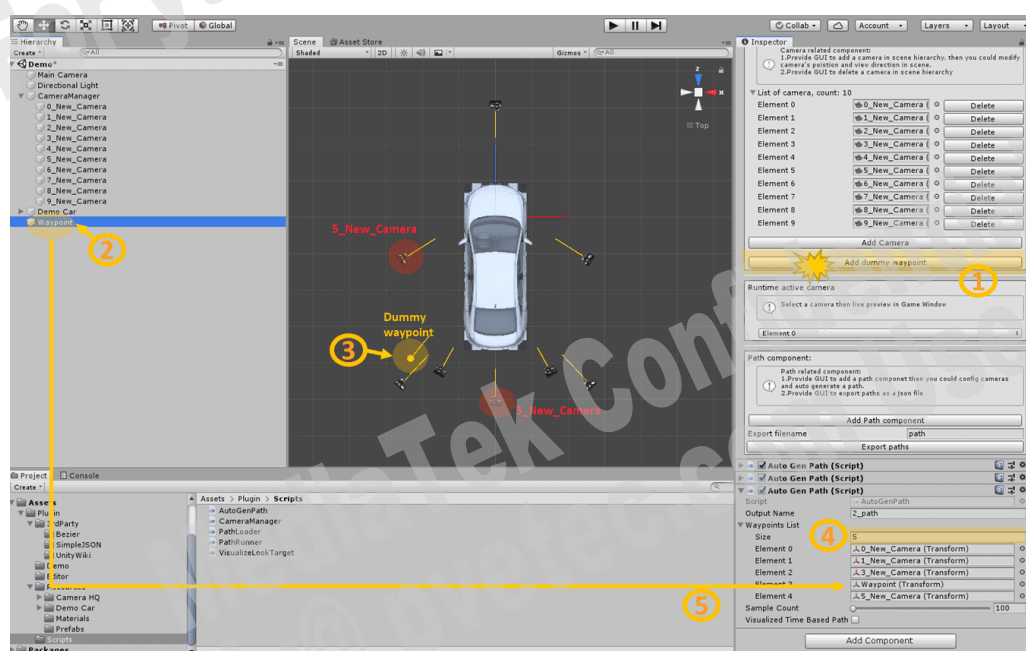


微调路径：虚拟航点

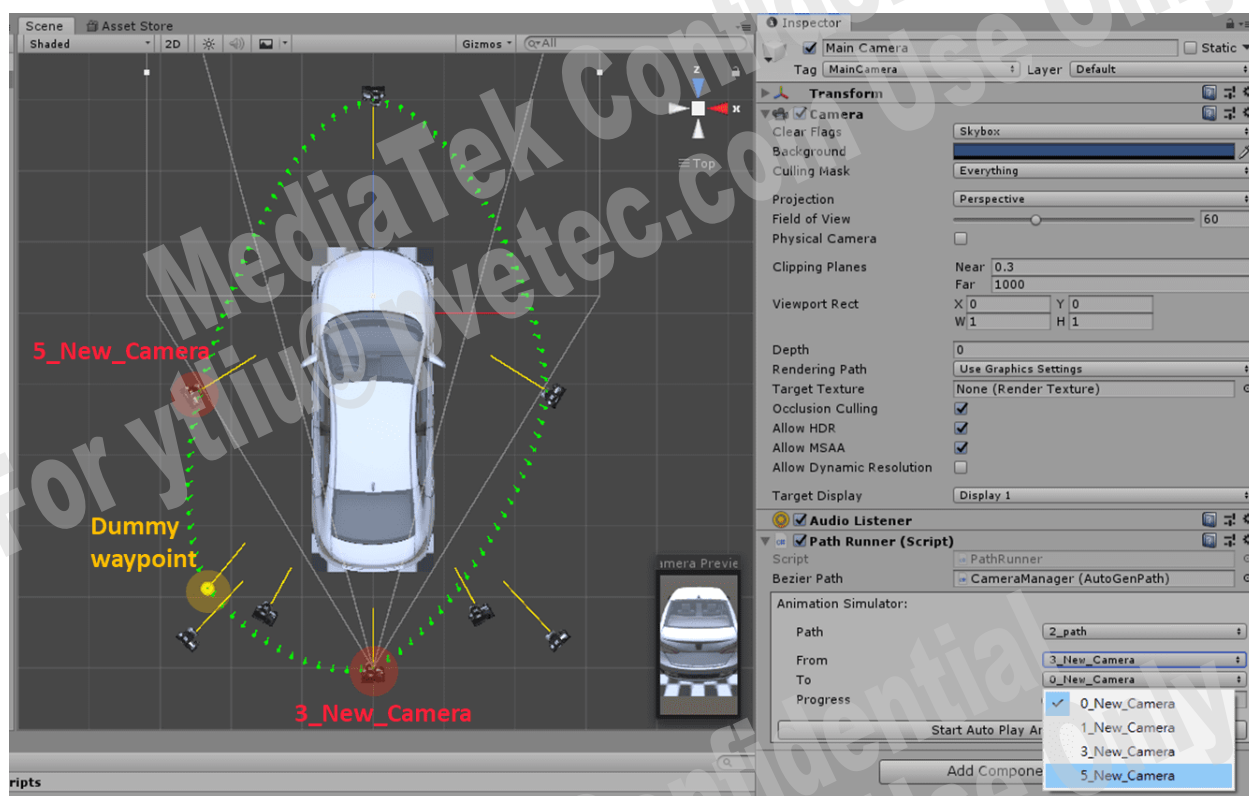
如上所述，路径将依次连接选定的虚拟相机，形成一条封闭路径。但如果 2 个相邻的虚拟相机相距较远，它们之间的路径可能会更直。下图是一个例子，如何调整 “5_New_Camera” 和 “3_New_Camera” 之间的路径，使它们之间的路径弯曲连接？虚拟航点是解决方案。



- 在 CameraManager (脚本) 中单击 “添加虚拟航点”。
- 你可以在 Scene Hierarchy 中找到一个名为 “Waypoint” 的对象。选择它，你可以在 Scene View 中找到它。
- 在 Scene View 中，将 'Waypoint' 放置在 '5_New_Camera' 和 '3_New_Camera' 之间。并确保 Waypoint 面向车辆。
- 修改航点列表的大小，为 “航点” 添加一个字段。
- 将 “Waypoint” 从 “Scene Hierarchy” 拖到字段中。注意顺序，将 “Waypoint” 插入 “5_New_Camera” 和 “3_New_Camera” 之间。



- 进入PLAY模式，检查路径。
 - 路径连接 “5_New_Camera”、“Waypoint” 和 “3_New_Camera”。
 - 路径更加弯曲。
 - “航点” 不是虚拟相机，因此您不会在 “从” 和 “到” 中找到它。它只是路径的中间点。



在 AVm 中使用定制的虚拟摄像机系统

• 如何使用？

- 检查完所有虚拟摄像机的视图和转换后，您可以按“导出路径”按钮将整个虚拟摄像机系统保存为 JSON。然后按照以下命令替换 AVm 中的旧 JSON：

```
$adb push path.json /etc/automotive/mt6771/avmdata/asset_3d/vehicle/
```

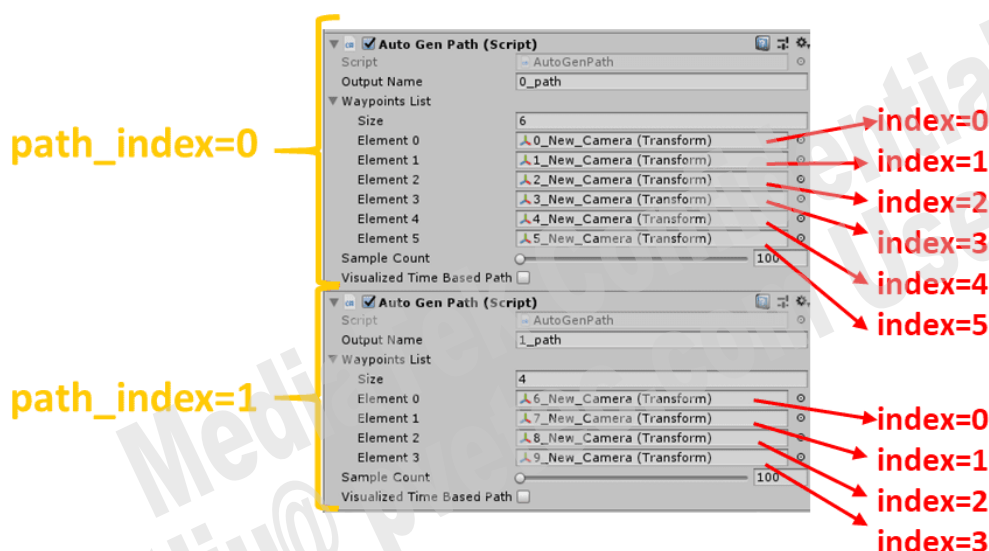
• 如何控制

- 您可以通过 `avm_surroundview_camera_state` 结构控制哪个虚拟摄像头处于活动状态以及转换进度。此结构在 `avm_runtime.h` 中定义。

```
typedef struct {
    int32_t path_index;           // path group index, from 0 ~ (Path group count - 1) in JSON asset
    int32_t active_from_index;    // starting position of virtual animation. Value range depends on JSON.
    int32_t active_to_index;     // end position of virtual camera animation. Value range depends on JSON.
    float active_progress;        // ranges from 0.0f(start) ~ 1.0f(end)
    int32_t expo_mode;           // Enters exhibition mode, virtual camera moves around the vehicle
    float expo_progress;          // Current position of expo camera. It's normalized value, virtual camera will
                                // walk around vehicle through 0~1.
} avm_surroundview_camera_state;
```

- `path_index`：选择要使用的路径。

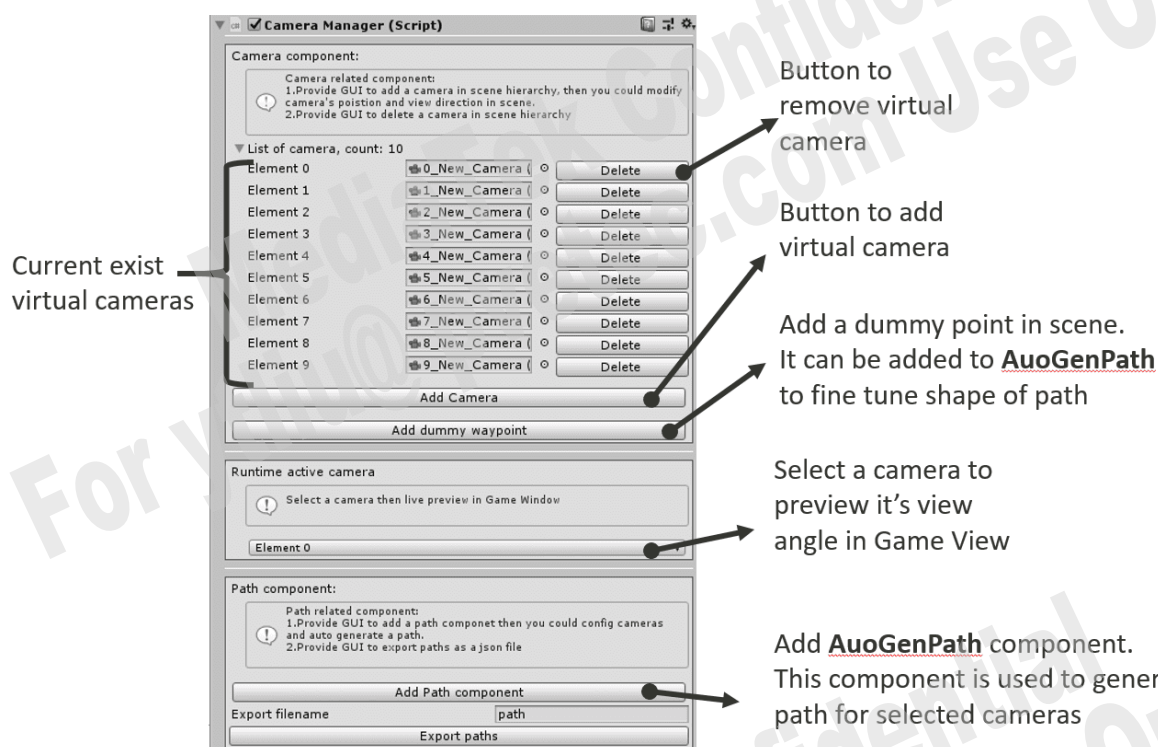
- Index 是 CameraManager 对象中附加的 AutoGenPath 的顺序。
- active_from_index: 选择哪个虚拟相机作为过渡的起点。
 - Index 是 AutoGenPath 中包含的虚拟相机的顺序。
- active_to_index: 选择哪个虚拟相机作为过渡的终点。
 - Index 是 AutoGenPath 中包含的虚拟相机的顺序。
- active_progress: 标准化的值 (0.0~1.0)。
 - 意味着虚拟摄像机被放置在过渡的开始处。
 - 表示虚拟相机位于过渡的末尾。
 - 过渡时每帧更新进度。



附录

CameraManager Script

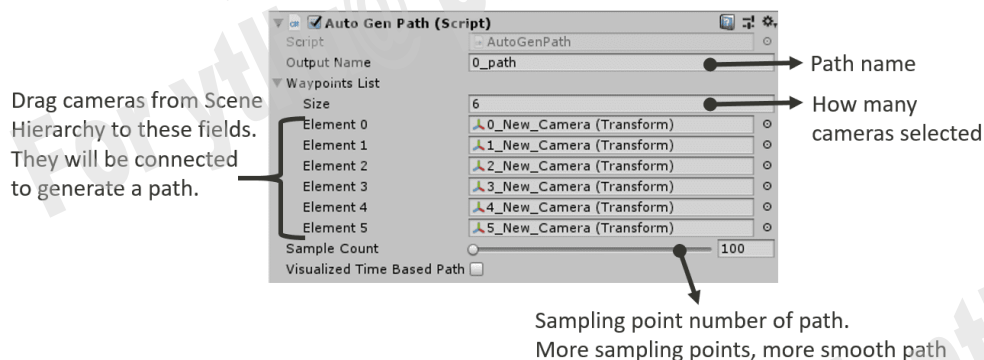
选择 CameraManager 对象后，可以在 Inspector Window 中找到 CameraManager(Script)。它是管理场景中所有虚拟相机的入口。即使您是 Unity Engine 的专家，最好也使用它来添加或删除虚拟相机



AutoGenPath Script

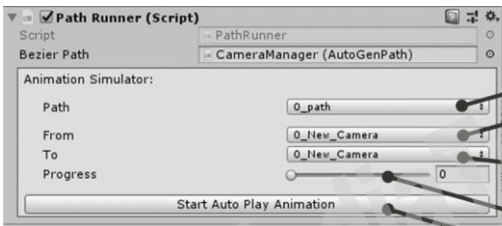
它允许您选择多个虚拟相机来生成路径，该路径将：

- 封闭路径
- 连接所有选定的虚拟摄像机



PathRunner Script

它是一个用于验证生成的路径和转换的模拟器（测试平台）。



- Which path to simulate
- Transition start from which virtual camera
- Transition end with which virtual camera
- Manually control progress of transition
- Auto play transition

11 源代码

源代码概述

以下是源代码结构的简要视图。

Android AVM 源代码树

```
# vendor/mediatek/proprietary/hardware/mtkavm
mtkavm
├── apps
│   ├── avmdemoapk    # AVM Demo APK
│   └── avmcalibapk   # AVM Calibration APK
│
├── libalgo # prebuild libraries
│   └── mt6873
│       └── arm64
│
├── services
│   ├── avmdaemon # AVM service
│   ├── avmclientdemo # mtkavmclient
│   ├── client
│   ├── evpp
│   ├── fastavmservice
│   ├── fastcalibservice
│   ├── inc
│   ├── mt6873
│   │   ├── avmdata
│   │   │   ├── asset_3d // 3D model
│   │   │   │   ├── Common
│   │   │   │   └── vehicle
│   │   │   └── test-480i
│   │   │       ├── test-720p
│   │   │       └── test-fa-customized
│   │   └── calibdata // calibration data, param*.bin
│   ├── mtkgpuproc
│   │   ├── inc
│   │   └── src
│   │       ├── guideline_2dview
│   │       ├── guideline_3dview
│   │       └── marker_3dview
│   └── test
├── doc
├── libguideline
└── include
```

└─src

Yocto AVM 源代码树

```
# src/multimedia/mtkavm_v2
mtkavm_v2
├─avmdaemon
│   └─avmclientdemo
│       └─client
│           └─fastavmservice
│               └─fastcalibservice
│                   └─inc
│                       └─mt8675
│                           └─avmdata
│                               └─asset_3d
│                                   └─Common
│                                       └─vehicle
│                                           └─calibdata
├─mtkgpuproc
│   └─inc
│       └─src
│           └─guideline_2dview
│               └─guideline_3dview
│                   └─marker_3dview
├─test
├─doc
├─libalgo
│   └─inc
│       └─mt8675
│           └─arm64
├─libguideline
│   └─include
│       └─src
```

12 调试 AVm

在 Android 上调试 AVm

日志级别

所有 AVm 日志都将通过 logcat 输出。您可以在 default_config.json 中更改日志级别，重新启动系统即可使其生效。

日志级别：

- LOG_LEVEL_CRITICAL 0
- LOG_LEVEL_ERROR 1
- LOG_LEVEL_INFO 3
- LOG_LEVEL_DEBUG 4
- LOG_LEVEL_VERBOSE 5

```
# /vendor/etc/automotive/xxxx/avmdata/default_config.json
{
  "mtkfastavm_config": {
    "version": "2.3",
    "log_level" : 5, // set loglevel here
    "car_dimensions" : {
      //...
    },
    // ...
  },
  //...
}
```

hwasan

Android官方页面: <https://source.android.com/docs/security/test/hwasan?hl=zh-cn>

Sanitizing individual targets

只要 libc.so 也经过清理，就可以在常规（未清理）构建中针对每个目标启用 HwASan。将 hwaddress: true 添加到 bionic/libc/Android.bp 中“libc_defaults”的清理块中。然后在您正在处理的目标中执行相同操作。

请注意，清理 libc 可在整个系统范围内标记堆内存分配，并检查 libc.so 内内存操作的标记。如果错误的内存访问发生在 libc.so 中（例如，delete() 互斥锁上的 pthread_mutex_unlock()），这甚至可以捕获未启用 HWASan 的二进制文件中的错误。

如果整个平台都是使用 HWASan 构建的，则无需更改任何构建文件。

```
# bionic/libc/Android.bp
cc_defaults {
  name: "libc_defaults",
  # ....
  sanitize: {
    address: false,
    hwaddress: true, // Add this line
    integer_overflow: false,
    fuzzer: false,
  },
  # ....
}

# Then enable hwasan for your module, here is avm
# LOCAL_SANITIZE := hwaddress (Android.mk) or sanitize: { hwaddress: true } (Android.bp).

# mtkavm/services/avmdaemon/Android.mk

LOCAL_MODULE := mtkfastavm
LOCAL_SANITIZE := hwaddress
....

LOCAL_MODULE := libmtkavmfunction
LOCAL_SANITIZE := hwaddress
```

修改AVM的libc/Android.bp和Android.mk后，完整编译包并升级。

13 版本

版本历史

AVM 版本	描述
v3.0-spm	3.0版

AVM v3.0-spm

全景监控器 v3.0 功能：

物品	描述	下载
Development Guide v3.0-spm	本网站	link
AVM PC Tool - Calibration Toolkit v2.3.0	包含工具可执行文件和示例输入文件的压缩包。	MediaTek Online
AVM PC Tool - Customization Toolkit v2.3.0	包含 3D 汽车模型定制工具和示例的压缩包。	MediaTek Online
AVM v3.0 Source Package	Android 源代码版本的附加 repo 项目。	SDK Page

此版本支持以下功能：

- AVM SDK 和演示应用程序
- AVM SDK 支持 2D 顶视图、侧视图
- AVM 演示应用程序支持车轮轨迹线示例
- 光度测定对准
- AVM 演示应用程序为单显示系统提供多种参考显示模式
- 3D AVM，包括：
 - 带有动画车轮、车门和车灯的虚拟车辆模型的 3D 视图
 - 带动画相机的 3D 视图
 - 带雷达的 3D 视图
 - 在 “Free3D” 显示模式下通过触摸屏控制摄像机视角
 - 带有 3D 相机动画和布局动画的演示显示模式
- 用于动画 3D 摄像机路径的离线 PC 编辑工具
- 标定
 - 支持PC离线标定的AVM SDK

- 支持目标端离线标定的AVM SDK
- AVM SDK 支持目标端一键内联标定

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿責任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不应承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。